

Tecnologias Web — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 1 (conteúdo processado de uma só vez, antes das aulas correspondentes)

Conteúdo

A World Wide Web e a Arquitetura Cliente-Servidor	4
Da Internet à World Wide Web	4
O modelo cliente-servidor	4
Navegadores: o cliente da Web	6
Servidores: a parte passiva	6
Panorama de tecnologias da Web	7
HTML	8
Introdução	8
Morfologia dos elementos	10
Estrutura do documento	13
Validação	19
Formulários	20
Tabelas	23
CSS — Regras, Seletores e Padrões	28
O que é uma regra CSS	28
Seletores e declarações: como agrupá-los	28
A cascata	30
Associar CSS ao HTML	33
Seletores de padrões: pseudo-classes e pseudo-elementos	37
Exemplos completos resolvidos	41
CSS — Propriedades e Layout	46
O modelo de caixa (box model)	46
Cor, texto e tipo de letra	49
Posicionamento clássico	52
Flexbox	57
CSS Grid	60
JavaScript — Fundamentos da Linguagem	63
As 3 formas de mostrar output (<code>document.write</code> , <code>alert</code> , <code>console.log</code> — e quando se usa cada uma)	63
Sintaxe básica	64
Objetos e arrays	71
Programação orientada a objetos	80
JavaScript — DOM, CSSOM e Eventos	88
Reflexão e widgets	88
CSSOM	89
A árvore DOM	90
Eventos	97
Exemplos completos	102
O Protocolo HTTP	107
Introdução	107
Examinar HTTP na prática	111
Recursos	112
Métodos HTTP	114
Códigos de estado	117
Cabeçalhos HTTP	120
Segurança	124
Comunicação Assíncrona Cliente-Servidor	125

A forma clássica: formulários e submissão de página completa	125
Serialização de dados	126
XMLHttpRequest	128
De callbacks a Promises a async/await	130
Comunicação em tempo real: SSE e WebSockets	137
Exemplos completos	141
Outras APIs do Browser	145
Web Storage	145
Canvas — fundamentos	147
Canvas — texto	149
Canvas — caminhos e formas poligonais	151
Canvas — propriedades de linha	153
Canvas — curvas	155
Canvas — animação	158
Canvas — embeber e eventos	159
Canvas — transformações	161
Exemplos completos	166
O Servidor: Node.js	171
Primeiro servidor Node (“Olá mundo”)	171
Estruturar código em Node (módulos)	172
Módulos internos do Node	174
Exemplos completos de servidores	183
CGI e Alternativas	194
O modelo CGI	194
Variáveis de ambiente CGI	195
Entrada e saída num programa CGI	196

A World Wide Web e a Arquitetura Cliente-Servidor

Da Internet à World Wide Web

A **Internet** é a infraestrutura física e lógica de redes que liga milhões de computadores em todo o mundo, capaz de transportar qualquer tipo de dados (email, ficheiros, streaming, etc.). A **World Wide Web** (WWW, ou simplesmente **Web**) não é a mesma coisa: é um dos serviços que corre por cima da Internet, especificamente vocacionado para organizar e disponibilizar informação.

Definição

A **World Wide Web** é um **espaço de informação global**, onde:

- os recursos (páginas, imagens, documentos, ...) são **identificados por URLs** (endereço único);
- a **gestão desses recursos** é feita por servidores espalhados pela Internet;
- a **navegação** entre recursos é feita por **hipertexto** — texto (ou outro conteúdo) que contém referências clicáveis para outros recursos.

O nome completo “World Wide Web” descreve bem o conceito (uma “teia” que cobre o mundo, espalhada pela Internet), mas na prática o nome que colou foi simplesmente “**Web**” — o slide de origem nota, em tom de brincadeira, que “a sigla [WWW] era pior que o nome”. A Web tornou-se:

- de utilização **massiva e ubíqua** — está presente em praticamente todos os dispositivos com ligação à Internet;
- a **base para aplicações distribuídas** — a maior parte do software que hoje usamos no dia a dia (redes sociais, email, bancos, streaming) é, por baixo, uma aplicação web;
- a **interface da “nuvem”** — quando falamos de serviços “na cloud”, é quase sempre através de um navegador Web (ou de uma API que segue os mesmos princípios) que lhes acedemos.

Atenção

Nota adicional (não estava explícito nos slides): é fácil confundir “Internet” com “Web” porque hoje em dia é a forma dominante de usar a Internet, mas convém reter a distinção para o resto da disciplina — outros serviços da Internet (email via SMTP, partilha de ficheiros via FTP, chat via IRC, etc.) não são “Web”, porque não usam URLs+HTTP+hipertexto como modelo de organização da informação. A Web é *um* serviço sobre a Internet, entre vários.

O modelo cliente-servidor

Toda a comunicação na Internet — não só na Web — assenta tipicamente no **modelo cliente-servidor**: dois papéis bem distintos que comunicam entre si.

Definição

No modelo cliente-servidor:

- o **cliente** é a parte **ativa**: é quem toma a iniciativa, **pede** um recurso, e depois **processa** esse recurso (por exemplo, mostra-o ao utilizador);
- o **servidor** é a parte **passiva**: fica **à espera**, **recebe** o pedido de um cliente, e **fornece** o recurso pedido como resposta.

Este modelo é comum a toda a Internet (por exemplo, um cliente de email também “pede” mensagens a um servidor de email), mas a Web tem a sua concretização própria e específica deste modelo: os navegadores fazem de cliente e os servidores web (Apache, Nginx, IIS, ...) fazem de servidor.

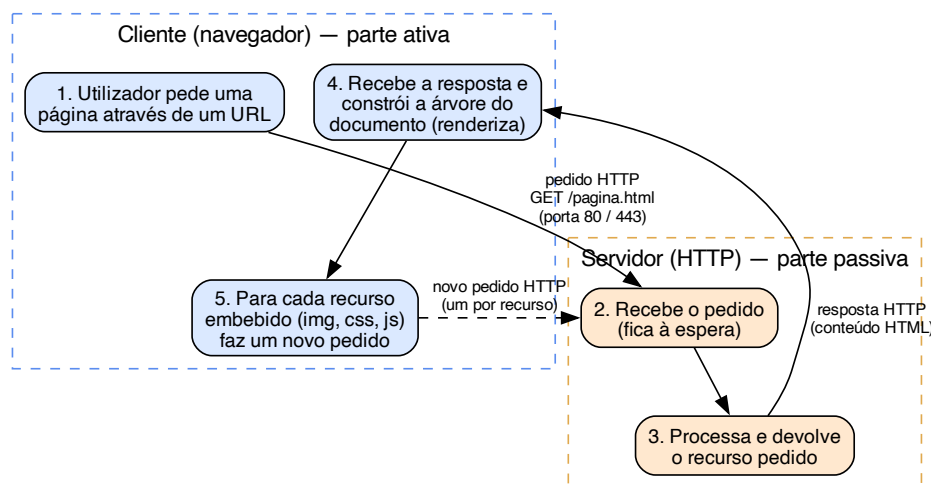
A comunicação entre cliente e servidor na Web é feita:

- sobre a **Internet**, usando a família de protocolos **TCP/IP** como camada de transporte de dados;
- usando um **protocolo aplicacional específico** por cima do TCP/IP — no caso da Web, esse protocolo é o **HTTP** (HyperText Transfer Protocol).

Atenção

Nota adicional (não estava explícito nos slides): os slides mencionam “HTTP” como o protocolo da Web mas não explicam o que lá acontece — vale a pena perceber isto já, porque é a base de tudo o que vem a seguir (URLs, formulários com **action/method**, etc.). Um **pedido HTTP** típico do cliente inclui, entre outras coisas, um **método** (o mais comum é **GET**, para simplesmente pedir um recurso; outro muito usado é **POST**, para enviar dados ao servidor, como acontece nos formulários) e o **caminho** do recurso pretendido. A **resposta HTTP** do servidor inclui um **código de estado** (ex: 200 = sucesso, 404 = não encontrado) e o **conteúdo** do recurso (por exemplo, o código HTML da página). Isto vai ser relevante mais à frente quando falarmos de formulários e dos atributos **action/method**.

O diagrama seguinte resume o ciclo completo de um pedido de página web, incluindo o facto de que uma única página normalmente dispara **vários** pedidos HTTP sucessivos — um pelo documento HTML e depois um por cada recurso que esse HTML referencia (imagens, folhas de estilo, scripts):

**Exemplo resolvido**

Passo a passo do que acontece quando o Gonçalo escreve `http://www.dcc.fc.up.pt/pagina.html` na barra de endereços e carrega Enter:

1. O **navegador** (cliente) interpreta o URL e sabe que tem de contactar o servidor `www.dcc.fc.up.pt` na porta por omissão do HTTP (80).
2. O navegador envia um **pedido HTTP** do tipo **GET /pagina.html** a esse servidor.
3. O **servidor** recebe o pedido, localiza o ficheiro `pagina.html` no seu sistema de ficheiros e prepara uma resposta.
4. O servidor devolve uma **resposta HTTP** contendo o código HTML de `pagina.html`.
5. O navegador recebe esse HTML e começa a **construir a árvore do documento** para o poder desenhar no ecrã.
6. Ao processar o HTML, o navegador encontra, por exemplo, `` — isto **não** veio incluído na resposta anterior, é apenas uma referência. O navegador faz então um **novo pedido HTTP**, desta vez por `logo.png`, ao mesmo servidor.
7. O servidor responde com os bytes da imagem, o navegador recebe-os e **insere a imagem** no local correto da página já renderizada.
8. Este ciclo repete-se para cada recurso adicional (CSS, JS, mais imagens), até a página estar completamente carregada.

Navegadores: o cliente da Web

Os **navegadores** (*browsers*) são o software que desempenha o papel de cliente na Web: pedem páginas, processam o HTML/CSS/JS recebido, e mostram o resultado ao utilizador.

Historicamente, a evolução dos navegadores passou por três fases:

- **Pioneiros** — os primeiros navegadores, como o **Mosaic**, o **tkWWW** e o **Lynx** (este último, em modo texto, ainda hoje usado em contextos muito específicos, como acesso sem interface gráfica).
- **Guerras de browsers** — período de forte concorrência e mudança de domínio de mercado:
 - **Netscape vs Internet Explorer (IE)** — a primeira grande “guerra”, ganha pelo IE (distribuído gratuitamente com o Windows);
 - **IE vs Firefox** — o Firefox, sucessor open-source do Netscape, recuperou quota de mercado ao IE;
 - **Firefox vs IE vs Chrome** — o Chrome, lançado pela Google, entrou mais tarde na disputa e acabou por se tornar dominante.
- **Evolução dos últimos anos** — caracterizada por:
 - **fim da hegemonia do IE** (entretanto descontinuado a favor do Microsoft Edge);
 - **múltiplos navegadores** com quotas de mercado relevantes em simultâneo (Chrome, Firefox, Safari, Edge, ...);
 - **preponderância do acesso móvel** — cada vez mais tráfego Web vem de navegadores em telemóveis e tablets, o que tem implicações diretas no design de páginas (ver mais tarde temas como *responsive design*).

Atenção

Nota adicional (não estava explícito nos slides): esta “guerra” entre navegadores é relevante para a disciplina porque é a raiz histórica do problema da **interoperabilidade** (ver secção “Validação” adiante) — cada navegador, ao longo do tempo, interpretou certas construções HTML de forma ligeiramente diferente, e por isso a *standardização* (através do W3C) e a **validação** do código são importantes para garantir que uma página funciona da mesma forma em todos os navegadores.

Servidores: a parte passiva

Do outro lado da comunicação estão os **servidores web**, o software que corre nas máquinas que alojam as páginas e recursos.

- **Pioneiros**: **NCSA HTTPd** e, mais tarde e com muito mais sucesso, **Apache** — que se tornou durante muito tempo o servidor web mais usado no mundo.
- **Guerra de servidores**: essencialmente **Apache vs Microsoft** (IIS, *Internet Information Services*).
- **Evolução dos últimos anos**:
 - a concorrência entre servidores é **menor** do que a que existe entre navegadores;
 - há **menos competidores** no mercado de servidores do que no de navegadores;
 - há uma **perceção muito menor por parte dos utilizadores finais** — ninguém, ao navegar na Web, sabe (nem geralmente precisa de saber) que servidor está a fornecer a página que está a ver, ao contrário do navegador, que o utilizador escolhe e vê diretamente.

Atenção

Nota adicional (não estava explícito nos slides): esta assimetria de visibilidade faz sentido à luz do modelo cliente-servidor explicado acima — o navegador é a parte **ativa e visível** (é a aplicação que o utilizador abre e usa diretamente), enquanto o servidor é a parte **passiva e invisível** (corre remotamente, sem interface para o utilizador final). Por isso a concorrência e a “visibilidade de marca” dos servidores é muito menor.

Panorama de tecnologias da Web

A Web não é uma tecnologia única e estática — é antes um conjunto de tecnologias relacionadas (HTML, CSS, JavaScript, HTTP, e muitas outras) que foram evoluindo ao longo do tempo, por vezes em paralelo e por vezes em conjunto. Os slides levantam duas ideias importantes sobre esta evolução, que vão ficando mais claras à medida que a disciplina avança:

- **Criação ou co-evolução?** — nem sempre é claro se uma tecnologia nova surge isolada ou como resposta/adaptação a outra tecnologia já existente (por exemplo, o CSS surgiu para separar formatação do HTML, que inicialmente misturava as duas coisas).
- **Profusão de versões, nem sempre alinhadas** — diferentes tecnologias da Web (HTML, CSS, JS, HTTP, ...) têm as suas próprias versões e ritmos de evolução, que nem sempre avançam a par — por exemplo, o HTML5 introduziu funcionalidades (como os novos tipos de `input`, ver secção Formulários) que dependem de suporte simultâneo do CSS e do JS, e a adoção pelos diferentes navegadores nem sempre foi imediata nem uniforme.

Atenção

Nota adicional (não estava explícito nos slides): esta ideia de “profusão de versões” vai aparecer concretamente mais à frente — por exemplo, quando se nota que o HTML5 **já não tem DTD** (ao contrário de versões anteriores do HTML) ou quando se avisa que certos tipos de `input` do HTML5 (como `tel` ou `url`) **não são suportados** em todos os navegadores (por exemplo, no Chrome). Isto é exatamente a tal falta de alinhamento entre tecnologias e entre implementações.

HTML

Introdução

O que é o HTML e para que serve

HTML (*HyperText Markup Language*) é a linguagem usada para descrever o conteúdo e a estrutura das páginas Web. Um documento HTML é, fisicamente, um simples **ficheiro de texto**, escrito com **anotações** especiais que dizem ao navegador como interpretar e apresentar esse texto.

Definição

O HTML serve, essencialmente, quatro propósitos:

1. **Apresentação de conteúdo** — mostra texto e outros conteúdos (imagens, vídeos, ...) ao utilizador.
2. **Anotação do texto** — o texto é “marcado” com anotações que lhe dão estrutura e significado (títulos, parágrafos, listas, ...).
3. **Navegação por hipertexto** — permite criar hiperligações que levam a outras páginas ou secções.
4. **Embeber conteúdo externo** — permite incluir ficheiros de outros tipos (imagens, CSS, JavaScript, vídeo, ...) dentro da página.

Todos os exemplos usados nesta secção de introdução partem sempre do mesmo documento HTML mínimo, que vale a pena decompôr desde já:

Exemplo resolvido

Documento HTML mínimo (usado repetidamente nos slides de introdução):

```
<!DOCTYPE html>
<html>
  <head>
    <title>Olá</title>
  </head>
  <body>
    <h1>Olá HTML 5</h1>
    
    <a href="https://www.w3.org/TR/html5/">
      Mais informação
    </a>
  </body>
</html>
```

Ao abrir isto num navegador, o resultado visual é: um título grande “Olá HTML 5”, seguido da imagem `logo.png`, seguido de uma hiperligação de texto “Mais informação” que aponta para a página do W3C sobre HTML5. Cada uma das próximas subsecções foca-se numa parte diferente deste mesmo exemplo.

Anotações, navegação e conteúdo embebido

A estrutura e formatação do documento são dadas por **anotações** (as construções entre `<` e `>`, como `<html>` ou `<h1>`). São porções de texto delimitadas por *parêntesis angulares* que funcionam como uma espécie de “comando” dentro do texto. Uma forma prática de ver isto no próprio browser é usar a opção “**Ver código fonte**”, que mostra o HTML tal como foi enviado pelo servidor, antes de ser processado.

No exemplo acima:

- a linha `<a href="https://www.w3.org/TR/html5/"> Mais informação </a>` é um **elemento ativo de navegação** — ao clicar no texto “Mais informação”, o navegador carrega uma nova página. Páginas Web são sempre referenciadas por **URLs**, e é exatamente isso que aparece no atributo `href`.
- a linha `` é um exemplo de **conteúdo embebido**: a imagem

`logo.png` não faz parte do texto do documento HTML — é um ficheiro à parte, que o navegador vai buscar separadamente (ver o diagrama de pedidos HTTP na secção anterior) e insere no local indicado. Além de imagens e vídeos, o HTML também permite embeber:

- folhas de estilo em **CSS** (*Cascading Style Sheets*), que controlam a formatação visual;
- programas em **JavaScript** (JS), que controlam o comportamento dinâmico da página.

Frequente em exame

É uma pergunta típica de exame perceber a diferença entre **navegação** (o `<a href=...>`, que troca de página) e **embeber** (o `<img src=...>`, ou CSS/JS, que traz conteúdo de outro ficheiro para dentro da página atual sem trocar de página). Ambos usam URLs, mas com efeitos completamente diferentes.

URLs absolutos vs relativos

Definição

Um **URL** (*Uniform Resource Locator*) é um identificador de recursos na Internet — não é exclusivo da Web (por exemplo, também se usam URLs para identificar recursos de email ou FTP). Um URL pode ser usado para **carregar um recurso**, e nas páginas Web pode assumir duas formas: **absoluto** ou **relativo**.

Um **URL absoluto** identifica um recurso de forma completa e independente de onde estamos, através de três partes:

Parte	Significado	Exemplo
protocolo	Sempre <code>http</code> ou <code>https</code> (a versão segura) na Web	<code>http</code>
servidor	Nome (ou IP) da máquina que fornece o recurso	<code>dcc.fc.up.pt</code>
caminho	Localização do recurso dentro do servidor (idêntico aos nomes-caminho do Linux — diretorias e ficheiros)	<code>/~zp/pagina.html</code>

O separador entre protocolo e servidor é sempre `://`. Opcionalmente, pode ainda indicar-se uma **porta** depois do nome do servidor, separada por `:` — por exemplo `http://www.dcc.fc.up.pt:80/`. Por omissão, os servidores HTTP usam a porta **80** (ou 443 para HTTPS), pelo que normalmente não é preciso escrevê-la explicitamente.

Um **URL relativo**, por outro lado, **não repete** protocolo nem servidor — apenas indica o caminho, **relativo a um URL base** (a diretoria da página que faz a referência), tal como acontece com nomes-caminho relativos no Linux. As vantagens de usar URLs relativos sempre que possível são:

- facilitam a **recolocação** de um conjunto de páginas inter-relacionadas (por exemplo, mudar todo o site para outro servidor sem ter de alterar nenhuma hiperligação interna);
- são mais **curtos** e mais fáceis de escrever.

Exemplo resolvido

Suponhamos que a página `http://dcc.fc.up.pt/~zp/index.html` contém a seguinte hiperligação relativa: `<a href="logo.png">`. Como é que o navegador resolve este URL relativo para saber a que recurso se refere?

1. O navegador identifica o **URL base** — a diretoria onde está a página atual. A página atual é `http://dcc.fc.up.pt/~zp/index.html`, logo a sua diretoria (o URL base) é `http://dcc.fc.up.pt/~zp/`.
2. O navegador **concatena** o URL base com o caminho relativo indicado: `http://dcc.fc.up.pt/~zp/ + logo.png`.
3. O resultado é o URL absoluto equivalente: `http://dcc.fc.up.pt/~zp/logo.png`.
4. É esse URL absoluto, já resolvido, que o navegador efetivamente usa para fazer o pedido HTTP ao servidor.

Se em vez disso a página estivesse em `http://dcc.fc.up.pt/~zp/galeria/index.html` e usasse a mesma hiperligação `<a href="logo.png">`, o resultado seria diferente: `http://dcc.fc.up.pt/~zp/galeria/logo.png` — porque o URL base muda consoante a localização da página que faz a referência. É exatamente esta dependência da localização da página-mãe que torna os URLs relativos tão úteis para mover sites inteiros sem quebrar hiperligações internas.

Morfologia dos elementos

Anotações e contentores

Definição

As **anotações** são o componente fundamental da formatação HTML:

- são delimitadas por chavetas angulares `<` e `>`;
- são iniciadas por um **nome**, que é o “vocabulário” do HTML (ex: `b`, `hr`);
- funcionam como uma espécie de *comando*;
- os nomes são frequentemente **mnemónicos** de uma palavra ou expressão em inglês — por exemplo `br` vem de *BReak* (quebra de linha) e `hr` vem de *Horizontal Rule* (régua horizontal).

Nem todas as anotações se comportam da mesma forma. Há uma distinção fundamental entre **anotações vazias** (como `<br>` ou `<hr>`, que não têm conteúdo nem anotação de fecho) e **contentores**:

Definição

Um **contentor** é delimitado por um **par** de anotações com o mesmo nome: uma anotação de **início** (ex: `<p>`) e uma anotação de **fecho** (ex: `</p>`). A anotação de fecho:

- é **iniciada por uma barra** (`/`);
- é **seguida do mesmo nome** da anotação de início.

Os contentores podem ser **aninhados**: podem conter outras anotações e outros contentores. Quando há aninhamento, uma anotação de fecho termina sempre a **última** anotação aberta com esse mesmo nome (ou seja, o fecho tem de respeitar a ordem LIFO/pilha da abertura).

Exemplo resolvido

`<p> Este contentor tem um <b>pequeno</b> texto </p>`

Aqui há dois contentores aninhados: o `<p>...</p>` (parágrafo) contém, no seu interior, um `<b>...</b>` (negrito) mais pequeno. A leitura correta é: “abre parágrafo, texto normal, abre negrito, texto em negrito, **fecha negrito** (o `</b>` fecha o `<b>` mais recentemente aberto, não o `<p>`), mais texto normal, fecha parágrafo”.

Atributos

Os **atributos** são uma espécie de “opções” das anotações, à semelhança das opções de um comando. Só podem aparecer nas anotações de **início** (nunca nas de fecho), são escritos como uma única palavra (sem espaços), e a **ordem dos atributos é irrelevante**.

Há dois géneros de atributos:

Definição

Atributos simples (também chamados *booleanos*): consistem apenas no seu nome, sem valor associado — por exemplo `<input disabled>` ou `<option selected>`. Têm um comportamento binário:

- **verdadeiro** se o atributo estiver presente;
- **falso** se estiver ausente.

Atributos como pares nome-valor: têm a forma **nome=valor**, separados por sinal de igual — por exemplo `<a href='http://www.up.pt/'>` ou `<img src=casa.png width=90 height=50 alt="Casa d'avó">`. Sobre os valores:

- são sempre *strings*, mesmo quando representam outro tipo de dados (ex: `width=90` representa um número, mas é escrito e interpretado como texto);
- podem ser delimitados por **aspas** ("...") ou **plicas** ('...');
- os caracteres delimitadores em si não fazem parte do valor;
- **têm de** ser delimitados por aspas/plicas se o valor contiver espaços (ex: `alt="Casa d'avó"`);
- podem conter aspas dentro do valor desde que o delimitador escolhido seja o outro (ex: usar aspas para delimitar se o valor tiver uma plica lá dentro, como em `"Casa d'avó"`, ou vice-versa).

Atenção

Nota adicional (não estava explícito nos slides): repare que, tal como no exemplo `<img alt=HTML5 src=../../html5-logo.png style="width: 3cm;">` usado em várias páginas de introdução, os valores **sem espaços** podem ser escritos sem aspas nenhuma (`alt=HTML5`). As aspas só se tornam **obrigatórias** quando o valor contém espaços ou outros caracteres especiais. Na prática, porém, é boa prática usar sempre aspas, mesmo quando não são estritamente necessárias, para tornar o código mais legível e evitar erros.

Entidades

Definição

As **entidades** são um mecanismo de substituição de sequências de caracteres (uma espécie de *macro*), cuja substituição ocorre durante o *parsing* do documento. Na formatação, são sempre iniciadas por `&` e terminadas por `;`. Servem sobretudo para introduzir caracteres que, de outra forma, seriam interpretados como caracteres de formatação (por exemplo, `<` e `>` não podem ser escritos literalmente no texto, porque seriam confundidos com o início/fim de uma anotação).

Entidade	Caráter	Significado
<code>&amp;</code>	<code>&</code>	O próprio <code>&</code> , que de outra forma iniciaria uma entidade
<code>&lt;</code>	<code><</code>	<i>Less Than</i> — evita confusão com início de anotação
<code>&gt;</code>	<code>></code>	<i>Greater Than</i> — evita confusão com fim de anotação

Entidade	Carácter	Significado
"	"	Aspas, úteis dentro de valores de atributos já delimitados por aspas
'	'	Plica, análoga à anterior mas para delimitadores de plica
 	(espaço)	<i>Non-Breaking Space</i> — um espaço que força a formatação (ver secção “Espaçar” adiante)
…	...	Reticências

Atenção

Nota adicional (não estava explícito nos slides): é exatamente por causa disto que, ao longo de todo este resumo (e nos ficheiros fonte originais), se vê tanto `<` e `>` dentro dos blocos de exemplo — é a forma de **mostrar código HTML como texto literal** dentro de uma página HTML, sem que o navegador o interprete como anotações reais. Sempre que um exemplo de código aparece “escapado” desta forma, o mecanismo por trás é o das entidades.

Texto e secções CDATA

O **texto** de um documento HTML é, por definição, todo o carácter que está fora de anotações e de entidades — é o conteúdo “normal” que aparece no ecrã.

Definição

Uma **secção CDATA** é um mecanismo que **inibe** o processamento de formatação dentro dela:

- é delimitada por `<![CDATA[` e `]]>`;
- pode incluir **todos** os caracteres, exceto a própria *string* de fecho `]]>`;
- **não pode haver blocos CDATA aninhados**;
- destina-se sobretudo a incluir fragmentos de **XML embebido**, ou programas, dentro de HTML;
- é relativamente **pouco usada** na prática.

Exemplo resolvido

```
<![CDATA[
  <p> Isto é <u> tudo </u> texto </p>
]]>
```

Dentro desta secção CDATA, o conteúdo `<p> Isto é <u> tudo </u> texto </p>` **não é interpretado como HTML** — é tratado literalmente como texto, e aparece no ecrã exatamente como está escrito (incluindo os sinais `<` e `>`), sem necessidade de o escapar com entidades `<`/`>`;

Comentários

Definição

Um **comentário** em HTML tem a forma `<!-- Isto é um comentário -->`. Regras importantes:

- pode conter anotações no seu interior, que ficam **inativas** (não são processadas);
- comentários **não podem ocorrer dentro** de outras anotações;
- a *string* `--` **não pode ocorrer** dentro do texto comentado;
- por isso mesmo, **comentários não podem ser aninhados** dentro de outros comentários (aninhar um comentário implicaria escrever `--` no meio, o que é proibido).

A declaração `<!DOCTYPE html>`

Definição

A **declaração de tipo** `<!DOCTYPE html>`:

- aparece sempre no **início** do documento HTML, antes de tudo o resto;
- **parece** uma anotação, mas **não é** — repara que tem um `!` antes do nome, o que nenhuma anotação normal tem;
- **parece** um comentário, mas também **não é** — tem um nome (`html`) em vez da sequência `--` que caracteriza os comentários;
- **declara o tipo do documento** como sendo HTML versão 5.

Existem declarações semelhantes para outras linguagens e versões (por exemplo, HTML 4 ou XHTML tinham declarações DOCTYPE bem mais longas e complexas, que remetiam para um DTD — ver secção “Validação”).

Estrutura do documento

Em linha vs bloco: a ideia geral

As anotações e os contentores conferem **estrutura** ao HTML: formam uma espécie de **árvore** que organiza o documento inteiro. Nas folhas dessa árvore podem estar anotações vazias (como `<br>` ou `<img>`) ou simplesmente texto.

Definição

Ao ser desenhado no ecrã, cada contentor é visualizado como uma **caixa**:

- ocupa uma porção retangular do ecrã;
- tem **dimensão** (largura e altura) e **posição** (x, y);
- contém, dentro de si, as caixas dos seus descendentes.

Há **dois géneros principais** de caixas:

- **Em linha** (*inline*) — são colocadas **no fluxo do texto**, lado a lado com o texto circundante, sem forçar quebra de linha antes/depois;
- **de Bloco** (*block*) — ocupam **toda a largura disponível**, o que provoca uma quebra de linha antes e depois do elemento.

Vale a pena ter sempre presente esta tabela-resumo, que junta todos os elementos vistos ao longo desta secção:

Categoria	Elemento	Nome/Significado	Efeito
Em linha	<code>b</code>	Bold	Negrito
Em linha	<code>i</code>	Italic	Itálico
Em linha	<code>u</code>	Underlined	Sublinhado
Em linha	<code>s</code>	Striketrough	Texto cortado
Em linha	<code>sup</code>	Superscript	Expoente

Categoria	Elemento	Nome/Significado	Efeito
Em linha	<code>sub</code>	Subscript	Índice
Em linha	<code>a</code>	Anchor	Âncora / hiperligação
Em linha	<code>br</code>	BReak	Quebra de linha (caixa “achatada”, sem largura útil)
Em linha	<code>code</code>	—	Texto com caracteres mono-espaçados
Bloco	<code>hr</code>	Horizontal Rule	Régua horizontal (bloco sem altura nem conteúdo)
Bloco	<code>p</code>	Paragraph	Parágrafo
Bloco	<code>h1...h6</code>	Heading	Cabeçalho/título, com destaque proporcional ao número
Bloco	<code>ul</code>	Unordered List	Lista não ordenada (<i>bullets</i>)
Bloco	<code>ol</code>	Ordered List	Lista ordenada (números)
Bloco	<code>li</code>	List Item	Item de lista (dentro de <code>ul/ol</code>)
Bloco	<code>dl</code>	Definition List	Lista descritiva
Bloco	<code>dt/dd</code>	Defined Text / Definition	Termo / definição, dentro de <code>dl</code>
Bloco	<code>img</code>	IMaGe	Imagem embebida

Elementos em linha

Os contentores **em linha** mantêm o **fluxo de formatação** do texto e são usados sobretudo em formatação de texto e hiperligações.

Fluxo do texto. Por omissão, o texto é posicionado da **esquerda para a direita**, com as linhas a quebrar automaticamente conforme o espaço disponível. Um ponto crucial (e fonte comum de confusão para quem vem de programação, onde espaços em branco costumam ser significativos):

Frequente em exame

Em HTML, os **caracteres brancos são, por omissão, irrelevantes** para a formatação:

- múltiplos espaços seguidos são reduzidos a um **único** espaço;
- mudanças de linha e caracteres de tabulação **não têm efeito** na formatação — são tratados como se fossem espaços simples.

Isto significa que escrever `Lorem ipsum` (vários espaços) no código fonte produz exatamente o mesmo resultado visual que `Lorem ipsum` (um espaço). Da mesma forma, quebrar uma frase em várias linhas no código fonte **não** cria quebras de linha visuais — o texto continua a fluir normalmente, como se estivesse tudo numa única linha lógica.

Além disso, por omissão, os **caracteres têm largura variável** — por exemplo, um “m” ocupa mais espaço horizontal do que um “i” (nos slides, dá-se a relação aproximada $|m| = 3 \cdot |i|$ e $|a| = 2 \cdot |i|$). O elemento `<code>`, em contraste, usa **caracteres mono-espaçados** (todos com a mesma largura) — mas continua a ignorar caracteres brancos extra na formatação, tal como o texto normal.

Atenção

Nota adicional (não estava explícito nos slides): esta regra de “colapsar” espaços em branco é uma das razões pelas quais indentar e organizar o código HTML livremente (para o tornar mais legível para nós, programadores) **não tem qualquer efeito visual** na página final — o navegador ignora essa formatação extra do código fonte. É também por isso que, para efetivamente forçar espaços ou quebras de linha visuais, é preciso recorrer a mecanismos explícitos (ver “Espaçar”, a seguir), em vez de simplesmente escrever mais espaços ou mudanças de linha no código.

Para **forçar** espaçamento ou quebras que o fluxo normal não permitiria, há dois recursos:

Definição

- O elemento `<br>` (*Break*) introduz uma **quebra de linha** explícita.
- A entidade ` ` (*Non-Breaking Space*) força um **espaço** que não é colapsado com os espaços vizinhos.

Ambos os recursos devem ser usados **com bastante parcimônia** — o HTML foi pensado para que a estrutura (parágrafos, cabeçalhos, ...) dite o espaçamento visual, não inserções manuais de `<br>`/` `.

Formatação de texto. Os contentores em linha **b**, **i**, **u** e **s** mudam o estilo visual do texto que envolvem (negrito, itálico, sublinhado, cortado, respetivamente), e podem ser **combinados encaixando contentores** uns dentro dos outros — por exemplo `<b><i>Negrito+Itálico</i></b>`.

Índices e expoentes. Os elementos `<sup>` (*SUPerscript*, expoente) e `<sub>` (*SUBscript*, índice) alteram a posição vertical do texto em relação à linha de base imaginária do texto normal, reduzindo também o tamanho de letra (para cerca de 75%). São usados sobretudo em formatação de fórmulas — por exemplo, `e = m.c<sup>2</sup>` produz “ $e = m.c^2$ ”.

Exemplo resolvido

Todos estes elementos em linha podem ser **combinados por aninhamento**, tal como os contentores em geral. Um exemplo com três níveis de aninhamento:

```
<b>Negrito
  <sup>expoente
    <sub>índice
  </sub></sup></b>
```

Lendo de fora para dentro: primeiro aplica-se negrito a todo o trecho, depois, dentro dele, a palavra “expoente” (e tudo o que se segue) é elevada como um expoente, e finalmente, dentro do expoente, a palavra “índice” é ainda mais reduzida e rebaixada como um índice — o resultado visual final é a palavra “expoente” em tamanho reduzido e deslocada para cima da linha de base (tudo a negrito), e dentro dela a palavra “índice” ainda mais pequena e deslocada para baixo (um índice dentro de um expoente).

Âncoras — a base das hiperligações. As âncoras, criadas com o elemento `<a>` (*Anchor*), são a base de todas as hiperligações usadas na Web.

Definição

O elemento `<a>` tem dois atributos usados em alternativa (embora um mesmo `<a>` possa, em teoria, usar ambos):

- **name** — **cria um alvo** para uma hiperligação (um ponto endereçável dentro da página);
- **href** — **cria a hiperligação** propriamente dita, apontando para um alvo.

Se o alvo de uma hiperligação for um **name** definido na mesma página (em vez de um URL externo), esse nome é precedido por `#` no valor de **href**.

Quando um `<a>` é usado para criar a hiperligação (com **href**), o seu contentor:

- serve como **ponto ativo** que pode ser clicado;
- o texto correspondente é, por omissão, formatado a **azul e sublinhado**.

Exemplo resolvido

Um padrão muito comum é o link “voltar ao topo” numa página longa:

```
<a name="top"> Vai ser muiiito longo ... </a>
```

```
<!-- ... muito conteúdo pelo meio ... -->
```

```
<a href="#top">Início</a>
```

Aqui, `<a name="top">` marca um ponto **dentro da própria página** chamado **top**. Mais abaixo, `<a href="#top">Início</a>` cria uma hiperligação que, ao ser clicada, não carrega uma página nova — em vez disso, o navegador salta diretamente para o ponto marcado como **top**, que está na mesma página. O `#` no início do `href` é o que diz ao navegador “isto é um nome interno à página atual, não um URL novo”.

Elementos de bloco

Os contentores de **bloco** ocupam toda a largura disponível e são usados para organizar o texto em unidades maiores: parágrafos, cabeçalhos e listas.

Réguas horizontais. O elemento `<hr>` (*Horizontal Rule*) introduz uma linha horizontal — ocupa toda a largura disponível (por isso é um bloco), mas não tem conteúdo nem altura própria (é descrito nos slides como uma “caixa achatada”). Permite criar uma separação visual mais notória do que o simples `<br>`.

Parágrafos. Delimitados pelo elemento `<p>` (*Paragraph*), contêm o texto de um parágrafo — que pode incluir texto simples e elementos em linha (negrito, hiperligações, etc.). Sendo blocos, ocupam toda a largura disponível, e **por omissão reservam espaço entre parágrafos** consecutivos.

Cabeçalhos. Os elementos `<h1>` a `<h6>` (*Headings*) são usados como títulos e subtítulos, com **destaque proporcional** ao número (`h1` é o maior/mais destacado, `h6` o menor).

Listas. Há dois tipos de listas simples:

Definição

- `<ul>` (*Unordered List*) — lista **itemizada**, com marcadores tipo *bullet*;
- `<ol>` (*Ordered List*) — lista **ordenada**, com marcadores numéricos.

Ambas são blocos que contêm **itens**, também eles blocos, delimitados pelo elemento `<li>` (*List Item*). O tipo de lista escolhido condiciona o tipo de marcador usado em cada item (bullet vs número).

Duas particularidades dos itens de lista:

- a anotação de **fecho** `</li>` pode ser **omitida** (o item seguinte, ou o fecho da lista, fecha implicitamente o anterior);
- em listas ordenadas, o atributo `value` num `<li>` permite **forçar** o número desse item específico (por exemplo, `<li value=4>` faz esse item aparecer como “4.”, independentemente da contagem natural anterior).

Atenção

Nota adicional (não estava explícito nos slides): a possibilidade de omitir `</li>` é um exemplo concreto do que a secção “Validação” mais à frente chama de tolerância dos navegadores a HTML “incompleto” — o parser HTML sabe que um novo `<li>` (ou o fecho `</ul>/</ol>`) implica automaticamente o fim do `<li>` anterior, por isso a anotação de fecho explícita é redundante e opcional. Isto **não** significa que seja boa prática omiti-la sempre — é apenas uma tolerância da linguagem.

Definição

Uma **lista descritiva**, criada com `<dl>` (*Definition List*), funciona como as entradas de um dicionário. Em vez de itens `<li>`, usa um par de elementos por entrada:

- `<dt>` (*Defined Text*) — o termo, em lugar do número ou *bullet*;
- `<dd>` (*Definition*) — a definição/texto associado, em lugar do conteúdo simples de um `<li>`.

Exemplo resolvido

As listas podem ser **combinadas** — por exemplo, colocar listas itemizadas dentro de uma lista descritiva, usando cada `<dd>` como contendor de uma sub-lista:

```
<dl>
  <dt> <b>Itemizada</b>
  <dd>
    <ul>
      <li> Primeiro
      <li> Segundo
    </ul>
  <dt> <b>Ordenada</b>
  <dd>
    <ul>
      <li> Primeiro
      <li> Segundo
    </ul>
</dl>
```

O resultado é uma lista de definições com dois termos (“Itemizada” e “Ordenada”, ambos a negrito), em que a “definição” de cada termo não é texto simples, mas sim uma lista itemizada completa com dois itens. Isto mostra bem como os blocos podem ser aninhados livremente dentro de outros blocos, desde que a estrutura de contentores/fecho seja respeitada.

Imagens. O elemento `<img>` insere **conteúdo embebido** a partir de um ficheiro externo, com dimensões (altura e largura) determinadas pelo próprio conteúdo (a não ser que sejam sobrepostas por atributos).

Definição

Atributos do elemento `<img>`:

- `src` (*SouRCe*) — o URL do ficheiro da imagem;
- `alt` (*ALternative*) — texto alternativo, mostrado se a imagem não puder ser carregada (e usado por leitores de ecrã para acessibilidade);
- `width` — largura em pixels da imagem;
- `height` — altura em pixels da imagem.

A estrutura de topo do documento

Além da estrutura interna de cada bloco/linha, o documento HTML como um todo tem também a sua própria estrutura obrigatória, hierárquica.

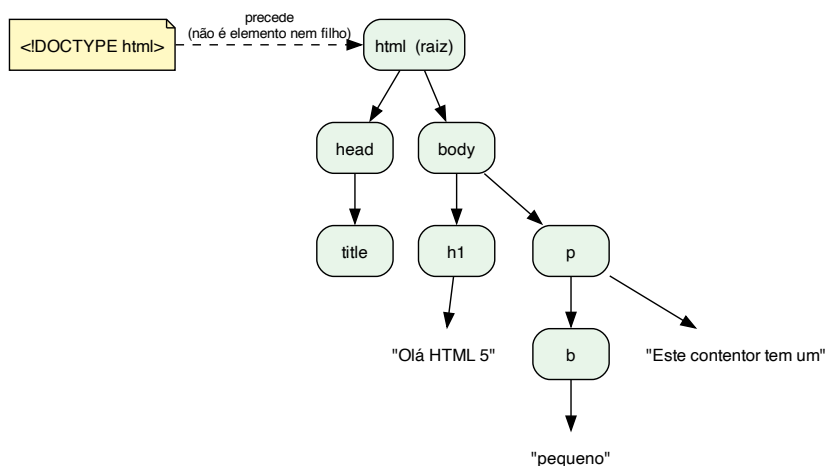
Definição

Todo o documento HTML:

- deve começar pela **declaração de tipo** `<!DOCTYPE html>`;
- tem uma **única raiz**, o elemento `<html>` (a declaração de tipo e eventuais comentários antes dela não contam como parte da árvore, nem como filhos de `<html>`);
- essa raiz é constituída por exatamente **dois elementos**:
 - `<head>` — o **cabeçalho** do documento (contém metadados, como o `<title>`, mas **não** é apresentado na área de trabalho do navegador);
 - `<body>` — o **corpo** do documento (contém a formatação efetivamente apresentada na área de trabalho do navegador — é aqui que vive a generalidade dos elementos que vimos até agora: parágrafos, cabeçalhos, listas, imagens, etc.).

Dentro do `<body>` não há uma estrutura rígida imposta — mas **certos elementos têm de estar contidos dentro de outros** (por exemplo, um `<li>` só faz sentido dentro de um `<ul>/<ol>`, um `<td>` só faz sentido dentro de um `<tr>`, etc., como se vê nas secções de listas e tabelas).

O diagrama seguinte mostra a árvore correspondente ao documento mínimo usado ao longo desta secção de introdução, deixando claro que o `<!DOCTYPE html>` **precede** a árvore mas não é, ele próprio, um nó dessa árvore:



Frequente em exame

Um erro comum é pensar que `<!DOCTYPE html>` é o elemento raiz do documento. **Não é.** A raiz é sempre `<html>` — o `DOCTYPE` é apenas uma declaração que antecede a árvore, informando o navegador de que tipo de documento se trata (ver secção Morfologia). Da mesma forma, é comum esquecer que `<head>` e `<body>` são os **únicos** dois filhos diretos permitidos de `<html>`.

Validação

Definição

HTML pode ser visto como um **tipo de documento anotado**: existe um conjunto bem definido de anotações válidas, e cada tipo de anotação:

- pode ter **certos atributos** específicos (nem todos servem em todas as anotações);
- pode conter **certos tipos de anotações** dentro de si (eventualmente nenhum, no caso de anotações vazias).

Todo o documento tem uma **única raiz** (o `<html>`, como visto acima). No HTML 4 e versões anteriores, esta gramática de tipos era formalizada através de um **DTD** (*Document Type Definition*). O **HTML5 não tem DTD** — a sua gramática é definida de outra forma pela especificação, mas a ideia de “tipos de anotações válidos” mantém-se.

Por que validar interessa: interoperabilidade. Os navegadores Web são construídos para serem **tolerantes** e aceitarem HTML tecnicamente inválido, em vez de simplesmente recusarem mostrar a página. O problema é que, quando o HTML é inválido, **diferentes navegadores podem interpretá-lo de forma diferente** — e por isso a experiência de visualização do utilizador pode variar de navegador para navegador.

Atenção

Nota adicional (não estava explícito nos slides): a figura de origem (“Interoperabilidade”) mostra vários exemplos de HTML tecnicamente incorreto que, ainda assim, os navegadores toleram de alguma forma — mas com resultados imprevisíveis ou inconsistentes. Os exemplos incluem:

- **omitir anotações de fecho** que deveriam estar presentes — por exemplo, não fechar `<head>`, `<body>` e `<html>`, ou não fechar um `<li>` dentro de uma lista (esta última, como vimos, é na realidade tolerada e bem definida pela especificação HTML5 — mas omitir o fecho de `<head>/<body>/<html>` já é um erro mais sério, mesmo que muitos navegadores o tolerem);
- **fechar anotações fora de ordem (sobreposição)** — por exemplo, escrever `<b><i> ... </b></i>` em vez de `<b><i> ... </i></b>`: as anotações ficam “cruzadas” em vez de corretamente aninhadas, o que viola a regra de que o fecho tem de respeitar a ordem inversa da abertura (ver secção “Morfologia”);
- **valores de atributos sem aspas que deveriam tê-las** — por exemplo, um `href` com um nome de ficheiro que, em certos contextos, precisaria de estar entre aspas.

A conclusão prática é: só porque uma página “parece funcionar” num navegador não quer dizer que o HTML está correto — pode estar simplesmente a ser tolerado de uma forma que **não é garantida** ser igual noutro navegador.

Como verificar a validade. Existem **validadores** de HTML que verificam automaticamente a correção do código face às regras da linguagem. O **Validador do W3C** (<https://validator.w3.org/>) é o exemplo de referência citado nos slides, e permite validar:

- páginas já publicadas (bastando indicar o URL);
- ficheiros HTML carregados diretamente;
- ou até formatação/texto colado diretamente na ferramenta.

Frequente em exame

Perguntas de exame sobre este tema costumam pedir para identificar **por que razão** um determinado trecho de HTML é inválido (tipicamente: anotação sem fecho, aninhamento incorreto/sobreposto, ou atributo mal formado) e explicar a **consequência prática** — comportamento inconsistente entre navegadores, mesmo que a página “pareça” funcionar num navegador específico.

Formulários

Os **formulários** permitem a introdução, por parte do utilizador, de valores de diferentes tipos, através de **widgets** especializados que restringem ou validam o que pode ser introduzido. Uma coleção de valores pode ser agregada num formulário e submetida a um servidor.

O elemento form

Definição

O elemento `<form>` agrega *widgets* que serão submetidos ao servidor. Regras principais:

- pode ser **omitido** se os *widgets* forem usados apenas do lado do cliente (sem necessidade de enviar nada a um servidor);
- pode existir **mais do que um** formulário na mesma página;
- tem dois atributos ligados diretamente à comunicação com o servidor:
 - **action** — o URL/recurso do servidor para onde os dados são enviados;
 - **method** — o método HTTP usado no envio (tipicamente **get** ou **post**).

Atenção

Nota adicional (não estava explícito nos slides): a diferença entre `method=get` e `method=post` não é explicada nos ficheiros fonte, mas é relevante para perceber **action/method** na prática. Com `method=get`, os pares nome-valor do formulário são anexados ao **próprio URL** do pedido (visíveis na barra de endereços, por exemplo `?Nome=Ana&Idade=20`) — adequado para pesquisas ou formulários sem dados sensíveis. Com `method=post`, os dados vão no **corpo** do pedido HTTP, não ficam visíveis no URL, e são o método recomendado para dados sensíveis ou quando se está a criar/alterar algo no servidor (como um registo).

Dentro de um `<form>`, os dois tipos de *widget* diretamente ligados ao próprio ato de submissão são:

- **reset** — reverte todos os valores do formulário para os seus valores iniciais;
- **submit** — submete todos os pares nome-valor recolhidos ao servidor indicado em **action**.

E duas propriedades são **comuns a todos os widgets** de um formulário: **name** (o nome do campo, usado como chave no par nome-valor enviado) e **value** (o valor atual desse campo).

Tipos de input básicos

O elemento `<input>` é o *widget* mais versátil de introdução de valores; o tipo concreto de widget mostrado é determinado pelo atributo **type**.

type	Tipo	Descrição
text	Texto	Valor por omissão se type for omitido
password	Texto oculto	Mostra caracteres mascarados
radio	Botão de rádio	Escolha única dentro de um grupo com o mesmo name
checkbox	Caixa de seleção	Escolha independente, ligado/desligado
button	Botão	Botão genérico, sem submissão automática
file	Ficheiro	Permite escolher um ficheiro local para envio

Inputs especializados em data e hora

O HTML5 introduziu tipos de `input` especializados em valores de tempo, que trazem duas vantagens sobre um simples campo de texto: só aceitam **valores válidos** (por exemplo, impedem um mês igual a 13) e mostram, de forma visual, tanto a **estrutura esperada** do valor (ex: `mm/dd/yyyy`) como **seletores** próprios de ajuda ao preenchimento (ex: um calendário para escolher uma data).

type	Tipo
date	Data
datetime-local	Data e tempo, sem indicação de fuso horário
month	Mês
time	Tempo (hora)
week	Semana

Outros tipos de input

Ainda dentro dos tipos especializados do HTML5, com o mesmo princípio de restringir/validar e oferecer seletores de apoio ao preenchimento (por exemplo, um seletor de cor, ou coordenadas recolhidas ao clicar numa imagem):

type	Tipo	Nota
color	Cor	Mostra um seletor de cor
image	Imagem	A imagem é dada pelo atributo <code>src</code> ; recolhe as coordenadas do ponto clicado na imagem
number	Número	Restringe a valores numéricos
range	Intervalo	<i>Slider</i> para escolher um valor num intervalo
email	Email	Valida formato de endereço de email
search	Procura	Campo de pesquisa (visual/semântico)
tel	Telefone	Não suportado no Chrome
url	URL	Não suportado no Chrome

Atenção

Nota adicional (não estava explícito nos slides): a ressalva “não suportado no Chrome” para `tel` e `url` é um exemplo prático e concreto da “profusão de versões, nem sempre alinhadas” mencionada na primeira secção deste resumo — mesmo dentro de uma única versão do HTML (o HTML5), diferentes navegadores podem escolher implementar (ou não) certas funcionalidades. Na prática, isto significa que, mesmo usando `type=tel`, é prudente não confiar cegamente em que o navegador vá validar o formato do número de telefone — pode ser necessário validar também do lado do servidor.

Seletores: `select`, `option` e `optgroup`

Definição

O elemento `<select>` agrega um conjunto de opções `<option>`. A forma como são apresentadas — como um **menu** suspenso ou como uma **lista** com vários valores visíveis — depende do atributo `size`:

size	Apresentação
1 (valor de omissão)	Menu suspenso
n > 1	Lista com n valores simultaneamente visíveis

Duas particularidades adicionais:

- as opções podem ser **agrupadas** visualmente usando `<optgroup label="...">` para envolver um conjunto de `<option>`, dando-lhes um cabeçalho de grupo (o `label`);
- uma opção pode ter um **valor diferente** do texto/etiqueta mostrado ao utilizador, usando o atributo `value` em `<option>` — por exemplo, mostrar “XXXL” ao utilizador mas enviar `value=9` ao servidor;
- a anotação de **fecho** `</option>` pode ser **omitida**, tal como acontecia com `</li>`.

Áreas de texto: `textarea`

Definição

O elemento `<textarea>` é usado para introdução de **texto multi-linha** (ao contrário de `<input type=text>`, que é sempre uma única linha). Tem dois atributos principais para controlar as suas dimensões visuais:

- `rows` — número de linhas visíveis;
- `cols` — número de colunas visíveis (a largura, em número de caracteres).

O texto colocado entre `<textarea>` e `</textarea>` é o **valor inicial** do campo, já preenchido quando a página carrega.

Exemplo completo de um formulário

Exemplo resolvido

Juntando o elemento `form`, vários tipos de `input`, e os widgets de submissão, um formulário de registo completo poderia ser:

```
<form action=registar method=post>
  Nome <input name=Nome><br>
  Idade <input type=number name=Idade> <br>
  Email <input type=email name=Email> <br>
  Data de nascimento <input name=Nasce type=date>
  <input type=reset>
  <input type=submit value=Registar>
</form>
```

Análise passo-a-passo:

1. `<form action=registar method=post>` — ao submeter, os dados serão enviados ao recurso `registar` do servidor atual, usando o método POST (dados no corpo do pedido, não visíveis no URL).
2. `<input name=Nome>` — campo de texto simples (tipo por omissão), cujo valor será enviado como `Nome=<o que o utilizador escrever>`.
3. `<input type=number name=Idade>` — restringe a entrada a valores numéricos, enviado como `Idade=<número>`.
4. `<input type=email name=Email>` — valida (ao nível do navegador) que o valor tem forma de endereço de email, enviado como `Email=<endereço>`.
5. `<input name=Nasce type=date>` — mostra um seletor de data (calendário), enviado como `Nasce=<data escolhida>`. Note-se que a ordem dos atributos (`name` antes ou depois de `type`) é irrelevante, como referido na secção de Morfologia.
6. `<input type=reset>` — botão que, se clicado, repõe todos os campos acima nos seus valores iniciais (vazios, neste caso).
7. `<input type=submit value=Registar>` — botão de submissão; o atributo `value` aqui não é um “valor de dados” mas sim o **texto mostrado no próprio botão** (“Registar” em vez do texto por omissão, tipicamente “Submit”).

Ao clicar em “Registar”, o navegador reúne todos os pares `nome=valor` dos campos preenchidos e envia-os, via POST, para o URL `registar` no mesmo servidor da página atual.

Tabelas

Tabelas permitem formatar conteúdo em **linhas e colunas**. Apesar de organizadas visualmente em linhas e colunas, a sua estrutura HTML subjacente é, tal como o resto do documento, uma **árvore**: uma tabela contém linhas, e cada linha contém células. As células podem conter texto, imagens, ou mesmo outras tabelas (aninhamento completo).

Estrutura básica: `table`, `tr`, `td`

Definição

A estrutura de uma tabela tem três níveis hierárquicos:

1. **tabela** — elemento `<table>`;
2. **linhas** — elemento `<tr>` (*Table Row*), dentro de `<table>`;
3. **células** — elemento `<td>` (*Table Data*), dentro de cada `<tr>`.

Exemplo resolvido

Um jogo do galo simples ilustra a estrutura mínima de uma tabela:

```
<table>
  <tr>
    <td>X</td>
    <td>&nbsp;</td>
    <td>&nbsp;</td>
  </tr>
  <tr>
    <td>0</td>
    <td>X</td>
    <td>0</td>
  </tr>
  <tr>
    <td>X</td>
    <td>0</td>
    <td>&nbsp;</td>
  </tr>
</table>
```

Cada `<tr>` corresponde a uma linha visual da grelha 3×3, e cada `<td>` dentro dela a uma célula dessa linha. As células vazias usam ` ` em vez de ficarem completamente vazias — isto porque, sem nenhum conteúdo, algumas combinações de formatação CSS poderiam colapsar a célula visualmente (a entidade garante que há sempre, no mínimo, um espaço “visível” para o motor de layout trabalhar).

Os bordos visíveis à volta de cada célula, tal como referido a propósito deste exemplo, **não** vêm do HTML em si — são obtidos por formatação em CSS (a tabela em HTML puro, sem CSS, não mostra bordos por omissão).

Cabeçalhos de tabela: `th`

Definição

O elemento `<th>` (*Table Header*) **substitui** `<td>` como célula quando se pretende marcar essa célula como um **cabeçalho** de linha ou de coluna. Por omissão, a formatação de `<th>` difere da de `<td>`:

- o texto aparece **centrado**;
- o texto aparece **destacado a negrito**.

Exemplo resolvido

```
<table>
  <tr>
    <th>Nome</th>
    <th>Idade</th>
  </tr>
  <tr>
    <td>Sofia Costa</td>
    <td>19</td>
  </tr>
  <tr>
    <td>João Almeida</td>
    <td>20</td>
  </tr>
</table>
```

A primeira linha usa `<th>` para as etiquetas de coluna (“Nome”, “Idade”), que aparecem centradas e a negrito, enquanto as linhas seguintes usam `<td>` normal para os dados efetivos de cada aluno.

Intervalos de linhas e colunas: `colspan` e `rowspan`

Definição

Uma célula pode **expandir-se** por várias linhas ou colunas contíguas, usando os atributos:

- `colspan` — número de **colunas** que a célula ocupa;
- `rowspan` — número de **linhas** que a célula ocupa.

O valor do atributo é sempre o número de linhas/colunas contíguas a abranger.

Exemplo resolvido

```
<table>
  <tr>
    <th colspan=3>Dados</th>
  </tr>
  <tr>
    <th>Sexo</th>
    <th>Nome</th>
    <th>Idade</th>
  </tr>
  <tr>
    <td rowspan=2>&#9792;</td>
    <td>Sofia Costa<td>19
  </tr>
  <tr>
    <td>Joana Dias<td>20
  </tr>
</table>
```

Análise passo-a-passo:

1. A primeira linha tem um único `<th colspan=3>Dados</th>` — esta célula **substitui** as três células que normalmente existiriam nessa linha, estendendo-se pelas três colunas da tabela e funcionando como um título geral.
2. A segunda linha define os três cabeçalhos de coluna normais: “Sexo”, “Nome”, “Idade”.
3. Na terceira linha, `<td rowspan=2>♀</td>` (o símbolo Unicode de Vénus, que representa sexo feminino) ocupa **duas linhas** — a linha atual e a seguinte — na coluna “Sexo”. É por isso que a quarta linha do código (`<td>Joana Dias<td>20`) só tem **duas** células: a célula de “Sexo” já está “ocupada” pelo `rowspan` da linha anterior, pelo que essa linha só precisa de indicar Nome e Idade.

Note-se ainda que, tal como aconteceu com `</li>` e `</option>`, os `</td>` de fecho foram omitidos em vários pontos deste exemplo (`<td>Sofia Costa<td>19`) — mais uma vez tolerado pela especificação HTML5, já que um novo `<td>` fecha implicitamente o anterior.

Posicionamento com tabelas

Antes da adoção generalizada de CSS para layout, era comum usar tabelas como mecanismo de **posicionamento** de outros elementos na página — a anotação `<table>` cria uma “grelha invisível” e as suas células podem ser usadas para posicionar texto, objetos, imagens, ou mesmo outras tabelas, aproveitando o alinhamento automático em linhas e colunas que uma tabela garante.

Atenção

Nota adicional (não estava explícito nos slides): esta técnica de “tabelas para layout” foi muito comum no passado, mas hoje em dia é considerada **má prática**, precisamente porque mistura estrutura de dados (o significado de uma tabela) com apresentação visual (posicionamento). O próprio slide seguinte (“Formatar”) já assinala que, em HTML5, a formatação deve ser feita em CSS — o mesmo princípio geral aplica-se ao layout: hoje usam-se mecanismos de CSS (como Flexbox ou Grid, fora do âmbito direto deste ficheiro) em vez de tabelas para posicionar elementos que não são, semanticamente, dados tabulares.

Formatação estrutural: `thead`, `tbody`, `tfoot`

Definição

No HTML5, a formatação visual de tabelas é feita inteiramente em **CSS** (ao contrário do HTML4, que tinha muitos atributos de formatação dedicados, diretamente nas anotações da tabela). Existem, no entanto, três elementos que **agrupam linhas** dentro de uma tabela, servindo exclusivamente para que o CSS os possa formatar de forma diferenciada:

- `<thead>` (*Table HEAD*) — agrupa a(s) linha(s) de **cabeçalho**;
- `<tbody>` (*Table BODY*) — agrupa as linhas de **corpo/dados**;
- `<tfoot>` (*Table FOOT*) — agrupa a(s) linha(s) de **rodapé** (ex: totais, médias).

Exemplo resolvido

```
<table>
  <thead>
    <tr><th>NCD</th><th>Nome</th><th>Classificação</th></tr>
  </thead>
  <tbody>
    <tr><td>209999999</td><td>Fulano de Tal</td><td>18</td></tr>
    <tr><td>209999999</td><td>Beltrano de Qual</td><td>17</td></tr>
    <!-- ... -->
  </tbody>
  <tfoot>
    <tr>
      <th colspan=2>Média</th><td>13.5</td>
      <th colspan=2>Inscritos</th><td>75</td>
    </tr>
  </tfoot>
</table>
```

Esta tabela de pauta ilustra bem o papel de cada agrupamento: o `<thead>` guarda apenas a linha com os nomes das colunas; o `<tbody>` guarda uma linha por aluno (podendo ter dezenas ou centenas de linhas, tipicamente geradas dinamicamente a partir de uma base de dados); e o `<tfoot>` guarda uma linha final de resumo (média da turma e total de inscritos), separada visualmente do corpo de dados graças a este agrupamento — por exemplo, aplicando-lhe um estilo CSS diferente (fundo mais escuro, negrito, etc.), coisa que não seria possível de forma tão direta sem estes elementos de agrupamento.

CSS — Regras, Seletores e Padrões

O que é uma regra CSS

O **CSS** (*Cascading Style Sheets*) é a linguagem usada para descrever a apresentação visual de documentos estruturados — historicamente aplicada a vários formalismos (por exemplo XML), mas sobretudo às várias versões do HTML, e em particular ao HTML5. Um documento CSS (uma *folha de estilos*) é constituído por um conjunto de **regras**.

Definição

Uma **regra CSS** associa **valores de propriedades** a um ou mais elementos do documento. Tem sempre a forma:

```
seletor {  
    propriedade: valor;  
}
```

O **seletor** indica *a que elementos* a regra se aplica; o bloco entre chavetas — a **declaração** — indica *que propriedades* mudar e *para que valores*.

Exemplo resolvido

```
H1 { font-weight: bold }
```

Aqui H1 é o seletor (aplica-se a todos os elementos <h1> do documento) e font-weight: bold é a única declaração dentro do bloco: a propriedade font-weight recebe o valor bold.

Repara na estrutura em duas colunas do exemplo: de um lado o **seletor** (H1), do outro a **declaração**, que por sua vez se decompõe em **propriedade** (font-weight) e **valor** (bold), separados por **dois-pontos** (:). Esta tripla — seletor, propriedade, valor — é o átomo de todo o CSS; o resto da linguagem consiste em formas de **combinar** e **agrupar** estes átomos de forma compacta (secção seguinte) e em regras para resolver **conflitos** quando várias regras tentam definir a mesma propriedade no mesmo elemento (secção “A cascata”).

Comentários

Tal como noutras linguagens, o CSS permite comentar código:

Definição

Os comentários em CSS são delimitados por /* e */ (não existe comentário de uma linha como //). Tudo o que estiver entre estes dois marcadores é ignorado pelo motor de CSS.

```
/* Títulos tamanho 1 a negrito */  
H1 { font-weight: bold }
```

Os comentários servem dois propósitos práticos:

- **Documentação** — explicar o porquê de uma regra, sobretudo quando o valor escolhido não é óbvio (ex: um número mágico de *z-index*, uma cor em hexadecimal que corresponde à identidade visual da marca).
- **Inativação temporária** de código — “comentar” uma regra para a desligar sem a apagar, útil durante testes e depuração.

Seletores e declarações: como agrupá-los

Escrever uma regra por propriedade seria extremamente verboso. O CSS oferece duas formas de compactar regras: **agrupar seletores** e **agrupar declarações** — e é possível combinar as duas ao mesmo tempo.

Agrupar seletores

Quando várias categorias de elementos devem receber exatamente as mesmas declarações, em vez de repetir o bloco de declarações uma vez por cada seletor, os seletores listam-se separados por **vírgula** , à frente de um único bloco de declarações:

```
H1, H2, H3 {  
    font-weight: bold;  
}
```

Esta regra é equivalente a escrever três regras separadas (uma para H1, outra para H2, outra para H3), cada uma com a mesma declaração `font-weight: bold`. A lista de seletores pode ter tamanho arbitrário (`seletor1, seletor2, ..., seletorn`) e **todas** as declarações do bloco se aplicam a **cada um** dos seletores da lista — não há distribuição seletiva.

Agrupar declarações

De forma simétrica, quando um mesmo (conjunto de) seletor(es) precisa de múltiplas propriedades, as declarações listam-se dentro das chavetas, separadas por **ponto e vírgula** ;:

```
H1 {  
    font-weight: bold;  
    font-family: helvetica;  
}
```

Atenção

É **boa prática** manter o ponto e vírgula também depois da **última** declaração do bloco (como no exemplo acima, depois de `helvetica`), mesmo sendo tecnicamente opcional nesse caso. A razão é conceptual: debes encarar o ; como um **terminador** de cada declaração, não como um mero **separador** entre declarações — isso evita esquecimentos quando mais tarde adicionares uma declaração a seguir à que hoje é a última.

Combinar as duas formas

As duas técnicas de agrupamento não são mutuamente exclusivas — uma regra pode ter **vários seletores** e **várias declarações** ao mesmo tempo, formando regras muito mais compactas do que escrever tudo por extenso:

```
H1, H2, H3 {  
    color: gray;  
    text-transform: capitalize;  
    font-family: arial, helvetica;  
}
```

Esta única regra é equivalente a nove regras separadas (3 seletores $\times$ 3 declarações): qualquer um dos seletores (H1, H2 ou H3) recebe **todas** as declarações do bloco (`color`, `text-transform` e `font-family`).

Frequente em exame

Uma pergunta típica de exame é pedir para “desdobrar” uma regra combinada nas regras simples equivalentes, ou o inverso — combinar um conjunto de regras simples repetitivas numa única regra compacta. O truque é sempre o mesmo: uma regra com n seletores e m declarações equivale a $n \times m$ pares seletor-declaração.

A cascata

Como se viu acima, as propriedades de um único elemento podem estar **dispersas por múltiplas regras** — e nada impede que duas regras diferentes definam valores **conflituosos** para a mesma propriedade no mesmo elemento. O mecanismo que resolve esta ambiguidade chama-se **cascata** (dá nome à sigla CSS: *Cascading Style Sheets*) e segue, por ordem de prioridade, quatro critérios sucessivos:

Definição — Ordem de resolução da cascata

1. **Origem da regra:** regras do **autor** do documento têm precedência sobre regras do **leitor**/navegador.
2. **Especificidade:** entre regras da mesma origem, a mais **específica** prevalece.
3. **Ordem de declaração:** se a especificidade for igual, prevalece a **última** regra declarada.
4. **!important:** pode forçar a precedência de uma declaração, passando à frente dos três critérios anteriores.

O exemplo que atravessa todos os ficheiros desta parte da matéria é o seguinte par de folhas de estilo (uma do autor do documento, outra do “user agent” — o navegador/leitor):

```
/* author */
h3.other { font-style: italic; }
h1, h2, h3 {
    font-family: san-serif;
    font-weight: bold;
    font-style: normal;
}
h1 { font-size: xx-large; }
h2 { font-size: x-large; }
h3 { font-size: large; font-style: italic; }
h2 { font-size: 110%; }

/* user agent (navegador) */
h1, h2, h3 {
    font-family: serif;
    font-weight: 500 !important;
}
```

Repara como `font-family`, `font-weight`, `font-style` e `font-size` aparecem definidas **várias vezes**, por vezes em folhas diferentes, por vezes dentro da mesma folha — é exatamente esta dispersão que a cascata tem de resolver, propriedade a propriedade.

1. Origem: autor vs. leitor

Definição

O **navegador** (*browser*) também pode definir as suas próprias regras CSS — quer como valores por omissão, quer como preferências/configurações do **leitor** (ex: aumentar o tamanho de letra global para acessibilidade). Por outro lado, cada **documento** carrega as suas próprias regras, definidas pelo **autor** do documento (a folha de estilos “normal” que achamos que é “o CSS”).

A regra geral (simplificada, tal como apresentada nos slides) é que **as regras do autor têm precedência sobre as do leitor**: no exemplo acima, sem mais nenhum critério a interferir, seria a folha `/* author */` a vencer sobre a `/* user agent */`.

Atenção

Nota adicional (não estava explícito nos slides): a especificação CSS real distingue **três** origens, não duas — folhas do **user agent** (navegador), do **utilizador** (preferências pessoais do leitor, raramente usadas hoje em dia) e do **autor** (o documento). A ordem de prioridade completa, da menor para a maior, é: **user agent < utilizador < autor < autor !important < utilizador !important < user agent !important**

Ou seja, o **!important** não só força a precedência dentro da mesma origem como **inverte completamente** a ordem entre origens — uma regra **!important** do leitor passa à frente de uma regra **!important** do autor. Para o que é avaliado nesta cadeira, a simplificação “autor vence o leitor, exceto quando há **!important**” chega, mas vale a pena saberes que existe esta nuance caso apareça uma pergunta mais especializada.

2. Especificidade

Quando duas regras da **mesma origem** definem valores conflituosos para a mesma propriedade, o critério seguinte é a **especificidade** de cada seletor — quão “preciso”/restrito ele é.

Definição

Existem quatro categorias de seletores, com pesos crescentes:

tipo < classe < ID < estilo em linha (**style=**)

Cada seletor tem um peso global que depende de **quantos** seletores de cada categoria inclui. Quando duas regras têm a mesma propriedade em conflito, prevalece a de **maior peso** (maior especificidade).

Atenção

Nota adicional (não estava explícito nos slides): os slides dizem apenas que “cada seletor tem um peso consoante as categorias que inclui”, sem dizer *como* esse peso se calcula. O método padrão é representar a especificidade como um **tuplo de três números** (b, c, d), contados a partir do seletor:

- b — número de **IDs** (**#algo**) no seletor;
- c — número de **classes** (**.algo**), **atributos** (**[algo]**) e **pseudo-classes** (**:hover**, **:first-child**, ...) no seletor;
- d — número de **elementos de tipo** (**h1**, **p**, **div**, ...) e **pseudo-elementos** (**::before**, **::after**) no seletor.

Compara-se **primeiro** b ; só se houver empate se compara c ; só se também houver empate se compara d . Um estilo em linha (**style="..."**) teria um quarto número $a = 1$ à frente de tudo, que vence sempre qualquer seletor externo (exceto contra **!important**). Nunca se “soma” tudo num único número — um seletor com 1 ID vence sempre um seletor com 50 classes, porque se compara posição a posição, não a soma.

Exemplo resolvido

Passo 1 — os seletores em confronto. Suponhamos três regras que definem a cor de fundo do mesmo item de um menu de navegação:

```
nav ul li.active      { background: yellow; } /* regra A */
#menu .active        { background: orange; } /* regra B */
.menu-item.active    { background: green; }  /* regra C */
```

Todas se aplicam ao mesmo elemento `<li id="menu" class="menu-item active">`. Qual delas vence?

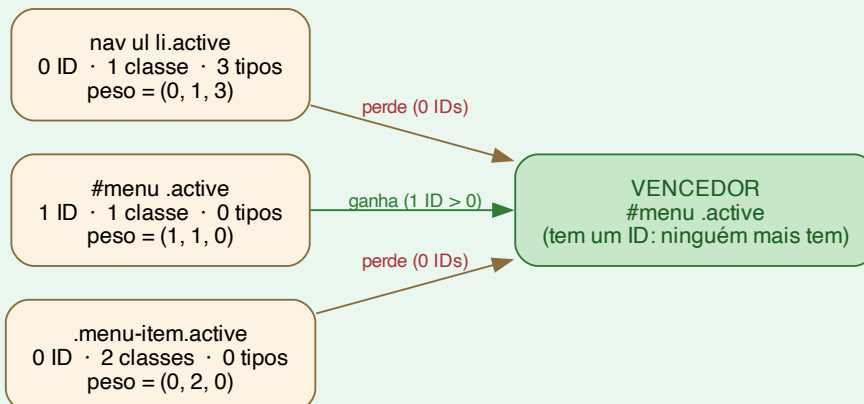
Passo 2 — contar cada categoria por seletor.

Seletor	IDs (<i>b</i>)	Classes/pseudo (<i>c</i>)	Tipos (<i>d</i>)	Peso (<i>b, c, d</i>)
nav ul li.active	0	1 (.active)	3 (nav, ul, li)	(0, 1, 3)
#menu .active	1 (#menu)	1 (.active)	0	(1, 1, 0)
.menu-item.active	0	2 (.menu-item, .active)	0	(0, 2, 0)

Exemplo resolvido

Passo 3 — comparar os tuplos, posição a posição. Comparamos primeiro o número de IDs: a regra B tem $b = 1$, as regras A e C têm $b = 0$. Como $1 > 0$, a regra B **já vence** nesta primeira posição — não é sequer necessário olhar para as classes ou os tipos, porque a comparação por posições é lexicográfica (a primeira posição onde há diferença decide).

Resultado: `#menu .active` (regra B) é a mais específica — o fundo fica **laranja**. Note-se que a regra C tem *mais* seletores de classe (2 contra 1), mas isso é irrelevante: um único ID pesa mais do que qualquer quantidade de classes.



3. Ordem de declaração

Quando **duas regras têm exatamente a mesma especificidade** e a mesma origem, o critério de desempate é puramente posicional:

Definição

Entre regras empatadas em especificidade, prevalece sempre a **última** a ser declarada (a que aparece mais abaixo na folha de estilos, ou a última folha de estilos carregada, se houver várias).

No exemplo introdutório da cascata, `h2 { font-size: x-large; }` aparece antes de `h2 { font-size: 110%; }` — ambos os seletores são idênticos (`h2`), logo têm a mesma especificidade (0, 0, 1). Como a segunda regra aparece **depois**, é ela que vence: o tamanho de letra final de `h2` é 110%, não `x-large`.

Atenção

Nota adicional (não estava explícito nos slides): isto explica também por que é que, na prática, colocar `<link rel="stylesheet">` **depois** de um `<style>` interno (ou vice-versa) importa — se as regras tiverem a mesma especificidade, ganha a folha que for **carregada por último** no documento, independentemente de ser externa ou interna.

4. A anotação `!important`

O último critério — e o único capaz de **saltar** todos os anteriores — é a anotação `!important`, colocada a seguir ao valor de uma declaração:

```
h1, h2, h3 {  
    font-weight: 500 !important;  
}
```

Definição

`!important` **força** a precedência de uma declaração específica, mesmo que a regra em que está inserida tenha menor especificidade ou apareça antes de outra regra concorrente (dentro da mesma origem — ver a nota sobre origens, acima, para o caso entre origens diferentes).

Atenção

`!important` deve ser usado com **muita parcimônia**. Ao contornar a cascata normal, torna o CSS mais difícil de depurar (uma declaração `!important` só pode ser sobreposta por *outra* declaração `!important` mais específica ou posterior) e é frequentemente sintoma de um problema de especificidade mal resolvido noutro sítio da folha de estilos, em vez de uma solução legítima.

Associar CSS ao HTML

Até agora vimos *como* uma regra CSS se escreve; falta ver *como* ela chega a “tocar” o HTML. Há três formas de embeber CSS num documento HTML, e depois há três formas de um seletor CSS apontar para partes do HTML (por tipo, por classe, por id), mais os contentores genéricos `div`/`span` para quando nenhum elemento semântico serve, mais os próprios elementos semânticos do HTML5.

As três formas de embeber CSS

Definição

- **Folha de estilo externa:** um ficheiro `.css` à parte, referenciado a partir do `<head>` com `<link>`.
- **Folha de estilo interna:** as regras escritas diretamente dentro de um elemento `<style>`, também no `<head>`.
- **Estilo local** (*inline*): declarações escritas diretamente no atributo `style` de um elemento HTML, sem seletor.

```
<!DOCTYPE html>
<html>
<head>
  <title>Olá mundo</title>
  <link rel="stylesheet" href="style.css" type="text/css">
  <style>
    h1 { color: orange; }
  </style>
</head>
<body>
  <h1 style="font-weight: 500;">Olá mundo</h1>
</body>
</html>
```

Este exemplo único mostra as três formas simultaneamente: o `<link>` carrega uma folha externa (`style.css`), o `<style>` define uma regra interna (`h1 { color: orange; }`) e o atributo `style="font-weight: 500;"` no próprio `<h1>` é o estilo local.

Folha externa (`<link rel="stylesheet" href="..." type="text/css">`):

- **Sintaxe:** embebida no `<head>` através do elemento `link`; o atributo `href` aponta para um documento que contém **apenas** regras CSS (sem HTML à mistura).
- **Aplicabilidade:** permite **reutilizar** as mesmas declarações entre vários documentos HTML e garantir **consistência visual** num site com várias páginas — é a forma recomendada em qualquer projeto real com mais do que uma página.

Folha interna (`<style>...</style>`):

- **Sintaxe:** o elemento `style` contém as regras só para **essa** página; no `<head>` é carregado antes de qualquer elemento a que se aplique. Os navegadores até reconhecem `<style>` no `<body>`, mas essa colocação **não é válida** — deve ficar sempre no `<head>`.
- **Aplicabilidade:** definir estilos específicos de uma única página, promovendo consistência **dentro** dessa página (mas sem partilha entre páginas, ao contrário da folha externa).

Estilo local (`style="..."` no próprio elemento):

- **Sintaxe:** é um **atributo**, não um elemento; o seu valor contém diretamente pares propriedade-valor. Não leva seletor (nem faria sentido — o “seletor” é o próprio elemento onde o atributo está).
- **Aplicabilidade:** formatar um único elemento sem recorrer a folhas de estilo; útil também para **substituir** pontualmente valores dados por regras gerais, já que o estilo local tem a maior especificidade possível.

Atenção

Nota adicional (não estava explícito nos slides, mas liga tudo à secção anterior): a razão pela qual o estilo local “ganha quase sempre” não é mágica — é exatamente o critério de **especificidade** da cascata: um `style=""` inline tem o peso mais alto de todas as categorias (equivalente a um quarto número $a = 1$ à frente do tuplo (b, c, d) visto atrás). Só uma declaração **!important** numa folha externa ou interna consegue vencer um estilo local.

Seletores por tipo, classe e id

Um documento HTML, além dos elementos e da sua estrutura, oferece dois mecanismos extra de “etiquetagem” que os seletores CSS podem explorar: **classes** e **identificadores (ids)**, através dos atributos `class` e `id`.

```
<style>
p { font-size: large; }
.strong { font-weight: bold; }
```

```
#important { color: red; }
</style>
...
<p class=strong> Lorem ipsum dolor sit amet, ... </p>
<p> Praesent vel elementum nunc... </p>
<p class=strong id=important> Donec cursus orci ... </p>
```

Definição

- **Seletor de tipo** (p, h1, div, ...): o nome do **tipo** de elemento. Aplica-se a **todos** os elementos desse tipo — no exemplo, p { font-size: large; } afeta os três parágrafos.
- **Seletor de classe** (.strong): precedido de um **ponto** (.). Aplica-se a **todos** os elementos cujo atributo class contenha esse nome — no exemplo, ambos os <p class=strong> ficam a negrito.
- **Seletor de id** (#important): precedido de um **cardinal** (#). Aplica-se apenas ao **único** elemento cujo atributo id tenha exatamente esse valor — no exemplo, só o terceiro parágrafo (o que tem id=important) fica vermelho.

Atenção

Nota adicional (não estava explícito nos slides): ao contrário de **class** (que pode repetir-se em vários elementos da página, e um mesmo elemento pode ter várias classes — ver abaixo), o valor de **id** deve ser **único** em todo o documento. Não é um erro de sintaxe repeti-lo, mas viola a semântica de HTML e pode causar comportamento inconsistente (ex: em JavaScript, `document.getElementById` só devolve o **primeiro** elemento com aquele id).

Contentores genéricos: div e span

Quando nenhum elemento HTML tem já um significado adequado para “agrupar isto para lhe aplicar CSS”, o HTML oferece dois contentores **sem** significado próprio, cuja única função é servir de gancho para regras CSS:

```
<div style="background: orange;">
  Em fundo laranja e
  <span style="color: white;">
    texto com cor branca
  </span> !!
</div>
```

Definição

- **div**: contentor **de bloco** — quebra linha antes e depois de si, como um parágrafo. Permite alterar a formatação da sua **caixa** inteira (largura, fundo, bordos, etc.).
- **span**: contentor **em linha** — não quebra o fluxo do texto à sua volta, funcionando como mais uma palavra dentro da frase. Muitas propriedades (como a cor do texto) são **herdadas** pelos elementos contidos dentro dele.

Ambos têm valores de propriedades por omissão neutros (praticamente “invisíveis” sem CSS aplicado) — a única razão de existir é dar um sítio onde pendurar um **id**, uma **class** ou um **style**.

Combinar seletores

Um elemento HTML não está limitado a **uma** classe, e uma classe CSS pode ser restringida a um tipo de elemento específico.

Múltiplas classes num elemento: o atributo **class** aceita uma lista de nomes separados por **espaço**:

```
<style>
  .strong { font-weight: bold; }
  .important { color: red; }
</style>
...
<p class=strong> Lorem ipsum dolor sit amet, ... </p>
<p class="strong important"> Donec cursus orci ... </p>
```

O segundo parágrafo recebe **ambas** as declarações (negrito e vermelho), porque pertence simultaneamente às classes `strong` e `important`.

Restringir uma classe a um tipo: escrevendo o tipo colado ao seletor de classe, separados por um **ponto** (sem espaço):

```
p.important { background: orange; }
.important { color: red; }

<p class=important> Lorem ipsum dolor sit amet, ... </p>
<p> Praesent vel elementum nunc... </p>
<p> Donec <span class=important>cursus</span> orci ... </p>
```

Aqui, `.important` (sem tipo) aplica `color: red` a **qualquer** elemento com essa classe (tanto o `<p>` como o `<span>`); mas `p.important` (tipo + classe) só se aplica quando a classe `important` está num elemento `<p>` — por isso só o parágrafo fica com fundo laranja, e o `<span class=important>` (dentro do terceiro parágrafo) fica só vermelho, sem fundo laranja.

Frequente em exame

Uma confusão comum: `p.important` (sem espaço) restringe a classe ao tipo `p` — lê-se “elementos `p` que também tenham a classe `important`”. Já `p .important` (com espaço) seria outra coisa completamente diferente: um seletor de **descendência** (ver a seção seguinte sobre padrões de contexto), que significa “qualquer elemento com classe `important` que esteja dentro de um `p`”. Nunca confundir os dois — o espaço muda completamente o significado.

Elementos semânticos do HTML5

Além de `div/span` (sem significado) e classes/ids (significado atribuído livremente pelo autor), o HTML5 introduziu **contentores semânticos**: elementos cujo **nome** já identifica o papel do seu conteúdo na página.

Definição

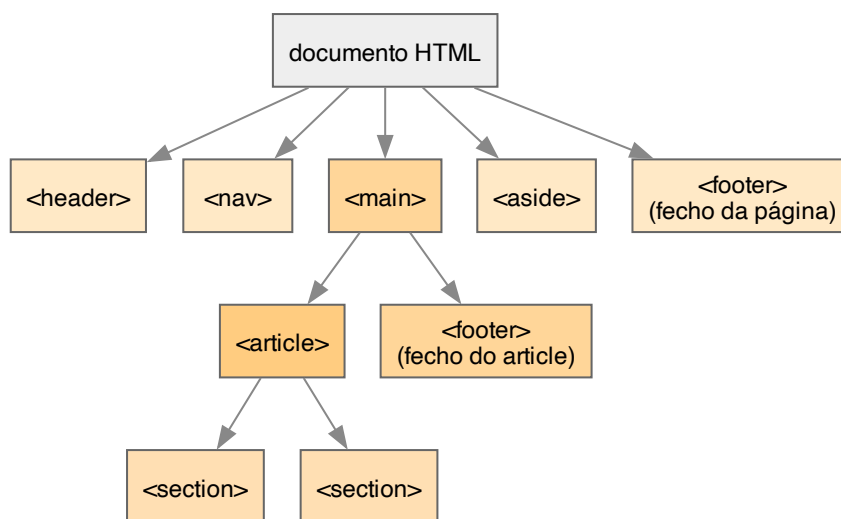
Um **elemento semântico** é um contentor com um significado pré-definido pela especificação HTML5, que indica o papel do conteúdo que envolve. Tal como `div/span`, geralmente não tem *look and feel* próprio (não aplica nenhuma formatação visual por omissão) — mas, ao contrário deles, **deve ser usado apenas quando o significado se adequa ao conteúdo**, nunca só pela conveniência de “mais um gancho para CSS”. Isto captura papéis recorrentes da publicação eletrônica e **melhora a acessibilidade** em navegadores para pessoas com deficiência visual (um leitor de ecrã consegue anunciar “está a entrar na navegação principal” ao encontrar um `<nav>`, coisa que não consegue com um `div` genérico).

Os elementos estruturais principais são:

- **<header>**: um bloco de cabeçalhos — títulos, imagens, *links* (ex: `<h1>`, `<img>`, `<a>`). Podem existir **vários** `<header>` numa mesma página (ex: um da página, outro de cada `<article>`), mas não é válido colocar um `<header>` dentro de outro `<header>`, de um `<footer>` ou de um `<address>`.
- **<nav>**: um bloco com *links* de navegação — tipicamente contém listas (`<ul>`, `<li>`, `<a>`). Destina-se à navegação **principal** do site, não a qualquer lista de links.
- **<main>**: contém o conteúdo **mais relevante** da página. Normalmente tem subcontentores semânticos lá dentro (como `<article>`), e deveria ter sempre um título (ex: `<h2>`).

- **<article>**: identifica um **tema específico e autônomo** — algo que faria sentido isolado do resto (ex: um post de blog, uma notícia). Contém outros elementos como **<section>** ou parágrafos, e normalmente tem um título próprio (ex: **<h3>**).
- **<section>**: identifica **uma seção** de um documento — geralmente existem **várias** dentro do mesmo **<article>** ou **<main>**. Contém parágrafos e normalmente tem também um título (ex: **<h4>**).
- **<aside>**: informação **complementar**, frequentemente mostrada a um dos lados do conteúdo principal (uma barra lateral, uma citação relacionada, publicidade contextual).
- **<footer>**: informação de **fecho** de uma parte do documento — pode fechar a página inteira ou um **<article>** individual. Tal como o **<header>**, podem existir **vários** **<footer>** no mesmo documento.

A figura seguinte ilustra como estes elementos tipicamente se aninham numa página: **<header>** e **<nav>** no topo, depois uma zona principal dividida entre **<main>** (que por sua vez contém um **<article>** com duas **<section>**, seguido do seu próprio **<footer>**) e um **<aside>** lateral, terminando com o **<footer>** da página inteira.



Além destes elementos estruturais, existem outros elementos semânticos mais pontuais:

Definição

- **Sem look and feel** (contentores puros): **<figure>** — contém uma figura (ex: imagem) e a sua legenda; **<figcaption>** — a própria legenda dessa figura.
- **Com look and feel** próprio: **<mark>** — texto a destacar (o navegador aplica um fundo amarelo por omissão); **<button>** — um botão clicável para iniciar uma ação; **<details>** — apresenta informação complementar que pode ser expandida/colapsada pelo utilizador; **<summary>** — a parte sempre visível de um **<details>** (o “título” clicável que abre/fecha o resto).

Seletores de padrões: pseudo-classes e pseudo-elementos

Os seletores vistos até agora (tipo, classe, id, e as suas combinações) já permitem apontar para muitos elementos, mas ainda deixam de fora situações que dependem do **contexto** do elemento na árvore do documento, do seu **conteúdo**, dos seus **atributos** ou do seu **estado** de interação. Para isso o CSS oferece **padrões de seletores** (mais conhecidos como *pseudo-classes* e *pseudo-elementos*), que foram sendo introduzidos ao longo dos vários níveis da linguagem (CSS Level 1, 2 e 3 — os slides marcam cada padrão com o nível em que foi introduzido).

Definição

- Uma **pseudo-classe** (sintaxe `seletor:nome`, um único `:`) seleciona elementos consoante uma condição — posição na árvore, estado de interação, etc. — sem que essa condição corresponda a uma classe real no HTML.
- Um **pseudo-elemento** (sintaxe `seletor::nome`, dois `::`) refere-se a uma **parte** de um elemento (ex: a primeira letra) ou permite **inserir** conteúdo que não existe no HTML original.

Atenção

Nota adicional (não estava explícito nos slides): historicamente (CSS2) tanto pseudo-classes como pseudo-elementos usavam um único `:` (por exemplo `:before`, `:after`). O CSS3 introduziu o duplo `::` só para pseudo-elementos, para os distinguir claramente das pseudo-classes — mas por compatibilidade, a forma antiga com um único `:` (`:before`, `:after`, `:first-line`, `:first-letter`) continua a funcionar em todos os navegadores modernos. Nos exemplos que se seguem uso a notação CSS3 (`::before/::after`), mas se vires a notação antiga num código de outra proveniência, é a mesma coisa.

Padrões de contexto

Estes padrões selecionam elementos com base na sua **posição relativa** a outros elementos na árvore do documento — para além da vírgula (lista de seletores, já vista) e do seletor universal:

Padrão	Significado
<code>elemento1, elemento2</code>	seleciona ambos os elementos (agrupamento, já visto)
<code>ancestral elemento</code>	elementos com um dado ancestral (combinador descendente)
<code>*</code>	todos os elementos (seletor universal)
<code>:not(seletor)</code>	inverte a seleção — tudo o que não corresponde a seletor

Exemplo resolvido

```
h2 b {
  color: orange;
}
```

```
<h2>Lorem <b>ipsum</b> </h2>
<p>Lorem <b>ipsum</b> dolor sit amet</p>
```

O seletor `h2 b` (com **espaço** entre `h2` e `b`) é um combinador **descendente**: seleciona qualquer `<b>` que esteja **dentro** de um `<h2>`, a qualquer profundidade de aninhamento. No exemplo, o `<b>` de dentro do `<h2>` fica laranja, mas o `<b>` de dentro do `<p>` **não** é afetado — não tem `<h2>` como ancestral.

Padrões de conteúdo

Selecionam (ou introduzem) **partes do conteúdo** de um elemento, não o elemento inteiro:

Padrão	Significado
<code>seletor:first-letter</code>	a primeira letra do elemento selecionado
<code>seletor:first-line</code>	a primeira linha do elemento selecionado
<code>seletor::before</code>	ponto antes do conteúdo do elemento selecionado
<code>seletor::after</code>	ponto depois do conteúdo do elemento selecionado

Padrão	Significado
--------	-------------

```
p:first-letter {
  font-size: larger;
  font-weight: bolder;
}
```

Este exemplo faz com que a primeira letra de cada parágrafo apareça maior e mais carregada — um efeito clássico de “letra capitular” tipográfica.

Atenção

Nota adicional (não estava explícito nos slides): `::before` e `::after`, sozinhos, **não mostram nada** — servem apenas para marcar uma posição. Para efetivamente inserirem conteúdo visível é obrigatório combiná-los com a propriedade `content`, por exemplo:

```
a:after {
  content: "\2197";
}
```

Isto insere uma seta diagonal (o caractere Unicode U+2197, escrito em CSS como `\2197`) a seguir a **todos** os links da página, sem alterar o HTML original — é a técnica usada, por exemplo, para adicionar ícones decorativos ou aspas tipográficas automaticamente.

Padrões de atributos

Selecionam elementos consoante a **presença** ou o **valor** dos seus atributos HTML:

Padrão	Significado
[atributo]	elementos com um dado atributo (independente do valor)
[atributo=valor]	elementos com um atributo igual a um dado valor exato
[atributo~=valor]	elementos cujo atributo contém esse valor (entre vários, separados por espaço)
[atributo^=valor]	elementos cujo atributo começa por um dado prefixo
elemento:lang(língua)	elementos com um dado atributo <code>lang</code>

Atenção

Nota adicional (não estava explícito nos slides): repara na diferença entre `[atributo=valor]` e `[atributo~=valor]`. O primeiro exige igualdade **total** do valor do atributo — `[class=strong]` só corresponde a `class="strong"` exato, **não** a `class="strong important"`. Já `[class~=strong]` corresponde a qualquer elemento cujo atributo `class` contenha **strong** como uma das palavras da lista separada por espaços — por isso `[class~=strong]` é o que realmente corresponde ao mesmo comportamento do seletor `.strong` visto atrás (que também “procura dentro” da lista de classes). Já agora, `[atributo^=valor]` é útil por exemplo para selecionar todos os links externos: `a[href^="http"]`.

Padrões de estado

Selecionam elementos consoante o seu **estado de interação**, que varia ao longo do uso da página — divide-se em dois grupos: **links** e **widgets** (elementos interativos genéricos).

Estado de links:

Padrão	Significado
<code>:link</code>	<i>links não visitados</i>
<code>:visited</code>	<i>links visitados</i>
<code>:active</code>	<i>links ativos</i> (a ser clicados, no momento do clique)

```
a:link { color: red; }  
a:visited { color: orange; }  
a:active { font-weight: bold; }
```

Estado de *widgets*:

Padrão	Significado
<code>elemento:focus</code>	elementos com focus (a receber input do teclado)
<code>:target</code>	elemento apontado por um link interno (#nome)
<code>:enabled</code>	objeto gráfico ativo (ex: um botão que se pode clicar)
<code>:disabled</code>	objeto gráfico inativo (ex: um botão desabilitado)
<code>:checked</code>	objeto gráfico escolhido (ex: uma <i>checkbox</i> marcada)
<code>::selection</code>	conteúdo atualmente selecionado pelo utilizador (arrastar o rato sobre texto)

Frequente em exame

`:hover` (o rato sobre o elemento) é o padrão de estado mais usado na prática e mais perguntado em exame, apesar de não aparecer explicitamente nas tabelas dos slides desta secção — é do mesmo género que `:focus/:active`: reage a um estado momentâneo de interação do utilizador, tipicamente usado para dar retorno visual em botões e links (`a:hover { text-decoration: underline; }`).

Padrões de estrutura

Selecionam elementos consoante a sua **posição estrutural** na árvore do documento — quantos irmãos tem, que posição ocupa entre eles, se está vazio, etc.

Combinadores estruturais e primeiro filho:

Padrão	Significado
<code>progenitor > elemento</code>	elementos com um dado progenitor direto (filho imediato)
<code>anterior + elemento</code>	elementos que seguem imediatamente o elemento anterior dado (irmão adjacente)
<code>seletor:first-child</code>	elemento que é o primeiro filho do seu progenitor

Atenção

Nota adicional (não estava explícito nos slides): é fácil confundir o combinador descendente (**ancestral elemento**, com espaço, visto nos padrões de contexto) com o combinador de **filho direto** (**progenitor > elemento**). `div p` seleciona **qualquer** `<p>` dentro de um `<div>`, não importa a que profundidade (pode estar dentro de vários outros elementos aninhados). Já `div > p` só seleciona `<p>` que sejam **filhos imediatos** do `<div>` — um `<p>` dentro de um `<span>` dentro do `<div>` já não seria selecionado por `div > p`, mas seria por `div p`.

Outros padrões estruturais (posição entre irmãos, tipo, vazio, raiz):

Padrão	Significado
<code>:root</code>	a raiz do documento (equivalente a <code><html></code> , mas com maior especificidade)
<code>elemento:empty</code>	elementos vazios (sem conteúdo nem sequer espaços em branco)
<code>anterior ~ elemento</code>	elementos que seguem o anterior dado, não necessariamente de forma imediata (irmão geral)
<code>elemento:first-of-type</code>	primeiro elemento deste tipo no seu progenitor
<code>elemento:last-of-type</code>	último elemento deste tipo no seu progenitor
<code>elemento:only-child</code>	único elemento deste tipo no seu progenitor
<code>elemento:nth-child(n)</code>	o enésimo elemento (de qualquer tipo) no seu progenitor
<code>elemento:nth-last-child(n)</code>	o enésimo elemento, contando a partir do fim
<code>elemento:last-child</code>	elemento que é o último filho do seu progenitor

Atenção

Nota adicional (não estava explícito nos slides): a fórmula `n` em `:nth-child(n)` e `:nth-last-child(n)` não é só um número fixo — aceita uma expressão da forma `an+b`, onde `n` conta 0, 1, 2, 3, ... Isto permite selecionar **padrões periódicos** de elementos, o que é muito usado na prática:

- `:nth-child(1)` — só o primeiro (equivalente a `:first-child`);
- `:nth-child(2n)` ou `:nth-child(even)` — todos os elementos **pares** (2º, 4º, 6º, ...) — típico para dar cores alternadas às linhas de uma tabela (efeito *zebra*);
- `:nth-child(2n+1)` ou `:nth-child(odd)` — todos os **ímpares** (1º, 3º, 5º, ...);
- `:nth-child(3n)` — de 3 em 3 (3º, 6º, 9º, ...).

Sem esta fórmula, `:nth-child` seria pouco mais útil do que `:first-child`/`:last-child`, por isso é importante saber que existe mesmo que os slides não a desenvolvam.

Exemplos completos resolvidos

Esta secção junta tudo o que foi visto — seletores de tipo e classe, combinação de classes, contentores genéricos, cascata — em dois exemplos completos e realistas: uma **caixa de diálogo** e uma **calculadora**, ambos construídos apenas com `div`s e CSS (sem nenhum elemento HTML especializado).

Atenção

Nota adicional (não estava explícito nos slides, mas necessária para perceber os dois exemplos): ambos os exemplos usam propriedades de layout (`float`, `clear`, `display: flex`) que não foram explicadas em mais nenhum ficheiro desta secção. Resumidamente:

- **float: left/float: right** tira um elemento do fluxo normal do documento e “empurra-o” para a esquerda ou direita, deixando o conteúdo seguinte fluir à volta dele (como texto à volta de uma imagem). **clear: right** (ou `left/both`) força um elemento a **não** subir para ao lado de um elemento flutuante anterior — “limpa” o efeito do `float`, empurrando o elemento para baixo dele.
- **display: flex** transforma um contêntor num *layout* flexível: os seus filhos diretos organizam-se automaticamente em linha (ou, com **flex-wrap: wrap**, passam para a linha seguinte quando não cabem). **flex-basis** define o tamanho “ideal” de um filho antes de o espaço sobranter ser distribuído.

Caixa de diálogo

O objetivo é construir uma caixa de diálogo com uma mensagem e dois botões, do tipo “Tem a certeza que quer sair?”, só com `div`s marcados por classes:

```
<div class="dialog">
  <div class="message"> Mesmo sair? </div>
  <div class="button"> Ok </div>
  <div class="button"> Cancelar </div>
</div>
```

Três classes caracterizam a estrutura: `dialog` (a caixa exterior), `message` (o texto da pergunta) e `button` (cada botão — reparem que os dois botões **partilham** a mesma classe, distinguindo-se apenas pelo texto interior).

Exemplo resolvido

Classe `dialog` — a caixa exterior:

```
.dialog {
  width: 90%;
  height: 120px;
  padding: 10px;
  margin: auto;

  border-width: 4px;
  border-style: double;
  border-color: #228;

  background: white;
  box-shadow: 3px 3px #AAA;
}
```

O que cada declaração contribui:

- `width/height/padding` definem as **dimensões** da caixa, incluindo o preenchimento interior.
- `margin: auto` **centra horizontalmente** a caixa dentro do seu contêntor — é a forma clássica de centrar um bloco com largura definida em CSS (as margens esquerda e direita são calculadas automaticamente para ficarem iguais).
- `border-style: double` desenha um contorno de **traço duplo**.
- `box-shadow: 3px 3px #AAA` desenha uma **sombra** deslocada 3px para a direita e 3px para baixo, na cor cinzenta `#AAA`, dando profundidade visual à caixa.

Exemplo resolvido

Classe `message` — a pergunta, e classe `button` — os botões:

```
.message {
  width: 50%;
  height: 100%;
  float: left;

  line-height: 120px;
  color: orange;
}

.button {
  float: right;
  clear: right;

  width: 145px;
  height: 40px;
  margin: 10px;

  line-height: 40px;
  text-align: center;

  background: orange;
  color: white;

  border-radius: 15px;
  border-width: 2px;
  border-style: outset;
}
```

O que cada declaração contribui:

- `.message` **flutua para a esquerda** (`float: left`) e ocupa 50% da largura da caixa de diálogo — o restante fica livre para os botões, que flutuam do lado oposto. Para **centrar verticalmente** o texto sem recorrer a *flexbox*, usa-se o truque clássico de igualar a **altura da linha** (`line-height`) à **altura total** da caixa (120px, igual à `height` de `.dialog`) — uma única linha de texto fica automaticamente centrada a meio dessa “linha” gigante.
- `.button` **flutua para a direita** (`float: right`); o `clear: right` em cada botão impede que o segundo botão suba para ao lado do primeiro — obriga-o a empilhar-se **por baixo** do anterior, ambos à direita. `line-height` igual à `height` centra verticalmente o texto do botão (mesmo truque de `.message`), e `text-align: center` centra-o horizontalmente. `border-radius: 15px` arredonda os cantos, e `border-style: outset` dá um efeito 3D “em relevo” à borda.

Calculadora

O segundo exemplo é uma calculadora básica, com um visor e um teclado numérico, novamente só com `div`s:

```
<div class="calculator">
  <div class="display">0</div>
  <div class="key">7</div> <div class="key">8</div> <div class="key">9</div>
  <div class="key op"> / </div>
  <div class="key">4</div> <div class="key">5</div> <div class="key">6</div>
  <div class="key op"> * </div>
  <div class="key">1</div> <div class="key">2</div> <div class="key">3</div>
  <div class="key op"> + </div>
```

```
<div class="key">,</div> <div class="key">0</div> <div class="key">C</div>
<div class="key op"> = </div>
</div>
```

Quatro classes caracterizam esta estrutura: **calculator** (o painel que contém tudo), **display** (o visor), **key** (cada tecla) e **op** (uma classe **adicional**, combinada com **key**, para marcar as teclas de operações — o que é o mesmo padrão de “múltiplas classes” visto na secção anterior: `class="key op"`).

Exemplo resolvido

Classe **calculator** — o painel de base:

```
.calculator {
  display: flex;
  flex-wrap: wrap;

  width: 200px;
  border-style: outset;
  border-width: 2px;
  padding: 10px;
  background: #DDD;
  margin: auto;
}
```

O painel é um bloco com **largura fixa** (200px); a **altura** é deliberadamente **omitida**, ajustando-se automaticamente ao conteúdo. `display: flex` combinado com `flex-wrap: wrap` é o que permite ao visor e às 16 teclas organizarem-se automaticamente em **linhas** (4 teclas por linha), passando para a linha seguinte sempre que a linha atual enche — sem isto, todos os elementos ficariam numa única fila horizontal. `margin: auto` centra o painel inteiro, tal como visto na caixa de diálogo.

Exemplo resolvido

Classe `display` — o visor, classe `key` — as teclas, e classe `op` — as teclas de operação:

```
.display {
  flex-basis: 90%;
  height: 1em;
  border-width: 2px;
  border-style: inset;
  margin: 2px;
  margin-bottom: 20px;
  padding: 2px;
  text-align: right;
  background: white;
  color: orange;
}

.key {
  width: 40px;
  height: 40px;
  border-width: 2px;
  margin-bottom: 4px;
  border-style: outset;
  text-align: center;
  line-height: 40px;
  background: orange;
  color: white;
}

.op {
  background: white;
  color: orange;
}
```

O que cada declaração contribui:

- `.display` usa `flex-basis: 90%` (em vez de `width`) para definir a sua largura dentro do *layout flex* — ocupa quase toda a linha sozinho, forçando as teclas a começarem todas numa nova linha por baixo dele (já que só sobra 10% de espaço na primeira linha). A altura é calculada a partir do tamanho dos caracteres (`height: 1em`), e o texto alinha-se à **direita** (`text-align: right`), como é hábito num visor de calculadora.
- `.key` define teclas quadradas (40px × 40px), com o texto centrado tanto horizontalmente (`text-align: center`) como verticalmente (`line-height` igual à `height`, o mesmo truque já visto). As quebras de linha entre grupos de teclas não são geridas aqui — são uma consequência automática do `flex-wrap: wrap` do contentor `.calculator`.
- `.op`, aplicada **em conjunto** com `.key` (`class="key op"`), troca simplesmente as cores de fundo e texto (inverte o laranja e o branco) — é a forma mais simples de dar destaque visual às quatro teclas de operação (`/`, `*`, `+`, `=`) sem duplicar todas as outras propriedades de `.key`, que continuam a aplicar-se por serem uma regra separada (a cascata resolve o conflito nas duas propriedades repetidas — `background` e `color` — a favor de `.op`, por ser declarada depois e ter especificidade igual a `.key` sozinha, ou por ser mais específica no caso combinado, dependendo da ordem das regras).

CSS — Propriedades e Layout

O CSS define **propriedades** para cada elemento HTML: cada propriedade tem um valor de **omissão** aplicado quando não é especificada, e algumas propriedades são **herdadas** automaticamente pelos elementos descendentes (tipicamente as de texto e tipo de letra), enquanto outras (como as de caixa ou posicionamento) não são herdadas — aplicam-se só ao elemento onde são declaradas. As grandes famílias de propriedades que vamos ver nesta secção são: caixa, cores, texto, tipo de letra e posicionamento (clássico, flexbox e grid).

O modelo de caixa (box model)

A ideia central: tudo é uma caixa

Todo o elemento HTML é formatado como uma **caixa retangular**. Essa caixa está sempre **contida** dentro da caixa do seu elemento progenitor, está **rodeada** por outro conteúdo (irmãos), e por sua vez **contém** as caixas dos seus próprios descendentes. As duas dimensões principais desta caixa são dadas pelas propriedades:

- **width** — a largura;
- **height** — a altura.

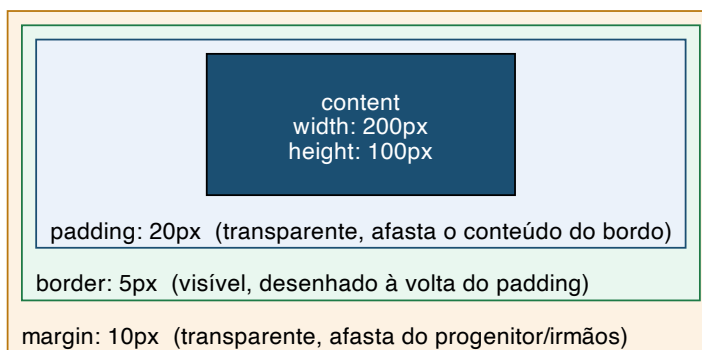
Só que a largura e a altura totais que uma caixa efetivamente ocupa no ecrã dependem de mais do que **width/height**: há espaço à volta da caixa, espaço à volta do conteúdo dentro da caixa, e o desenho do limite da própria caixa. É a isto que se chama o **modelo de caixa**.

As quatro camadas: content, padding, border, margin

Definição

O modelo de caixa organiza cada elemento em **quatro camadas concêntricas**, como as camadas de uma cebola, de dentro para fora:

- **content** — o conteúdo em si (texto, imagem, etc.), com as dimensões dadas por **width/height**;
- **padding** — preenchimento entre o conteúdo e o bordo; espaço **transparente** (não é desenhado, mas mostra o fundo do elemento);
- **border** — o bordo/contorno da caixa; é a única destas três camadas que **pode ser desenhada** visualmente (linha, cor, espessura);
- **margin** — margem entre esta caixa e as caixas vizinhas (o progenitor ou irmãos); espaço **transparente** e nunca desenhado.



Repara na ordem: **padding** fica **dentro** do **border**, e **margin** fica **fora** dele. Isto é importante para perceber, por exemplo, que dar **background-color** a um elemento pinta o **content** e o **padding** (porque estão dentro do bordo), mas nunca o **margin** (que fica sempre transparente, mostrando o que está por trás).

Bordo: só é visível depois de definido o estilo

O bordo tem três sub-propriedades relevantes:

- **border-width** — espessura do bordo;
- **border-style** — estilo da linha (ex: **solid**, **dotted**);
- **border-radius** — arredondamento dos cantos.

Atenção

O bordo só fica **visível** depois de se definir **border-style** com um valor diferente de **none** (o valor de omissão). Isto não é dito explicitamente nos slides, mas é um erro comum: definir **border-width** e **border-color** sem **border-style** não produz bordo nenhum visível, porque por omissão o estilo é “sem bordo”.

Com **border-radius** é possível arredondar um só canto da caixa, usando o nome desse canto no meio do nome da propriedade — por exemplo **border-top-left-radius** arredonda apenas o canto superior esquerdo, sobrepondo-se ao valor geral de **border-radius** só nesse canto.

Especificar um lado individualmente

Qualquer uma destas medidas (margem, bordo, preenchimento) pode ser aplicada a **um único lado** da caixa, em vez dos quatro lados ao mesmo tempo. Fazem-se isso inserindo o nome do lado a seguir ao nome da propriedade, separado por hífen:

- **top** — topo;
- **bottom** — fundo;
- **left** — esquerda;
- **right** — direita.

Por exemplo: **margin-top**, **border-right**, **padding-left**. Isto aplica-se a **margin**, **border** e **padding** (e também às suas sub-propriedades, como **border-right-width** ou **border-top-style**).

Unidades de medida

Definição

Toda a dimensão em CSS tem de vir seguida de uma **unidade**, que pode ser **absoluta** (um valor convencional e fixo, independente do resto da página) ou **relativa** (calculada a partir de alguma característica da formatação envolvente).

Absolutas:

- **cm** — centímetros;
- **mm** — milímetros;
- **in** — polegadas (*inches*) — **1in = 2,54cm**;
- **pt** — pontos (*points*) — **1pt = 1/96in**; **não são pixels**, apesar de serem frequentemente confundidos;
- **pc** — picas (*picas*) — **1pc = 12pt**.

Relativas:

- **%** — percentagem, relativa à dimensão correspondente do **progenitor**;
- **px** — pixels, cujo tamanho real no ecrã depende da **resolução** do dispositivo (por isso é tecnicamente “relativa”, apesar de na prática ser a unidade mais usada como se fosse absoluta);
- **em** — relativo ao **tamanho de letra atual** do próprio elemento;
- **rem** — *root em*, relativo ao tamanho de letra do elemento **raiz** (**html**), o que evita o efeito de acumulação em cascata que **em** tem quando há elementos aninhados com tamanhos de letra diferentes;
- **vw** — percentagem da **largura do viewport** (área visível da janela);
- **vh** — percentagem da **altura do viewport**.

Atenção

Nota adicional (não estava explícito nos slides): a diferença prática entre **em** e **rem** só importa quando há **aninhamento**. Se um elemento com `font-size: 1.5em` estiver dentro de outro que também usa **em**, os valores multiplicam-se em cascata (efeito composto), o que rapidamente produz tamanhos de letra inesperados. Usar **rem** evita este problema porque ancora sempre a mesma referência (a raiz do documento), independentemente de quantos elementos aninhados existirem.

Exemplo resolvido: calcular a largura total ocupada por uma caixa

Exemplo resolvido

Considera a seguinte declaração CSS para um elemento `div`:

```
div {  
  width: 200px;  
  padding: 20px;  
  border-width: 5px;  
  border-style: solid;  
  margin: 10px;  
}
```

Queremos calcular o **espaço total** que este elemento ocupa no layout da página (incluindo tudo o que “empurra” os elementos vizinhos).

Passo 1 — largura do conteúdo. `width: 200px` define apenas a largura da camada **content**: **200px**.

Passo 2 — somar o padding (dos dois lados). O `padding: 20px` sem indicação de lado aplica-se aos **quatro lados**; como a largura tem lado esquerdo e lado direito, soma-se **20px** duas vezes:

$$200 + 20 + 20 = 240\text{px}$$

Esta é a largura da caixa até ao limite exterior do **padding** (ou seja, o limite interior do **border**).

Passo 3 — somar o border (dos dois lados). `border-width: 5px` aplica-se também aos dois lados (esquerdo e direito):

$$240 + 5 + 5 = 250\text{px}$$

Esta é a largura **visível** da caixa, incluindo o bordo desenhado.

Exemplo resolvido

Passo 4 — somar a margin (dos dois lados). `margin: 10px` aplica-se aos dois lados:

$$250 + 10 + 10 = 270\text{px}$$

Este valor de **270px** é o espaço total que o elemento reserva no layout — é a distância mínima que separa o limite exterior da margem deste elemento do limite exterior da margem do elemento seguinte.

Resumo do cálculo:

Camada	Contributo (cada lado)	Largura acumulada
content	<code>width: 200px</code>	200px
+ padding	<code>20px + 20px</code>	240px
+ border	<code>5px + 5px</code>	250px
+ margin	<code>10px + 10px</code>	270px

O mesmo raciocínio aplica-se de forma independente à **altura**, somando `height` com o `padding`, `border` e `margin` verticais (topo + fundo). Nota que, por omissão neste modelo (chamado *content-box*, que é o comportamento por omissão do CSS2/CSS3 sem a propriedade `box-sizing`), `width/height` referem-se **só** ao conteúdo — `padding` e `border` **acrescentam-se** sempre a essas dimensões, nunca são subtraídos delas.

Atenção

Nota adicional (não estava explícito nos slides, mas é essencial para não haver confusões ao calcular layouts): existe uma propriedade `box-sizing`, com valor de omissão `content-box` (o comportamento descrito acima). Se for definida como `box-sizing: border-box`, o valor de `width` passa a representar a largura **total** até ao limite exterior do `border` (ou seja, `content` encolhe automaticamente para caber `padding+border` dentro dos `200px`, em vez de os somar). Isto não apareceu explicitamente nestes ficheiros, mas é o tipo de detalhe que é preciso saber para não errar contas de layout como a do exemplo acima.

Cor, texto e tipo de letra

Propriedades coloríveis

Definição

Nem todas as propriedades aceitam cor — apenas as que estão associadas a algo que é desenhado. As três propriedades coloríveis mais comuns são:

- `color` — cor dos elementos desenhados diretamente (tipicamente o texto);
- `background-color` — cor de fundo do elemento;
- `border-color` — cor do bordo (só visível se `border-style` e `border-width` também estiverem definidos, como visto acima).

Representação de valores de cor

Uma cor em CSS pode ser expressa de três formas equivalentes:

- **Nome predefinido** em inglês (ex: `red`, `green`, `blue`);
- **RGB** (*Red Green Blue*) — por componentes de vermelho, verde e azul, seja em hexadecimal (`#FF0000`) seja em função (`rgb(255,0,0)`); os dois formatos representam exatamente a mesma cor;
- **HSL** (*Hue Saturation Lightness*) — matiz, saturação, luminosidade) — por exemplo `hsl(0,100%,50%)` é vermelho puro, porque o matiz 0 corresponde ao vermelho na roda de cores, com saturação e luminosidade máximas/médias.

Atenção

Nota adicional (não estava explícito nos slides): a vantagem prática do HSL sobre o RGB é que é muito mais fácil de **ajustar visualmente à mão** — para escurecer uma cor mantendo o mesmo tom, basta reduzir o terceiro valor (luminosidade), enquanto em RGB seria preciso recalcular os três componentes proporcionalmente. Por isso o HSL costuma ser preferido quando se está a afinar uma paleta de cores manualmente.

A qualquer uma destas representações (RGB ou HSL) pode ser adicionado um canal extra chamado **alpha**, que controla a opacidade dessa cor especificamente (ex: `rgba(255,0,0,0.5)` ou `hsla(0,100%,50%,0.5)`) — não confundir com a propriedade `opacity`, vista a seguir, que afeta o elemento inteiro.

Opacidade

Definição

A propriedade `opacity` controla a opacidade do **elemento inteiro** (incluindo todo o seu conteúdo e descendentes), com valores entre 0 (totalmente transparente, invisível) e 1 (totalmente opaco). Quanto **menor** o valor de opacidade, mais **transparente** fica o elemento — o fundo por trás dele começa a transparecer.

Atenção

Nota adicional (não estava explícito nos slides): a diferença entre `opacity: 0.5` num elemento e usar `rgba(...,0.5)` só no `background-color` é que a primeira torna **tudo** dentro do elemento transparente (incluindo texto, imagens, bordos), enquanto a segunda só afeta a cor de fundo especificamente — o texto dentro continua 100% opaco. Escolher qual usar depende de se se quer o efeito de transparência só no fundo ou no elemento completo.

Texto: espaçamento

Duas propriedades controlam o espaçamento dentro de um bloco de texto:

- `word-spacing` — espaço extra entre **palavras**;
- `letter-spacing` — espaço extra entre **letras** (carateres).

Ambas tomam um valor numérico com unidade de comprimento (ex: `3mm`), ou o valor `normal` (que é a omissão, sem espaçamento extra).

Texto: alinhamento horizontal e vertical

Definição

Alinhamento horizontal — propriedade `text-align`, aplicada a um elemento de **bloco** (precisa de ter uma largura definida para o alinhamento ter efeito visível). Valores possíveis:

- `left` — alinhado à esquerda (omissão);
- `right` — alinhado à direita;
- `center` — centrado;
- `justify` — justificado (espaço distribuído para as linhas ocuparem toda a largura, exceto a última).

Alinhamento vertical — propriedade `vertical-align`, usada sobretudo em elementos **em linha** (como `span`), porque altera a posição vertical de um elemento em linha relativamente à linha de texto que o rodeia (não tem efeito em elementos de bloco isolados). Valores possíveis:

- nomes predefinidos: `baseline`, `sub` (subscrito), `super` (sobrescrito), `top`, `text-top`, `middle`, `bottom`, `text-bottom`;
- valores numéricos: distâncias (ex: `20px`) ou percentagens, deslocando o elemento para cima (valor positivo) ou para baixo (valor negativo) relativamente à linha de base.

Texto: decoração

A propriedade `text-decoration` desenha uma linha associada ao texto, com os valores:

- `underline` — sublinhado (linha por baixo);
- `overline` — linha por cima;
- `line-through` — rasurado (linha a atravessar o texto);
- `blink` — texto intermitente (obsoleto, praticamente não suportado nos navegadores modernos);
- `none` — sem decoração (omissão, e também usado para remover o sublinhado por omissão de hiperligações `<a>`).

Texto: altura de linha

A propriedade `line-height` controla o espaço vertical reservado para cada linha de texto. Pode tomar:

- o nome predefinido `normal` (omissão, calculado automaticamente a partir do tipo de letra);
- um número (fator multiplicador do tamanho de letra atual, ex: `1.5` significa $1.5 \times$ o tamanho da letra) ou uma percentagem (sobre a altura de linha corrente);
- uma distância com unidade explícita (ex: `15px`).

Frequente em exame

`line-height` vai reaparecer mais à frente como técnica para **centrar texto verticalmente** dentro de uma caixa de altura fixa: basta igualar o `line-height` à `height` da caixa — ver secção de centrar elementos, mais abaixo.

Tipo de letra: família

Definição

A propriedade `font-family` seleciona o tipo de letra, podendo indicar:

- uma **família genérica**: `serif` (com serifas), `sans-serif` (sem serifas), `cursive` (cursiva), `fantasy` (decorativa), `monospace` (largura fixa, todos os caracteres com a mesma largura);
- um **nome de família específico**: `times`, `courier`, `arial`, etc.

Atenção

Nota adicional (não estava explícito nos slides): na prática usa-se quase sempre uma **lista** de valores separados por vírgula (ex: `font-family: "Helvetica", Arial, sans-serif;`), em que o navegador tenta cada fonte da lista pela ordem indicada e usa a primeira que tiver disponível no sistema, terminando sempre com uma família genérica como rede de segurança caso nenhuma das fontes específicas exista.

Tipo de letra: estilo, espessura, tamanho e variante

Definição

Estilo — `font-style`, controla a inclinação:

- **normal** — sem inclinação;
- **italic** — versão **itálica** da fonte, desenhada especificamente pelo tipógrafo (glifos diferentes, não é apenas inclinação mecânica);
- **oblique** — inclinação **falsa**, gerada automaticamente pelo navegador a partir da versão normal (quando não existe uma versão itálica real da fonte).

Espessura — `font-weight`, controla a grossura do traço:

- nomes predefinidos: **normal**, **bold**, **bolder**, **lighter**;
- valores numéricos, de 100 a 900 (múltiplos de 100), em que 400 corresponde a **normal** e 700 a **bold**.

Tamanho — `font-size`:

- nomes predefinidos, numa escala relativa: **xx-small**, **x-small**, **small**, (tamanho normal), **large**, **x-large**, **xx-large**;
- percentagem (relativa ao tamanho de letra do elemento progenitor);
- tamanho absoluto, com qualquer unidade de comprimento (ex: **24pt**).

Variante — `font-variant`, com valores **normal** ou **small-caps**. No valor **small-caps**, as letras **minúsculas** são automaticamente transformadas em **maiúsculas**, mas desenhadas com um tamanho **menor** do que as maiúsculas normais do texto — é o efeito tipográfico chamado “versaletes”.

Posicionamento clássico

O posicionamento clássico usa apenas propriedades herdadas do CSS1/CSS2 (antes de existirem flexbox e grid, que veremos a seguir). Cobre: fazer elementos **flutuar** da sua posição normal, controlar a **classificação** (e por consequência as dimensões) dos elementos, posicionar por **coordenadas** em diferentes **referenciais**, controlar a **sobreposição** e o **transbordo**, e técnicas para **centrar** conteúdo.

Flutuar (`float`) e folgas (`clear`)

Definição

A propriedade **float** retira um elemento do fluxo normal do documento e faz com que **flutue** para um dos lados, sendo rodeado pelo resto do conteúdo (texto e elementos em linha) que continua a fluir à sua volta. Valores: **left** (flutua para a esquerda) ou **right** (flutua para a direita).

Elementos flutuantes foram originalmente pensados para **imagens** e **tabelas** — o típico “texto a envolver uma imagem” de um jornal. Note-se que imagens e tabelas são, por si só, elementos de **bloco** que ocupam toda a largura disponível; **float** é precisamente o que lhes permite deixar de ocupar a largura toda e serem rodeadas por conteúdo.

Colocando vários elementos consecutivos todos com **float: left**, estes **empilham-se lado a lado** (em vez de um por baixo do outro, como aconteceria com blocos normais) — é assim que se conseguem criar **colunas** usando apenas **divs**. O sentido da flutuação (**left** ou **right**) também determina a **ordem** com que os elementos se dispõem: com **float: left**, o primeiro elemento fica mais à esquerda e os seguintes encaixam à sua direita; com **float: right** a ordem inverte-se (o primeiro elemento fica mais à direita). É também possível **misturar** os dois sentidos na mesma linha: os elementos com **float: left** acumulam-se a partir da esquerda, e os elementos com **float: right** acumulam-se a partir da direita, ficando o espaço restante no meio.

Exemplo resolvido

Duas colunas com float. Para criar duas colunas lado a lado a partir de dois `div`s:

```
<div style="float: left; width: 45%;">
  Coluna esquerda...
</div>
<div style="float: left; width: 45%;">
  Coluna direita...
</div>
```

Como ambos têm `float: left`, o segundo `div` encaixa-se à direita do primeiro em vez de ir para a linha seguinte — resultado: duas colunas lado a lado, cada uma ocupando 45% da largura do progenitor.

Atenção

Um elemento **não flutuante** que venha a seguir a elementos flutuantes pode acabar por **sobrepôr-se** a eles (o texto ou bloco seguinte flui para o espaço ainda disponível ao lado/por baixo dos flutuantes, em vez de esperar que eles acabem). Para evitar isto, força-se uma “folga” com a propriedade `clear`, cujo valor indica de que lado(s) o elemento seguinte **não** deve ter elementos flutuantes: `left`, `right` ou `both` (nenhum dos dois lados). É frequente usar `clear: both` num `div` vazio logo a seguir ao último elemento flutuante de um grupo, só para “fechar” a zona de flutuação e garantir que o conteúdo seguinte começa numa linha nova, limpa.

Classificação (`display`) e dimensões

Definição

A propriedade `display` determina como um elemento é posicionado no fluxo, e isso por sua vez determina como as suas **dimensões** são controladas. Os dois valores base são:

- **inline** (em linha) — o elemento segue o **fluxo do texto**, colocando-se lado a lado com outros elementos em linha; as suas dimensões (largura e altura) são determinadas **pelo próprio conteúdo** — não se pode fixar `width/height` num elemento **inline** (o navegador ignora esses valores);
- **block** (bloco) — o elemento ocupa toda a **largura disponível** do seu progenitor e força uma quebra de linha antes e depois de si; `width` e `height` são respeitados normalmente.

Existe ainda **inline-block**, que combina características de ambos: comporta-se como **inline** para efeitos de posicionamento (fica lado a lado no fluxo, sem forçar quebra de linha), mas comporta-se como **block** para efeitos de dimensões (`width/height` são respeitados). Isto torna **inline-block** numa forma simples de criar colunas lado a lado sem recorrer a `float`.

Exemplo resolvido

Comparação entre três `div`s consecutivos com `display: block` (omissão), `inline` e `inline-block`, todos com `width: 100px; height: 300px;`:

- Com `display: block` (omissão de um `div`): cada `div` ocupa uma linha inteira — ficam empilhados verticalmente, um por baixo do outro, mesmo que a largura definida (100px) seja muito menor que a largura disponível.
- Com `display: inline`: os `div`s ficam lado a lado no fluxo de texto, mas o `width: 100px` e `height: 300px` são **ignorados** — cada `div` ocupa apenas o espaço do seu conteúdo real.
- Com `display: inline-block`: os `div`s ficam lado a lado (como `inline`) e respeitam `width: 100px; height: 300px;` (como `block`) — o resultado são três colunas de 100px de largura, lado a lado.

`display` tem ainda outros valores para posicionamento especializado que imitam elementos HTML nativos (úteis quando se quer o comportamento de layout de uma tabela/lista sem usar as tags reais):

- **Tabelas:** `table` (como `<table>`), `table-cell` (como `<td>`), `table-row` (como `<tr>`);
- **Listas:** `list-item` (como `<li>`);
- **Caixas flexíveis** (CSS3): `flex`, `inline-flex` — ver secção Flexbox;
- **Grelha** (CSS3): `grid`, `inline-grid` — ver secção CSS Grid.

Esconder elementos: `display:none` vs `visibility:hidden`

Atenção

Existem **duas** formas de esconder um elemento, com efeitos diferentes no layout:

- **`display: none`** — o elemento é **removido** da formatação como se não existisse: as suas dimensões tornam-se nulas e o espaço que ocuparia é **libertado** para o resto do conteúdo. É a forma habitual de esconder elementos dinamicamente com JavaScript (ex: caixas de diálogo que só aparecem quando necessário).
- **`visibility: hidden`** — o elemento fica **invisível**, mas as suas **dimensões são preservadas**: continua a ocupar o mesmo espaço no layout, só que sem se ver. É útil quando se quer esconder algo sem que o resto do conteúdo “salte” para preencher o espaço.

Coordenadas de posicionamento

Uma vez que um elemento está **posicionado** (com `position` diferente de `static` — ver a seguir), pode ser deslocado usando até quatro propriedades de coordenadas: `top`, `right`, `bottom` e `left`. Cada uma indica a distância entre o lado correspondente da caixa e o lado correspondente do seu **referencial** (que depende do valor de `position`, como vamos ver). Não é preciso especificar as quatro — por exemplo, `top` e `left` sozinhas já posicionam o elemento a partir do canto superior esquerdo do referencial.

Referencial de posicionamento: os 5 valores de `position`

Frequente em exame

Esta é tipicamente a parte mais confusa (e mais testada em exame) do posicionamento clássico: cada valor de `position` estabelece um **referencial diferente** para as coordenadas `top/right/bottom/left`, e também difere em se o **espaço da posição estática original** do elemento fica ou não reservado no layout.

position	referencial de top/right/bottom/left	espaço da posição estática fica reservado?
static	nenhum — propriedades de posição são ignoradas	sim (é o próprio fluxo normal)
relative	a própria posição estática do elemento (desloca-se a partir dela)	sim
absolute	o progenitor “posicionado” mais próximo (position diferente de static); se não houver, o bloco inicial do documento	não
fixed	a janela do navegador (viewport) — não se move com o scroll	não
sticky	alterna entre relative (posição normal visível) e fixed (ao contentor com scroll, quando essa posição saíria do ecrã)	sim

Definição

static — é o valor de **omissão**. O elemento é posicionado inteiramente de acordo com a sua classificação (**display**) e a ordem no documento; as propriedades **top/right/bottom/left** são **irrelevantes** e ignoradas por completo. Não há “referencial” a falar propriamente — é o próprio fluxo normal do documento.

relative — o elemento continua a ocupar o seu espaço normal no fluxo (o espaço da posição estática é **mantido**, ao contrário dos casos seguintes), mas depois é **deslocado visualmente** a partir dessa mesma posição estática, segundo os valores de **top/right/bottom/left**. Ou seja, o referencial é a **própria posição estática do elemento**. É também o valor usado internamente para implementar os efeitos de **vertical-align: sub/super** vistos atrás.

absolute — o elemento é retirado do fluxo normal (o espaço da posição estática **não** é mantido — os outros elementos comportam-se como se ele não existisse). O referencial passa a ser o **primeiro progenitor “posicionado”** que se encontrar subindo na árvore — ou seja, o primeiro antepassado com **position** diferente de **static** (seja **relative**, **absolute**, **fixed** ou **sticky**). Se nenhum progenitor estiver posicionado, o referencial acaba por ser o bloco inicial do próprio documento.

fixed — semelhante ao **absolute** (também retirado do fluxo, espaço não reservado), mas o referencial é sempre a **janela do navegador** (*viewport*) — não um progenitor qualquer. Por isso um elemento **fixed** mantém sempre a mesma posição no ecrã mesmo quando se faz *scroll* à página.

sticky — comportamento **híbrido**, que alterna entre dois modos consoante a posição de *scroll*: comporta-se como **relative** (mantendo o espaço normal e a posição dentro do fluxo) enquanto essa posição está visível dentro do seu contentor; mas assim que essa posição ficaria “tapada” pelo *scroll* (sairia da área visível do contentor), passa a comportar-se como **fixed relativamente ao contentor com scroll**, ficando “colado” a essa borda enquanto o resto do conteúdo continua a deslocar-se por baixo/cima dele. É especialmente útil para cabeçalhos de secção ou de tabela que devem ficar sempre visíveis durante o *scroll*.

Atenção

Nota adicional (não estava explícito nos slides, mas é um truque de uso muito comum): como o referencial de **absolute** é “o primeiro progenitor posicionado”, uma técnica frequente é dar **position: relative**; a um elemento progenitor **sem** lhe atribuir nenhum **top/left** (ou seja, sem realmente o deslocar visualmente) só para o tornar “posicionado” e assim servir de referencial (*viewport* local) para um descendente com **position: absolute**. Sem este truque, o descendente **absolute** iria procurar um progenitor posicionado mais longe na árvore (ou cair no documento todo), o que normalmente não é o efeito pretendido.

Sobreposição (z-index)

Definição

Quando elementos posicionados se sobrepõem uns aos outros, a propriedade **z-index** controla a **ordem de sobreposição** — imagina-se uma pilha de níveis transparentes, e cada elemento fica associado a um desses níveis pelo valor (numérico) de **z-index**. Elementos com índice **mais baixo** ficam **por baixo** (sobrepostos) dos elementos com índice mais alto.

Atenção

z-index não tem qualquer efeito em elementos com **position: static** (a omissão) — só funciona em elementos que estejam **posicionados** (**relative**, **absolute**, **fixed** ou **sticky**). Tentar usar **z-index** num elemento estático é ignorado silenciosamente pelo navegador.

Transbordar (overflow)

Quando o conteúdo de um elemento **excede** as dimensões que lhe foram atribuídas, diz-se que o conteúdo **transborda**. A propriedade **overflow** controla o que acontece nesse caso:

- **visible** — o conteúdo continua **visível**, ultrapassando os limites da caixa (é a omissão);
- **hidden** — o conteúdo em excesso é **escondido** (cortado no limite da caixa);
- **scroll** — é adicionada uma barra de deslocamento (*scrollbar*), sempre visível, para aceder ao conteúdo em excesso;
- **auto** — comporta-se como **scroll**, mas a barra de deslocamento **só aparece se for realmente necessária** (ou seja, só se o conteúdo transbordar).

Centrar texto vs. centrar elementos

Centrar em CSS2 (sem flexbox nem grid) exige técnicas diferentes consoante o que se quer centrar.

Exemplo resolvido

Centrar texto (horizontal e vertical) dentro de uma caixa. Considera um botão com `width: 200px; height: 100px;`:

```
div.botao {  
  width: 200px;  
  height: 100px;  
  text-align: center;    /* centra o texto na horizontal */  
  line-height: 100px;   /* centra o texto na vertical    */  
}
```

- **Horizontalmente:** `text-align: center` centra o texto dentro da largura do bloco (tal como visto na secção de texto, acima).
- **Verticalmente:** o truque é igualar `line-height` à `height` da caixa. Como uma linha de texto se centra sempre verticalmente dentro da sua própria altura de linha, ao fazer essa altura de linha igual à altura *total* da caixa (aqui, 100px em ambas), o texto acaba centrado na caixa inteira. É essencial que os dois valores sejam **iguais** — se `line-height` fosse diferente de `height`, o texto deslocar-se-ia para cima ou para baixo do centro.

Exemplo resolvido

Centrar um elemento (bloco) na horizontal e na vertical. Horizontalmente, um bloco com largura fixa centra-se com a já conhecida técnica `margin: auto;` (o navegador distribui a margem esquerda e direita automaticamente e de forma igual). Verticalmente, sem flexbox nem grid, o mais comum é combinar posicionamento relativo com uma compensação manual:

```
div.botao {  
  width: 100px;  
  height: 100px;  
  margin: auto;           /* passo 1: centra na horizontal */  
  position: relative;     /* passo 2: fica posicionado, para poder usar top */  
  top: 50%;               /* passo 3: desloca 50% da altura do contentor */  
  margin-top: -50px;      /* passo 4: compensa metade da ALTURA DO PRÓPRIO BOTÃO */  
}
```

Passo a passo: `top: 50%` desloca o elemento para baixo, a partir da sua posição normal, numa distância igual a 50% da altura do **contentor** (não do próprio elemento) — isto sozinho colocaria o **topo** do botão a meio do contentor, o que ainda não é centrar o botão (o botão ficaria deslocado meio botão abaixo do centro). Por isso o passo seguinte, `margin-top: -50px`, sobe o elemento de volta metade da **sua própria altura** ($100\text{px} / 2 = 50\text{px}$), compensando esse desvio e deixando agora o **centro** do botão exatamente a meio do contentor.

Flexbox

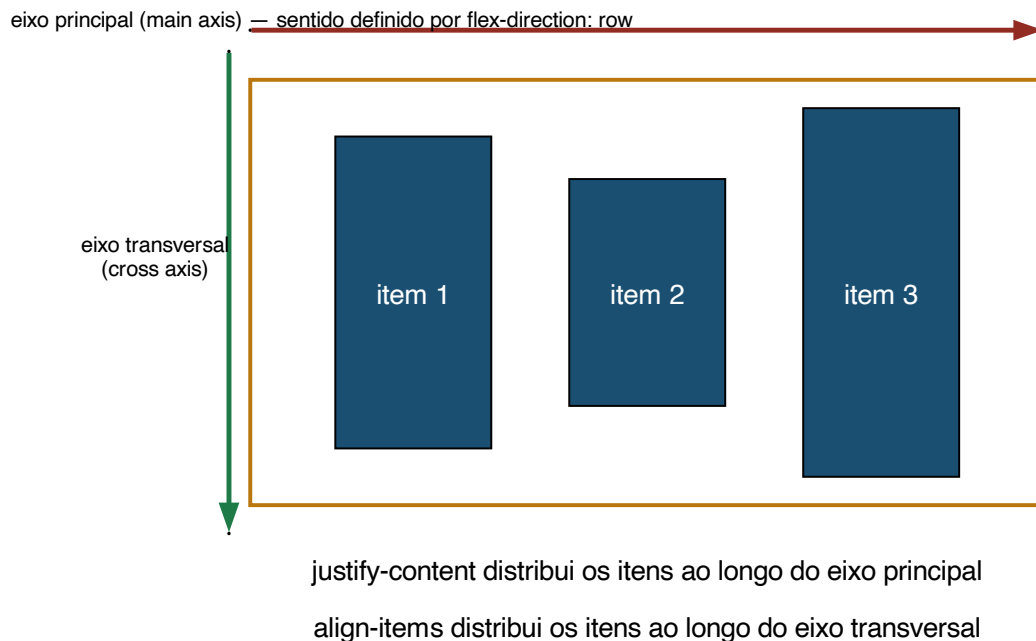
O conceito: eixo principal e eixo transversal

Definição

O **flexbox** (caixas flexíveis) é uma estratégia de posicionamento em **fluxo**, mas muito mais flexível do que o posicionamento clássico — baseia-se em conceitos usados há muito tempo em *toolkits* gráficos de interfaces (preenchimento de espaço, fluxo com quebra, alinhamento). Ao contrário do posicionamento clássico, o flexbox distingue explicitamente dois **papéis**: o elemento **contentor** (que ativa o flexbox com `display: flex`) e os seus **itens** (todos os filhos diretos do contentor, que se tornam itens automaticamente).

O flexbox organiza os itens ao longo de dois eixos:

- **eixo principal** (*main axis*) — o eixo ao longo do qual os itens são colocados em sequência; a sua direção é definida por `flex-direction`;
- **eixo transversal** (*cross axis*) — o eixo perpendicular ao eixo principal.



Atenção

Nota adicional (não estava explícito nos slides mas é fundamental para não confundir as propriedades seguintes): **qual eixo é “principal” e qual é “transversal” depende de flex-direction**. Com `flex-direction: row` (omissão), o eixo principal é **horizontal** e o transversal é **vertical**. Com `flex-direction: column`, isto inverte-se: o eixo principal passa a ser **vertical** e o transversal **horizontal**. Isto significa que `justify-content` (que atua sempre no eixo principal) distribui os itens na horizontal com `row`, mas na vertical com `column` — e vice-versa para `align-items` (que atua sempre no eixo transversal).

Propriedades do contentor

Definição

Sobre o **contentor flex** existem as seguintes propriedades (o prefixo **flex** é usado quando já existe uma propriedade “genérica” com esse nome, para evitar ambiguidade):

- **Exibição** — `display`;
- **Direção** — `flex-direction`;
- **Quebra** — `flex-wrap`;
- **Fluxo** — `flex-flow` (atalho para as duas anteriores);
- **Justificação** — `justify-content`;
- **Alinhar itens** — `align-items`;
- **Alinhar conteúdo** — `align-content`.

Exibição (`display`) identifica o elemento como contentor flex: **flex** posiciona o contentor como um bloco (como um `div`); **inline-flex** posiciona-o como elemento em linha (como um `span`). Em ambos os casos, **todos os descendentes diretos** tornam-se automaticamente itens flex.

Direção (`flex-direction`) controla a orientação do eixo principal:

- **row** — em linha, da esquerda para a direita (omissão);
- **row-reverse** — em linha, da direita para a esquerda;
- **column** — em coluna, de cima para baixo;
- **column-reverse** — em coluna, de baixo para cima.

Quebra (`flex-wrap`) controla o que acontece quando os itens excedem o espaço disponível ao longo do eixo principal:

- **nowrap** — sem quebra; os itens **encolhem** para caber todos na mesma linha (omissão);
- **wrap** — quebra para a linha seguinte;
- **wrap-reverse** — quebra para a linha anterior (inverte o sentido em que as novas linhas se acumulam).

Fluxo (`flex-flow`) é apenas um atalho (*shorthand*) que combina `flex-direction` e `flex-wrap` num só valor, permitindo misturar livremente qualquer direção com qualquer modo de quebra (ex: `flex-flow: column-reverse wrap`);).

Exemplo resolvido

justify-content — **distribuição ao longo do eixo principal**. Com `flex-direction: row` (eixo principal horizontal) e três itens dentro do contentor, cada valor produz:

- **flex-start** — itens encostados ao **início** do eixo principal (à esquerda), com o espaço livre todo acumulado no fim (à direita);
- **flex-end** — itens encostados ao **fim** do eixo principal (à direita), com o espaço livre todo no início;
- **center** — itens agrupados no **centro** do eixo principal, com espaço livre repartido igualmente antes do primeiro e depois do último item;
- **space-between** — o **primeiro** item encosta ao início e o **último** encosta ao fim; o espaço livre restante é distribuído em partes iguais **entre** os itens (não há espaço nas pontas);
- **space-around** — o espaço livre é distribuído em partes iguais **à volta** de cada item (cada item recebe a mesma margem de ambos os lados) — visualmente, os espaços entre itens ficam com o dobro do espaço nas pontas (porque nas pontas só há a margem de um item, entre itens há a soma da margem de dois).

Exemplo resolvido

align-items — alinhamento ao longo do eixo transversal. Com `flex-direction: row` (eixo transversal vertical), controla onde os itens ficam **verticalmente** dentro da altura do contentor:

- **stretch** — os itens **esticam-se** para preencher toda a altura do contentor (omissão; só tem efeito visível se os itens não tiverem **height** própria fixada);
- **flex-start** — itens alinhados ao **topo** do eixo transversal;
- **flex-end** — itens alinhados ao **fundo** do eixo transversal;
- **center** — itens centrados **verticalmente**;
- **baseline** — itens alinhados pela **linha de base do texto** (útil quando os itens têm tamanhos de letra diferentes e se quer que o texto fique alinhado, independentemente da altura de cada caixa).

align-content funciona de forma semelhante a **align-items**, mas só tem efeito quando há **várias linhas** de itens (`flex-wrap: wrap` ativo, com o contentor a ter altura suficiente para mostrar essas várias linhas) — em vez de alinhar cada item individualmente, distribui as **linhas inteiras** ao longo do eixo transversal (aceitando também `space-around`, tal como `justify-content`).

Propriedades do item

Definição

Sobre cada **item** flex existem as seguintes propriedades (o prefixo `flex` é usado quando já existe uma propriedade “genérica” com esse nome):

- **Crescer** — `flex-grow`;
- **Base** — `flex-basis`;
- **Encolher** — `flex-shrink`;
- **Reúne** — `flex` (atalho para as três anteriores);
- **Ordem** — `order`;
- **Alinha** — `align-self`.

Base (flex-basis) define o tamanho **base** de um item ao longo do eixo principal, antes de qualquer crescimento ou encolhimento ser aplicado. Se omitido (ou com valor `auto`), o tamanho base é o tamanho natural do próprio conteúdo do item. `flex-basis` tem **prioridade** sobre `width` (com `flex-direction: row`) ou `height` (com `flex-direction: column`) — ou seja, quando ambos estão definidos, `flex-basis` é o que realmente conta.

Exemplo resolvido

Crescer (flex-grow). Define a capacidade de um item **crescer** para ocupar o espaço livre restante no contentor, depois de todos os itens terem o seu tamanho base atribuído. O valor de omissão é 0 (não cresce, não ocupa espaço livre extra). Quando vários itens têm `flex-grow` maior que zero, o espaço livre é distribuído **proporcionalmente** aos valores:

- Três itens todos com `flex-grow: 1` — o espaço livre é dividido em **três partes iguais**, uma para cada item.
- Itens com `flex-grow: 1`, `flex-grow: 2` e `flex-grow: 3` — o espaço livre é dividido em $1+2+3 = 6$ partes; o primeiro item recebe $1/6$ desse espaço extra, o segundo $2/6$, o terceiro $3/6$ (o dobro do primeiro).

Nota que `justify-content` deixa de ter qualquer efeito visível quando `flex-grow` está ativo nos itens: se os itens já absorvem todo o espaço livre a crescer, não sobra espaço livre nenhum para `justify-content` distribuir.

Exemplo resolvido

Encolher (`flex-shrink`). É o oposto de `flex-grow`: controla o **encolhimento** dos itens quando, ao somar os seus tamanhos base (`flex-basis`), excedem o espaço disponível no contentor. Valores:

- 0 — o item **não encolhe**, mesmo que o conjunto extravase o contentor;
- 1 — encolhimento “normal” (é o valor de omissão);
- um valor n maior — o item encolhe n vezes mais depressa do que um item com `flex-shrink: 1`.

Por exemplo, com três itens de `flex-basis` 100px, 200px e 300px e `flex-shrink` 1, 2 e 3 respetivamente, se for preciso encolher o conjunto, o terceiro item (com `flex-shrink: 3`) perde proporcionalmente mais largura do que o primeiro (com `flex-shrink: 1`).

Reúne (`flex`) é o atalho (*shorthand*) que atribui `flex-grow`, `flex-shrink` e `flex-basis` de uma só vez, nessa mesma ordem (ex: `flex: 1 0 200px`; — cresce, não encolhe, base de 200px). O valor de omissão implícito de `flex` é 0 1 auto. Não é obrigatório indicar os três valores — por exemplo `flex: 2`; define apenas `flex-grow: 2`, deixando `flex-shrink` e `flex-basis` nos seus valores de omissão.

Ordem (`order`) permite reordenar visualmente os itens sem alterar a ordem em que aparecem no HTML. O valor de omissão é 0 (ordem natural, de ocorrência no documento); itens com `order` mais baixo aparecem antes dos de `order` mais alto, e são permitidos valores **negativos** (para colocar um item antes de todos os que têm `order: 0`).

Alinha (`align-self`) sobrepõe, apenas para um item específico, o valor de `align-items` definido no contentor — permite que um único item tenha um alinhamento diferente dos restantes no eixo transversal. Aceita os mesmos valores de `align-items` (`flex-start`, `center`, `baseline`, `flex-end`, `stretch`).

CSS Grid

O conceito: linhas e colunas da grelha

Definição

O **CSS Grid** é uma estratégia de posicionamento **bidimensional** (diferente do flexbox, que é unidimensional): organiza os elementos numa grelha explícita de **linhas** e **colunas**. Tal como no flexbox, existem dois papéis: o **contentor** (ativado com `display: grid`) e os **itens** (os seus filhos diretos, dispostos automaticamente nas células da grelha).

As fronteiras entre células — tanto na horizontal como na vertical — são chamadas **linhas de grelha** (*grid lines*) e são numeradas a partir de 1: uma grelha com 3 colunas tem 4 linhas de grelha verticais (antes da primeira coluna, entre cada duas colunas, e depois da última); o mesmo raciocínio aplica-se às linhas de grelha horizontais consoante o número de linhas de conteúdo.

	linha de coluna 1	2	3	4
linha de linha 1				
	1,1	1,2	1,3	
2	2,1	item com grid-column: span 2		
3				
	3,1	3,2	3,3	
4				

Numeros a cinza = linhas de grelha (fronteiras); células a laranja = item expandido por 2 colunas

Propriedades do contentor

Definição

Sobre o **contentor** grid existem as seguintes propriedades (o prefixo **grid** é usado na generalidade dos casos):

- **Exibição** — `display`;
- **Modelos** — `grid-template-columns`, `grid-template-rows`;
- **Separações** — `row-gap`, `column-gap`;
- **Fluxos** — `grid-auto-flow`;
- **Dimensões** — `grid-auto-rows`, `grid-auto-columns`;
- **Abreviaturas** — `grid`, `gap`, `grid-template`.

Exibição (`display`) identifica o elemento como contentor de grelha: `grid` posiciona-o como bloco; `inline-grid` posiciona-o como elemento em linha. Todos os descendentes diretos tornam-se automaticamente itens da grelha.

Modelos (`grid-template-columns` / `grid-template-rows`) definem o **número** e a **dimensão** das colunas/linhas da grelha, através de uma lista de valores separados por espaços — cada valor da lista corresponde a uma coluna (ou linha). Por exemplo, `grid-template-columns: 100px 40% auto;` cria exatamente **três** colunas: a primeira com **100px** fixos, a segunda com 40% da largura do contentor, e a terceira a ocupar o espaço restante (`auto`). Valores possíveis para cada entrada:

- um **comprimento** (absoluto ou relativo, incluindo percentagens);
- **none** — omissão; as colunas/linhas são criadas apenas conforme necessário (isto é, sem um modelo explícito, ver “dimensões” abaixo);
- **auto** — dimensão determinada automaticamente pelo conteúdo;
- **max-content** / **min-content** — dimensão igual à do maior/menor elemento dessa linha/coluna;
- **initial** / **inherit** — valor de omissão / herdado do progenitor.

Separações (`row-gap` / `column-gap`) definem o espaço (uma espécie de “corredor”) entre linhas e entre colunas da grelha, respetivamente — funcionam de forma semelhante a uma margem, mas só entre células,

nunca nas bordas exteriores da grelha. Aceitam um comprimento, ou **normal** (omissão, sem espaço extra).

Fluxos (grid-auto-flow) controla a ordem pela qual os itens são automaticamente colocados nas células da grelha, quando não têm posição explícita: **row** (omissão) insere os itens **linha a linha** (preenche a primeira linha toda antes de passar à seguinte); **column** insere os itens **coluna a coluna**.

Atenção

Nota adicional (não estava explícito nos slides): a diferença entre **grid-template-columns** (que define um modelo **explícito** com um número fixo de colunas) e **grid-auto-columns/grid-auto-rows** (que definem a dimensão das linhas/colunas **implícitas**, criadas automaticamente quando há mais itens do que células no modelo explícito) é fácil de confundir. Se a grelha tiver **grid-template-rows: auto auto;** (só duas linhas explícitas) mas os itens ocuparem uma terceira linha extra (por haver itens a mais), essa terceira linha é **implícita** e a sua altura é controlada por **grid-auto-rows** — não por **grid-template-rows**, que só cobre as duas linhas que foram explicitamente declaradas.

As **dimensões** das linhas/colunas implícitas (**grid-auto-rows / grid-auto-columns**) aceitam os mesmos tipos de valor que **grid-template-columns/grid-template-rows**: um comprimento, **auto** (omissão, determinado pelo elemento maior), **max-content** ou **min-content**.

Por fim, existem três propriedades que **reúnem** (atalhos) várias das anteriores: **grid-template** reúne **grid-template-columns** e **grid-template-rows**; **gap** reúne **row-gap** e **column-gap**; e **grid** reúne **todas** as declarações de contentor da grelha numa só propriedade.

Propriedades do item

Definição

Sobre cada **item** de grelha existem propriedades para o **posicionar** e **expandir** através de várias células:

- **Posicionar** — **grid-row-start**, **grid-row-end** (linha), **grid-column-start**, **grid-column-end** (coluna);
- **Expandir** — usar o valor **span** seguido de um número inteiro, em qualquer das propriedades de posição acima;
- **Reúne** — **grid-row** reúne **grid-row-start/grid-row-end**; **grid-column** reúne **grid-column-start/grid-column-end**.

Posicionar um item significa indicar explicitamente entre que **linhas de grelha** ele deve ficar, usando os números dessas linhas (não o número da célula/coluna em si, mas o número da fronteira). Por exemplo, **grid-row-start: 2;** coloca o topo do item exatamente na linha de grelha horizontal número 2 (ou seja, começa na segunda linha de conteúdo); combinado com **grid-column-start: 2;**, o item é colocado na célula da segunda linha e segunda coluna, saltando a posição que a ordem automática lhe daria.

Expandir um item por **várias células** de uma só vez faz-se com o valor **span** seguido do número de células a ocupar, em vez de um número de linha de grelha absoluto — por exemplo **grid-row-start: span 3;** faz o item ocupar **3 linhas** de altura (em vez de uma célula só), e **grid-column-start: span 2;** faz-o ocupar **2 colunas** de largura. Este mecanismo de **span** funciona da mesma forma quer se aplique a linhas quer a colunas, e é o que está representado no diagrama acima (o item a laranja ocupa duas colunas com **grid-column: span 2**).

Por fim, **grid-row** e **grid-column** são atalhos que reúnem, respetivamente, o par início/fim de linha (**grid-row-start + grid-row-end**) e o par início/fim de coluna (**grid-column-start + grid-column-end**) numa só declaração — por exemplo **grid-row: span 3;** tem o mesmo efeito que escrever só **grid-row-start: span 3;** sem **grid-row-end**.

JavaScript — Fundamentos da Linguagem

As 3 formas de mostrar output (`document.write`, `alert`, `console.log` — e quando se usa cada uma)

O JavaScript é integrado no HTML através de elementos `<script>`, que podem estar tanto no `body` como no `head` do documento. A posição importa para quando o código corre:

Definição

- Um `<script>` dentro do `body` é executado **durante** a formatação (*parsing*) da página, no ponto exato em que aparece — se escrever algo para o documento nesse momento, o resultado aparece ali, no meio do HTML à sua volta.
- Um `<script>` dentro do `head` é executado **antes** da formatação do corpo da página começar (a página em si ainda não existe visualmente).
- Código também pode correr mais tarde, **em reação a eventos** (*event handlers*), como o clique num elemento (`onclick="..."`, atributo HTML que corre JavaScript quando o evento ocorre).

Há três formas distintas de “mostrar” algo a partir de JavaScript, cada uma com um propósito diferente:

Definição — As três formas de output

1. **`document.write(x)` / `document.writeln(x)`** — escrevem o argumento diretamente na própria formatação da página (isto é, inserem HTML/texto no documento tal como ele está a ser construído). `writeln` faz o mesmo que `write` mas acrescenta também uma mudança de linha (`\n`) no fim. Anotações HTML dentro do texto escrito (ex: `<h3>Olá</h3>`) são interpretadas e formatadas normalmente, como se estivessem no HTML original.
2. **`alert(x)`** — mostra uma caixa de diálogo modal ao utilizador com o argumento. É tipicamente usada em resposta a eventos. Duas funções irmãs, do mesmo estilo de caixa de diálogo:
 - **`prompt(mensagem)`** — mostra uma caixa com uma pergunta e um campo de texto; **recolhe e retorna** o valor (**string**) que o utilizador escreveu (ou `null` se cancelar).
 - **`confirm(mensagem)`** — mostra uma caixa com “OK”/“Cancelar”; **recolhe e retorna um booleano** consoante a escolha do utilizador.

Nenhuma destas três caixas de diálogo suporta anotações HTML — o texto é mostrado literalmente.

3. **`console.log(x)`** — escreve o argumento na **consola** do navegador (uma das ferramentas de programador, não visível na página em si nem ao utilizador comum). É a ferramenta natural para depuração (*debugging*), e a consola também permite executar comandos JavaScript interativamente ali mesmo, útil para testar pequenos trechos de código sem alterar a página.

Atenção

Nota adicional (não estava explícito nos slides): a função que **recolhe um booleano** de uma caixa de diálogo chama-se `confirm()`, não `message()` — não existe nenhuma função global `message()` em JavaScript. As três caixas de diálogo nativas do navegador são exatamente estas três: `alert()` (só mostra), `confirm()` (pergunta sim/não → booleano) e `prompt()` (pede texto → string ou `null`).

Frequente em exame

Em prática/exame é fácil confundir os três mecanismos porque todos “mostram” algo, mas servem propósitos muito diferentes: `document.write` manipula o **conteúdo da página**, `alert/confirm/prompt` interrompem a execução para **interagir com o utilizador**, e `console.log` é uma ferramenta de **depuração** que nunca deve ficar em código de produção visível ao utilizador final.

Sintaxe básica

Tipos primitivos, expressões e variáveis

O JavaScript tem 3 tipos primitivos básicos:

Definição

- **Number** — números (inteiros e decimais não são distinguidos), ex: 123
- **String** — cadeias de caracteres, ex: "Olá"
- **Boolean** — valores lógicos, ex: true/false

As expressões são muito semelhantes às de Java — mesmos operadores, mesmas precedências e associatividades. Duas particularidades importantes:

Exemplo resolvido

```
document.writeln(1 + 1);      // 2      (soma numérica)
document.writeln("1" + "1");  // "11"    (+ também concatena strings)
document.writeln(1 && 1);      // 1      (ver nota abaixo)
document.writeln(1 && true);    // true
document.writeln(1 == "1");    // true    (compara só valor, converte tipo)
document.writeln(1 === "1");   // false   (compara valor E tipo)
```

Explicação linha a linha:

- `1 + 1` → ambos são **Number**, `+` soma normalmente → 2.
- `"1" + "1"` → como pelo menos um operando é **String**, `+` funciona como **concatenação**, não soma → "11".
- `1 && 1` → o operador `&&` (E lógico) em JavaScript **não converte sempre o resultado para booleano**: se o primeiro operando for “verdadeiro” (*truthy*), o `&&` devolve o **segundo operando tal como é**, sem o converter. Como 1 é *truthy*, o resultado é o valor 1 (não true).
- `1 && true` → pela mesma regra, o primeiro operando (1) é *truthy*, logo devolve o segundo operando, que já é true.
- `1 == "1"` → o operador `==` (igualdade **fraca**) converte os tipos antes de comparar: converte a string "1" para número 1, compara `1 == 1` → true.
- `1 === "1"` → o operador `===` (igualdade **estrita**) não converte nada: compara tipo e valor. Como um é **Number** e o outro é **String**, são imediatamente diferentes → false.

Atenção

Nota adicional (não estava explícito nos slides): esta particularidade do `&&/||` — devolverem um dos operandos originais e não um booleano — é chamada de **avaliação de curto-circuito com valor**. `a && b` devolve `a` se `a` for *falsy* (sem sequer avaliar `b`), senão devolve `b`. `a || b` devolve `a` se `a` for *truthy*, senão devolve `b`. É uma técnica comum em JavaScript para dar valores por omissão, ex: `nome = nomeDado || "Anônimo"`.

Quanto a variáveis:

Definição

- Uma variável **não precisa de ser declarada** para existir: é criada (mas **não declarada**) implicitamente pela sua primeira **inicialização** (atribuição). Isto cria uma variável **global**, mesmo que a atribuição esteja dentro de uma função — má prática, evitar.
- Podem e **devem** ser declaradas com uma destas três palavras-chave:
 - **const** — não pode ser reatribuída depois de inicializada. *Usar sempre que possível* (o valor não vai mudar).
 - **let** — âmbito restrito ao **bloco** { } em que é declarada.
 - **var** — âmbito alargado à **função** (ou global, se fora de qualquer função). *Evitar usar* — ver a secção de âmbito abaixo para perceber porquê é problemático.
- Podem ser inicializadas na própria declaração.
- O **tipo** de uma variável é **inferido dinamicamente** a partir do valor atual — a mesma variável pode passar a conter um valor de outro tipo mais tarde (ao contrário de Java).

Exemplo resolvido

```
x = 1;           // sem declaração -> cria variável global implicitamente

const y = 2;
let z;           // declarada, ainda sem valor (fica undefined)
var w;           // declarada, ainda sem valor (fica undefined)

z = 3;
w = 4;

console.log(x + y + z + w);
```

Trace: x=1 (global, implícita), y=2 (constante), z declarada como **undefined** e depois passa a 3, w declarada como **undefined** e depois passa a 4. No `console.log` final: `x+y+z+w = 1+2+3+4 = 10`.

Blocos, comentários e separação de instruções

Definição

- Comentários como em Java: `//` para uma linha, `/* ... */` para vários.
- Um **bloco** é uma instrução **composta**: várias instruções entre chavetas { }, tal como em Java.
- O **âmbito** de uma variável declarada com **let** é exatamente o bloco { } que a contém (mais detalhe na secção “Âmbito de variáveis”).
- Instruções são separadas por ponto e vírgula ;, mas este separador **pode ser omitido** — o motor de JavaScript infere-o automaticamente (ver ASI, “Automatic Semicolon Insertion”, abaixo).

Exemplo resolvido

```
{
  let x = 1;
  let y = 2;

  // bloco interior, aninhado no exterior
  {
    let x = 3;
  }
}
```

Trace: o bloco exterior declara $x=1$ e $y=2$ no seu próprio âmbito. O bloco interior declara **outro** x , com valor 3 — como `let` tem âmbito de bloco, este $x=3$ é uma variável **completamente distinta** da $x=1$ de fora, e existe apenas dentro das chavetas internas. Ao sair do bloco interior, essa variável deixa de existir; o x do bloco exterior nunca foi alterado e continua a valer 1.

A omissão do `;` funciona através de um mecanismo chamado **inserção automática de ponto e vírgula** (*Automatic Semicolon Insertion*, ASI): o compilador infere o fim de uma instrução a partir da mudança de linha, mas só em certas condições. O `;` é necessário sempre que há ambiguidade entre duas instruções — e essa ambiguidade é mais comum do que parece:

Exemplo resolvido

```
const a = 1
const b = 2
const c = a + b
(a + b).toString()
```

Isto lança `TypeError: b is not a function`. Não é o comportamento que a leitura superficial sugere (quatro instruções independentes).

Trace do que o motor de JavaScript realmente vê: o ASI só insere um `;` automaticamente quando a linha seguinte, lida tal como está, **tornaria a instrução inválida**. A linha `(a + b).toString()` começa com `(`, e `a + b (a + b).toString()` é **sintaticamente válido** como uma única expressão — uma chamada de função! Por isso o ASI **não** insere `;` ali, e o parser junta as linhas 3 e 4 numa só instrução:

```
const c = a + b(a + b).toString()
```

Ou seja, é lido como “ c recebe a mais o resultado de invocar b como função, passando-lhe $(a+b)$ como argumento, e depois chamando `.toString()` ao resultado”. Como b é o número 2, e números não são funções, tentar invocar `b(...)` lança `TypeError: b is not a function`.

Atenção

Regra geral do ASI (as condições em que a próxima linha força a inserção do `;`, mesmo sem ambiguidade de chamada de função):

- a próxima linha, lida a seguir sem `;`, tornaria a instrução inválida (caso mais comum);
- a próxima linha começa por `{` (abertura de bloco);
- a linha atual contém apenas `return`, `break`, `throw` ou `continue` (nestes casos o `;` é sempre inserido logo a seguir à palavra-chave, **mesmo que isso quebre a intenção do programador** — ex: um `return` seguido de uma expressão na linha seguinte é interpretado como um `return;` sozinho, e a expressão seguinte é ignorada);
- chegou ao fim do ficheiro (EOF).

O `;` pode ser omitido, mas é **boa prática usá-lo sempre como terminador**, exatamente para evitar armadilhas como o exemplo acima.

Controlo de fluxo

O controlo de fluxo é muito semelhante ao de Java, com as mesmas palavras-chave: `if/else`, `while/do`, `switch/case/break`, `for/break/continue`. As variáveis de controlo em ciclos devem usar `let` (não `var` — ver âmbito abaixo). Existe ainda um ciclo **estendido**, com sintaxe ligeiramente diferente: `for (variável in objeto) { ... }` (tratado em detalhe na secção “Objetos e arrays”).

Atenção

Nota adicional (não estava explícito nos slides): o código de exemplo original desta parte dos slides tinha uma chaveta em falta antes do `else`, o que é sintaticamente inválido em JavaScript (ao contrário de Python, o `if/else` do JavaScript segue as mesmas regras de chavetas do Java — se o corpo do `if` está entre `{ }`, o `else` correspondente tem de vir a seguir a esse `}` fechado). Versão corrigida:

```
for (let x = 1; x < 6; x++) {  
  document.write(x + " é ");  
  if (x % 2 == 0) {  
    document.writeln("par");  
  } else {  
    document.writeln("ímpar");  
  }  
}
```

Trace (x de 1 a 5): 1 é ímpar, 2 é par, 3 é ímpar, 4 é par, 5 é ímpar. A condição `x < 6` deixa de ser verdadeira quando x chega a 6, terminando o ciclo.

Declarar e invocar funções

Definição

- **Declaração:** `function nome(parâmetros) {definição}`. Não carece de `;` a seguir por não ser uma instrução simples, mas uma **declaração**. O nome fica associado à função. Os parâmetros não têm tipo (tal como as variáveis). A definição **pode** conter uma instrução `return`; o valor da expressão em `return` é o que fica disponível para quem invocar a função.
- **Invocação:** a função é invocada pelo nome, com uma lista de **argumentos** entre parêntesis. Os argumentos são atribuídos, por posição, aos parâmetros correspondentes. A definição da função é então avaliada com esses valores. O valor de retorno **substitui** a própria expressão de invocação no código que a chamou.

Exemplo resolvido

```
function soma(a, b) {  
  return a + b;  
}
```

```
document.writeln(soma(2, 3));
```

Trace: `soma(2,3)` é invocada → dentro da função, `a=2` e `b=3` → `return a+b` avalia `2+3=5` → a expressão `soma(2,3)` no código do chamador é substituída pelo valor 5 → `document.writeln(5)` escreve 5.

Âmbito de variáveis — função vs. bloco

Uma variável pode ter âmbito **local** (visível apenas dentro da função onde foi declarada) ou **global** (se não foi declarada dentro de nenhuma função). Cada definição de função cria um **novo âmbito**, e variáveis locais sobrepõem-se (fazem *shadowing*) a variáveis globais com o mesmo nome.

A diferença crucial entre `var` e `let/const` é que **`var` ignora blocos**: o seu âmbito é sempre a função (ou o

global) mais próxima, nunca um simples `{ }`. Já `let/const` respeitam o bloco onde foram declaradas. Isto é fácil de ler mal — o código *parece* criar uma variável nova dentro do `if`, mas com `var` não cria:

Exemplo resolvido

```
function testarAmbito() {  
  var a = "função";  
  let b = "função-let";  
  
  if (true) {  
    var a = "bloco";      // MESMA variável 'a' de cima (var é de função)  
    let b = "bloco-let";  // variável NOVA (let é de bloco, esconde a de fora)  
    console.log("dentro do bloco:", a, b);  
  }  
  
  console.log("fora do bloco:", a, b);  
}  
  
testarAmbito();
```

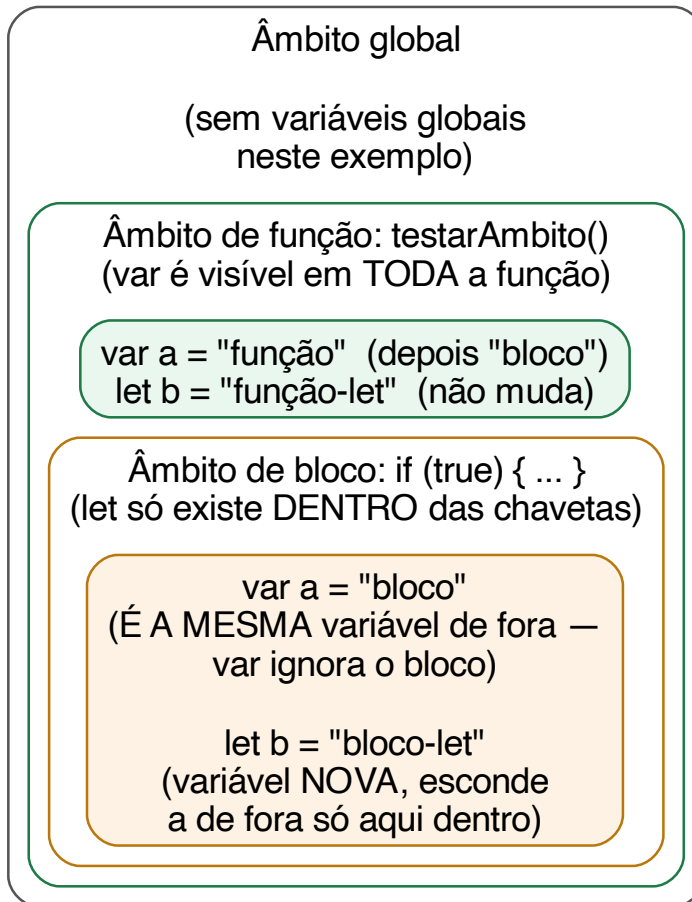
Trace linha a linha:

1. Entra em `testarAmbito()`. Cria-se o âmbito de função. `var a` é hoisted para o topo da função (ver secção seguinte) — existe desde já, mas sem valor.
2. `var a = "função"` → `a` (âmbito de função) passa a `"função"`.
3. `let b = "função-let"` → `b` (âmbito de função, porque está fora de qualquer bloco `{ }` interior) passa a `"função-let"`.
4. Entra no bloco `if (true) { ... }`. Cria-se um **novo âmbito de bloco**.
5. `var a = "bloco"` → como `var` **ignora** este bloco, isto reatribui a **mesma** variável `a` de fora (não cria uma nova) → `a` passa a `"bloco"` **globalmente dentro da função**, não só dentro do `if`.
6. `let b = "bloco-let"` → como `let` respeita o bloco, isto cria uma variável **nova**, `b`, que só existe dentro deste `if` e **esconde** (faz *shadow* de) o `b` de fora enquanto durar o bloco.
7. `console.log("dentro do bloco:", a, b)` → imprime `a="bloco"` (a única `a` que existe) e `b="bloco-let"` (a `b` de dentro do bloco, que está a esconder a de fora).
8. Sai do bloco `if`. A `b` de bloco deixa de existir; volta a estar visível a `b` de função, que **nunca foi tocada**.
9. `console.log("fora do bloco:", a, b)` → imprime `a="bloco"` (foi mesmo alterada pelo `if`, porque era a mesma variável) e `b="função-let"` (ficou intacta, porque o `let` de dentro do bloco era outra variável).

Resultado:

```
dentro do bloco: bloco bloco-let  
fora do bloco: bloco função-let
```

Âmbito aninhado: global -> função -> bloco



Frequente em exame

Este é o argumento central para **evitar var**: o facto de **var** “escapar” de dentro de um `if/for/while` para o resto da função é uma fonte clássica de bugs (variáveis de ciclo que “vazam” para fora do ciclo, por exemplo). `let/const` evitam este problema por construção.

Elevação / Hoisting

Declarações de variáveis (`var`) e de funções (`function nome(...) { ... }`) são **elevadas** (*hoisted*) automaticamente para o início do âmbito onde estão — **como se tivessem sido escritas ali**, embora a sua **inicialização/definição não mude de posição**. Isto permite usar uma função ou variável **antes** da linha onde textualmente aparece declarada — mas a declaração em si não pode ser omitida (dá erro de execução se a variável/função nunca chegar a ser declarada nalgum lado).

Exemplo resolvido

```
document.writeln(x);
document.writeln(getX());
x = 1;
document.writeln(x);

function getX() {
    var x = 2;
    return x;
}
```

```
var x;
```

O que é “elevado” antes de correr qualquer código:

- `var x;` (a declaração, sem o valor) → sobe para o topo do âmbito global.
- `function getX() { ... }` → sobe **por inteiro** (declaração e corpo) para o topo, porque declarações de função são sempre elevadas completamente, ao contrário de `var`.

Portanto o motor “vê” efetivamente este código, antes de o executar:

```
var x;                                // hoisted, ainda undefined
function getX() { var x = 2; return x; } // hoisted, completa

document.writeln(x);
document.writeln(getX());
x = 1;
document.writeln(x);
```

Trace linha a linha:

1. `var x` já existe (elevada), mas ainda não foi atribuída → `x === undefined`.
2. `document.writeln(x)` → escreve `undefined`.
3. `document.writeln(getX())` → invoca `getX()`. **Dentro** de `getX()` há um `var x = 2` **local** a essa função — este `x` local não tem nada a ver com o `x` global (é um âmbito de função diferente, ver secção anterior) → `getX()` faz `return x` com o `x` local, que vale 2 → escreve 2.
4. `x = 1` → agora sim, atribui-se 1 à variável `x` **global** (a elevada no passo 1). O `x` dentro de `getX()` nunca foi tocado por esta linha, porque é outra variável.
5. `document.writeln(x)` → escreve 1.

Resultado: undefined, 2, 1.

Atenção

A elevação (hoisting) só eleva a **declaração**, nunca a **inicialização/valor**. É por isso que a primeira leitura de `x` dá `undefined` e não erro — a variável já “existe” (foi declarada), mas ainda não foi atribuída. Se a variável não estivesse declarada em lado nenhum do âmbito (nem sequer com `var`), a mesma leitura daria `ReferenceError`, não `undefined`. `let/const` também são hoisted tecnicamente, mas ficam numa “zona morta temporal” (*temporal dead zone*) até à linha da sua declaração — usá-las antes dessa linha dá erro, ao contrário de `var`. Este detalhe de `let/const` não constava explicitamente nos slides, mas é essencial para não confundir com o comportamento de `var` ilustrado acima.

Funções anónimas

Funções também podem ser criadas como **expressões**, em vez de declaradas com nome — chamam-se **funções anónimas**, porque não têm nome associado. Podem ser atribuídas a uma variável (que passa a poder ser invocada como se fosse a própria função) ou invocadas **diretamente**, logo que são criadas, passando-lhes argumentos entre parêntesis a seguir à definição — um padrão conhecido como IIFE (*Immediately Invoked Function Expression*). São também uma forma de criar um âmbito próprio para variáveis (útil, por exemplo,

para isolar variáveis temporárias sem poluir o âmbito à volta). Podem ainda usar a sintaxe mais compacta da seta (*arrow function*), `=>`.

Exemplo resolvido

```
let somar = function(a, b) { return a + b; };

document.writeln(somar(2, 3));
document.writeln((function(a, b) { return a + b; })(2, 3));
document.writeln(((a, b) => a + b)(2, 3));
```

Trace:

- `somar(2,3)` → invoca a função anónima guardada em `somar`, com `a=2`, `b=3` → `2+3=5`.
- `(function(a,b){return a+b;})(2,3)` → cria a função anónima e invoca-a **imediatamente** (IIFE) com `a=2`, `b=3` → 5.
- `((a,b) => a+b)(2,3)` → o mesmo, mas com sintaxe de seta: `(a,b) => a+b` é uma função anónima que devolve `a+b` implicitamente (sem `return` nem chavetas, porque o corpo é uma única expressão), também invocada de imediato → 5.

Todas as três linhas escrevem 5.

Atenção

Nota adicional (não estava explícito nos slides): repare no parêntesis a **envolver toda a função** antes de a invocar — `(function(...){...})(...)` e `((a,b)=>a+b)(...)`. Sem esse parêntesis exterior, o motor de JavaScript tenta interpretar a palavra-chave `function` (ou os parêntesis da seta) como o **início de uma declaração**, o que causa um erro de sintaxe quando encontra a chamada logo a seguir. Envolver a função inteira entre parêntesis torna-a uma **expressão**, que pode então ser chamada como qualquer outra expressão que produza uma função.

Modo estrito ("use strict")

O JavaScript é, por omissão, uma linguagem sintaticamente **relaxada**. É possível torná-lo mais estrito com a diretiva `"use strict"` (uma string literal — em versões antigas do motor que não reconhecem a diretiva, é simplesmente ignorada como uma string sem efeito, daí ser escrita entre aspas). A diretiva pode aplicar-se em dois modos:

Definição

- **Global** — colocada no início do ficheiro/script, aplica-se a todo o código a seguir.
- **Local** — colocada no início do corpo de uma função, aplica-se só a essa função.

Em modo estrito deixa de ser permitido, entre outras coisas:

- Usar variáveis e funções **sem as declarar** (ou seja, elimina a criação implícita de globais vista na secção de variáveis acima — em modo estrito, `x = 1` sem declaração prévia dá erro).
- **Remove** (`delete`) variáveis e declarações de função.
- A construção `with` (que permite tratar as propriedades de um objeto como se fossem variáveis do âmbito — fonte de ambiguidade, por isso banida em modo estrito).
- Usar como identificadores certas palavras reservadas para futuras versões da linguagem.

Objetos e arrays

Criar objetos, `undefined` e `null`

Objetos são tipicamente alocados com o construtor `new`. Existem muitas classes predefinidas de objetos — `String` (cadeias de caracteres), `Date` (datas e tempo), `RegExp` (expressões regulares), `Object` (objeto genérico) — cada uma com os seus próprios **métodos** (ex: `hoje.getFullYear()`) e **propriedades** (ex: `'ola mundo'.length`).

Atenção

As aspas/plicas em torno de uma cadeia de caracteres ('ola mundo') são **açúcar sintático** para `new String('ola mundo')` — é por isso que se pode chamar `.length` diretamente sobre um literal de string.

Há duas formas distintas de “ausência de valor”, que é fácil confundir:

Definição

- **undefined** — uma variável ou propriedade que existe mas **nunca foi atribuída**, ou uma propriedade que simplesmente **não existe** no objeto.
- **null** — um objeto **explicitamente** marcado como não instanciado ou inexistente (é o programador que atribui `null`, ao contrário de `undefined`, que aparece “sozinho”).

Atenção

Nota adicional (não estava explícito nos slides, e o exemplo original dos slides tinha um erro): comparar um objeto real (ex: uma instância de `Date`) com `null` usando `==` ou `===` dá sempre `false` — `null` só é “igual” (com `==`) a `undefined`, a mais nada. A regra completa de `==` envolvendo `null/undefined` é:

```
const hoje   = new Date();
let  amanhã = null;

console.log(hoje.meteorologia); // undefined (propriedade não existe)
console.log(amanha == undefined); // true - null e undefined são "iguais" com ==
console.log(amanha === undefined); // false - tipos diferentes (null vs undefined)
console.log(hoje == amanhã); // false - hoje é um objeto Date, não null nem undefined
console.log(hoje === amanhã); // false - idem, e ainda por cima tipos diferentes
```

Ou seja: `null == undefined` é o **único** par de valores diferentes que `==` considera iguais sem qualquer conversão de tipo numérica ou de string envolvida — é uma regra especial da linguagem, não uma coincidência de conversão de tipos.

Propriedades, arrays e a equivalência entre ambos

Definição

- Propriedades de objetos são criadas **dinamicamente** no momento em que são inicializadas — **não** precisam de ser declaradas antecipadamente numa classe ou no próprio objeto.
- Existem apenas no objeto onde foram criadas (dois objetos da mesma classe podem ter propriedades completamente diferentes).
- Ler uma propriedade que não existe não dá erro — devolve `undefined`.
- O operador `in` verifica se uma propriedade existe de facto no objeto (distingue “existe e vale `undefined`” de “não existe”, ao contrário de simplesmente ler a propriedade e comparar com `undefined`).

Exemplo resolvido

```
const obj1 = new Object();
const obj2 = new Object();

obj1.nome = 'Objeto um';

document.writeln(obj1.nome);      // 'Objeto um'
document.writeln(obj2.nome);      // undefined (obj2 nunca teve esta propriedade)
document.writeln('nome' in obj1);  // true
document.writeln('nome' in obj2);  // false
```

Arrays são, eles próprios, instâncias (objetos) da classe **Array**. Na construção, a **dimensão** é passada como parâmetro (ou omitida). São acedidos com índices entre parêntesis retos []. É possível criar/referir índices **para lá** da dimensão inicialmente definida — o array cresce automaticamente. Índices nunca inicializados devolvem **undefined** ao serem lidos.

Exemplo resolvido

```
const array = new Array(2);

array[0] = 'Início';
array[1] = 123;
array[3] = true;           // salta o índice 2 -> array cresce até lá
```

```
for (let i = 0; i < 5; i++)
  document.writeln(i + ':' + array[i]);
```

Trace: array[0]='Início', array[1]=123, o índice 2 nunca é atribuído, array[3]=true. No ciclo (i de 0 a 4):

0:Início

1:123

2:undefined

3:true

4:undefined

array[2] nunca foi definido (ficou “vazio”, devolve **undefined**), e array[4] está para além de tudo o que foi atribuído (também **undefined**).

O facto de os arrays serem apenas uma classe de objetos explica porque os índices e as propriedades são, no fundo, o **mesmo mecanismo**, só com sintaxe diferente:

Exemplo resolvido

```
const obj = new Object();
const prop = 'nome';

obj.nome = 'Um objeto';

document.writeln(obj.nome);    // 'Um objeto'
document.writeln(obj['nome']); // 'Um objeto'
document.writeln(obj[prop]);   // 'Um objeto'
```

- **Notação de propriedade** — *objeto.chave*: a chave tem de ser um **literal** (um nome fixo, escrito diretamente).
- **Notação de índice** — *array[chave]*: a chave é uma **expressão** de qualquer tipo (pode vir de uma variável, como `prop` acima, ou de uma expressão calculada) — e por isso é mais flexível.
- Propriedades seguem as regras de nomes de variáveis (não podem ter espaços, começar por dígito, etc.); índices podem ser **qualquer string** (ou até outro tipo, convertido para string).

As três linhas do exemplo produzem exatamente o mesmo resultado — são só formas sintáticas diferentes de aceder à **mesma** propriedade `nome`.

Propriedades e índices podem ser **removidos** com a instrução `delete`, cujo argumento é uma expressão que referencia a propriedade a remover (pode usar tanto a notação de ponto como a de índice):

Exemplo resolvido

```
const obj = new Object();

obj.nome = 'Um objeto';
obj['estado'] = 'ativo';

delete obj['nome'];
delete obj.estado;

document.writeln(obj.nome);    // undefined
document.writeln(obj.estado);  // undefined
```

Depois do `delete`, a propriedade passa a estar `undefined` — mas agora por **não existir mesmo** (`'nome' in obj` passaria a `false`), e não só por nunca ter sido inicializada.

Atenção

Nota adicional (não estava explícito nos slides): usar `delete` num **índice de array** não faz o array “encolher” nem reindexa os elementos seguintes — apenas deixa esse índice como um “buraco” (o `.length` do array mantém-se igual, e ler esse índice dá `undefined`, tal como um índice nunca atribuído). Para remover mesmo um elemento de um array e fechar o buraco (deslocando os seguintes), usa-se antes o método `array.splice(indice, 1)`.

Igualdade: `==` vs `===`

Já se viu que `==` (igualdade **fraca**) converte tipos antes de comparar, e `===` (igualdade **estrita**) nunca converte — compara sempre tipo e valor ao mesmo tempo. Os casos em que realmente **diferem** merecem destaque, porque são fonte comum de bugs:

Exemplo resolvido

```

console.log(1 == "1");           // true   -> "1" convertida para número 1
console.log(1 === "1");         // false  -> tipos diferentes (Number vs String)

console.log(null == undefined); // true   -> regra especial da linguagem
console.log(null === undefined); // false -> tipos diferentes (Object vs Undefined)

console.log(0 == "");           // true   -> "" convertida para número 0
console.log(0 === "");          // false  -> tipos diferentes

console.log(NaN == NaN);        // false  -> NaN nunca é igual a nada, nem a si próprio
console.log(NaN === NaN);       // false  -> idem

```

Regra prática: `===` só dá `true` quando os dois valores já têm o **mesmo tipo** à partida e o mesmo valor. `==` tenta primeiro **converter** um dos lados para o tipo do outro (seguindo regras específicas da linguagem) e só depois compara — por isso pode surpreender.

Frequente em exame

`NaN == NaN` (e `NaN === NaN`) serem **ambos false** é um dos casos mais citados em exame: `NaN` (*Not a Number*, resultado de operações numéricas inválidas como `0/0`) é o único valor em JavaScript que nunca é igual a si próprio, por definição da norma IEEE 754 de vírgula flutuante que o JavaScript segue para números. Para testar se um valor é `NaN`, usa-se a função `Number.isNaN(x)`, nunca `x == NaN`.

Iteração sobre objetos e arrays

Os ciclos “estendidos” permitem iterar sobre o conteúdo de objetos e arrays sem gerir índices manualmente. Há duas variantes, com comportamentos **diferentes**:

Definição

- **for (variável in objeto)** — itera sobre as **chaves** (nomes das propriedades) do objeto. A variável deve ser declarada (`let`), mas também pode ser apenas referida, sem `let/var`, se já tiver sido declarada antes.
- **for (variável of iterável)** — itera sobre os **valores** de uma coleção iterável (arrays, Maps, strings, etc.).

Exemplo resolvido

```

const pessoa = new Object();
pessoa.nome = 'Ninguém';
pessoa.idade = 99;
pessoa.ativo = true;

for (let campo in pessoa)
    document.writeln(campo + ' = ' + pessoa[campo]);

```

Trace: `campo` percorre sucessivamente `'nome'`, `'idade'`, `'ativo'` (as chaves do objeto). Em cada iteração, `pessoa[campo]` lê o valor associado a essa chave (notação de índice, ver secção anterior).
Resultado:
nome = Ninguém
idade = 99
ativo = true

Em arrays, tanto `in` como `of` funcionam, mas com significados diferentes — `in` dá as **chaves** (que num array

são inteiros, os índices), **of** dá os **valores**:

Exemplo resolvido

```
const numeros = [null, 'primeiro', 'segundo', 'terceiro'];

for (let indice in numeros)
  document.writeln(indice);
// 0
// 1
// 2
// 3

for (let valor of numeros)
  document.writeln(valor);
// null
// primeiro
// segundo
// terceiro
```

Atenção

Nota adicional (não estava explícito nos slides): além de **for-in/for-of**, os arrays têm vários **métodos de iteração** próprios, muito usados na prática (mais do que os ciclos estendidos, em código moderno):

- **array.forEach(funcao)** — invoca **funcao(valor, indice, array)** para cada elemento, sem produzir um novo array (só executa efeitos, como escrever ou acumular externamente).
- **array.map(funcao)** — invoca **funcao(valor, indice, array)** para cada elemento e constrói um **novo array** com os valores devolvidos (não altera o array original).
- **array.filter(funcao)** — constrói um **novo array** só com os elementos para os quais **funcao(valor)** devolve *truthy*.
- **array.reduce(funcao, valorInicial)** — “acumula” o array num único valor, aplicando **funcao(acumulado, valor)** sucessivamente.

```
const numeros = [1, 2, 3, 4];

numeros.forEach(n => document.writeln(n * 10)); // escreve 10,20,30,40 (nada devolvido)
const dobros = numeros.map(n => n * 2);           // dobros = [2,4,6,8] (array novo)
const pares = numeros.filter(n => n % 2 == 0);    // pares = [2,4] (array novo)
const soma = numeros.reduce((acc, n) => acc + n, 0); // soma = 10 (1+2+3+4, a partir de 0)
```

Ao contrário de **forEach**, tanto **map** como **filter** e **reduce** **não alteram** o array original — devolvem sempre um valor novo (array ou, no caso do **reduce**, um valor qualquer).

Map: mapas associativos

Um objeto tem muitas chaves possíveis, mas nem todas devem necessariamente ser iteradas como “dados” (por exemplo, métodos herdados de classes predefinidas também aparecem como chaves nalguns contextos). A classe **Map** cria arrays associativos mais controláveis e previsíveis:

Exemplo resolvido

```
const pessoa = new Map();

pessoa.set('nome', 'Ninguém');
pessoa.set('idade', 99);
pessoa.set('ativo', true);

for (let dado of pessoa)
  document.writeln(dado[0] + " = " + dado[1] + "\n" + pessoa.get(dado[0]));
```

Trace: iterar um Map com for-of dá, em cada iteração, uma mini-lista [chave, valor] — dado[0] é a **chave**, dado[1] é o **valor**. `pessoa.get(chave)` é a forma direta de consultar um valor pela chave (equivalente ao `.set()` usado para escrever). Resultado:

```
nome = Ninguém
Ninguém
idade = 99
99
ativo = true
true
```

Atenção

O ciclo **for-in** **não funciona** em Maps (só em objetos e, para as chaves, em arrays) — para iterar um Map, usa-se sempre **for-of**.

A diferença entre Map e objeto simples resume-se a três pontos:

Definição — Map vs objeto

Chaves:

- Num Map, as chaves podem ser **qualquer valor** (incluindo objetos, funções, etc.).
- Num objeto, as chaves só podem ser tipos simples (essencialmente strings, ou símbolos).

Ordem de inserção:

- Num Map, a ordem de inserção é sempre **preservada** ao iterar.
- Num objeto, a ordem “nominal” costuma ser descrita como lexicográfica das chaves.

Tipo:

- Um Map é sempre um objeto (é uma classe como outra qualquer).
- Um objeto em geral **não** é um Map (não tem `.set()`/`.get()`).

Atenção

Nota adicional (não estava explícito nos slides, precisão sobre a “ordem lexicográfica” referida acima): na prática, os motores modernos de JavaScript preservam a **ordem de inserção** também para chaves que são strings normais num objeto — a única exceção é que chaves que **parecem números inteiros** (ex: “0”, “1”, “42”) são sempre colocadas **primeiro**, por ordem numérica crescente, antes de todas as chaves de string “normais” (que aí sim seguem a ordem de inserção). Por isso, na prática, um Map continua a ser a escolha mais previsível quando a ordem importa de facto.

Notação de objetos, construção e desconstrução

A notação de objetos é a forma direta de escrever estruturas de dados compostas, com objetos e arrays à mistura:

Exemplo resolvido

```
const pessoa = {
  nome: 'Fulano de tal',
  idade: 99,
  morada: {
    endereco: 'Rua Nova',
    codPostal: '4000-001 PORTO'
  },
  hobbies: ['sky diving', 'graffiti']
};

document.writeln(pessoa.nome);           // Fulano de tal
document.writeln(pessoa.idade);          // 99
document.writeln(pessoa.morada.endereco); // Rua Nova
document.writeln(pessoa.morada.codPostal); // 4000-001 PORTO
document.writeln(pessoa.hobbies[0]);      // sky diving
document.writeln(pessoa.hobbies[1]);      // graffiti
```

Notação geral: objetos usam { nome: valor, ..., nome: valor }; arrays usam [valor, ..., valor]. Um valor **estruturado** (dentro de um objeto ou array) tem de seguir o mesmo formato — é por isso que morada é escrito com { } e hobbies com [], dentro do mesmo literal. O acesso a valores aninhados encadeia propriedades e índices à vontade (pessoa.morada.endereco, pessoa.hobbies[0]).

Quando as variáveis usadas para inicializar um objeto têm o **mesmo nome** da propriedade que vão preencher, existe uma notação abreviada:

Exemplo resolvido

```
const nome      = 'Fulano de tal';
const idade     = 99;
const morada    = {};

const original = { nome: nome, idade: idade, morada: morada };
const copia    = { nome, idade, morada }; // forma abreviada

for (let campo in original)
  if (original[campo] !== copia[campo])
    document.writeln('diferentes!');
```

Explicação: { nome, idade, morada } é **exatamente equivalente** a { nome: nome, idade: idade, morada: morada } — quando só se escreve um nome dentro do literal, esse nome serve simultaneamente de **nome da propriedade** e de **nome da variável** cujo valor a preenche. O ciclo **for-in** final não escreve nada, porque **original** e **copia** têm propriedades com os mesmos valores em cada campo (nada de “diferentes!” é escrito).

A **desconstrução** (*destructuring*) é o mecanismo inverso: em vez de construir uma estrutura a partir de variáveis, extrai variáveis a partir de uma estrutura já existente, colocando um “molde” do lado esquerdo de uma atribuição:

Atenção

Nota adicional (não estava explícito nos slides — e o exemplo original tinha dois problemas): um erro tipográfico no nome da variável (**hobbies** vs **hobies**), e uma tentativa de trocar duas variáveis repetindo **let** na segunda desconstrução — o que na realidade dá **SyntaxError: Identifier 'segundo' has already been declared**, porque **let** não permite redeclarar a mesma variável no mesmo âmbito. Para trocar valores entre variáveis já existentes com desconstrução, a segunda atribuição tem de ser feita **sem** **let/const** (são apenas atribuições a variáveis que já existem, não novas declarações). Versão corrigida e tracejada abaixo.

Exemplo resolvido

```
let hobbies = ['sky diving', 'graffiti'];
let [primeiro, segundo] = hobbies;

console.log(primeiro, segundo);           // sky diving graffiti

[segundo, primeiro] = [primeiro, segundo]; // troca, sem 'let'

console.log(primeiro, segundo);           // graffiti sky diving
```

Trace:

1. `let [primeiro, segundo] = hobbies` — desconstrução de **array**: as variáveis são emparelhadas com os valores **pela posição**: `primeiro = hobbies[0] = 'sky diving'`, `segundo = hobbies[1] = 'graffiti'`.
2. `[segundo, primeiro] = [primeiro, segundo]` — cria-se primeiro o array do lado direito com os valores **atuais**: `['sky diving', 'graffiti']` (porque `primeiro='sky diving'` e `segundo='graffiti'` nesse instante). Depois desconstrói-se esse array nas variáveis do lado esquerdo, na ordem em que aparecem: `segundo = 'sky diving'`, `primeiro = 'graffiti'`. Resultado: os valores das duas variáveis ficaram **trocados**.

Exemplo resolvido

```
const pessoa = {
  nome: 'Fulano de tal',
  idade: 99,
  morada: { endereco: 'Rua Nova', codPostal: '4000-001 Porto' }
};

let { nome, idade, morada } = pessoa;
let { nome: name, morada: { endereco: street, codPostal: zip } } = pessoa;

console.log(nome, idade, morada); // Fulano de tal 99 { endereco: ..., codPostal: ... }
console.log(name, street, zip);    // Fulano de tal Rua Nova 4000-001 Porto
```

Explicação: na desconstrução de **objetos**, o emparelhamento é feito **por nome**, não por posição — `{ nome, idade, morada }` extrai as propriedades com esses nomes exatos para variáveis com o mesmo nome. É possível **renomear** durante a extração com **propriedade: novoNome** (ex: `nome: name` cria uma variável `name` com o valor de `pessoa.nome`), e também aninhar o molde para entrar em estruturas profundas (`morada: { endereco: street, codPostal: zip }` desce um nível dentro de `morada` na mesma desconstrução). Tal como nos arrays, propriedades podem ser omitidas do molde se não forem necessárias.

Programação orientada a objetos

O JavaScript suporta o paradigma de **orientação a objetos** (OOP) desde sempre através de objetos com propriedades e métodos, mas o **ECMAScript 2015 (ES6)** acrescentou uma sintaxe dedicada baseada em **class**, mais próxima da de outras linguagens (Java, etc.), que permite:

Definição

- **Estruturação** — organizar componentes numa hierarquia.
- **Reutilização** — reutilizar componentes de forma controlada (herança).
- **Encapsulação** — controlar o acesso a componentes internos.
- Definir **classes** de objetos, **instanciar** objetos a partir delas, associar **campos** e **métodos** a instâncias, e **estender** classes.

Atenção

Apesar da sintaxe, **class** em JavaScript é **apenas açúcar sintático** sobre o mecanismo de objetos e protótipos que já existia antes do ES6 — não é um mecanismo novo por baixo. Consequências importantes: continua a ser **opcional** organizar o código em classes (o exemplo abaixo com notação de objetos, sem **class** nenhuma, é igualmente válido); **não existem mecanismos reais de encapsulação** (não há **private** “a sério” na sintaxe de classes base do JavaScript — tudo o que está em **this** é acessível de fora); e não é possível reutilizar **declarações** (como interfaces ou assinaturas), só **definições** completas.

Objetos sem classe, class, construtor e this

Um objeto pode ser construído diretamente com a notação de objetos (sem nenhuma classe), incluindo propriedades cujo valor é uma **função** (métodos “ad-hoc”). É útil para criar objetos únicos (*singletons*), sem necessidade de instanciar mais do que um:

Atenção

Nota adicional (não estava explícito nos slides): o exemplo original tinha vários erros de sintaxe (ponto e vírgula a fechar uma propriedade em vez de vírgula, vírgula em falta antes do método). Versão corrigida:

```
let joao = {
  nome: "João",
  nasce: new Date(1999, 3 - 1, 15), // mês é 0-indexado em Date: 3-1 = Março
  hobbies: ["programação", "mergulho"],

  idade: function() {
    var agora = new Date();
    var msAno = 1000 * 60 * 60 * 24 * 365.25;
    return Math.floor((agora.getTime() - this.nasce.getTime()) / msAno);
  }
};
```

Note-se **this.nasce** dentro do método **idade** — **this** refere-se ao próprio objeto **joao** no momento em que **joao.idade()** é invocado. Objetos criados desta forma **não têm uma classe própria** — só existe este objeto concreto, sem “molde” reutilizável para criar outros iguais.

Quando se querem **vários** objetos com a mesma estrutura, usa-se a sintaxe de classe:

Definição

- Uma **class** é definida por um bloco iniciado pela palavra-chave **class** e um **nome**.
- O método especial **constructor(...)** é invocado automaticamente para inicializar cada nova instância.
- Dentro dos métodos, **this** refere-se ao **objeto corrente** — a instância concreta sobre a qual o método foi invocado.
- Os **campos** (propriedades) de um objeto são simplesmente propriedades atribuídas a **this**, tipicamente dentro do **constructor** — **não existe uma declaração explícita e separada de campos** da classe, ao contrário de Java.
- Instâncias são criadas com **new NomeDaClasse(argumentos)**, os argumentos são passados ao **constructor**.

Exemplo resolvido

```
class Pessoa {  
  constructor(nome, ano, mes, dia) {  
    this.nome = nome;  
    this.nasce = new Date(ano, mes - 1, dia);  
  }  
}
```

```
let joao = new Pessoa("João", 1999, 3, 15);  
let maria = new Pessoa("Maria", 2000, 5, 21);
```

Trace: `new Pessoa("João",1999,3,15)` cria um objeto novo, invoca **constructor** com `nome="João"`, `ano=1999`, `mes=3`, `dia=15`, e dentro dele **this** refere-se a esse objeto novo — `this.nome` fica "João", `this.nasce` fica a data 15 de março de 1999 (note que `mes-1=2`, porque o construtor de `Date` numera os meses a partir de 0 — Janeiro é 0, Março é 2). O mesmo processo repete-se, independentemente, para `maria`.

Métodos de instância

Métodos são funções definidas dentro da classe, que se referem a campos e a outros métodos do objeto corrente através de **this**, e podem devolver um valor com **return**, tal como qualquer função:

Exemplo resolvido

```
class Pessoa {
  constructor(nome, ano, mes, dia) { /* ... */ }

  saudacao(outro) {
    return 'Olá ' + this.nome + ', eu sou o ' + outro.nome;
  }

  saudar(outro) {
    console.log(this.saudacao(outro));
  }
}
// ...
joao.saudar(maria); // escreve na consola: "Olá Maria, eu sou o João"
```

Trace: joao.saudar(maria) invoca saudar com this=joao e outro=maria. Dentro de saudar, this.saudacao(outro) invoca saudacao **também com this=joao** (invocar um método a partir de this mantém o mesmo this) e outro=maria — devolve 'Olá ' + joao.nome + ', eu sou o ' + maria.nome = "Olá Maria, eu sou o João". saudar passa esse valor a console.log.

Chamar um método de um objeto sobre outro objeto (outro.nome, para ler um dado de outra instância) é, conceptualmente, uma forma de **troca de mensagens** entre objetos — a ideia central da orientação a objetos.

Acessores (get/set)

Os **acessores** (*getters* e *setters*) são métodos especiais que medeiam o acesso a uma propriedade, mas são usados como se fossem a **própria propriedade** (sem parêntesis de invocação):

Exemplo resolvido

```
const MS_ANO = 1000 * 60 * 60 * 24 * 365.25;

class Pessoa { // ...
  get idade() {
    let agora = new Date();
    return Math.floor((agora.getTime() - this.nasce.getTime()) / MS_ANO);
  }

  set idade(anos) {
    let agora = new Date();
    this.nasce = new Date(agora.getTime() - anos * MS_ANO);
  }
}

joao.idade = 99; // invoca o SET
document.writeln("0 " + joao.nome + " tem " + joao.idade); // invoca o GET
```

Trace: joao.idade = 99 **não** cria uma propriedade literal idade — invoca automaticamente o método set idade(anos) com anos=99, que recalcula this.nasce (a data de nascimento) de forma a corresponder a alguém com 99 anos **hoje**. Na linha seguinte, ler joao.idade invoca o método get idade(), que calcula a idade atual a partir de this.nasce — dando de volta (aproximadamente) 99.

Atenção

Um acessor **não deve modificar a própria propriedade que representa** dentro de si mesmo — por exemplo, o `get idade()` não deve fazer `this.idade = ...`, porque isso invocaria de novo o `set idade`, potencialmente entrando num ciclo infinito de invocações mútuas entre `get` e `set`. Os acessores devem, em vez disso, manipular **campos internos diferentes** (aqui, `this.nasce`) para armazenar o estado real.

Membros estáticos

Um método **estático** (`static`) é avaliado no contexto da **classe** em si, não de uma instância — por isso **não pode** usar `this` para se referir a um objeto concreto (não há nenhum “objeto corrente” quando se invoca um método estático). É invocado usando o **nome da classe** como recetor, nunca uma instância:

Exemplo resolvido

```
class Pessoa {
  constructor(nome, ano, mes, dia) { /* ... */ }

  static duracao(inicio, fim) {
    return (fim.getTime() - inicio.getTime()) / MS_ANO;
  }

  idade() {
    return Pessoa.duracao(new Date(), this.nasce);
  }
}
```

Trace: `joao.idade()` invoca o método de **instância** `idade`, com `this=joao`. Dentro dele, `Pessoa.duracao(...)` invoca o método **estático** `duracao`, usando o **nome da classe** `Pessoa` (não `this`) como recetor — dentro de `duracao`, não existe `this` associado a `joao` nem a nenhuma instância, só os parâmetros `inicio` e `fim` recebidos.

Herança (`extends`, `super`) — hierarquia com 2 níveis

Uma classe pode **estender** outra com `extends`, criando uma **subclasse** que herda tudo da superclasse e pode redefinir métodos (*sobreposição*, ou *override*) e/ou acrescentar novas definições. A referência `super` dá acesso às definições da **superclasse imediata** (um nível acima na cadeia de `extends`, não necessariamente “o topo de tudo”).

Para perceber bem como `this` e `super` se comportam quando há **mais do que um nível** de `extends`, vale a pena seguir um exemplo com três classes encadeadas — `Pessoa` → `Aluno` → `Finalista`:

Exemplo resolvido

```
class Pessoa {
  constructor(nome) {
    this.nome = nome;
  }
  saudacao() {
    return `Olá, sou ${this.nome}`;
  }
}

class Aluno extends Pessoa {
  constructor(nome, curso) {
    super(nome); // invoca o constructor de Pessoa
    this.curso = curso;
  }
  saudacao() {
    return `${super.saudacao()} e estudo ${this.curso}`;
  }
}

class Finalista extends Aluno {
  constructor(nome, curso, anoFinal) {
    super(nome, curso); // invoca o constructor de Aluno
    this.anoFinal = anoFinal;
  }
  saudacao() {
    return `${super.saudacao()} (ano final: ${this.anoFinal})`;
  }
}
```

Trace da construção — let f = new Finalista("Marco", "Eng. Informática", 2026):

1. new Finalista(...) cria um objeto novo e invoca o constructor de Finalista com this já apontado a esse objeto (chamemos-lhe f).
2. Primeira instrução: super(nome, curso) — invoca o constructor de Aluno (a superclasse **imediate** de Finalista), **com o mesmo this** (continua a ser f, não se cria outro objeto).
3. Dentro do constructor de Aluno: super(nome) invoca o constructor de Pessoa (superclasse imediata de Aluno), também com o mesmo this=f. Dentro de Pessoa, this.nome = nome atribui f.nome = "Marco".
4. Volta ao constructor de Aluno, depois do super(nome): this.curso = curso atribui f.curso = "Eng. Informática".
5. Volta ao constructor de Finalista, depois do super(nome,curso): this.anoFinal = anoFinal atribui f.anoFinal = 2026.
6. No fim, f é um único objeto com **três** campos, todos atribuídos ao mesmo this ao longo da cadeia: { nome: "Marco", curso: "Eng. Informática", anoFinal: 2026 }.

Exemplo resolvido

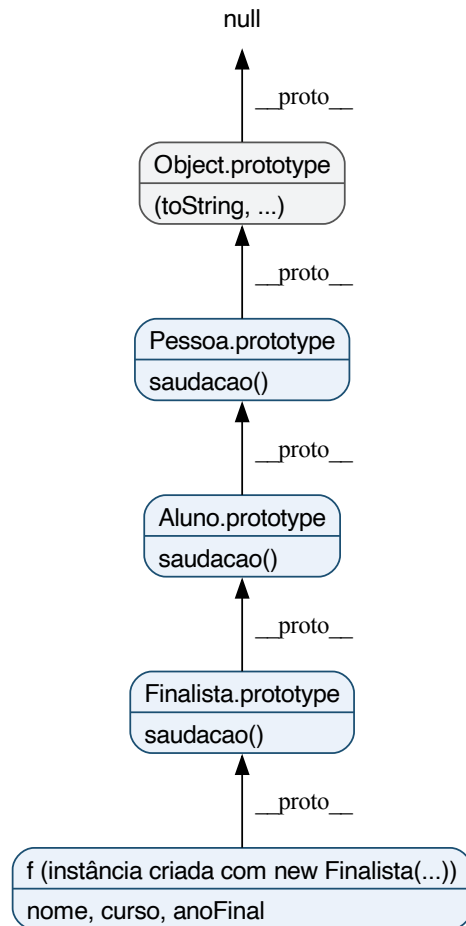
Trace da invocação de método — `f.saudacao()`:

1. A procura do método `saudacao` começa em `Finalista.prototype` (a classe mais específica de `f`) — encontra-o logo aí, porque `Finalista` também define `saudacao()`.
2. Dentro desse `saudacao()` de `Finalista`, `this` continua a ser `f` (invocar `f.saudacao()` liga sempre `this` ao objeto à esquerda do ponto, `f`, independentemente de em que nível da cadeia o método está fisicamente definido). `super.saudacao()` invoca a versão de `saudacao` **um nível acima** — a de `Aluno` — mas **continua com `this=f`**.
3. Dentro do `saudacao()` de `Aluno`: `super.saudacao()` invoca a versão de `saudacao` de `Pessoa` (um nível acima de `Aluno`), sempre com `this=f`. Devolve ``Olá, sou ${f.nome}`` = "Olá, sou Marco".
4. De volta ao `saudacao()` de `Aluno`: junta isso com `` e estudo ${this.curso}`` = " e estudo Eng. Informática" → devolve "Olá, sou Marco e estudo Eng. Informática".
5. De volta ao `saudacao()` de `Finalista`: junta isso com `` (ano final: ${this.anoFinal})`` → resultado final:

Olá, sou Marco e estudo Eng. Informática (ano final: 2026)

Regra a reter: `this` refere-se **sempre** ao objeto concreto que recebeu a chamada original (`f`), **em qualquer nível** da cadeia de `extends` — nunca muda. Já `super` refere-se sempre à versão do método definida **um nível acima na classe onde o `super` está escrito** (não “o topo da hierarquia”) — por isso `super.saudacao()` dentro de `Aluno` vai a `Pessoa`, mas o mesmo `super.saudacao()` dentro de `Finalista` vai a `Aluno`, não a `Pessoa`.

Cadeia de protótipos criada por: `class Finalista extends Aluno extends Pessoa`



Atenção

Nota adicional (não estava explícito nos slides): por baixo do açúcar sintático de `class/extends`, o que liga `Finalista` a `Aluno` e a `Pessoa` é a **cadeia de protótipos** — cada objeto tem uma referência interna (`__proto__`) para o `.prototype` da sua classe, e cada `.prototype` tem por sua vez a sua própria referência `__proto__` para o `.prototype` da superclasse. Quando se invoca `f.saudacao()`, o motor procura `saudacao` primeiro no próprio `f`, depois em `Finalista.prototype`, depois em `Aluno.prototype`, etc., subindo a cadeia até encontrar a primeira definição — é essa procura ao longo da cadeia que dá a impressão de “herança”.

Referir métodos como *callbacks*, e classes anónimas

Uma função pode ser passada como argumento a outra (por exemplo, `setTimeout(funcao, milisegundos)`, que invoca `funcao` de forma diferida ao fim desse tempo). Isto levanta um problema com `this` que **não estava explícito nos slides originais**:

Exemplo resolvido

```
class Pessoa { // ...
  alerta(quando, mensagem) {
    let callback = () => this.alertar(mensagem);
    let millisecs = (quando.getTime() - (new Date()).getTime());

    setTimeout(callback, millisecs);
  }

  alertar(mensagem) {
    alert("Olá " + this.nome + "\nSão horas de " + mensagem);
  }
}
```

`setTimeout` recebe uma **função** a invocar (`callback`) e o número de **milissegundos** a esperar antes de a invocar. Aqui, `callback` é uma **função seta** (`=>`), não uma função normal, e isso não é um detalhe estético.

Atenção

Nota adicional (não estava explícito nos slides): se `callback` fosse uma função normal (`function() { this.alertar(mensagem); }`), o `this` lá dentro **não seria** a instância de `Pessoa` quando `setTimeout` a invocasse mais tarde — seria `undefined` (em modo estrito) ou o objeto global (fora de modo estrito), porque funções normais recebem o seu `this` de **como são invocadas**, e `setTimeout` invoca-a como uma chamada solta, sem nenhum objeto à esquerda de um ponto. As **funções seta**, pelo contrário, **não têm o seu próprio `this`** — usam sempre o `this` do âmbito onde foram **escritas** (aqui, dentro do método `alerta`, onde `this` já era a instância de `Pessoa`), e mantêm essa ligação mesmo quando invocadas mais tarde, por outra função, sem nenhum objeto à esquerda do ponto. É exactamente por isto que o exemplo usa `() => this.alertar(mensagem)` em vez de uma função normal.

Por fim, tal como as funções, também as classes podem ser **anónimas**: definidas como uma **expressão** (sem nome a seguir a `class`), e atribuídas a uma variável ou passadas como argumento:

Exemplo resolvido

```
let Pet = class {
  constructor(nome) {
    this.nome = nome;
  }
};

let bola = new Pet("Bola");
```

Todos os exemplos anteriores de classes (`Pessoa`, `Aluno`, `Finalista`) eram **declarações** de classes (com nome, `class NomeAqui { ... }`). Aqui, a classe é uma expressão sem nome próprio — só existe através da variável `Pet` a que foi atribuída, tal como uma função anónima só existe através da variável que a guarda.

JavaScript — DOM, CSSOM e Eventos

Reflexão e widgets

Definição

Reflexão (no contexto do JavaScript no cliente) é a técnica pela qual a formatação de uma página é representada como **estruturas de objetos** manipuláveis a partir de código. Permite **ler** o estado atual da página e **alterá-lo** programaticamente. Existem vários modelos de reflexão, historicamente sobrepostos:

- **Widgets** (modelo legado, deve ser evitado hoje em dia);
- **CSSOM** (CSS Object Model) — reflexão do estilo aplicado a um elemento;
- **DOM** (Document Object Model) — reflexão de toda a estrutura do documento HTML/XML.

O modelo de **widgets** foi a versão mais antiga de reflexão em JavaScript. Reflete apenas **partes** da formatação — nomeadamente formulários e os seus campos (*widgets*, no sentido de controlos de interface: `input`, `select`, `textarea`). É simples de usar mas pouco versátil, porque só cobre elementos de formulário, e o seu uso deve ser evitado em código novo (foi superado pelo DOM, que cobre qualquer elemento).

Cada campo de formulário (`input`, `select`, `textarea`) é refletido como um objeto com atributos como `name` (o nome pelo qual é referido dentro do formulário) e `value` (o seu valor corrente). A partir de um *callback* (por exemplo associado a um botão), a palavra `this` refere o objeto corrente — no exemplo clássico de um botão “=” que soma dois campos, `this` dentro do `onclick` do botão refere o próprio botão, e `this.form` refere o formulário a que pertence. O formulário, por sua vez, tem uma propriedade com o *nome* de cada um dos seus campos, o que permite aceder-lhes por nome: `this.form.a.value`, `this.form.b.value`, etc.

Exemplo resolvido

Formulário HTML com três campos e um botão que soma os dois primeiros e escreve o resultado no terceiro:

```
<form>
  <input name=a size=2 value=1> +
  <input name=b size=2 value=1>
  <input type=button value="=" onclick="
    this.form.c.value =
      parseInt(this.form.a.value) +
      parseInt(this.form.b.value);">
  <input name=c size=2>
</form>
```

Quando o botão é clicado: `this` é o botão; `this.form` é o formulário (porque o botão está dentro dele); `this.form.a`, `this.form.b` e `this.form.c` são os três campos, acedidos pelo `name`; `this.form.a.value` e `this.form.b.value` são convertidos de *string* para inteiro com `parseInt()`, somados, e o resultado (ainda número) é atribuído a `this.form.c.value` — a atribuição converte implicitamente de volta para *string* ao ser mostrada no campo.

CSSOM

Definição

O **CSSOM** (*CSS Object Model*) é o modelo de reflexão do **estilo** aplicado a um elemento. Todo o elemento tem uma propriedade **style** que é um objeto refletindo as regras CSS que lhe são aplicadas diretamente (estilo *inline*). As propriedades desse objeto **correspondem** às propriedades CSS, e permitem **inspecionar** e **alterar** os seus valores a partir de JavaScript. A sintaxe JavaScript do CSSOM tem correspondência direta com a sintaxe CSS, com uma diferença de nomenclatura para propriedades compostas (ver abaixo).

```
function paint(element) {  
    element.style.background = "orange";  
    element.style.color = "white";  
}
```

Há duas situações a distinguir na correspondência de nomes:

- **Nomes simples** (uma só palavra) — são **iguais** em CSS e em JavaScript. Exemplos: `color`, `background`, `width`, `height`. Assim, `elemento.style.color = "orange"` corresponde exatamente à regra CSS `color: orange`;
- **Nomes com hífen** (várias palavras separadas por -) — em JavaScript são convertidos para **lowerCamelCase**, ou seja, o hífen é removido e a palavra seguinte passa a começar por maiúscula. Exemplos: `font-size` → `fontSize`; `font-color` → `fontColor`; `border-style` → `borderStyle`; `border-width` → `borderWidth`.

Atenção

Nota adicional (não estava explícito nos slides): esta conversão é necessária porque o hífen (-) não é um carácter válido dentro de um identificador em JavaScript — `elemento.style.font-size` seria interpretado como a subtração `elemento.style.font MENOS size`, o que não faz sentido e gera um erro (ou um resultado incorreto, se `font` e `size` existirem como variáveis). A conversão para *camelCase* resolve este problema de sintaxe mantendo o acesso à propriedade como `elemento.style.fontSize`.

Exemplo resolvido

Um `<select>` que altera a cor do texto do formulário onde está inserido, usando um nome simples:

```
<select onchange="this.form.style.color = this.value;">  
    <option></option>  
    <option>orange</option>  
    <option>DarkBlue</option>  
</select>
```

Ao escolher “orange”: `this` é o `<select>`; `this.form` é o formulário que o contém; `this.form.style` é o objeto CSSOM desse formulário; `this.value` é o texto da opção escolhida (“orange”); a atribuição `this.form.style.color = "orange"` altera diretamente o estilo *inline* do formulário, e o browser repinta o texto de imediato.

Um segundo `<select>`, agora com um nome de propriedade composto por duas palavras (`font-size`), ilustra a conversão para *camelCase*:

```
<select onchange="this.form.style.fontSize = this.value;">  
    <option>24pt</option>  
    <option>36pt</option>  
    <option>12pt</option>  
</select>
```

O mecanismo é idêntico, mas a propriedade acedida é `style.fontSize` (equivalente a `font-size` em CSS) em vez de `style.color`.

A árvore DOM

Definição

O **DOM** (*Document Object Model*) reflete o documento HTML **inteiro** como uma estrutura de objetos (uma árvore), com uma API para consultar e manipular essa estrutura. Ao contrário do modelo de widgets, o DOM cobre **qualquer** elemento do documento, não só formulários — e a mesma API aplica-se também a documentos XML (o HTML tem apenas algumas propriedades/métodos adicionais, específicos dele). O DOM define vários **tipos de objeto**:

- **Document** — o documento HTML/XML como um todo;
- **Node** — o tipo mais genérico de nó do documento;
- **NodeList** — uma lista/coleção de nós;
- **Element** — os nós criados por pares de anotações (tags) HTML/XML;
- **Text** — os nós de texto “pendurados” em elementos.

A estrutura do documento é uma **árvore** (hierarquia): cada nó tem **um único** progenitor (exceto a raiz). Cada par de anotações HTML (por exemplo `<p>...</p>`) dá origem a **um nó** do tipo **Element**; cada fragmento de texto dá origem a **um nó** do tipo **Text**; e mesmo espaços em branco entre tags (mudanças de linha, indentação) geram nós de texto próprios (nós de “carateres brancos”), o que é uma fonte comum de confusão ao percorrer `childNodes`.

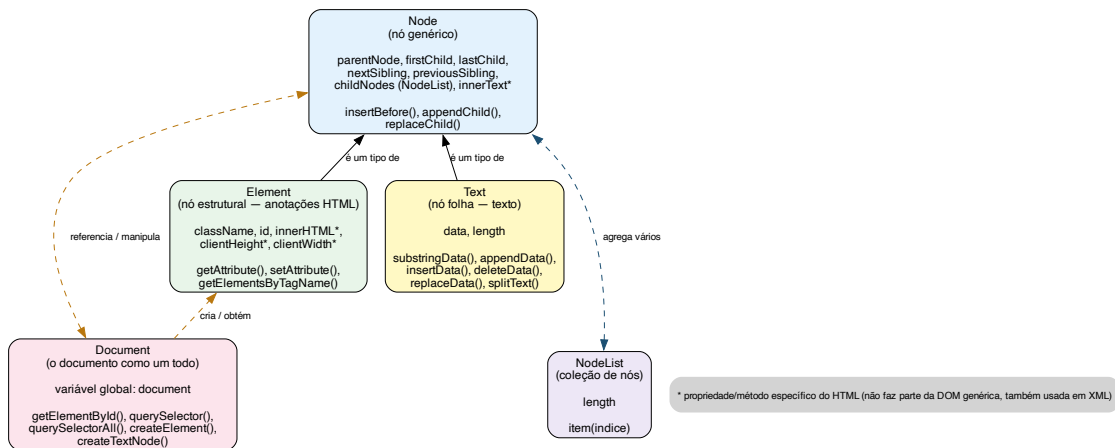
Exemplo resolvido

Para o documento:

```
<html>
  <head>
    <title>Uma página</title>
  </head>
  <body>
    <h1>Uma página</h1>
    <p>Uma linha</p>
    <p>Ainda <b>outra</b> linha</p>
  </body>
</html>
```

a árvore de nós correspondente tem `html` como raiz, com dois filhos (`head` e `body`); `head` tem um filho `title`, que por sua vez tem um nó de texto filho “Uma página”; `body` tem três filhos: `h1` (com o nó de texto “Uma página”), o primeiro `p` (com o nó de texto “Uma linha”), e o segundo `p` (com dois filhos: o nó de texto “Ainda ” e o elemento `b`, que por sua vez tem o nó de texto “outra” — a palavra “linha” a seguir ao `</b>` seria mais um nó de texto irmão de `b`, filho do segundo `p`).

A árvore DOM tem uma **hierarquia de tipos**: **Element** e **Text** são casos particulares (mais especializados) de **Node** — herdam todas as propriedades e métodos genéricos de **Node** e acrescentam as suas próprias. **Document** não é um nó da árvore no mesmo sentido, mas é o objeto que dá acesso a toda a árvore e permite criar/procurar nós. **NodeList** não é um tipo de nó — é um tipo de **coleção** que agrega vários nós (por exemplo, os filhos de um elemento, ou o resultado de uma pesquisa).



O tipo Document

Document representa o documento como um **todo**, e é acessível através da variável global predefinida `document` (disponível em qualquer script que corra no browser). Disponibiliza dois grupos de métodos: métodos para **obter** (procurar) elementos já existentes na árvore, e métodos para **criar** novos nós de diferentes tipos.

Obter elementos. O método mais direto é `document.getElementById(id)`, que devolve o elemento com o `id` dado como argumento (ou `null` se não existir nenhum). A referência devolvida pode depois ser usada para efetuar alterações ao elemento (por exemplo ao seu estilo, via CSSOM).

```
function colorize(color) {
  document.getElementById("base").style.background = color;
}
```

Além da procura por identificador, o DOM disponibiliza **procura por seletores CSS** (mais poderosa, porque aceita qualquer seletor válido de CSS, não só `#id`):

- `document.querySelector(seletor)` — devolve apenas o **primeiro** elemento que corresponde ao seletor;
- `document.querySelectorAll(seletor)` — devolve uma **lista de nós** (tipo `NodeList`) com **todos** os elementos correspondentes.

```
// Primeiro parágrafo do primeiro DIV
const firstParagraph = document.querySelector('div > p');

// Primeiro parágrafo de cada DIV
const allFirstParagraphs = document.querySelectorAll('div > p');
```

Estes dois métodos de procura por seletor estão também disponíveis nos próprios **elementos** (não só em `document`), o que permite uma pesquisa **localizada** — por exemplo `algumDiv.querySelector('p')` procura apenas dentro de `algumDiv`, não no documento inteiro.

Criar nós. `document.createElement(tag)` cria um novo nó **Element** do tipo de tag indicado (por exemplo `"div"`), e `document.createTextNode(texto)` cria um novo nó **Text** com o conteúdo dado. Em ambos os casos, os nós criados existem **isolados** — ainda não estão pendurados em lado nenhum da árvore; é preciso inseri-los explicitamente com os métodos de **Node** descritos a seguir.

```
function create(color) {
  const base = document.getElementById("base");
  const box = document.createElement("div");
```

```
const space = document.createTextNode(" ");

box.setAttribute("style", "background: " + color + "; width: 1em; margin: auto;");
box.setAttribute("onclick", "create('" + color + "')");
box.appendChild(space);
base.appendChild(box);
base.appendChild(document.createTextNode("\n"));
}
```

Atenção

Nota adicional (não estava explícito nos slides): a ordem das operações neste exemplo importa. `document.createElement("div")` e `document.createTextNode(" ")` só criam objetos em memória — não produzem efeito nenhum visível na página até serem ligados à árvore com `appendChild()`. Só depois de `base.appendChild(box)` é que o novo div aparece de facto no documento renderizado.

O tipo Node

Node é o tipo **mais geral** de nó pendurado no documento. Tanto `Element` como `Text` (e outros tipos mais específicos) herdam dele as propriedades e métodos **estruturais** — isto é, os que dizem respeito à posição do nó dentro da árvore, independentemente do tipo concreto do nó.

Propriedades de Node:

- `parentNode` — o nó progenitor;
- `firstChild` — o primeiro filho;
- `lastChild` — o último filho;
- `nextSibling` — o irmão seguinte (mesmo progenitor);
- `previousSibling` — o irmão anterior;
- `childNodes` — uma `NodeList` com todos os filhos;
- `innerText*` — o texto do próprio nó e de todos os seus descendentes, concatenado (específico do HTML, não faz parte da DOM genérica).

Métodos de Node:

- `appendChild(no)` — adiciona `no` como **último filho** do nó onde o método é invocado;
- `insertBefore(no, referencia)` — insere `no` como filho, **antes** do filho `referencia` já existente (se `referencia` for null, o comportamento equivale a inserir no fim);
- `replaceChild(depois, antes)` — substitui o filho **antes** pelo nó **depois**.

Exemplo resolvido

Concatenar com `appendChild()`. Cada clique cria uma nova caixa colorida e junta-a sempre ao fundo:

```
function create(color) {  
  const base = document.getElementById("base");  
  const box = document.createElement("div");  
  const space = document.createTextNode(" ");  
  
  box.style.background = color;  
  box.style.width = "1em";  
  box.style.margin = "auto";  
  box.onclick = function() { create(color); };  
  box.appendChild(space);  
  base.appendChild(box);  
  base.appendChild(document.createTextNode("\n"));  
}
```

Traço da execução ao clicar numa caixa #f90 já existente dentro de #base: (1) `create("#f90")` corre; (2) `base` é obtido por `getElementById`; (3) `box` é um novo `div` vazio, ainda sem pai; (4) `space` é um novo nó de texto " ", também sem pai; (5) o estilo de `box` é configurado via CSSOM (`style.background`, `style.width`, `style.margin`); (6) `box.onclick` é atribuído a uma nova função anónima que, quando executada mais tarde, volta a chamar `create` com a mesma cor — ou seja, a nova caixa também fica clicável e cria outra igual; (7) `box.appendChild(space)` pendura o nó de texto dentro de `box` (agora `box` tem um filho); (8) `base.appendChild(box)` pendura `box` como **último filho** de `base` — é neste passo que a caixa aparece no ecrã, no fundo da lista já existente; (9) `base.appendChild(document.createTextNode("\n"))` acrescenta ainda um nó de texto de mudança de linha a seguir à caixa, para que a próxima caixa apareça visualmente separada.

Exemplo resolvido

Inserir com `insertBefore()`. Em vez de acrescentar sempre ao fundo, esta variante insere a nova caixa a seguir à caixa em que se clicou, usando `this` para saber qual foi:

```
function create(color, current) {
  const base = document.getElementById("base");
  const box = document.createElement("div");
  const space = document.createTextNode(" ");
  const nl = document.createTextNode("\n");
  const next = current.nextSibling.nextSibling;

  box.style.background = color;
  box.style.width = "1em";
  box.style.margin = "auto";
  box.onclick = function() { create(color); };
  box.appendChild(space);
  base.insertBefore(box, next);
  base.insertBefore(nl, next);
}
```

registada como `onclick="create('#f90', this);"` em cada caixa (`this` refere a caixa clicada). Traço da execução ao clicar na caixa `current`: (1) `next = current.nextSibling.nextSibling` — como cada caixa é seguida de um nó de texto `"\n"` (ver exemplo anterior), é preciso avançar **dois** irmãos para chegar ao próximo elemento estrutural (o primeiro `nextSibling` dá o nó de texto, o segundo dá de facto a caixa seguinte, ou `null` se `current` for a última); (2) `box` e `space/nl` são criados como antes; (3) `base.insertBefore(box, next)` insere `box` antes de `next` — ou seja, imediatamente a seguir a `current` (se `next` for `null`, insere no fim, tal como `appendChild`); (4) `base.insertBefore(nl, next)` insere a mudança de linha, também antes de `next`, ficando entre a caixa nova e a caixa que era `next`.

Atenção

Nota adicional (não estava explícito nos slides): `insertBefore(no, null)` tem exatamente o mesmo efeito que `appendChild(no)` — inserir “antes de nada” equivale a inserir no fim da lista de filhos. Isto explica porque é seguro usar a mesma chamada `insertBefore` mesmo quando `current` é o último elemento (nesse caso `next` acaba por ser `null`).

O tipo `NodeList`

`NodeList` agrega **coleções de nós**, tipicamente os descendentes diretos de um elemento, ou o resultado devolvido por funções de pesquisa (por exemplo `getElementsByTagName()` ou `querySelectorAll()`).

- **Propriedade `length`** — número de elementos da lista;
- **Método `item(indice)`** — devolve o nó na posição `indice` (a numeração começa em 0).

Exemplo resolvido

Contador de cliques que percorre todos os `div` dentro de `#base` e atualiza o texto de cada um:

```
let conta = 0;

function conta() {
  const base = document.getElementById("base");
  const lista = base.getElementsByTagName("div");

  conta++;

  for (let i = 0; i < lista.length; i++)
    lista.item(i).innerText = conta;
}
```

Traço de execução ao clicar numa das caixas: (1) `conta()` é invocada (registada como `onclick`); (2) `base` é obtido; (3) `base.getElementsByTagName("div")` devolve uma `NodeList` com todos os `div` descendentes de `base`; (4) a variável global `conta` (partilhada entre chamadas) é incrementada; (5) o ciclo `for` usa `lista.length` como condição de paragem e `lista.item(i)` para aceder a cada nó por índice numérico; (6) o texto de **cada uma** das caixas (não só a clicada) é atualizado para o novo valor de `conta` — o que revela que o clique em qualquer caixa atualiza todas.

O tipo `Element`

Os elementos são os nós **estruturais** do HTML — os restantes tipos de nó (nomeadamente `Text`) são folhas da árvore, sem filhos próprios. Os objetos `Element` são tipicamente obtidos através das funções de pesquisa de `Document` (`getElementById`, `querySelector`, etc.) ou criados com `document.createElement()`.

Métodos de `Element`:

- `getAttribute(nome)` — obtém o valor de um atributo HTML do elemento;
- `setAttribute(nome, valor)` — altera (ou cria) o valor de um atributo;
- `appendChild(no)` — herdado de `Node`, adiciona um nó filho;
- `getElementsByTagName(nome)` — devolve uma `NodeList` com todos os elementos descendentes com essa tag.

Propriedades de `Element`:

- `className` — as classes CSS do elemento (uma *string*, tal como no atributo `class` do HTML);
- `childNodes` — herdado de `Node`, lista dos descendentes diretos;
- `clientHeight*` / `clientWidth*` — altura/largura **interna** do elemento (específico do HTML);
- `id` — o identificador do elemento;
- `innerHTML*` — a formatação HTML do **conteúdo** do elemento, como *string* (específico do HTML);
- `firstChild` / `lastChild` — herdados de `Node`.

Exemplo resolvido

Usar `setAttribute()` para configurar tanto o estilo como o comportamento de um novo elemento (alternativa ao acesso direto via propriedades, útil quando o valor do atributo é montado dinamicamente como *string*):

```
function create(color) {
  const base = document.getElementById("base");
  const box = document.createElement("div");
  const space = document.createTextNode(" ");

  box.setAttribute("style", "background: " + color + "; width: 1em; margin: auto;");
  box.setAttribute("onclick", "create('" + color + "');");
  box.appendChild(space);
  base.appendChild(box);
  base.appendChild(document.createTextNode("\n"));
}
```

`setAttribute("style", "...")` define o atributo `style` do elemento diretamente como texto CSS (equivalente a escrever esse `style="..."` no HTML), em vez de configurar propriedade a propriedade via CSSOM (`box.style.background = ...`). De igual forma, `setAttribute("onclick", "...")` define o *callback* de clique como uma *string* de código a interpretar — tal como escrever `onclick="..."` diretamente em HTML.

Atenção

Nota adicional (não estava explícito nos slides): `setAttribute("onclick", "código")` regista o *handler* como texto, que só é interpretado no momento do clique. Isto é diferente de atribuir uma função diretamente à propriedade (`box.onclick = function() {...}`), que é o padrão preferido no JavaScript moderno — o registo por atributos de *string* mistura HTML dentro de JavaScript e dificulta a deteção de erros de sintaxe antes da execução (ver secção seguinte sobre modos de registo de eventos).

O tipo Text

Os nós de texto representam o conteúdo textual “pendurado” em elementos (ou seja, as folhas da árvore que não são anotações).

Propriedades:

- `data` — o texto propriamente dito (tipo *String*);
- `length` — o comprimento do texto (tipo inteiro).

Métodos (todos operam sobre `data`, com posições/`offset` a começar em 0):

- `substringData(offset, count)` — devolve uma parte da *string*, com `count` caracteres a partir de `offset`;
- `appendData(string)` — concatena `string` ao fim do texto existente;
- `insertData(offset, string)` — insere `string` na posição `offset`;
- `deleteData(offset, count)` — remove `count` caracteres a partir de `offset`;
- `replaceData(offset, count, string)` — substitui `count` caracteres a partir de `offset` por `string`;
- `splitText(offset)` — divide o nó de texto em dois nós de texto na posição `offset` (o nó original fica só com a primeira parte, e é devolvido um novo nó `Text` com a segunda parte, que é inserido como irmão seguinte).

Frequente em exame

Estes métodos de manipulação fina de `Text` (`substringData`, `splitText`, etc.) raramente são necessários na prática — na maioria dos casos basta recriar o nó de texto com `document.createTextNode()` com o conteúdo já corrigido, ou manipular a *string* em JavaScript puro e só depois atribuir. Mas é uma pergunta típica de exame identificar corretamente a que **tipo** (`Node`, `Element`, `Text`, `NodeList`, `Document`) pertence cada propriedade/método listado nesta secção, porque a distribuição não é óbvia à primeira vista (por exemplo `childNodes` está em `Node`, mas também é referido como propriedade de `Element`, por herança).

Eventos

Definição

O modelo tradicional de programação é **proativo**: a execução começa numa função predefinida (ex: `main()`), essa função invoca outras (incluindo operações de I/O), o controlo passa da função que invoca para a invocada, e regressa quando esta termina. A **programação por eventos** é, em contraste, **reativa**: a execução de um troço de código só se inicia **em resposta a um evento**. O evento é corporizado num **objeto**, passado como argumento a uma função que foi previamente **registada** para tratar esse tipo de evento nessa origem. O JavaScript no browser é fundamentalmente orientado a eventos: os elementos do documento são a origem dos eventos, há vários tipos de evento (mas comuns entre modos de registo) e várias formas alternativas de fazer esse registo.

O registo do processamento de um evento tem sempre **três partes**: o **elemento** que vai receber o evento, o **tipo** de evento a processar, e o **callback** (a função) a executar em resposta. Há três modos diferentes de fazer este registo em JavaScript, mais antigos os primeiros:

Modo 1 — atributo HTML

A forma mais antiga suportada. O elemento tem um **atributo** no próprio HTML para receber o evento; o **nome** do atributo indica o tipo do evento (começa sempre por `on` seguido do nome do evento, ex: `onclick`); o **valor** do atributo é uma *string* com o código a executar. Qualquer elemento pode ter estes atributos (embora nem todos os tipos de evento sejam de facto entregues a todos os elementos). Dentro do *callback*, `this` refere o próprio elemento.

```
<div style="..." onclick="with(this.style) {  
  let tmp = color;  
  color = background;  
  background = tmp;  
}"> Don't click me! </div>
```

Modo 2 — propriedade do elemento DOM

Separa a apresentação (HTML) da interação (JavaScript): em vez de escrever o *callback* como atributo no HTML, atribui-se uma **função** à **propriedade** do elemento (o nome da propriedade é igual ao do atributo HTML, ex: `elemento.onclick`), tipicamente usando uma função anónima. A referência ao elemento é obtida por `getElementById`, e este registo só pode ser feito depois de a janela estar carregada (evento `window.onload`, já que antes disso o elemento pode ainda não existir na árvore).

```
window.onload = function() {  
  const button = document.getElementById("button");  
  button.onclick = function() {  
    const tmp = button.style.color;  
    button.style.color = button.style.background;  
    button.style.background = tmp;  
  };  
};
```

```
};
```

Atenção

Nota adicional (não estava explícito nos slides): tanto no modo 1 como no modo 2, atribuir um novo *callback* a `onclick` **substitui** qualquer *callback* anterior registado dessa forma — só é possível ter **um** *handler* por propriedade de evento. Isto é uma limitação importante que o modo seguinte (`addEventListener`) resolve.

Modo 3 — `addEventListener()` (DOM Level 2)

`addEventListener()`, invocado sobre um elemento DOM, regista um *callback* para um tipo de evento. Ao contrário do modo por propriedade, **permite registar múltiplos *callbacks* no mesmo elemento** para o mesmo tipo de evento (todos são executados). Recebe três argumentos:

1. **Nome** do evento, como *string*, **sem** o prefixo "on" (ex: "click", não "onclick");
2. **Callback** — a função a executar;
3. **Fase** — um booleano que controla em que fase da propagação o *handler* é acionado: `true` regista na fase de *capturing*, `false` (o mais comum) regista na fase de *bubbling* (ver adiante).

```
window.addEventListener("load", function() {  
  const button = document.getElementById("button");  
  button.addEventListener("click", function() {  
    const tmp = button.style.color;  
    button.style.color = button.style.background;  
    button.style.background = tmp;  
  }, false);  
}, false);
```

Frequente em exame

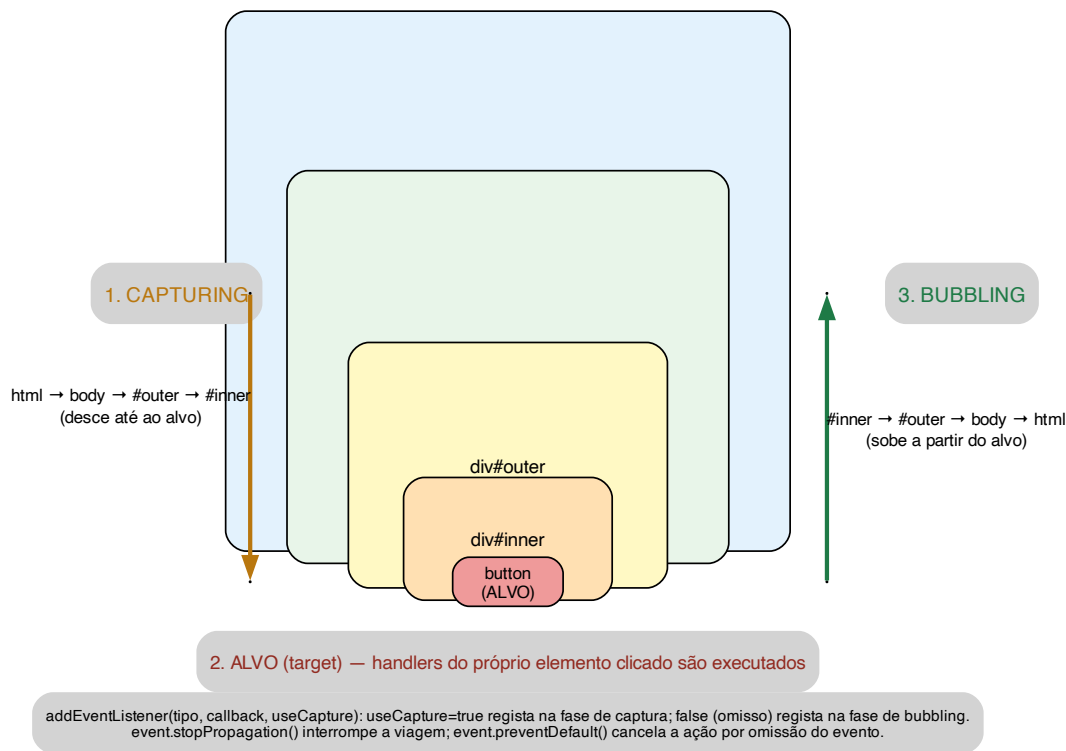
Comparação rápida dos três modos, típica de pergunta de exame: o modo por **atributo HTML** mistura marcação e comportamento e só permite um *handler*; o modo por **propriedade DOM** separa HTML de JS mas continua só a permitir um *handler* por evento/elemento; **`addEventListener`** é o único que permite múltiplos *handlers* no mesmo elemento e evento, e é o único que dá controlo explícito sobre a fase de propagação (captura vs. bubbling).

Fases de propagação de um evento

Um evento ocorre numa posição concreta do ecrã. Como os elementos se sobrepõem uns aos outros (um elemento contém os seus descendentes, que estão “por cima” dele visualmente), o evento não é entregue só ao elemento mais interno — é enviado a **todos** os elementos ao longo do caminho da raiz até esse elemento, cada um com o seu próprio conjunto de *handlers*. A ordem por que essa entrega acontece pode seguir duas convenções historicamente distintas:

- **Capturing** — do mais **externo** para o mais **interno** (à la Netscape, um dos dois browsers que originalmente disputavam o *standard* do DOM);
- **Bubbling** — do mais **interno** para o mais **externo** (à la Microsoft/Internet Explorer).

O DOM moderno resolve a disputa histórica **combinando as duas**, numa sequência de três fases: primeiro a captura desce da raiz até ao elemento alvo, depois o próprio alvo processa o evento, e por fim a propagação sobe de volta (*bubbling*) do alvo até à raiz. O terceiro argumento de `addEventListener()` é o que permite escolher se um dado *handler* deve ser acionado durante a fase de captura (`true`) ou durante a fase de *bubbling* (`false`).



Atenção

Nota adicional (não estava explícito nos slides): a maioria dos *handlers* que se escrevem na prática regista-se na fase de *bubbling* (**false**, que é também o valor por omissão quando o terceiro argumento é deixado de fora). A fase de captura só costuma ser usada quando se quer **interceptar** um evento antes que chegue ao elemento alvo — por exemplo para o cancelar globalmente antes que qualquer *handler* mais específico seja executado.

O objeto Event

Todo o *callback* de evento recebe (ou pode receber, consoante o modo de registo) um objeto **Event** como argumento, com informação sobre o evento e a possibilidade de controlar a sua propagação:

Propriedades (exemplos):

- **bubble** — booleano, indica se o evento está atualmente na fase de *bubbling*;
- **cancelBubble** — permite cancelar a fase de *bubbling* a partir desse ponto.

Métodos (exemplos):

- **preventDefault()** — cancela a **ação por omissão** do evento, se possível (por exemplo, evita que um clique num link efetivamente navegue para outra página, ou que um **mousedown** inicie uma seleção de texto);
- **stopPropagation()** — impede que o evento continue a propagar-se (interrompe a viagem de captura/bubbling nesse ponto, não chegando aos restantes elementos do caminho).

Além destas propriedades/métodos genéricos, existem **classes especializadas** do objeto **Event** para cada categoria de evento (rato, teclado, etc.), cada uma com métodos e propriedades próprios — descritos nas subsecções seguintes.

Eventos de janela

Alguns eventos têm origem na própria janela do browser, não num elemento específico do documento — o exemplo mais relevante já usado nesta seção é `load` (a página e todos os seus recursos terminaram de carregar), acedido como `window.onload = function() {...}` (modo 2) ou `window.addEventListener("load", callback, false)` (modo 3). Como já notado, é necessário esperar por este evento sempre que o código precisa de aceder a elementos do documento por referência (`getElementById`, etc.) a partir de um script incluído antes do corpo da página estar completo.

Eventos de rato

Tipos de eventos relacionados com o rato:

- `click` — clique num botão (início e fim de pressão, no mesmo local);
- `doubleclick` — dois cliques em sequência rápida;
- `mousedown` — início da pressão sobre um botão do rato;
- `mouseup` — fim da pressão sobre um botão do rato;
- `mousemove` — movimento do rato.

O objeto `Event` de eventos de rato acrescenta propriedades como a posição (`x`, `y`), o botão pressionado (`button`), e se a tecla *control* estava pressionada (`ctrlKey`).

Exemplo resolvido

Arrastar um elemento com o rato, combinando `mousedown`, `mousemove` e `mouseup`. Código completo (registado sobre um `div#dot` já existente, depois de a janela carregar):

```
window.onload = function() {
    const dot = document.getElementById("dot");

    let x;
    let y;
    let moving = false;

    dot.onmousedown = function(event) {
        x = event.x;
        y = event.y;
        moving = true;
        event.preventDefault();
    };

    dot.onmousemove = function(event) {
        if (!moving) return;
        dot.style.left = (parseInt(dot.style.left) + (event.x - x)) + "px";
        dot.style.top = (parseInt(dot.style.top) - (y - event.y)) + "px";
        x = event.x;
        y = event.y;
        event.preventDefault();
    };

    dot.onmouseup = function() {
        moving = false;
    };
};
```

Contexto: quando a página carrega, `dot` é obtido por `getElementById`; `x` e `y` guardam a última posição conhecida do cursor; `moving` (booleano) guarda se o ponto está atualmente a ser arrastado.

Exemplo resolvido

Execução traçada — o utilizador prime o botão do rato sobre o ponto, arrasta-o, e larga o botão:

1. **mousedown** dispara **dot.onmousedown**: **x** e **y** são atualizados para a posição atual do cursor (**event.x**, **event.y**); **moving** passa a **true**; **event.preventDefault()** evita o comportamento por omissão do browser para este evento (por exemplo, iniciar uma seleção de texto ao arrastar).
2. O utilizador move o rato sem largar o botão — cada movimento dispara um novo **mousemove**, que chama **dot.onmousemove**. Como **moving** é **true**, a função **não** retorna cedo (**if (!moving) return;**) e prossegue: calcula o deslocamento horizontal como a diferença entre a posição atual do rato (**event.x**) e a última posição guardada (**x**), soma esse deslocamento à posição **left** atual do ponto (convertida de *string* para número com **parseInt**, já que **style.left** é uma *string* do tipo "75px"), e escreve o novo valor de volta em **dot.style.left** (concatenando "px"); o mesmo raciocínio aplica-se ao eixo vertical com **top** (note a subtração invertida **y - event.y**, porque o eixo **y** do ecrã cresce para baixo); no fim, **x** e **y** são atualizados para a posição atual, para que o próximo **mousemove** calcule o deslocamento a partir daqui, e não da posição inicial.
3. **mouseup** dispara **dot.onmouseup**: **moving** volta a **false**. A partir daqui, novos eventos **mousemove** voltam a cair no **return** antecipado e deixam de mover o ponto, mesmo que o rato continue a mover-se sobre o ecrã.

Se o **mouseup** tivesse sido omitido, o ponto continuaria “colado” ao cursor mesmo depois de largado o botão, porque **moving** nunca voltaria a **false**.

Eventos de teclado

Tipos de eventos relacionados com o teclado:

- **keypress** — tecla pressionada (gera um carácter);
- **keydown** — início de pressão de uma tecla;
- **keyup** — fim de pressão de uma tecla.

O objeto **Event** de eventos de teclado acrescenta propriedades como o carácter (**key**), o código do carácter (**keyCode**), e se as teclas modificadoras estavam pressionadas: **ctrlKey**, **shiftKey**, **altKey**.

Exemplo resolvido

Mover um ponto com as teclas I/J/K/M (cima/esquerda/direita/baixo), registrando o evento em `document` (não no ponto em si, já que o teclado não tem uma posição no ecrã à qual associar o evento a um elemento específico):

```
const dot = document.getElementById("dot");

document.onkeypress = function(event) {
  let left = parseInt(dot.style.left);
  let top = parseInt(dot.style.top);

  switch (event.key) {
    case "k": case "K": left += 10; break;
    case "j": case "J": left -= 10; break;
    case "i": case "I": top -= 10; break;
    case "m": case "M": top += 10; break;
  }
  dot.style.left = left + "px";
  dot.style.top = top + "px";
};
```

Traço da execução ao premir a tecla “K”: (1) o browser gera um evento `keypress`, entregue a `document` (o alvo por omissão de eventos de teclado, já que não há um elemento visualmente associado ao próprio teclado); (2) `document.onkeypress` é invocado com o `event`; (3) `left/top` são lidos da posição atual de `dot` (convertidos para inteiro); (4) `event.key` vale “k” ou “K” (dependendo de *shift*), ambos os casos do `switch` incrementam `left` em 10; (5) `dot.style.left` e `dot.style.top` são reescritos com os novos valores (com “px” concatenado), movendo visualmente o ponto 10 pixels para a direita.

Exemplos completos

Contador

Componente de contagem construído inteiramente por JavaScript + DOM (sem HTML pré-escrito para o botão/visor), sobre uma `div` vazia. Ilustra: criação de nós com `createElement`, ligação à árvore com `appendChild`, associação de estilo via `className` (e uma folha CSS externa), e uso de **métodos de uma classe como *callback***.

Reconstrução do código completo, juntando `contexto.mdown` (carregamento e instanciação), `classe.mdown` (estrutura da classe), `construtor1.mdown` (construção do DOM) e `construtor2.mdown` (classes CSS, texto e ligação dos eventos):

```
class Counter {
  constructor(parentId) {
    const parent = document.getElementById(parentId);
    const counter = document.createElement("div");
    const incr = document.createElement("div");
    const reset = document.createElement("div");

    this.display = document.createElement("div");
    this.count = 0;

    parent.appendChild(counter);
    counter.appendChild(incr);
    counter.appendChild(reset);
    counter.appendChild(this.display);
```

```
    counter.className = "counter";
    incr.className = "button";
    reset.className = "button";
    this.display.className = "display";

    incr.innerText = " Incr ";
    reset.innerText = " Reset ";
    this.display.innerText = this.count;

    incr.onclick = () => this.increment();
    reset.onclick = () => this.reset();
  }

  increment() {
    this.count++;
    this.display.innerText = this.count;
  }

  reset() {
    this.count = 0;
    this.display.innerText = this.count;
  }
}

window.onload = function() {
  const counter = new Counter("base");
};

<script src="counter.js"></script>
<link rel="stylesheet" href="counter.css" type="text/css">
...
<div id=base></div>
```

Atenção

Nota adicional (não estava explícito nos slides): os ficheiros fonte apresentam **duas variantes** para ligar `increment/reset` como *callback* mantendo a referência correta a `this`: `construtor1.mdown` usa `incr.onclick = this.increment.bind(this)`, enquanto `construtor2.mdown` usa a forma mais moderna com **arrow function**, `incr.onclick = () => this.increment()`. Ambas resolvem o mesmo problema — dentro de um método normal (`increment() {...}`) usado diretamente como *callback*, `this` deixaria de referir a instância de `Counter` e passaria a referir o elemento que disparou o evento (comportamento por omissão de `onclick`). `.bind(this)` cria uma nova função com `this` fixado explicitamente; a *arrow function* `() => this.increment()` resolve o mesmo problema porque **não tem o seu próprio this** — usa sempre o `this` do contexto onde foi definida (o construtor, onde `this` é a instância). Nesta reconstrução optou-se pela forma com *arrow function*, por ser a mais recente das duas apresentadas.

O construtor apenas guarda como propriedades da instância `this.display` (o elemento onde o valor é mostrado) e `this.count` (a contagem atual) — os restantes elementos criados (`counter`, `incr`, `reset`) são variáveis locais do construtor, porque não precisam de ser acedidos depois de construído o *widget*.

Exemplo resolvido

Execução traçada — a página carrega e o utilizador clica em “Incr” duas vezes seguidas:

1. **Carregamento:** `window.onload` dispara; `new Counter("base")` corre o construtor: obtém `parent (div#base)`; cria os quatro elementos (`counter`, `incr`, `reset`, `this.display`) desligados da árvore; liga-os com `appendChild` (primeiro `incr` e `reset` dentro de `counter`, depois `counter` dentro de `parent`); atribui as classes CSS (que dão o aspeto visual, definido em `counter.css`, não incluído nos ficheiros fonte lidos); define o texto inicial dos botões (" Incr ", " Reset ") e do visor (`this.count`, que é 0); regista os dois `onclick` como *arrow functions*. No final deste passo, o ecrã mostra dois botões e um visor a “0”.
2. **Primeiro clique em “Incr”:** dispara-se `incr.onclick`, que executa `() => this.increment()` — como é uma *arrow function*, `this` continua a ser a instância de `Counter` criada no passo anterior; `increment()` corre: `this.count++` passa `count` de 0 para 1; `this.display.innerText = this.count` escreve "1" no visor, atualizando o ecrã.
3. **Segundo clique em “Incr”:** o mesmo *handler* dispara de novo; `this.count` passa de 1 para 2; o visor é atualizado para "2".

Se o utilizador clicasse depois em “Reset”, `this.count` voltaria a 0 e o visor mostraria "0" de novo — a mesma instância e as mesmas propriedades são reutilizadas em todos os cliques, porque `this` dentro das *arrow functions* refere sempre a mesma instância de `Counter` criada uma única vez no `onload`.

Jogo do Galo

Tabuleiro interativo de jogo do galo (jogo da velha, *tic-tac-toe*), também construído como classe sobre uma `div` vazia. Ilustra a mesma técnica do contador (DOM + CSS + classe), mas com **métodos como *callback* com parâmetros** — cada célula precisa de saber a **sua própria posição** quando é clicada.

Reconstrução do código completo, juntando `contexto.mdown`, `estilo.mdown` (CSS do tabuleiro), `classe.mdown` (estrutura de dados) e `tabuleiro.mdown/pecas.mdown` (construção do DOM e ligação dos eventos):

```
class TicTacToe {
  constructor(id) {
    const parent = document.getElementById(id);
    const board = document.createElement("div");

    board.className = "board";
    parent.appendChild(board);

    this.content = new Array(9);
    this.board = new Array(9);
    this.current = "X";

    for (let i = 0; i < 9; i++) {
      let cell = document.createElement("div");
      cell.className = "cell";
      board.appendChild(cell);

      cell.onclick = () => this.play(i);

      this.board[i] = cell;
    }
  }

  play(pos) {
    this.content[pos] = this.current;
```

```

        this.board[pos].innerHTML = this.current;

        this.current = (this.current == "X" ? "O" : "X");
    }
}

window.onload = function() {
    const ticTacToe = new TicTacToe("base");
};

.board {
    width: 295px;
    height: 295px;
    display: grid;
    grid-template-columns: auto auto auto;
    grid-template-rows: auto auto auto;
    gap: 5px;
    background: #228;
}

.cell {
    width: 95px;
    height: 95px;
    background: #FAFAFA;
    font-family: sans-serif;
    font-size: 95px;
    line-height: 95px;
    text-align: center;
    color: orange;
    cursor: pointer;
}

```

O estado do jogo é guardado em **duas listas paralelas** de 9 posições (uma por célula do tabuleiro 3×3): `this.content` guarda o valor lógico de cada célula ("X", "O" ou vazio) para eventual processamento (ex. verificar vencedor — não implementado nos ficheiros lidos), e `this.board` guarda a **referência ao elemento DOM** de cada célula, para poder atualizar o ecrã. `this.current` alterna entre "X" e "O" a cada jogada. O tabuleiro visual é posicionado com CSS **grid** (`display: grid`), com a cor de fundo do `.board` a servir de “grelha” — o espaçamento (`gap: 5px`) entre células deixa ver essa cor por baixo, simulando linhas divisórias sem as desenhar explicitamente.

Atenção

Nota adicional (não estava explícito nos slides): a forma como cada célula fica associada à **sua** posição (`i`) no `onclick` é subtil. `cell.onclick = () => this.play(i)` é definido **dentro** de um ciclo `for` (`let i = 0; ...`). Isto só funciona corretamente porque a variável de ciclo é declarada com **let**, não **var**: com `let`, cada iteração do ciclo cria uma **nova ligação** da variável `i`, e a *arrow function* “fecha sobre” (captura) o valor de `i` **dessa iteração específica** — ou seja, a célula 0 guarda para sempre `i = 0`, a célula 1 guarda `i = 1`, etc. Se o código usasse `var` em vez de `let`, todas as células partilhariam a **mesma** variável `i`, que no fim do ciclo valeria 9 (o valor que fez a condição `i < 9` falhar) — todos os cliques chamariam `this.play(9)`, e `this.content[9]/this.board[9]` estariam fora dos limites do array (posições válidas são 0–8). O ficheiro `pecas.mdown` confirma isto ao assinalar explicitamente que `let` “retém o valor da iteração em que foi definida”, precisamente para alertar para esta diferença face a `var`.

Exemplo resolvido

Execução traçada — o tabuleiro acabou de ser criado (vazio, é a vez de “X”) e o utilizador clica na célula central (posição 4, linha 1 coluna 1, numerando de 0 a 8 da esquerda para a direita e de cima para baixo):

1. **Construção:** `new TicTacToe("base")` corre o construtor: `parent` é `div#base`; `board` é criado e recebe a classe "board" (que lhe dá o `display: grid` de 3×3); `board` é pendurado em `parent`; `this.content` e `this.board` são inicializados como *arrays* vazios de 9 posições; `this.current` começa a "X"; o ciclo `for` cria 9 células (`div.cell`), pendura cada uma em `board`, regista o `onclick` de cada uma (capturando o seu próprio índice `i`, como explicado acima), e guarda a referência em `this.board[i]`.
2. **Clique na célula central:** o `onclick` da célula de índice 4 dispara `() => this.play(4)` — `this` continua a ser a instância de `TicTacToe` (mesma razão que no contador: *arrow function* sem `this` próprio); `play(4)` corre: `this.content[4] = this.current` guarda "X" na posição lógica 4; `this.board[4].innerHTML = this.current` escreve "X" dentro do `div` correspondente a essa célula, tornando-o visível no ecrã; `this.current = (this.current == "X" ? "O" : "X")` troca o jogador corrente para "O", preparando a próxima jogada.
3. **Segundo clique**, agora noutra célula (por exemplo posição 0): o mesmo mecanismo corre com `pos = 0`, mas desta vez `this.current` já vale "O" — a célula 0 é marcada com "O", e `this.current` volta a alternar para "X".

Se o utilizador voltasse a clicar na **mesma** célula já preenchida, o código tal como apresentado **não impede** a jogada — `play()` sobrescreve `this.content[pos]` e o conteúdo visual da célula sem verificar se já estava ocupada; essa validação não está implementada nos ficheiros lidos (nem a deteção de vencedor), o que é consistente com o objetivo pedagógico do exemplo (ilustrar DOM + eventos + classes), não o de ser um jogo completo e validado.

O Protocolo HTTP

Introdução

HTTP (*HyperText Transfer Protocol*) é o protocolo **aplicacional** usado pelo serviço WWW (World Wide Web). Está assente no modelo **cliente-servidor** subjacente à Internet e permite a troca de **mensagens** entre navegadores e servidores. Essas mensagens referem-se sempre a representações de **recursos** (páginas, imagens, dados, ...). Tal como outros protocolos aplicacionais (ex: FTP), tem características próprias que o distinguem, e tem vindo a evoluir — ainda que lentamente — estando correntemente na versão 2 (com a versão 3 já a ganhar adoção).

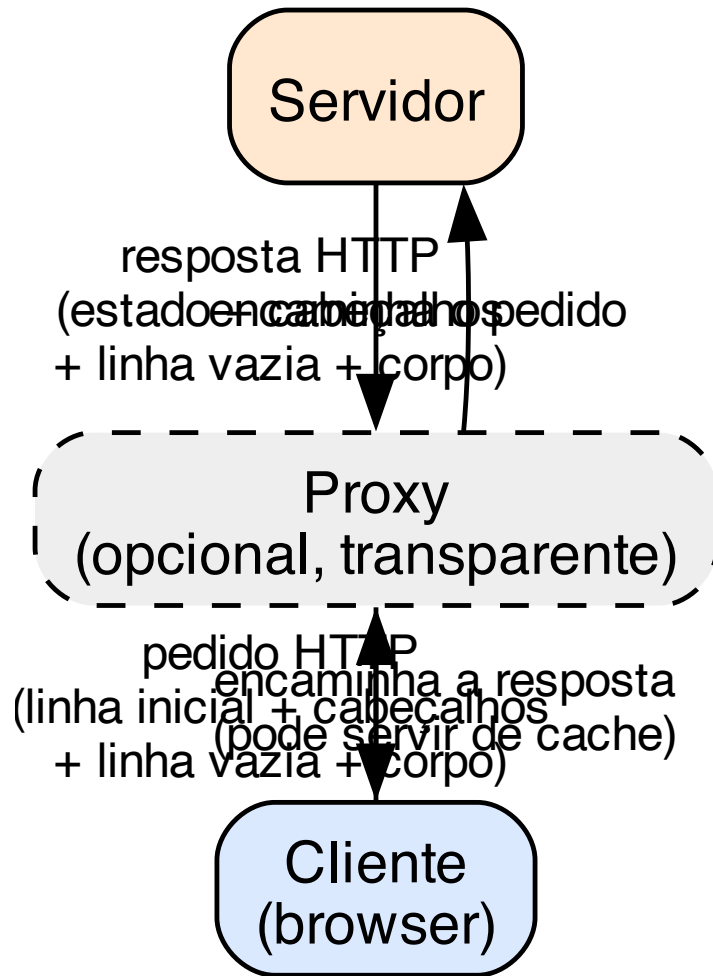
Agentes envolvidos

Três tipos de agentes participam tipicamente numa comunicação HTTP:

- **Navegadores** (*browsers*) — operados por utilizadores humanos, pedem conteúdo aos servidores em diferentes formatos: texto (HTML, CSS, JS), imagem e vídeo (PNG, JPEG, MPEG, Ogg, ...) ou dados (JSON, XML, TXT).
- **Servidores** — funcionam automaticamente e fornecem conteúdos, que podem ser **estáticos** (ficheiros guardados em disco) ou **dinâmicos** (gerados por programas no momento do pedido).
- **Intermediários** (*proxies*) — encaminham pedidos e respostas entre cliente e servidor, podendo implementar políticas de acesso, registar tráfego, ou guardar cópias de recursos para evitar pedidos redundantes (**cache**).

Ciclo pedido-resposta

O ciclo pedido-resposta é o **átomo** da comunicação HTTP: todo o protocolo se constrói a partir desta unidade básica. Os pedidos HTTP são sempre iniciados pelo **cliente** (o *browser*) e de imediato respondidos pelo **servidor**. Entre os dois pode haver *proxies* que são, em princípio, **transparentes** à comunicação (não alteram o significado da troca, apenas a encaminham).



Características do protocolo

Definição

Sem conexão (*connectionless*): a conexão entre cliente e servidor é, em princípio, refeita a cada pedido — não há uma conexão persistente dedicada a um “diálogo” contínuo entre os agentes. Isto foi pensado para facilitar a mudança de servidor entre pedidos sucessivos.

Sem estado (*stateless*): o protocolo não mantém, por si, estado da interação entre cliente e servidor. Apenas os próprios agentes guardam internamente o seu estado; cada pedido é, para o protocolo, independente dos anteriores. Ainda assim, existe capacidade para *simular* estado da interação (ex: cookies, ver secção de cabeçalhos).

Independente de media (*media independent*): qualquer tipo de recurso pode ser transportado pelo protocolo, desde que os agentes envolvidos saibam interpretá-lo. O protocolo transporta metainformação (cabeçalhos) que caracteriza o tipo de recurso trocado, para que o agente que recebe saiba como o processar.

Atenção

Nota adicional (não estava explícito nos slides): a característica “sem conexão” tem de ser lida à luz da evolução das versões do protocolo. Nas versões 0.9/1.0, cada pedido HTTP implicava de facto abrir e fechar uma nova ligação TCP — daí “connectionless”. A partir da versão 1.1, passou a existir suporte para **conexões persistentes** (*keep-alive*): a mesma ligação TCP pode ser reaproveitada para vários pedidos/respostas sucessivos, o que reduz a sobrecarga de abrir uma ligação nova a cada vez. Isto **não contradiz** o carácter *stateless* do protocolo: mesmo reaproveitando a ligação de transporte, o HTTP continua sem memória de interação ao nível aplicacional — reaproveitar a ligação é apenas uma otimização de desempenho, não introduz estado protocolar.

Evolução das versões

- **Versão 0.9** (1991) — primeira versão operacional; apenas com o método **GET**; ainda é suportada por muitos servidores por compatibilidade.
- **Versão 1.0** (1996) — introduz a generalidade dos métodos atuais, cabeçalhos com meta informação, e mecanismos de segurança.
- **Versão 1.1** (1997) — suporte para múltiplos pedidos relacionados pela mesma conexão (ligações persistentes), e clarificação de vários detalhes do protocolo.
- **Versão 2.0** (2015) — baseada no protocolo SPDY da Google; mantém a sintaxe (semântica de métodos, cabeçalhos, códigos) da versão 1.1, mas altera a forma como é implementada por baixo — passa a usar **pacotes binários** em vez de texto simples, o que permite, por exemplo, multiplexar vários pedidos na mesma ligação.

Estrutura geral de uma mensagem HTTP

As mensagens HTTP são baseadas em **texto**, mas podem transportar dados **binários** no corpo. A estrutura é semelhante em pedidos e em respostas, e compõe-se sempre de quatro partes, nesta ordem:

Definição

1. **Linha inicial** — uma única linha, terminada por EOL. É diferente em pedidos (método + URL + versão) e em respostas (versão + código + descrição de estado).
2. **Cabeçalhos** — uma linha por cabeçalho, no formato **nome: valor**, cada uma terminada por EOL.
3. **Linha vazia** — dois EOL consecutivos, separa cabeçalhos do corpo.
4. **Corpo** — texto ou binário, e pode ser vazio.

Atenção

Nota adicional (não estava explícito nos slides): o EOL (fim de linha) usado em HTTP é a sequência **CR+LF** (`\r\n`, carácter *carriage return* seguido de *line feed*), e não apenas `\n` como é costume em texto Unix. Isto é relevante na prática (ex: ao usar **telnet** manualmente) porque um terminal que só envie `\n` pode não ser reconhecido como fim de linha pelo servidor.

Estrutura de um pedido HTTP**Definição**

- **Linha inicial:** MÉTODO URL VERSÃO
 - **Método** — o verbo do pedido, ex: GET.
 - **URL** do recurso — ex: `/~zp/about/index.html`.
 - **Versão** HTTP — ex: HTTP/1.1.
- **Cabeçalhos** — metadados do pedido, por exemplo:
 - Capacidades do navegador/utilizador (**Accept***).
 - Características do navegador (**User-Agent**).
 - Página de onde o apontador foi seguido (**Referer**).
- **Corpo** — dados enviados no pedido (ex: conteúdo de um formulário).

Estrutura de uma resposta HTTP**Definição**

- **Linha inicial:** VERSÃO CÓDIGO DESCRIÇÃO
 - **Código** de estado — ex: 200.
 - **Descrição** de estado — ex: OK.
- **Cabeçalhos** — metadados da resposta, por exemplo:
 - Data — **Date**.
 - Tipo de conteúdo — **Content-Type**.
 - Descrição do servidor — **Server**.
- **Corpo** (pode não existir) — o recurso pedido, ex: uma página HTML.

Exemplo resolvido

Exemplo completo de um pedido HTTP a GET `/~zp/about/index.html` HTTP/1.1, byte a byte / linha a linha, tal como circularia na rede (EOL = `\r\n`):

```
GET /~zp/about/index.html HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate, sdch, br
Accept-Language: en-US,en;q=0.8,pt;q=0.6,pt-PT;q=0.4
Connection: keep-alive
Host: localhost
Referer: http://www.dcc.fc.up.pt/~zp/about/index.html
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
```

Note-se a linha inicial (GET + URL + versão), seguida das linhas de cabeçalho, seguida de uma linha vazia — este pedido não tem corpo (é um GET), por isso a mensagem termina logo a seguir à linha vazia.

Exemplo resolvido

E a resposta correspondente a esse pedido, também linha a linha:

```
HTTP/1.1 200 OK
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
Date: Tue, 24 Oct 2017 17:51:01 GMT
Keep-Alive: timeout=5, max=100
Server: Apache/2.4.25 (Fedora)
Transfer-Encoding: chunked
```

```
<!DOCTYPE html>
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <title>José Paulo Leal</title>
  ...
```

Aqui a linha inicial indica a versão HTTP, o código de estado (200) e a sua descrição (OK). Seguem-se os cabeçalhos, a linha vazia, e finalmente o corpo — neste caso o documento HTML pedido.

Examinar HTTP na prática

Como o HTTP é um protocolo de transferência, não fica diretamente visível ao utilizador — por vezes é necessário examinar as mensagens realmente transacionadas. Isso pode ser feito de três formas: pelo **navegador**, por **telnet**, ou pelo comando **curl**.

No navegador

Nas *developer tools* do navegador (ex: Chrome), no separador **Network**, é possível observar, para cada pedido feito pela página: a informação da linha inicial, os cabeçalhos de pedido e de resposta, e os próprios recursos recebidos (quando são texto).

Com telnet

O **telnet** é um serviço da Internet que permite dialogar diretamente com a porta de um servidor. Basta executar **telnet servidor porta**, escrever à mão um pedido HTTP, e terminar com dois EOL (sem mais cabeçalhos) — a resposta aparece diretamente na consola.

Exemplo resolvido

```
$ telnet www.up.pt 80
Trying 193.137.55.13...
Connected to www.up.pt.
Escape character is '^]'.
OPTIONS / HTTP/1.0

HTTP/1.1 200 OK
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache
Allow: OPTIONS,GET,HEAD,POST
Content-Length: 0
Connection: close
Content-Type: text/html

Connection closed by foreign host.
O utilizador escreveu manualmente a linha OPTIONS / HTTP/1.0 seguida de uma linha vazia (sem cabeçalhos); o servidor respondeu com um 200 OK e o cabeçalho Allow, que lista os métodos que suporta.
```

Com curl

O `curl` é um comando Linux que transfere um URL “falando” HTTP diretamente. A opção `-X` (ou `--request`) *método* permite escolher um método diferente de `GET`; a opção `-D` (ou `--dump-header`) *ficheiro* envia os cabeçalhos da resposta para esse ficheiro — usando `-` como nome de ficheiro, os cabeçalhos vão para o `stdout` (e assim aparecem no terminal). Mais informação: `man curl`.

Exemplo resolvido

```
$ curl -D - -X OPTIONS http://www.up.pt/
HTTP/1.1 200 OK
Date: Tue, 24 Oct 2017 15:51:54 GMT
Server: Apache
Allow: OPTIONS,GET,HEAD,POST
Content-Length: 0
Content-Type: text/html
Este comando faz um pedido OPTIONS ao servidor www.up.pt e, graças a -D -, mostra os cabeçalhos da resposta no terminal — resultado equivalente ao obtido por telnet, mas sem ter de escrever o pedido à mão.
```

Recursos

As mensagens HTTP **atuam** sobre recursos alojados no servidor: podem **obter** um recurso existente, **criar** um novo (enviando-o ou enviando os seus dados), **modificar** um recurso já existente, ou **remover** um recurso do servidor. Os recursos podem ser **referidos** — como URLs, na linha de pedido ou em cabeçalhos — ou **transferidos** — no corpo das mensagens, quer em pedidos quer em respostas. Os dados enviados usam **formatos** específicos do HTTP, e os **metadados** sobre recursos e dados (como o tamanho ou o tipo) são colocados nos cabeçalhos.

Estrutura de um URL

Um **URL** (*Uniform Resource Locator*) identifica um recurso transferido pelo protocolo. Além do **servidor**, pode indicar-se a **porta** (por omissão, 80 para HTTP); o **caminho** localiza o recurso no servidor, como um caminho de ficheiro; e existe uma **query string** opcional, separada por `?`, usada para enviar dados (tipicamente vindos de formulários, num **formato** próprio — ver secção seguinte).

Exemplo resolvido

Decomposição do URL `http://localhost:8080/regista?nome=zp&id=123`:

Parte	Valor
Protocolo (esquema)	<code>http</code>
Servidor (host)	<code>localhost</code>
Porta	<code>8080</code>
Caminho	<code>/regista</code>
<i>Query string</i>	<code>nome=zp&id=123</code>

Nas URLs colocadas na linha inicial de um pedido HTTP, o protocolo, o servidor e a porta são **omitidos** — o cliente já sabe para onde está a ligar-se — exceto em pedidos dirigidos a um agente intermediário (*proxy*), em que o URL absoluto (completo) é necessário para o proxy saber para onde encaminhar o pedido.

Atenção

Nota adicional (não estava explícito nos slides): um URL pode ainda ter uma quinta parte, o **fragmento**, introduzido por `#` e colocado a seguir à *query string* (ex: `http://localhost:8080/pagina.html?id=123#seccao2`). O fragmento identifica uma posição *dentro* do recurso (tipicamente uma âncora HTML). É processado inteiramente do lado do **cliente** depois do recurso ser recebido — **nunca é enviado ao servidor** como parte do pedido HTTP.

Media types

Um **media type** (também conhecido por tipo **MIME** — *Multipurpose Internet Mail Extensions*) é um identificador bipartido: um **tipo** (ex: `text`) e um **subtipo** (ex: `html`), separados por `/` (ex: `text/html`). Representa o tipo de um recurso, e serve de metadado tanto em pedidos (ex: cabeçalho `Accept`) como em respostas (ex: cabeçalho `Content-Type`).

Exemplos mencionados:

- `application/`: `javascript`, `json`, `x-www-form-urlencoded`, `msword (.doc)`, `vnd.ms-excel (.xls)`;
- `audio/`: `mpeg`, `vorbis`;
- `multipart/`: `form-data`;
- `text/`: `css`, `html`, `plain`;
- `image/`: `png`, `jpeg`, `gif`.

Codificações de dados

Os dados enviados nos pedidos têm origem tipicamente em HTML — em **âncoras** (que geram pedidos `GET`) ou em **formulários** (que podem gerar pedidos `GET` ou `POST`) — e podem ser colocados no **URL** da linha de pedido (método `GET`) ou no **corpo** da mensagem (método `POST`). Estes dados são codificados num de dois formatos (media types): `application/x-www-form-urlencoded` e `multipart/form-data`.

Definição — `application/x-www-form-urlencoded`

É a codificação de dados por **omissão** do HTTP. Campos e valores são colocados numa única *string*:

- Nos **valores**: os espaços são substituídos por `+`; letras e dígitos mantêm-se literalmente; outros caracteres são substituídos pelo seu valor hexadecimal, precedido de `%`.
- Na **estrutura**: nomes de campos e valores são separados por `=`, e os pares `campo=valor` são ligados entre si por `&`.

Em JavaScript, existem funções para realizar esta codificação/decodificação: `encodeURIComponent/decodeURI` e `encodeURIComponent/decodeURIComponent`.

Definição — multipart/form-data

Destinada a dados mais complexos, nomeadamente quando incluem **ficheiros**. Transporta os dados no **corpo** do pedido, organizados em várias partes, separadas por uma *string* aleatória escolhida como delimitador (*boundary*). Cada parte começa com `--boundary`, tem os seus próprios metadados (nome do campo, e opcionalmente tipo e nome de ficheiro), e a mensagem completa termina com `--boundary--`.

Exemplo resolvido

Considere-se o envio do mesmo formulário — campos **de**, **para**, **texto** (com o valor Olá Mundo) — nos dois formatos.

application/x-www-form-urlencoded (tudo numa linha, no corpo do pedido):

`de=Jos%C3%A9+Paulo+Leal¶=Mundo&texto=Ol%C3%A1+Mundo`

Repare-se na codificação: o espaço em “José Paulo Leal” torna-se +, e o é acentuado (fora do ASCII imprimível simples) é convertido para a sua representação percentual (%C3%A9, o é em UTF-8).

multipart/form-data (com um campo adicional **foto**, um ficheiro, que não seria sequer possível representar de forma prática em *urlencoded*):

`Content-Type: multipart/form-data; boundary=AaB03x`

`--AaB03x`

`Content-Disposition: form-data; name="de"`

José Paulo Leal

`--AaB03x`

`Content-Disposition: form-data; name="para"`

Mundo

`--AaB03x`

`Content-Disposition: form-data; name="texto"`

Olá Mundo

`--AaB03x`

`Content-Disposition: form-data; name="foto"; filename="foto.jpeg"`

`Content-Type: image/jpeg`

`... conteúdo da imagem ...`

`--AaB03x--`

Note-se como cada parte é auto-descrita (tem o seu próprio **Content-Disposition**, e no caso do ficheiro também o seu **Content-Type**), o que é exatamente o que falta ao formato *urlencoded* para poder transportar dados binários como uma foto.

Métodos HTTP

Existem vários métodos HTTP, cada um associado a uma finalidade diferente sobre um recurso — são o **verbo** do pedido. Nem todos são suportados por omissão pelos servidores. Os métodos básicos — **GET**, **HEAD**, **POST**, **OPTIONS** — estão disponíveis em todos os servidores e são requeridos por todos os navegadores; os restantes métodos não estão necessariamente disponíveis por omissão.

Método	O que faz
GET	Método mais usado; pede o recurso indicado no URL, que é devolvido no corpo da resposta (metadados no cabeçalho). Não deve ter qualquer efeito colateral.

Método	O que faz
HEAD	Pedido idêntico a um GET, mas a resposta não inclui o corpo — apenas os cabeçalhos. Serve para obter metadados sobre um recurso sem o transferir.
POST	Transporta dados no corpo do pedido, relacionados com o recurso do URL: podem ser critério de pesquisa, dados para acrescentar/modificar esse recurso, ou dados para criar um novo recurso. Os dados vêm codificados em <code>application/x-www-form-urlencoded</code> ou <code>multipart/form-data</code> .
OPTIONS	Devolve, no cabeçalho <code>Allow</code> da resposta, a lista de métodos suportados pelo servidor para aquele recurso — permite saber se um método é suportado antes de o tentar usar.
PUT	Recebe um recurso no corpo do pedido e guarda-o no servidor, associado ao URL indicado.
DELETE	Remove do servidor o recurso referenciado no URL.
TRACE	Ecoa (devolve) o próprio pedido recebido — útil para ver que modificações ou adições foram introduzidas por <i>proxies</i> pelo caminho.
CONNECT	Usado para estabelecer comunicações sobre SSL/TLS através de um proxy.
PATCH	Aplica modificações parciais a um recurso já existente.

Exemplo resolvido

Pedido POST a submeter um formulário para /registar, e a resposta do servidor:

```
POST /registar HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate, sdch, br
Accept-Language: en-US,en;q=0.8,pt;q=0.6,pt-PT;q=0.4
Connection: keep-alive
Host: localhost
Referer: http://www.dcc.fc.up.pt/~zp/about/index.html
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Content-Length: 56
Content-Type: application/x-www-form-urlencoded; charset=UTF-8

de=Jos%C3%A9+Paulo+Leal&para=Mundo&texto=01%C3%A1+Mundo
HTTP/1.1 200 OK
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
Date: Tue, 24 Oct 2017 17:51:01 GMT
Keep-Alive: timeout=5, max=100
Server: Apache/2.4.25 (Fedora)
Transfer-Encoding: chunked

<!DOCTYPE html>
<html><head><title>Recebido</title><head>
<body><h1>Recebido</h1><head>
</html>
```

Exemplo resolvido

Pedido OPTIONS genérico ao servidor, para descobrir que métodos suporta:

```
OPTIONS / HTTP/1.1
```

```
Accept: */*
```

```
Accept-Encoding: gzip, deflate, sdch, br
```

```
Accept-Language: en-US,en;q=0.8,pt;q=0.6,pt-PT;q=0.4
```

```
Connection: keep-alive
```

```
Host: localhost
```

```
Referer: http://www.dcc.fc.up.pt/~zp/about/index.html
```

```
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
```

```
HTTP/1.1 200 OK
```

```
Date: Tue, 24 Oct 2017 17:51:01 GMT
```

```
Server: Apache/2.4.25 (Fedora)
```

```
Allow: HEAD,HEAD,GET,HEAD,POST,OPTIONS,TRACE
```

```
Content-Length: 0
```

```
Connection: close
```

```
Content-Type: httpd/unix-directory
```

O cabeçalho Allow da resposta lista os métodos aceites pelo servidor para este recurso.

Propriedades: seguro, idempotente, cachável

Os métodos podem ser categorizados segundo três características — no entanto, **nenhuma delas é imposta pelo protocolo**: é uma convenção que as implementações devem respeitar, mas nada as obriga tecnicamente a isso.

Atenção

Nota adicional (não estava explícito nos slides): os slides listam quais os métodos com cada propriedade, mas não definem formalmente os três termos. As definições são:

- **Seguro** (*safe*): um método é seguro se, por definição, o seu propósito é apenas obter informação — **não deve** alterar o estado do servidor. Não significa que seja **totalmente** inofensivo (ex: um pedido GET pode acionar um programa CGI que, por erro de implementação, apague um recurso; muitos pedidos GET em simultâneo podem causar uma negação de serviço; TRACE pode ser usado em ataques de *cross-site tracing*) — é apenas uma **convenção semântica** sobre a intenção do método.
- **Idempotente**: um método é idempotente se fazer o **mesmo** pedido múltiplas vezes produz sempre o **mesmo efeito líquido** no servidor que fazê-lo uma única vez (ainda que as respostas individuais possam diferir). É diferente de “seguro”: PUT e DELETE alteram o estado do servidor mas são idempotentes (repetir um DELETE sobre o mesmo recurso continua a deixá-lo removido). Pelo contrário, POST tipicamente **não** é idempotente (ex: repetir um POST de compra pode criar duas encomendas) — por isso os navegadores costumam alertar o utilizador quando este tenta reenviar um formulário POST.
- **Cachável**: a resposta a um pedido cachável pode ser guardada por um cliente ou por um *proxy* intermédio e reutilizada para responder a pedidos idênticos futuros, evitando repetir o pedido ao servidor de origem — sujeito a regras adicionais (ver cabeçalhos de cache, mais abaixo) que podem forçar revalidação ou impedir a cache.

Note-se ainda que é **arriscado** inferir estas propriedades apenas pelo método: uma aplicação pode alterar o comportamento esperado (ex: um GET com parâmetros que tenha efeitos secundários no servidor).

Método	Seguro	Idempotente	Cachável
GET	Sim	Sim	Sim
HEAD	Sim	Sim	Sim
POST	Não	Não	Sim (sob condições)

Método	Seguro	Idempotente	Cachável
PUT	Não	Sim	Não
DELETE	Não	Sim	Não
OPTIONS	Sim	Sim	Não
TRACE	Sim	Sim	Não
CONNECT	Não	Não	Não
PATCH	Não	Não	Não

Códigos de estado

Os códigos de estado são **números de 3 dígitos**, organizados em 5 grupos pelo algarismo das centenas. Cada código tem uma mensagem (frase) associada para facilitar a leitura humana — mas essa mensagem é **opcional** e pode ser alterada por quem implementa o servidor; apenas o **código numérico** é normativo (parte da especificação). A lista de códigos apresentada nos slides, e reproduzida abaixo, **não é exaustiva** — existem mais códigos em cada classe.

Classe	Significado geral
1XX	Respostas informativas
2XX	Sucesso
3XX	Redireção
4XX	Erros do cliente
5XX	Erros do servidor

Código	Nome	Quando ocorre
100	Continue	Indica ao cliente que os cabeçalhos do pedido já enviados estão corretos e pode continuar a enviar o corpo (usado quando o cliente pergunta antecipadamente, ex: antes de enviar um upload grande).
101	Switching Protocols	O servidor aceita mudar para um protocolo diferente do pedido pelo cliente (ex: para <i>WebSockets</i>).
102	Processing	Indica que o pedido foi recebido mas o servidor ainda está a processá-lo, sem resposta final ainda disponível.
200	OK	O código de sucesso mais frequente — o pedido foi processado com sucesso e a resposta contém o resultado esperado.
201	Created	O pedido teve sucesso e resultou na criação de um novo recurso (tipicamente resposta a um POST ou PUT).

Código	Nome	Quando ocorre
202	Accepted	O pedido foi aceite para processamento, mas esse processamento ainda não terminou (útil para operações assíncronas/demoradas).
204	No Content	O pedido teve sucesso, mas a resposta não tem corpo (não há conteúdo a devolver).
300	Multiple Choices	Existem várias respostas/representações possíveis para o recurso pedido, e o cliente (ou utilizador) tem de escolher uma.
301	Moved Permanently	O recurso foi movido definitivamente para um novo URL, indicado no cabeçalho Location .
302	Found	O recurso está temporariamente disponível noutro URL; não deve ser retido em cache como se fosse permanente.
303	See Other	A resposta ao pedido pode ser encontrada noutro URL, a obter tipicamente com um GET ; comum após um POST , para evitar reenvio do formulário.
307	Temporary Redirect	Redireção temporária que, ao contrário do 302/303, exige repetir o pedido com o mesmo método original no novo URL.
400	Bad Request	O servidor não conseguiu processar o pedido devido a um erro de sintaxe do próprio pedido.
401	Unauthorized	O pedido requer autenticação, que não foi fornecida (ou foi inválida) — ver secção de autenticação abaixo.
402	Payment Required	Reservado para uso futuro relacionado com sistemas de pagamento digital; raramente usado na prática.
403	Forbidden	O servidor entendeu o pedido, mas recusa-se a autorizá-lo (independentemente de autenticação).
404	Not Found	O recurso pedido simplesmente não existe no servidor.
500	Internal Server Error	O erro genérico mais frequente do lado do servidor — este não conseguiu responder devido a um erro interno (ex: exceção não tratada num programa).

Código	Nome	Quando ocorre
501	Not Implemented	O servidor não reconhece o método do pedido, ou não tem capacidade para o satisfazer.
502	Bad Gateway	Um servidor a atuar como <i>gateway</i> ou <i>proxy</i> recebeu uma resposta inválida do servidor a montante.
503	Service Unavailable	O servidor está temporariamente incapaz de responder (ex: sobrecarga, manutenção).
504	Gateway Timeout	Um servidor a atuar como <i>gateway</i> ou <i>proxy</i> não recebeu resposta a tempo do servidor a montante.

Exemplo resolvido

```
Exemplo de resposta com código de sucesso:
HTTP/1.1 200 OK
Date: Fri, 21 Apr 2017 15:23:55 GMT
Server: Apache/2.4.25 (Fedora)
Last-Modified: Fri, 21 Apr 2017 15:23:55 GMT
ETag: "116f-54daed884be86"
Accept-Ranges: bytes
Content-Length: 4463
Connection: close
Content-Type: text/html; charset=UTF-8
Exemplo de resposta de redireção permanente:
HTTP/1.1 301 Moved Permanently
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache/2.4.25 (Fedora)
Location: http://khato.dcc.fc.up.pt/~zp/
Content-Length: 238
Connection: close
Content-Type: text/html; charset=iso-8859-1
Exemplo de erro do cliente (recurso inexistente):
HTTP/1.1 404 Not Found
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache/2.4.25 (Fedora)
Content-Length: 210
Connection: close
Content-Type: text/html; charset=iso-8859-1
Exemplo de erro do servidor:
HTTP/1.1 500 Internal Server Error
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache/2.4.25 (Fedora)
Content-Length: 238
Connection: close
Content-Type: text/html; charset=iso-8859-1
```

Cabeçalhos HTTP

A informação principal de uma mensagem HTTP é transportada no corpo, mas há muita informação relevante que vai nos **cabeçalhos**. Alguns cabeçalhos destinam-se apenas a pedidos, outros apenas a respostas, e outros a ambos; em alguns casos, um cabeçalho de pedido e um de resposta fazem parte do mesmo “diálogo” (ex: cache, autenticação, CORS).

Identificação

Cabeçalhos que dão informação genérica sobre os agentes envolvidos e o contexto do pedido:

- **User-Agent** (pedido) — descrição do cliente: navegador, sistema operativo, sistema de janelas.
- **Server** (resposta) — descrição do servidor.
- **Referer** (pedido) — página de onde o pedido teve origem, se existir.
- **Date** (resposta) — data e hora da resposta.

Exemplo resolvido

```
GET /index.html HTTP/1.1
Referer: http://www.dcc.fc.up.pt/~zp/about/index.html
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
HTTP/1.1 200 OK
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache/2.4.25 (Fedora)
```

Metadados do corpo

Cabeçalhos que caracterizam a informação transportada no corpo da mensagem (podem aparecer quer em pedidos quer em respostas):

- **Content-Type** — tipo de conteúdo (media type) do corpo. Nos pedidos, o tipo é tipicamente um dos dois formatos de dados de formulário (*urlencoded* ou *multipart*); nas respostas pode ser qualquer media type.
- **Content-Length** — tamanho do corpo, em bytes.
- **Transfer-Encoding: chunked** — usado quando é difícil determinar o tamanho do corpo antecipadamente (ex: conteúdo gerado dinamicamente); o conteúdo é enviado aos bocados, cada um precedido do seu próprio tamanho.

Exemplo resolvido

```
POST /registra HTTP/1.1
Content-Length: 56
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
```

Redireção

As redireções corrigem contextos para URLs relativos, indicam novas localizações de páginas e diretorias, ou informam sobre alternativas existentes (ex: traduções). Consoante o caso, é usado um código de estado 3XX diferente (ver tabela acima); o novo URL vai no cabeçalho **Location**.

Exemplo resolvido

Um pedido a uma diretoria sem barra final é corrigido por redireção:

```
GET /~zp HTTP/1.1
HTTP/1.1 301 Moved Permanently
Location: http://khato.dcc.fc.up.pt/~zp/
GET /~zp/ HTTP/1.1
```

Gestão de estado (cookies)

O estado da comunicação **não** é mantido pelo protocolo HTTP em si — *cookies* são o mecanismo que permite simular esse estado, e servem para rastrear utilizadores, personalizar conteúdo, ou criar sessões. Usam dois cabeçalhos:

- **Set-Cookie:** `nome=valor` — usado na **resposta**, é sempre o **primeiro** a aparecer (o servidor “planta” o cookie no cliente).
- **Cookie:** `nome=valor` — usado no **pedido**, sempre **depois** de o cliente já ter recebido o cookie numa resposta anterior.

Um cookie pode ter uma validade limitada, indicada por **Expires**; por omissão, expira quando o *browser* é fechado. Pode também ser restrito a uma sub-árvore do servidor através do atributo **Path**. Quando há vários cookies no cabeçalho de um pedido, são separados por ; .

Exemplo resolvido

Sequência de trocas ilustrando o ciclo de vida de cookies numa sessão:

```
GET /index.html HTTP/1.1
200 OK
Set-cookie: id=123; Expires=Wed, 22 Sep 2027 10:00:00 GMT
Set-cookie: sessao=12345
O cliente guarda os dois cookies e passa a enviá-los em pedidos seguintes ao mesmo servidor:
GET /loja/index.html HTTP/1.1
Cookie: id=123; sessao=12345
200 OK
Set-cookie: prefere=A; Path=/loja/
O cookie prefere só é reenviado em pedidos dentro de /loja/:
GET /loja/regista HTTP/1.1
Cookie: id=123; sessao=12345; prefere=A
Fora dessa sub-árvore, prefere não é enviado:
GET /index.html HTTP/1.1
Cookie: id=123; sessao=12345
Se o utilizador desligar e voltar a ligar o browser, o cookie sessao (sem Expires, logo de sessão) perde-se, e é recriado com um novo valor:
GET /index.html HTTP/1.1
Cookie: id=123
200 OK
Set-cookie: sessao=4321
```

Cache

Cabeçalhos que dão informação relativa a mecanismos de *cache*:

- **Cache-control** — fornece diretivas de controlo:
 - **max-age** — tempo de vida em segundos.
 - **no-store** — evitar guardar em cache (ex: informação sensível).
 - **no-cache** — pode guardar, mas tem de confirmar (revalidar) antes de reutilizar.
 - **private** — a resposta requer autenticação para ser reutilizada.

- **public** — a resposta não requer autenticação.
- Controlo de modificações por *hash*:
 - **ETag** — *hash* identificando a versão do recurso, enviado na resposta.
 - **If-None-Match** — enviado no pedido seguinte, com o valor do **ETag** anteriormente recebido, para validar se o recurso mudou.
- **If-Modified-Since** — controlo alternativo, por data da última modificação conhecida pelo cliente.

Exemplo resolvido

Pedido inicial e resposta cachável:

```
GET /index.html HTTP/1.1
```

```
HTTP/1.1 200 OK
```

```
Cache-control: max-age=120
```

```
ETag: xpto
```

Pedido de revalidação, depois de o **max-age** expirar, usando o **ETag** guardado:

```
GET /index.html HTTP/1.1
```

```
If-None-Match: xpto
```

```
HTTP/1.1 304 Not Modified
```

O **304 Not Modified** confirma que o recurso não mudou, e o cliente pode reutilizar a cópia que já tinha em cache, sem o servidor ter de reenviar o corpo.

Partilha de recursos entre origens (CORS)

Pedidos vindos de uma página são, geralmente, dirigidos ao mesmo servidor de onde a página veio. Pedidos **cruzados** (para outra origem) são muito usados em ataques informáticos, pelo que os navegadores **limitam** este tipo de pedidos por omissão. Como, no entanto, há servidores que legitimamente fornecem serviços web a múltiplos clientes de origens diferentes, existe o mecanismo **CORS** (*Cross-Origin Resource Sharing*): o servidor usa o cabeçalho **Access-Control-Allow-Origin** para indicar que origens autoriza — um asterisco (*) para qualquer origem, ou uma lista de URLs válidos.

Exemplo resolvido

```
GET /recurso HTTP/1.1
```

```
HTTP/1.1 200 OK
```

```
Access-Control-Allow-Origin: *
```

Preparação de CORS (*preflight request*)

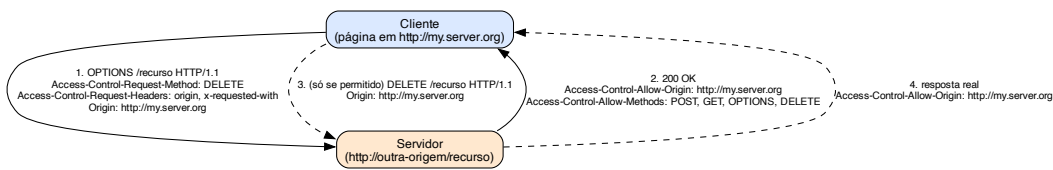
Antes de um pedido “não simples” entre origens (ex: usando métodos como **DELETE**, ou cabeçalhos personalizados), o navegador envia primeiro um pedido de **preflight**, para verificar se o servidor o vai permitir. É um pedido com o método **OPTIONS**, e os seguintes cabeçalhos:

- **Access-Control-Request-Method** — o método que o pedido real irá usar.
- **Access-Control-Request-Headers** — os cabeçalhos que o pedido real irá enviar.
- **Origin** — a origem (esquema+servidor+porta) de onde partirá o pedido.

A resposta do servidor ao **preflight** inclui:

- **Access-Control-Allow-Origin** — a origem que o servidor aceita.
- **Access-Control-Allow-Methods** — a lista de métodos aceites (deve incluir o método pedido, para o pedido real ser depois permitido).

Só se o **preflight** for aceite é que o navegador chega a enviar o pedido real.



Exemplo resolvido

Pedido de *preflight*, completo e cru:

```
OPTIONS /recurso HTTP/1.1
```

```
Access-Control-Request-Method: DELETE
```

```
Access-Control-Request-Headers: origin, x-requested-with  
Origin: http://my.server.org
```

Exemplo resolvido

Resposta do servidor ao *preflight*, autorizando o pedido:

```
HTTP/1.1 200 OK
```

```
Access-Control-Allow-Origin: http://my.server.org
```

```
Access-Control-Allow-Methods: POST, GET, OPTIONS, DELETE
```

Como DELETE está incluído em Access-Control-Allow-Methods, e a origem `http://my.server.org` corresponde à indicada em Access-Control-Allow-Origin, o navegador prossegue e envia o pedido DELETE real que motivou este preflight.

Autenticação

O processo de autenticação começa tipicamente com um acesso a um recurso protegido, ao qual o servidor responde com 401 **Unauthorized** e um cabeçalho **WWW-Authenticate** tipo `realm="reino"`, onde o **tipo** indica o esquema de autenticação (geralmente Basic) e o **reino** é um texto descritivo do motivo da autenticação. O navegador reage a este desafio mostrando uma caixa de autenticação ao utilizador, e repete o pedido com um cabeçalho **Authorization: tipo credenciais**, em que o tipo é o mesmo pedido pelo servidor. No esquema Basic, as credenciais vão codificadas em **base64**.

Atenção

Nota adicional (não estava explícito nos slides): base64 é apenas uma codificação para transmitir dados binários em canais de 7 bits — **não é cifragem**, e é trivialmente reversível (ex: com o comando Linux `base64 -d`). Por isso, a autenticação Basic só deve ser usada sobre um canal já cifrado (HTTPS) — caso contrário, as credenciais viajam, na prática, em texto legível por qualquer interveniente no percurso da mensagem.

Exemplo resolvido

```
GET /segredos/index.html HTTP/1.1
```

```
HTTP/1.1 401 Unauthorized
```

```
WWW-Authenticate: Basic realm="Pequenos segredos"
```

```
GET /segredos/index.html HTTP/1.1
```

```
Authorization: Basic enA6c291IGV1Cg==
```

```
HTTP/1.1 200 OK
```

Segurança

Segurança do canal — HTTP vs. HTTPS

O protocolo HTTP é intrinsecamente **inseguro**: as suas mensagens de texto são **visíveis** a quem intercetar a comunicação (incluindo dados de autenticação), tornando-o vulnerável a ataques *man-in-the-middle*; além disso, as linhas iniciais dos pedidos (que podem conter dados, ex: parâmetros num **GET**) ficam frequentemente registadas em ficheiros de log intermédios (ex: em proxies).

HTTPS é o HTTP sobre uma camada de transporte segura — identificado por uma variante do esquema no URL (**https://**). Cria um **canal seguro cifrado** entre cliente e servidor, mantendo-se permeável à passagem por agentes intermediários (*proxies*, que já não conseguem ler o conteúdo). A cifragem direcional das mensagens impossibilita tanto a **leitura** das mensagens no seu trajeto como a sua **alteração** entre emissor e recetor. É usado tipicamente para proteger: transações financeiras (inicialmente sobre SSL), dados de autenticação (palavras-passe), e a identidade e privacidade dos utilizadores. A sua utilização é cada vez mais generalizada — hoje é a norma, mesmo para sites sem dados sensíveis óbvios.

Certificados digitais

O canal seguro assenta em **certificados digitais**, tanto de clientes como de servidores.

- **Certificados de servidor** — são específicos do **domínio** e **validáveis**; permitem ao cliente estabelecer a **autenticidade** do servidor com quem está a comunicar. São validados junto de **autoridades certificadoras**, e as cadeias de certificação permitem distribuir essa validação (uma autoridade pode certificar outra, que por sua vez certifica o servidor). As autoridades de topo (raiz) vêm **pré-configuradas** nos navegadores, que confiam nelas por omissão.
- **Certificados de cliente** — habitualmente **gerados automaticamente** pelos próprios navegadores, mas também podem ser certificados **pessoais**, carregados manualmente para o navegador. Fornecem uma forma mais segura de autenticação do que, por exemplo, o esquema **Basic** visto acima.

Atenção

Nota adicional (não estava explícito nos slides): os slides não detalham *como* um certificado estabelece um canal cifrado — apenas que “HTTPS é baseado em certificados”. Em traços largos, ao estabelecer uma ligação HTTPS ocorre um *handshake* (aperto de mão) TLS: o servidor apresenta o seu certificado (que inclui a sua chave pública e é assinado por uma autoridade certificadora); o cliente valida essa assinatura contra as autoridades em que já confia; se for válida, cliente e servidor usam criptografia de chave pública para negociar em segurança uma **chave simétrica** partilhada, que passa a cifrar todo o tráfego HTTP seguinte (a criptografia simétrica é usada para o tráfego em si por ser muito mais rápida do que a de chave pública). É por isto que os certificados “garantem” simultaneamente duas coisas distintas: a **identidade** do servidor (assinatura da autoridade certificadora) e a **confidencialidade** do canal (a chave simétrica negociada a partir do certificado).

Comunicação Assíncrona Cliente-Servidor

A forma clássica: formulários e submissão de página completa

O modelo básico de comunicação na Web é o **ciclo pedido-resposta**: o cliente envia um pedido HTTP e o servidor devolve uma resposta que, na forma mais simples, **substitui integralmente** a página no cliente. Este modelo foi pensado para navegação hipertexto (documentos ligados por hiperligações) e não facilita, por si só, o desenvolvimento de aplicações interativas.

Definição

Âncoras. Uma hiperligação simples

```
<a href="http://some.site.com">Um sítio qualquer</a>
```

ao ser ativada origina um pedido HTTP com método **GET** e o URL indicado no atributo **href**. O servidor responde com uma página HTML completa que substitui a anterior. É a forma mais básica de atualização de conteúdo.

Envio de dados por URL. Um pedido **GET** pode transportar dados através da *query string*, o sufixo do URL a seguir a **?**, com pares atributo-valor codificados em **urlencoded**:

```
<a href="http://some.site.com/execute?command=remove&id=123">Remover</a>
```

Esta forma de envio tem limitações importantes: só é prática para um número reduzido de pares nome-valor, não permite dados livremente introduzidos pelo utilizador (não há widgets de formulário associados) e, por definição do protocolo HTTP, os pedidos **GET** devem ser **seguros** (não alteram estado), **idempotentes** (repetir o pedido tem o mesmo efeito que fazê-lo uma vez) e **cacheáveis** — pelo que não são apropriados para operações que modifiquem dados no servidor.

Definição

Formulários. Um **<form>** dá maior controlo sobre o envio de dados:

```
<form action=http://some.site.com/execute method=post>
  <input type=hidden name=command value=remove>
  <input type=hidden name=id value=123>
  <input type=submit value=Remover>
</form>
```

- O atributo **method** indica o método HTTP a usar: **POST** deve ser usado sempre que houver modificação de estado no servidor; **GET** é o valor por omissão.
- Os dados enviados são pares **nome-valor** recolhidos dos widgets do formulário: o **nome** vem do atributo **name**, o **valor** do atributo **value** (ou do que o utilizador escreveu/selecionou). O tipo **hidden** permite incluir valores fixos que não aparecem na interface.
- Por omissão, os dados são codificados em **urlencoded**.

Codificação dos dados. O atributo **enctype** do formulário controla como os dados são serializados no pedido:

- **application/x-www-form-urlencoded** — a codificação por omissão (espaços viram **+**, caracteres especiais são escapados).
- **multipart/form-data** — obrigatória para *upload* de ficheiros; permite incluir o conteúdo de campos **type=file** como parte do corpo do pedido; só está disponível com o método **POST**.
- **text/plain** — sem codificação real (apenas espaços viram **+**).

```
<form action=http://some.site.com/execute method=post enctype=multipart/form-data>
  <input type=hidden name=command value=insert>
  <input type=text name=name>
  <input type=file name=photo>
```

```
<input type=submit value=Inserir>
</form>
```

Definição

Frames. Antes do DOM permitir manipulação fina da página, os *frames* eram a forma de ter zonas da página atualizáveis **independentemente** do resto:

```
<a href=p1.html target=frame>Uma página</a>
<a href=p2.html target=frame>Outra página</a>
<iframe name=frame></iframe>
```

- O elemento `iframe` cria uma sub-janela de navegação, identificada por um `name` e que pode ser inicializada com um `src`.
- Âncoras (`a`) e formulários (`form`) podem indicar, no atributo `target`, o nome de um *frame* para onde a resposta do servidor deve ser direcionada, em vez de substituir a página inteira.

Atenção

Evitar atualizar a formatação inteira. Substituir a página completa a cada pedido-resposta provoca mudanças abruptas na interface (a página “pisca”, perde-se posição de *scroll*, estado de widgets, etc.). Usando o DOM diretamente é possível fazer **mudanças localizadas**, refletindo apenas os dados recebidos do servidor sem recarregar tudo. Antes de existirem mecanismos como XHR e `fetch`, usavam-se alternativas *hacky* para obter dados do servidor sem recarregar a página:

- **iframe invisível + script** — submeter dados para um `iframe` escondido, evitando a navegação da página principal.
- **JSONP** (*JSON with padding*) — gerar dinamicamente, via DOM, um elemento `<script>` cujo atributo `src` aponta para um URL do servidor; o recurso devolvido tem tipo `application/javascript` e consiste tipicamente numa invocação de uma função já definida no cliente, passando-lhe um objeto de dados (JSON) como argumento. Isto contorna a *same-origin policy* porque scripts podem ser carregados de qualquer origem — mas por isso mesmo é um **hack** (exige confiar completamente no servidor remoto, que passa a poder executar código arbitrário no cliente).

Estes mecanismos foram entretanto superados por XHR/`fetch` (secções seguintes), que permitem pedir dados em segundo plano sem qualquer destas gambiarras.

Serialização de dados

Definição

Necessidade de serialização. O corpo de uma mensagem HTTP transporta um recurso como uma simples fila de *bytes* (com um tipo de meio associado). Só que os dados manipulados por um programa em execução são uma **rede de referências a objetos**, potencialmente dispersa por regiões não contíguas de memória, com valores binários cuja representação depende da plataforma (*endianness*, alinhamento, etc.). Para transportar esses dados entre processos — possivelmente em linguagens e máquinas diferentes — é preciso **serializá-los**: convertê-los para uma representação linear (tipicamente texto) que possa ser reconstruída no destino. Formatos de texto têm a vantagem de serem independentes de linguagem/plataforma e legíveis por humanos.

Há várias famílias de formatos de serialização, cada uma com vantagens e desvantagens:

Formato	Exemplo	Vantagens	Desvantagens
Texto simples	Fulano 99	Muito simples	Não estruturado — apenas dados separados por um carácter convencional

Formato	Exemplo	Vantagens	Desvantagens
XML	<code><data> <name>Fulano</name> <age>99</age> </data></code>	Estruturado, permite definir linguagens e validar (schemas)	Mais complexo de analisar e gerar
JSON	<code>{"name": "Fulano", "age": 99}</code>	Estruturado e compacto	Mais difícil de <i>standardizar</i> e validar (sem um schema universal tão maduro como XML)

Definição

JSON — JavaScript Object Notation. Formato de serialização baseado diretamente na sintaxe de objetos literais do JavaScript:

```
{ "name": "Fulano", "age": 99, "active": true, "lucky numbers": [7, 13] }
```

Tipos de dados suportados:

- **Números** — inteiros ou vírgula flutuante.
- **Strings** — sequências de caracteres Unicode, sempre delimitadas por aspas duplas (com sequências de escape para caracteres especiais).
- **Booleanos** — `true` e `false`.
- **Listas** — ordenadas, delimitadas por parêntesis retos `[ ]`.
- **Objetos** — coleções de pares chave-valor, delimitados por chavetas `{ }`.
- **null** — valor nulo explícito.

Num par chave-valor, a chave é sempre uma *string* (com aspas) separada do valor por dois pontos `:`. Elementos de listas e pares chave-valor sucessivos são separados por vírgulas `,`. Repare-se que **JSON não é um subconjunto perfeito de JS** — por exemplo, chaves sem aspas ou vírgulas finais (*trailing commas*), válidas em objetos literais JS, não são válidas em JSON.

Exemplo resolvido

Converter entre JSON e objetos JS.

```
let objt = { a: 1, b: [2, 3] };
let json = JSON.stringify(objt); // '{"a":1,"b":[2,3]}' (uma string)
let copy = JSON.parse(json);    // { a: 1, b: [2, 3] } (um novo objeto)
```

- `JSON.stringify(valor)` converte um valor/objeto JS na sua representação textual JSON.
- `JSON.parse(texto)` faz o inverso: converte uma *string* JSON num valor/objeto JS.
- Historicamente, antes destes métodos existirem em todos os navegadores, convertia-se texto para JS com `eval()` — protegendo a expressão com parêntesis, `eval("(" + jsonString + ")")` — mas esta abordagem é **insegura**: se a *string* não for JSON mas sim código JS arbitrário (por exemplo, vindo de uma fonte não confiável), esse código é executado tal e qual. `JSON.parse/JSON.stringify` são a forma preferível e segura.
- Nota lateral: como `JSON.parse(JSON.stringify(objt))` produz um objeto novo e independente, com os mesmos valores mas sem partilhar referências com o original, este par de chamadas é também uma forma simples (embora não muito eficiente) de fazer uma **cópia profunda** (*deep copy*) de um objeto composto apenas por dados serializáveis em JSON.

XMLHttpRequest

Definição

Cenário motivador. Considere-se um contador distribuído: uma interface com dois botões (**Incr**, **Reset**) e um visor, mas em que o valor do contador é mantido num **servidor central** e não localmente no cliente — para que múltiplos clientes vejam (e alterem) o mesmo valor. Isto exige comunicar com o servidor a partir do JavaScript da página, **sem recarregar a página**, o que motiva o objeto `XMLHttpRequest` (XHR). O cenário usa três operações remotas: dois métodos `POST` que alteram o estado do servidor (**incr** incrementa, **reset** repõe a zero) e uma forma de obter o valor corrente.

Criar o objeto. Navegadores antigos podem não disponibilizar o construtor `XMLHttpRequest`, pelo que é boa prática testar a sua existência antes de o instanciar:

```
if (!XMLHttpRequest) { console.log('XHR não é suportado'); return; }
const xhr = new XMLHttpRequest();
```

Cada instância de `XMLHttpRequest` deve ser usada para **apenas uma** comunicação (um pedido/resposta) — não se reutiliza a mesma instância para vários pedidos sucessivos. A instância fornece o contexto (métodos e propriedades) para toda a comunicação.

Definição

Método `open(método, url, assíncrono)`.

```
xhr.open('POST', 'http://' + host + ':' + port + '/' + command, false);
```

Recebe três argumentos: o **método HTTP** (GET, POST, ...), o **URL absoluto** do pedido, e um booleano que indica se o pedido é **assíncrono** (`true` por omissão). `open()` apenas configura o pedido — não o envia.

Definição

Método `send([dados])`. Inicia efetivamente o pedido configurado por `open()`. Pode receber, opcionalmente, dados para o corpo do pedido. O comportamento depende do terceiro argumento de `open()`:

- Num pedido **síncrono**, `send()` só retorna quando a resposta chega — bloqueia.
- Num pedido **assíncrono**, `send()` retorna **de imediato**, antes de a resposta estar disponível.

Síncrono vs. assíncrono

```
// Versão síncrona
increment() { this.do('incr'); }
reset()     { this.do('reset'); }

do(command) {
  if (!XMLHttpRequest) { console.log('XHR não é suportado'); return; }
  const xhr = new XMLHttpRequest();
  xhr.open('POST', 'http://' + host + ':' + port + '/' + command, false);
  xhr.send();
  if (xhr.status == 200)
    this.display.innerText = xhr.responseText;
}
```

Neste código, como o pedido é síncrono (`false`), quando `send()` retorna a resposta **já chegou**, e as propriedades `xhr.status` e `xhr.responseText` já estão preenchidas — o fluxo lê-se como uma chamada de função normal.

Atenção

Porque é que os pedidos síncronos são maus? Enquanto `send()` está bloqueado à espera de resposta, **o programa não processa mais nenhum evento** — não há repaint da UI, não se pode clicar noutro botão, o separador do navegador pode aparecer “pendurado”. Isto degrada gravemente a experiência de utilização (UX). Por esta razão, `XMLHttpRequest` síncrono está **descontinuado** (*deprecated*) quando usado na *thread* principal. Continua a poder usar-se a partir de *web workers* (que correm numa *thread* separada) e, tradicionalmente, era por vezes aceite no evento `onbeforeunload`, para garantir que um registo termina antes da página fechar. A alternativa geral é usar sempre **pedidos assíncronos**, em que `send()` retorna de imediato — mas nesse caso as propriedades da resposta ainda estarão por preencher no momento em que `send()` retorna, sendo necessário registar uma função de *callback* para ser notificado quando a resposta (ou uma mudança de estado) ocorrer.

```
// Versão assíncrona
do(command) {
  const xhr = new XMLHttpRequest();
  const display = this.display;

  xhr.open('POST', 'http://' + host + ':' + port + '/' + command, true);
  xhr.onreadystatechange = function() {
    if (xhr.readyState < 4) return;
    if (xhr.status == 200)
      display.innerText = xhr.responseText;
  };
  xhr.send();
}
```

Definição

Estado de prontidão (`readyState`). O objeto XHR expõe o progresso do ciclo pedido-resposta através de duas propriedades associadas:

- `xhr.readyState` — inteiro com o estado atual: 0 — por enviar; 1 — enviado; 2 — cabeçalhos recebidos; 3 — corpo a chegar; 4 — **completo**.
- `xhr.onreadystatechange` — função (*callback*) invocada sempre que `readyState` muda. Por isso o padrão típico é verificar `readyState == 4` dentro deste *callback*, para só reagir quando a resposta estiver totalmente disponível.

Receber dados. Depois de a resposta chegar (e só depois — as propriedades são só de leitura e ficam indefinidas antes disso), lêem-se os resultados a partir de propriedades do próprio objeto XHR:

- `xhr.status` — código de estado da resposta HTTP (ex.: 200).
- `xhr.responseText` — corpo da resposta como texto.
- `xhr.responseXML` — corpo da resposta interpretado como um documento DOM (quando o tipo do conteúdo o permite).

Exemplo resolvido

Processar dados JSON via XHR.

```
do(command, value) {
  const xhr = new XMLHttpRequest();
  const display = this.display;

  xhr.open('POST', 'http://' + host + ':' + port + '/process', true);
  xhr.onreadystatechange = function() {
    if (xhr.readyState == 4 && xhr.status == 200) {
      const data = JSON.parse(xhr.responseText);
      display.innerText = data.value;
    }
  };
  xhr.send(JSON.stringify({ command: command, value: value }));
}
```

O corpo do pedido é construído serializando um objeto JS em JSON com `JSON.stringify` (em ES6, `{command, value}` é uma forma abreviada de criar um objeto cujas chaves têm o mesmo nome das variáveis); o corpo da resposta é desserializado de volta para um objeto JS com `JSON.parse`. Ou seja, `send()` pode receber texto (ou dados serializáveis) para o corpo do pedido, e `responseText` devolve o corpo da resposta como texto — o formato dos dados em si (JSON, neste caso) é uma convenção entre cliente e servidor, independente do XHR.

Cabeçalhos. Podem ser manipulados explicitamente através de métodos do próprio objeto:

- `xhr.setRequestHeader(nome, valor)` — adiciona um cabeçalho ao **pedido**, antes de chamar `send()` (ex.: `xhr.setRequestHeader('Content-Type', 'text/plain')`).
- `xhr.getResponseHeader(nome)` — lê um cabeçalho da **resposta**, depois de esta ter chegado (ex.: `xhr.getResponseHeader('Content-Type')`).

Definição

CORS do lado do pedido. A *same-origin policy* do navegador impede, por omissão, que um pedido XHR feito a partir de uma página seja dirigido para um domínio diferente do da própria página — a menos que o servidor de destino o permita explicitamente através de cabeçalhos CORS. Mesmo quando isso é permitido, por omissão os navegadores **não enviam automaticamente cookies** (nem outras credenciais) nesses pedidos entre origens. Isto pode ser contornado com a propriedade `xhr.withCredentials = true`, que faz o navegador incluir os cabeçalhos relevantes (como `Cookie`) automaticamente. Se o pedido não depender de *cookies*, o mais seguro e simples é deixar `withCredentials` no valor por omissão (`false`).

De callbacks a Promises a async/await

O problema: encadear pedidos

Muitas vezes é preciso fazer um segundo pedido cujo conteúdo depende do resultado do primeiro (encadeamento). Com pedidos **síncronos**, isto é trivial — lê-se como chamadas de função normais, em sequência linear:

```
const xhr1 = new XMLHttpRequest();
xhr1.open('POST', url1, false);
xhr1.send(JSON.stringify(x));

const xhr2 = new XMLHttpRequest();
xhr2.open('POST', url2, false);
```

```
xhr2.send(xhr1.responseText);  
  
continuar(xhr2.responseText);
```

A 1ª chamada é invocada; quando termina, o seu resultado é injetado na 2ª, que só então é invocada. A sequência de código coincide com a sequência de execução — é o habitual num programa. Mas isto tem o problema já discutido: bloqueia a *thread* principal duas vezes.

Atenção

O “*callback hell*”. Passando os mesmos dois pedidos para a forma **assíncrona**, o encadeamento fica assim:

```
const xhr1 = new XMLHttpRequest();  
xhr1.open('POST', url1, true);  
xhr1.onreadystatechange = function() {  
  // ...  
  const xhr2 = new XMLHttpRequest();  
  xhr2.open('POST', url2, true);  
  xhr2.onreadystatechange = function() {  
    // ...  
    continuar(xhr2.responseText);  
  };  
  xhr2.send(xhr1.responseText);  
};  
xhr1.send(JSON.stringify(x));
```

Repare-se que a **sequência do código já não coincide com a sequência das invocações**: `xhr2` só é criado e enviado dentro do *callback* de `xhr1`, e `continuar` só é chamado dentro do *callback* de `xhr2`. O resultado final é o mesmo do caso síncrono (sem bloquear a *thread*), mas cada novo pedido encadeado acrescenta **mais um nível de indentação e de aninhamento** de funções anónimas dentro de funções anónimas. Com 4 ou 5 pedidos em cadeia, ou com tratamento de erros em cada nível, o código torna-se rapidamente ilegível — este problema é conhecido informalmente como *callback hell* (nota adicional: este termo e a explicação do porquê de ser um problema não estavam explícitos nos slides, mas é essencial para perceber a motivação das Promises a seguir).

fetch e o objeto Request/Response

A API `fetch` é a alternativa moderna ao `XMLHttpRequest` para obter recursos por HTTP, e já **devolve uma Promise** nativamente (o que a torna o ponto de partida natural para o encadeamento com Promises).

```
fetch(url, {  
  method: 'POST',  
  headers: {  
    'Content-type': 'application/x-www-form-urlencoded; charset=UTF-8'  
  },  
  body: 'de=zp&para'  
})  
  .then(response => response.json())  
  .then(process)  
  .catch(console.log);
```

`fetch` é invocado com dois argumentos: o **URL** (obrigatório) e um objeto de configuração `Request` (opcional). Retorna uma **promessa de um Response**.

Definição

Objeto Request (configuração do pedido). Passado como segundo argumento de `fetch`, controla os detalhes do pedido HTTP:

- `method` — *string* com o método HTTP.
- `headers` — objeto com os cabeçalhos HTTP a enviar.
- `body` — corpo da mensagem HTTP.

```
fetch('http://some.site.org/authenticate', {  
  method: 'POST',  
  headers: { 'Content-type': 'application/x-www-form-urlencoded; charset=UTF-8' },  
  body: 'nick=cave&pass=word'  
})  
.then(response => console.log(response))  
.catch(console.log);
```

Definição

Objeto Response (a resposta). É o valor com que a promessa devolvida por `fetch` resolve. Propriedades:

- `ok` — booleano: verdadeiro se o estado da resposta está entre 200 (inclusive) e 300 (exclusive).
- `status` — inteiro com o estado da resposta.
- `statusText` — *string* com a mensagem de estado.
- `headers` — cabeçalhos da resposta.
- `type` — `basic` ou `cors`.

Métodos (ambos retornam uma nova *Promise*, porque ler o corpo da resposta é em si uma operação assíncrona):

- `response.json()` — promessa que resolve com o corpo interpretado como JSON.
- `response.text()` — promessa que resolve com o corpo como texto.

```
fetch('http://some.site.org/resource')  
.then(function(response) {  
  if (response.ok) {  
    response.text().then(console.log);  
  } else  
    console.log('erro: ' + response.statusText);  
})  
.catch(console.log);
```

Promises

Definição

O que é uma Promise. Um objeto `Promise` representa o resultado (eventual) de uma operação assíncrona. É construído a partir de uma função que recebe duas **continuações** (também funções): `resolve` e `reject`. A função implementa a funcionalidade “prometida” e, no final, invoca `resolve` (sucesso) ou `reject` (falha) consoante o caso:

```
function vamosSairLogo(quando) {
  return new Promise(function(resolve, reject) {
    setTimeout(function() {
      if (aindaApetece()) resolve(); else reject();
    }, quando.getTime() - (new Date()).getTime());
  });
}
```

```
vamosSairLogo(new Date("..."))
  .then(() => irASitioAltamente())
  .catch(() => console.log('És sempre o mesmo :-('));
```

Uma instância de `Promise` serve de **contexto** para dois métodos principais:

- `.then(fn)` — regista a continuação a executar quando a promessa **resolve** (equivalente ao `resolve`).
- `.catch(fn)` — regista a continuação a executar quando a promessa é **rejeitada** (equivalente ao `reject`).

Além disso, há métodos **estáticos** de `Promise` úteis para encadeamento:

- `Promise.resolve(valor)` — cria uma promessa já resolvida com `valor`.
- `Promise.reject(erro)` — cria uma promessa já rejeitada com `erro`.

Encadear com `.then`. Como `.then()` **retorna sempre uma nova Promise**, é possível encadear vários `.then()` em sequência — cada um processa o resultado do anterior:

```
function status(response) {
  if (response.ok)
    return Promise.resolve(response);
  else
    return Promise.reject(new Error('Invalid status'));
}

fetch(url)
  .then(status)
  .then(response => response.json())
  .then(json => document.getElementById('out').innerHTML = json.result)
  .catch(console.log);
```

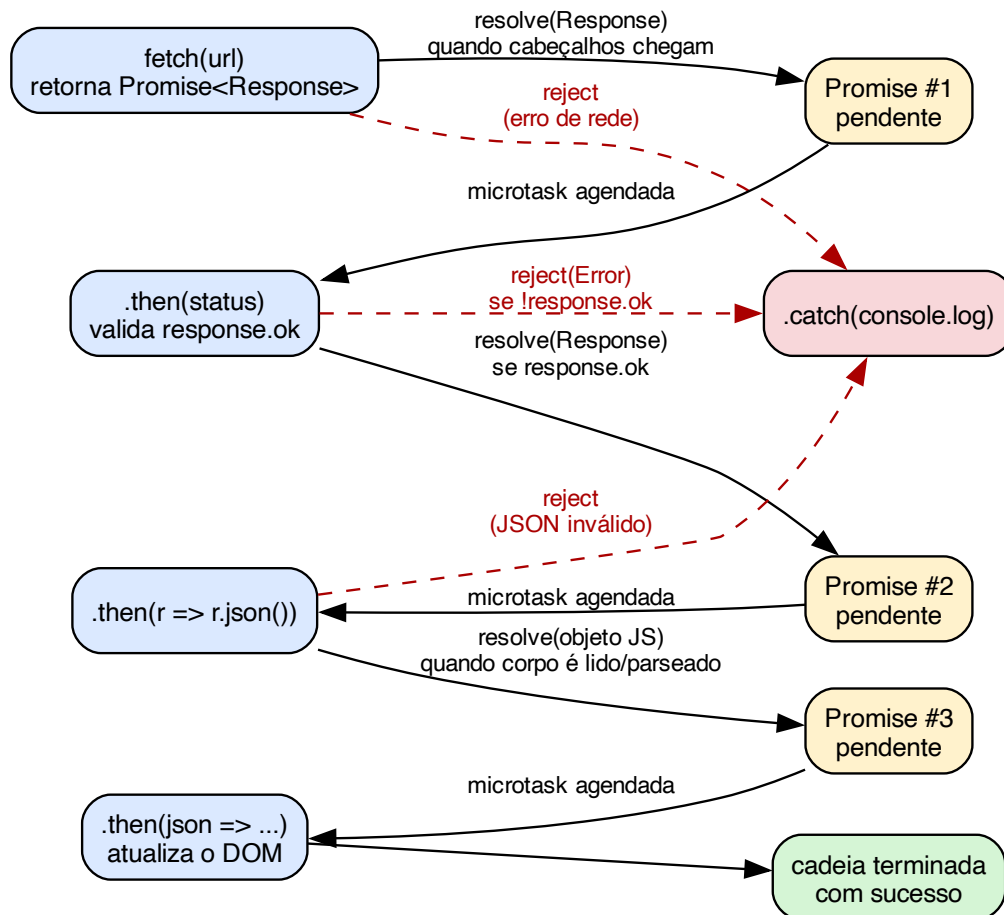
Isto funciona porque tanto `response.json()/response.text()` (métodos de `Response`) como `status()` (que devolve explicitamente `Promise.resolve/Promise.reject`) retornam `Promises` — e sempre que a função passada a `.then()` retorna uma `Promise`, o `.then()` seguinte só dispara quando **essa** promessa resolver (encadeamento “achatado”, em vez de aninhado).

Exemplo resolvido

Execução traçada de uma cadeia de 3 `.then()`. Considere-se exatamente a cadeia acima, e trace-se a ordem real de execução, distinguindo código **síncrono**, o **event loop**/tarefas de I/O, e a **fila de microtasks** (onde as continuações de Promises são colocadas):

```
console.log('A');
fetch(url)
  .then(status)
  .then(response => response.json())
  .then(json => console.log('D', json))
  .catch(console.log);
console.log('B');
```

1. **Síncrono.** `console.log('A')` corre imediatamente. `fetch(url)` é invocado: isto **inicia** o pedido de rede (uma operação assíncrona gerida pelo navegador, fora da *thread* de JS) e retorna **de imediato** uma **Promise** ainda pendente — o código não espera pela rede aqui. `.then(status)`, `.then(...)` e `.catch(...)` limitam-se a **registrar** as respectivas continuações nessa cadeia de promessas (ainda nada executa). `console.log('B')` corre a seguir, **antes** de qualquer `.then()`. Ordem até aqui: A, B.
 2. **Resposta de rede chega** (evento de I/O, gerido pelo navegador, ocorre “algures no futuro” depois de B). Isto resolve a primeira **Promise** (a de `fetch`) com o **Response**. Como há um `.then(status)` à espera, a chamada a `status(response)` é colocada na **fila de microtasks** — não corre imediatamente, mas assim que a *thread* principal ficar livre (o que já está, pois A e B terminaram).
 3. **Microtask 1** — `status(response)` corre: se `response.ok`, devolve `Promise.resolve(response)`. Como esta função devolveu uma **Promise** (mesmo já resolvida), a segunda **Promise** da cadeia só resolve **depois** dessa promessa interna resolver — o que agenda mais uma microtask para propagar o valor.
 4. **Microtask 2** — a segunda **Promise** resolve com `response`; dispara-se `response => response.json()`. Este método começa a ler e fazer *parse* do corpo da resposta (também assíncrono) e devolve **outra Promise** pendente — a terceira **Promise** da cadeia só resolve quando o *parse* do JSON terminar.
 5. **Corpo lido/parseado** (mais um passo assíncrono de I/O) → agenda-se nova microtask.
 6. **Microtask 3** — a terceira **Promise** resolve com o objeto JS `json`; dispara-se `json => console.log('D', json)`, que finalmente imprime D.
 7. Se, em qualquer ponto da cadeia, uma promessa for **rejeitada** (por exemplo, `status` devolve `Promise.reject` porque `!response.ok`, ou o `fetch` falha por erro de rede), a execução **salta diretamente** para o `.catch()` mais próximo a seguir na cadeia, ignorando os `.then()` intermédios.
- Ordem final observada na consola: A, B, ..., D — ou seja, **todo** o código dentro dos `.then()` corre sempre depois de todo o código síncrono que se segue à chamada a `fetch`, mesmo que a resposta de rede chegasse “instantaneamente” (as microtasks só correm depois de a *thread* principal esvaziar a pilha de chamadas síncrona atual).



Funções assíncronas: `async/await`

Definição

async. Uma função declarada com o modificador `async` retorna automaticamente uma `Promise`, mesmo que o seu corpo não construa uma explicitamente:

- Uma expressão `return valor;` é convertida, para efeitos da promessa devolvida, em `Promise.resolve(valor)`.
- Uma expressão `throw erro;` é convertida em `Promise.reject(erro)`.

```

async function status(response) {
  if (response.ok)
    return response;
  else
    throw new Error('Invalid status');
}

```

Isto significa que `status(response)` (assíncrona) é usável exatamente como a versão anterior que devolvia `Promise.resolve/ Promise.reject` explicitamente — `async` é açúcar sintático sobre a construção manual de Promises.

Definição

await. Só pode ser usado **dentro de uma função async**. Obriga a execução dessa função a **esperar pela terminação** de uma Promise, “desembrulhando” o seu valor:

```
async function message(response) {
  if (response.ok) {
    let message = await response.json();
    if ('error' in message)
      throw new Error(message.error);
    else
      return message;
  } else
    throw new Error('Invalid status');
}
```

await promessa devolve o valor passado a **resolve** (fica com esse valor, como se fosse uma chamada síncrona) ou **lança** (throw) o valor passado a **reject**, que pode ser apanhado com um **try/catch** normal.

Exemplo resolvido

Reescrever a cadeia de Promises em async/await — equivalência.

Cadeia em Promises (a mesma do exemplo anterior, resumida ao encadeamento de dois pedidos):

```
function combina(x) {
  return fetch(url1, { method: 'POST', body: JSON.stringify(x) })
    .then(response => response.json())
    .then(y => fetch(url2, { method: 'POST', body: JSON.stringify(y) }))
    .then(response => response.json());
}
combina(x).then(continuar);
```

Equivalente em async/await:

```
async function combina(x) {
  let respostaY = await fetch(url1, { method: 'POST', body: JSON.stringify(x) });
  let y = await respostaY.json();

  let respostaZ = await fetch(url2, { method: 'POST', body: JSON.stringify(y) });
  let z = await respostaZ.json();

  return z;
}
combina(x).then(continuar);
```

Porque são equivalentes: cada **await** corresponde a um ponto de sincronização que, na versão em Promises, seria um **.then()** — o **await** “pausa” a função **combina** (sem bloquear a *thread* principal: outras tarefas continuam a poder correr enquanto se espera) até a promessa de **fetch(url1, ...)** resolver, atribuindo o **Response** a **respostaY**; o segundo **await** faz o mesmo para o **.json()**; e assim sucessivamente. No final, como **combina** é **async**, o **return z** é automaticamente convertido em **Promise.resolve(z)** — por isso **combina(x)** continua a poder ser usado com **.then(continuar)** a partir de código que não seja ele próprio **async**. A diferença é puramente de **legibilidade**: o código com **await** lê-se de cima a baixo como uma sequência síncrona normal, sem aninhar funções de *callback*, apesar de continuar assíncrono por baixo.

Atenção

Nota adicional (não estava explícito nos slides): callback vs. Promise vs. async/await — a mesma coisa, três sintaxes. As três formas resolvem exatamente o mesmo problema (encadear operações assíncronas sem bloquear a *thread*) e são, em última análise, inter-convertíveis — **async/await** é açúcar sintático sobre Promises, e Promises internamente ainda usam *callbacks* (**resolve/reject**) para serem notificadas do resultado. A tabela resume as diferenças práticas:

Aspetto	Callbacks (XHR clássico)	Promises (<code>.then/.catch</code>)	<code>async/await</code>
Encadeamento	Aninhamento crescente (<i>callback hell</i>)	Achatado, uma cadeia de <code>.then()</code>	Leitura linear, como código síncrono
Tratamento de erros	Tem de ser verificado manualmente em cada nível	Um único <code>.catch()</code> apanha qualquer rejeição da cadeia	<code>try/catch</code> normal à volta dos <code>await</code>
Valor de retorno	Não há — usa-se sempre uma continuação explícita	A função devolve sempre uma Promise	A função async devolve sempre uma Promise (automaticamente)
Composição com outras promessas	Difícil (é preciso “prometizar” manualmente)	Natural — <code>.then()</code> devolve nova Promise	Natural — <code>await</code> funciona com qualquer Promise
Quando usar	Legado / APIs antigas (XHR pré-Promise)	Quando se compõe muitas operações de forma funcional (<code>Promise.all</code> , etc.)	Quando se quer o código mais legível possível para lógica sequencial

Na prática, hoje escreve-se **async/await** sempre que possível (mais legível), recorrendo a `.then()/catch()` explícitos apenas quando se quer compor várias promessas em paralelo (ex.: `Promise.all([...])`) ou quando não se está dentro de uma função **async**.

Comunicação em tempo real: SSE e WebSockets

O modelo pedido-resposta (mesmo assíncrono) tem uma limitação: é sempre o **cliente** que inicia a comunicação. Se o servidor precisar de notificar o cliente de algo que aconteceu (ex.: outro utilizador jogou uma jogada, chegou uma mensagem nova), o cliente não tem forma de “ser avisado” — só pode voltar a perguntar.

Definição**Alternativas anteriores ao HTML5.**

- **Polling** — o cliente faz pedidos regulares (ex.: a cada segundo) a perguntar se há novidades. A maioria dos pedidos é feita em vão (não há nada de novo), com custo de tempo de ligação e largura de banda desperdiçados.
- **Long polling** — usa-se a mesma ligação HTTP para repetir pedidos: o servidor só responde (parcialmente, ou quando tem algo) e o cliente imediatamente reabre outro pedido idêntico. Reduz o desperdício face ao *polling* simples, mas ainda assim reabre ligações constantemente.
- **Comet** — designação para o conjunto de técnicas (incluindo *long polling*) usadas antes do HTML5 para simular comunicação em tempo real sobre HTTP.

Com HTML5 surgem duas APIs nativas para isto, com um padrão de utilização semelhante: um **construtor** que cria um objeto de comunicação, **propriedades** com o estado da ligação, e **eventos** para receção (e, no caso de WebSockets, envio) de dados.

Server-Sent Events (SSE)

Definição

Construtor EventSource.

```
const eventSource = new EventSource(url);
eventSource.onmessage = function(event) {
    const data = JSON.parse(event.data);
};
// ...
eventSource.close();
```

- O construtor cria o objeto de comunicação usando o GET do HTTP sobre o url dado.
- Propriedades (só de leitura): **withCredentials** (booleano — enviar credenciais em pedidos CORS?), **url** (o URL usado na ligação), **readyState** (0 — CONNECTING; 1 — OPEN; 2 — CLOSED).

Receção de eventos. Os eventos do servidor são expostos como eventos do próprio objeto EventSource:

- **onopen/onstart** — a ligação foi estabelecida.
- **onmessage** — chegou uma nova mensagem (chamado **múltiplas vezes**, uma por mensagem recebida).
- **onerror** — ocorreu um erro na comunicação.

Os dados vêm sempre como **texto** na propriedade **event.data** do objeto de evento, e podem ser convertidos para um objeto com **JSON.parse()**.

Fecho. A ligação SSE é mantida **indefinidamente** por omissão; se quebrar, é **automaticamente re-estabelecida** pelo próprio navegador. Como o número de ligações simultâneas que um navegador mantém com o mesmo servidor é limitado, no final da utilização a ligação deve ser fechada explicitamente com **eventSource.close()**. A propriedade **readyState** reflete sempre o estado atual da ligação.

Definição

Como funciona sobre HTTP. SSE não é um protocolo à parte — é HTTP normal, mas com a ligação mantida aberta:

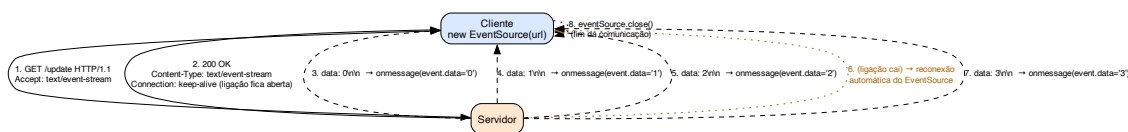
- O cliente faz um GET normal, indicando **Accept: text/event-stream**.
- O servidor responde 200 OK com **Content-Type: text/event-stream**, **Cache-Control: no-cache**, e mantém a ligação aberta (**Connection: keep-alive**).
- O corpo da resposta é composto por sucessivas **partes**, cada uma terminada por **duas quebras de linha** (uma linha vazia a separar). Cada parte tem uma ou mais linhas, cada uma começando por um campo — tipicamente **data:** (o conteúdo da mensagem) ou **id:** (identificador do evento, útil para retomar após reconexão). Uma mensagem **data:** pode ocupar várias linhas consecutivas e transportar, por exemplo, texto JSON.

data: 0

data: 1

data: 2

Cada uma destas partes dispara uma invocação de **onmessage** no cliente.



WebSockets

Definição

Construtor WebSocket.

```
const url = "ws://" + host + ":" + port + "/";
const websocket = new WebSocket(url);
websocket.onmessage = function(event) {
    const data = JSON.parse(event.data);
    // ...
};
// ...
websocket.send(JSON.stringify(data));
// ...
websocket.close();
```

- O URL usa um protocolo próprio: **ws** (não seguro) ou **wss** (equivalente a **https**, sobre TLS).
- Um segundo argumento opcional do construtor permite indicar **sub-protocolos** a negociar com o servidor.
- Propriedades: **protocol** (o sub-protocolo escolhido pelo servidor, resultado da negociação) e **readyState** (0 CONNECTING, 1 OPEN, 2 CLOSED — os mesmos valores que em SSE).

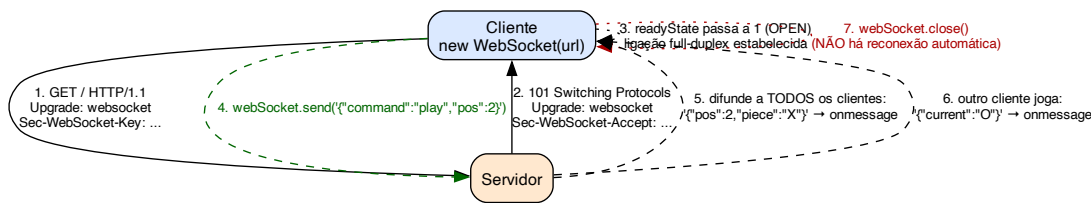
Receção e envio de dados. Tal como em SSE, os eventos disponíveis são **onopen/onstart**, **onmessage** (chamado múltiplas vezes) e **onerror**, e os dados chegam sempre em **event.data**, convertíveis com **JSON.parse()**. A diferença fundamental é que WebSocket também permite **enviar** dados a qualquer momento com **websocket.send(dados)** — isto não tem paralelo em SSE (que é só receção; para enviar dados ao servidor sobre uma ligação SSE seria preciso um **XHR/fetch** à parte). Os dados enviados são sempre convertidos para *string* (tipicamente com **JSON.stringify** a partir de um objeto JS). O conjunto de mensagens que cliente e servidor trocam entre si define, na prática, uma espécie de **protocolo aplicacional** próprio da aplicação.

Fecho. Ao contrário de SSE, se uma ligação WebSocket quebrar, **não** é automaticamente reiniciada — cabe à aplicação decidir se e como reconectar. No final da utilização, a ligação deve ser fechada com **websocket.close()**.

Definição

Como funciona sobre HTTP. A ligação WebSocket começa como um pedido HTTP normal que pede para ser “atualizado”:

- O cliente faz um **GET** com cabeçalhos **Upgrade: websocket**, **Connection: Upgrade**, **Sec-WebSocket-Version** e uma chave aleatória **Sec-WebSocket-Key**.
- Se o servidor aceitar, responde com o estado 101 **Switching Protocols** e cabeçalhos **Upgrade: websocket**, **Connection: Upgrade** e um **Sec-WebSocket-Accept** calculado a partir da chave do cliente (prova de que o servidor entende o protocolo WebSocket).
- A partir daqui, a ligação deixa de falar HTTP e passa a trocar **mensagens no protocolo WebSocket** (*frames* binários próprios), em ambos os sentidos — as ferramentas de desenvolvimento do navegador permitem inspecionar essas mensagens.



Frequente em exame

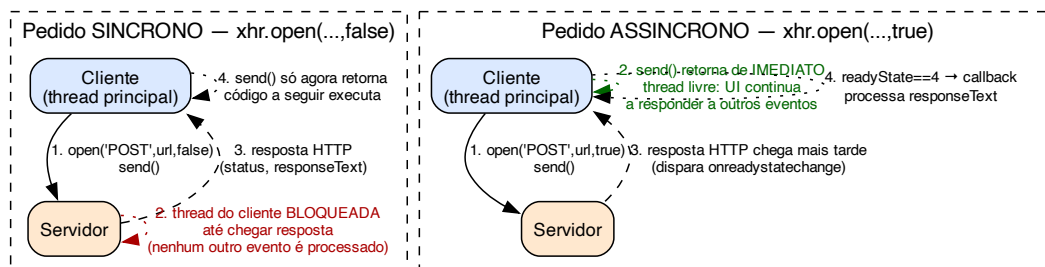
Tabela comparativa SSE vs. WebSocket.

Aspetto	Server-Sent Events	WebSocket
Direção da comunicação	Servidor → Cliente (unidirecional; <code>fetch</code> /XHR complementam para o sentido inverso)	Bidirecional (full-duplex)
Protocolo subjacente	Sobre HTTP normal (<code>text/event-stream</code>)	Protocolo próprio (<code>ws/wss</code>), negociado via <code>upgrade</code> HTTP
Reconexão em caso de falha	Automática, pelo navegador	Não automática — a aplicação tem de tratar disso
Duplex	<i>Half-duplex</i> (só um sentido ativo de cada vez, e mesmo assim só servidor→cliente)	<i>Full-duplex</i> (ambos os lados enviam a qualquer momento)
Tipos de dados	Apenas texto UTF-8	Texto UTF-8 e dados binários
Sub-protocolos	Usa sempre HTTP	Pode negociar protocolos aplicativos próprios
Complexidade do servidor	Mais simples	Mais complexa
Exemplo típico	<i>Ticker</i> de cotações, notificações	<i>Chat</i> , jogos multiutilizador

Quando usar cada um: se o fluxo de informação é essencialmente “o servidor avisa o cliente de algo” (notificações, atualizações de estado, *feeds*), SSE é mais simples e já resolve o problema, aproveitando infraestrutura HTTP existente (proxies, *load balancers*, etc.) e com reconexão automática “de graça”. Se há necessidade de comunicação nos **dois sentidos** com baixa latência (o cliente também precisa de enviar dados frequentemente, como jogadas num jogo ou mensagens de chat), WebSocket é a escolha adequada, ainda que exija mais trabalho do lado do servidor e não tenha reconexão automática embutida.

Atenção

Nota adicional (não estava explícito nos slides): SSE e `fetch`/XHR não são mutuamente exclusivos. Como SSE só recebe dados, uma aplicação típica que usa SSE para atualizações continua a precisar de `fetch`/XHR para **enviar** comandos ao servidor (é exatamente o padrão usado no exemplo do Contador, a seguir): os comandos vão por `fetch`/XHR (pedido-resposta pontual) e as atualizações de estado voltam por SSE (fluxo contínuo). Com WebSocket, pelo contrário, uma única ligação basta para os dois sentidos.



Exemplos completos

Contador distribuído (XHR/fetch + SSE)

Juntando os fragmentos das páginas de contexto, envio e atualização, reconstrói-se a classe `Counter` completa:

```
class Counter {
  constructor(parentId) {
    const parent = document.getElementById(parentId);
    const counter = document.createElement("div");
    const incr = document.createElement("div");
    const reset = document.createElement("div");
    this.display = document.createElement("div");

    parent.appendChild(counter);
    counter.appendChild(incr);
    counter.appendChild(reset);
    counter.appendChild(this.display);

    counter.className = "counter";
    incr.className = "button";
    reset.className = "button";
    this.display.className = "display";

    incr.innerText = " Incr ";
    reset.innerText = " Reset ";

    incr.onclick = this.increment.bind(this);
    reset.onclick = this.reset.bind(this);

    // Atualização por Server-Sent Events: o servidor é que manda
    // o valor corrente, o cliente nunca calcula o valor sozinho.
    const url = "http://" + host + ":" + port + "/update";
    const source = new EventSource(url);
    source.onmessage = event => this.display.innerText = event.data;
  }

  increment() { this.do("incr"); }
  reset() { this.do("reset"); }

  // Envio assíncrono de comandos por fetch - não lê resposta,
```

```
// a atualização do visor vem sempre por SSE.
do(command) {
  const url = "http://" + host + ":" + port + "/" + command;
  fetch(url, { method: 'POST' })
    .catch(console.log);
}
}
```

Pontos de desenho relevantes: a contagem **não** é mantida no cliente (ao contrário da versão original de um só utilizador) — é mantida no servidor, e o cliente limita-se a mostrar o que o servidor lhe manda. Há uma separação clara entre **comandos** (`incr/reset`, enviados por `fetch`, assíncronos, sem ler a resposta) e **atualizações** (recebidas por SSE, que atualizam o visor sempre que o valor mudar no servidor — mesmo que a mudança tenha sido causada por outro cliente). Note-se o uso de `=>` (*arrow function*) em `source.onmessage`, que garante que `this` dentro do *callback* continua a referir-se à instância de `Counter` (e não ao objeto `EventSource`).

Exemplo resolvido

Traçar uma interação completa no Contador. Suponha-se dois clientes, A e B, ambos com a página aberta e ligados via SSE, contador a começar em 5.

1. O utilizador de A clica em “Incr”. Dispara `incr.onclick`, que chama `this.increment()`, que chama `this.do('incr')`.
2. `do('incr')` chama `fetch(url, {method: 'POST'})` — isto **inicia** um pedido `POST /incr` ao servidor e retorna de imediato uma `Promise`; o `.catch(console.log)` só interviria se o pedido falhasse. O cliente A não faz mais nada com a resposta deste pedido.
3. O servidor recebe o `POST /incr`, incrementa o seu contador interno (de 5 para 6) e responde `200 OK` ao pedido `fetch` (que A ignora).
4. O servidor, tendo mudado de estado, **envia uma mensagem SSE** com o novo valor (`data: 6`) a **todos os clientes ligados à ligação /update** — não só a A, mas também a B.
5. No cliente A, o `onmessage` do `EventSource` dispara: `this.display.innerText = event.data` → o visor de A passa a mostrar 6.
6. No cliente B — que não clicou em nada — o `onmessage` do **seu próprio** `EventSource` (ligação SSE independente, mas todas a receber a mesma difusão do servidor) dispara **exatamente na mesma altura**, e o visor de B também passa a mostrar 6, sem que B tenha feito qualquer pedido.

Ou seja, do clique em A até à atualização em B, a informação percorre: clique → `fetch POST /incr` (A→servidor) → servidor atualiza estado → mensagem SSE `data: 6` difundida (servidor→A e servidor→B) → ambos os `onmessage` atualizam o respetivo DOM. Em nenhum momento o `fetch` de A leu diretamente o novo valor — o valor só chega pelo canal SSE, exatamente como chegaria a qualquer outro cliente.

Jogo do Galo multiutilizador (WebSocket)

Juntando os fragmentos de contexto, envio (`enviar/enviar2`), mensagens e receção, reconstrói-se a lógica de comunicação do jogo:

```
class TicTacToe {
  constructor(id) {
    const parent = document.getElementById(id);
    const board = document.createElement("div");
    board.className = "board";
    parent.appendChild(board);

    this.cells = new Array(9);

    for (let i = 0; i < 9; i++) {
```

```
const cell = document.createElement("div");
cell.className = "cell" + (((i + 1) % 3) == 0 ? " line" : "");
board.appendChild(cell);
cell.innerHTML = "&nbsp;";

// Cada célula, ao ser clicada, envia um comando "play"
// com a sua posição - não altera o tabuleiro localmente.
cell.onclick = () =>
    websocket.send(JSON.stringify({ command: "play", pos: i }));

this.cells[i] = cell;
}

const reset = document.createElement("div");
reset.className = 'button';
parent.appendChild(reset);
reset.onclick = function() {
    websocket.send(JSON.stringify({ command: "reset" }));
};
}
}

// Ligação WebSocket e recepção de mensagens do servidor
const url = "ws://" + host + ":" + port + "/";
const websocket = new WebSocket(url);

websocket.onmessage = function(event) {
    const message = JSON.parse(event.data);

    if ('board' in message)
        for (const b in message.board)
            ticTacToe.cells[b].innerHTML = message.board[b];

    if ('pos' in message && 'piece' in message)
        ticTacToe.cells[message.pos].innerHTML = message.piece;

    if ('current' in message)
        showCurrent.innerHTML = "Playing " + message.current;
};

const ticTacToe = new TicTacToe("base");
```

Definição

Protocolo de mensagens do jogo. As mensagens são simples objetos JSON, que definem, na prática, um pequeno protocolo aplicativo — típico de **arquiteturas orientadas a serviços**:

Pedidos do cliente para o servidor (comandos, com argumentos quando necessário):

```
{ "command": "play", "pos": 2 }  
{ "command": "reset" }
```

Respostas/difusões do servidor para o(s) cliente(s) (estado do jogo — completo ou apenas a alteração):

```
{ "board": [ "X", " ", "O" ], "current": "O" }  
{ "pos": 2, "piece": "X" }
```

O servidor mantém o estado (tabuleiro, de quem é a vez de jogar) e decide o que difundir: pode enviar o tabuleiro completo (por exemplo, a um cliente que acabou de se ligar) ou só a alteração (uma jogada individual, mais eficiente para clientes já sincronizados).

Exemplo resolvido

Traçar uma interação completa no Jogo do Galo. Dois clientes, A e B, ligados por WebSocket ao mesmo servidor; é a vez de “X” jogar (A joga com “X”).

1. O utilizador de A clica na célula de posição 2. O `onclick` dessa célula chama `websocket.send(JSON.stringify({command:"play", pos:2}))` — a mensagem é enviada de imediato pela ligação WebSocket de A; o tabuleiro de A **não** é alterado localmente neste passo (o clique não escreve X na célula diretamente).
2. O servidor recebe a mensagem `{"command":"play","pos":2}` na sua ligação com A. Valida a jogada (posição livre, é a vez de “X”), atualiza o seu estado interno do tabuleiro (posição 2 passa a “X”) e troca o jogador corrente para “O”.
3. O servidor **difunde** (envia pela ligação WebSocket de cada cliente ligado, incluindo A e B) uma mensagem com a jogada: `{"pos": 2, "piece": "X"}` — e, possivelmente na mesma ou noutra mensagem, `{"current": "O"}`.
4. Em A: `websocket.onmessage` dispara, `JSON.parse` decompõe a mensagem; como tem `pos` e `piece`, executa `ticTacToe.cells[2].innerHTML = "X"` — só **agora** a célula 2 do tabuleiro de A mostra “X” (o efeito visual do próprio clique só chega depois de ir e voltar do servidor). Se vier também `current`, atualiza o indicador “Playing O”.
5. Em B — que não clicou em nada — o `onmessage` da **sua** ligação WebSocket dispara com a mesma mensagem, e o tabuleiro de B atualiza-se da mesma forma, mostrando “X” na posição 2 e “Playing O” a seguir.
6. Se B clicar entretanto numa célula ocupada ou fora da sua vez, cabe ao **servidor** decidir se ignora o pedido ou responde com um erro — a validação da jogada não está no cliente, está centralizada no servidor (é o servidor quem decide o que difundir, os clientes limitam-se a refletir o que recebem).

Note-se o paralelo com o Contador: em ambos os casos, o clique no cliente **nunca** altera a interface diretamente — envia sempre um comando ao servidor, e é a mensagem de volta (SSE no Contador, WebSocket no Galo) que efetivamente atualiza o DOM, garantindo que todos os clientes ligados vêem sempre o mesmo estado.

Outras APIs do Browser

O HTML5 trouxe um conjunto de APIs de JavaScript que não estão diretamente relacionadas com HTTP nem com a estrutura da página, mas que são hoje essenciais em aplicações web: a **Web Storage** (persistência simples do lado do cliente) e o **Canvas** (desenho de gráficos 2D imperativo, ao nível do pixel).

Web Storage

Definição

A **Web Storage** é um mecanismo de persistência de dados do lado do cliente, pensado para superar as limitações dos *cookies*:

- Capacidade tipicamente entre **5MB e 10MB** por origem (muito mais do que os poucos KB de um cookie).
- **Não há transferência automática para o servidor** — ao contrário dos cookies, que são enviados em todos os pedidos HTTP, os dados ficam só no browser a menos que o código JS decida explicitamente enviá-los.
- Facilmente acessível a partir do código JS através de um objeto simples.
- Estrutura muito simples: um dicionário indexado por **chaves** (sempre *strings*), com **valores simples ou objetos** (desde que serializados).

Existem duas variantes, que partilham exatamente a mesma interface e diferem apenas na duração e âmbito dos dados guardados:

Definição — localStorage vs sessionStorage

	localStorage	sessionStorage
Duração	indefinida — sobrevive a fechar o browser	apenas durante a sessão do separador (<i>tab</i>)
Âmbito	partilhado por todos os separadores/janelas da mesma origem	privado a cada separador; um separador novo (mesmo que aberto para o mesmo site) começa vazio
Sobrevive a reload?	sim	sim
Sobrevive a fechar o separador?	sim	não — é apagado
API	idêntica	idêntica

Atenção

Nota adicional (não estava explícito nos slides): a diferença de âmbito entre **localStorage** e **sessionStorage** costuma confundir mais do que a diferença de duração. **sessionStorage** não é “guardado durante X minutos” — está ligado ao *ciclo de vida do separador do browser*: se o utilizador abrir a mesma página noutro separador, esse separador tem o seu **próprio sessionStorage**, vazio, mesmo estando a sessão “anterior” ainda aberta noutro separador. Já duplicar um separador (ou restaurar separadores fechados, em alguns browsers) pode copiar o **sessionStorage** existente, porque nesse caso o browser considera que é a “mesma” navegação. Isto é relevante nas práticas sempre que se guarda estado que não deve “vazar” entre separadores (ex: um carrinho de compras por sessão).

Antes de usar qualquer uma das duas, convém verificar se a API está disponível — nem todas as APIs do HTML5 são suportadas por todos os *browsers* (embora hoje em dia o Web Storage seja praticamente universal):

Exemplo resolvido

```
if (typeof(Storage) === 'undefined') {  
    // API Web Storage não disponível - usar alternativa  
    // (ex: cookies, ou simplesmente não persistir)  
}
```

A verificação testa se o *browser* define o tipo `Storage` de que `localStorage` e `sessionStorage` são instâncias. Se não existir, qualquer tentativa de usar `localStorage.setItem(...)` lançaria um erro.

Guardar e ler valores simples

Ambos os objetos expõem a mesma API, indexada por chave:

- `localStorage.setItem(chave, valor)` / `sessionStorage.setItem(chave, valor)` — guarda **valor** associado a **chave**. Ambos os argumentos são convertidos para *string* com `toString()` antes de serem guardados — mesmo que se passe um número, ele é guardado (e devolvido) como texto.
- `localStorage.getItem(chave)` / `sessionStorage.getItem(chave)` — devolve o valor (sempre como *string*, ou `null` se a chave não existir).

Exemplo resolvido

```
localStorage.setItem('name', 'fulano de tal');  
localStorage.setItem('age', 99);  
  
localStorage.getItem('name'); // 'fulano de tal'  
localStorage.getItem('age'); // '99' (nota: string, não número!)
```

O mesmo código com `sessionStorage` em vez de `localStorage` comporta-se de forma idêntica — a única diferença é *quando* os dados desaparecem (ver tabela acima).

Guardar objetos complexos

Como os valores são sempre convertidos para *string*, guardar diretamente um objeto ou *array* não funciona como seria de esperar (ficaria guardado o resultado de `Object.toString()`, tipicamente a inútil string `"[object Object]"`). É necessário **serializar** o objeto antes de o guardar, e **desserializar** ao recuperar — e a forma mais conveniente de o fazer é usar JSON:

Exemplo resolvido

```
// Guardar um objeto (ou array de objetos) complexo  
const data = [{ name: 'fulano', age: 99 }];  
  
localStorage.setItem('data', JSON.stringify(data));  
  
// ... mais tarde, mesmo depois de recarregar a página ...  
  
const data = JSON.parse(localStorage.getItem('data'));  
// data volta a ser o array de objetos original
```

`JSON.stringify(objeto)` converte a estrutura JS (objetos, *arrays*, números, *strings*, booleanos, `null`) numa *string* de texto no formato JSON. `JSON.parse(string)` faz o inverso, reconstruindo a estrutura JS a partir dessa *string*. Sem este par de conversões, qualquer objeto ou *array* guardado no Web Storage perde-se ou corrompe-se.

Atenção

Nota adicional (não estava explícito nos slides): funções, `undefined`, referências circulares e tipos especiais (como `Date` ou `Map`) **não sobrevivem** a `JSON.stringify/JSON.parse` sem cuidado extra — uma `Date` por exemplo é convertida numa *string* de texto e, ao ser recuperada com `JSON.parse`, continua a ser uma *string*, não volta a ser um objeto `Date` automaticamente (é preciso voltar a fazer `new Date(string)` manualmente). Isto é relevante para as práticas sempre que se guarde estado que inclua datas ou estruturas mais ricas do que texto/números.

Canvas — fundamentos

Definição

O **canvas** é um elemento HTML5 que fornece uma **tela em branco** para “pintar” gráficos de forma imperativa (ao contrário de SVG, que é declarativo). A “pintura” é feita invocando funções **JavaScript** sobre um **contexto gráfico**, controlado por propriedades com sintaxe inspirada no CSS (ex: cores, tipos de letra).

O elemento <canvas>

- O desenho fica associado à área ocupada pelo elemento `canvas`.
- É um elemento **vazio** — qualquer conteúdo colocado entre `<canvas>...</canvas>` é ignorado pelo motor de desenho (só é mostrado como alternativa em *browsers* muito antigos que não suportem `canvas`).
- As dimensões do bloco de desenho definem-se com os atributos HTML `width` e `height` do próprio elemento — **nunca** com propriedades CSS (ver mais abaixo, secção Dimensões).
- O atributo `id` é essencial, pois é a forma habitual de o JS obter uma referência ao elemento (`document.getElementById(id)`).

Exemplo resolvido

```
<canvas id="tela" width="300" height="200"></canvas>

const tela = document.getElementById('tela');
const gc = tela.getContext('2d');

gc.fillStyle = '#228';
gc.font = '48px serif';
gc.fillText('Olá mundo!', 100, 100);
```

Obter o contexto gráfico

O **contexto gráfico** (abreviado habitualmente `gc` nestes apontamentos) é o objeto que efetivamente desenha sobre o `canvas`. Obtém-se a partir do elemento com:

- `canvas.getContext(tipo)` — devolve o contexto do tipo pedido. O argumento é uma *string* que seleciona entre diferentes contextos possíveis:
 - `"2d"` — desenho a 2 dimensões, o **mais usual** e o único abordado nestes apontamentos.
 - `"webgl"` — desenho a 3 dimensões (fora do âmbito).

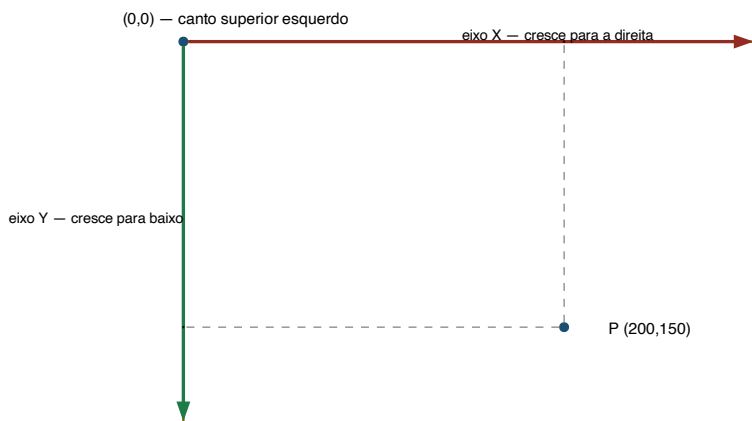
O contexto gráfico expõe:

- **Propriedades**, que controlam o estado do desenho (ex: `fillStyle`, `font`, `lineWidth`) — ficam ativas para todas as operações seguintes, até serem alteradas de novo.
- **Métodos**, que são as primitivas gráficas propriamente ditas (ex: `fillText()`, `fillRect()`, `arc()`).

Sistema de coordenadas

Atenção

Nota adicional (não estava explícito nos slides, mas é fundamental e frequentemente confuso para quem vem de matemática/geometria tradicional): no **canvas**, a origem (0,0) está no **canto superior esquerdo**, o eixo X cresce para a **direita** (como seria de esperar) mas o eixo Y cresce para **baixo** — o oposto do referencial cartesiano habitual. As coordenadas são valores em píxeis. Isto significa que um ângulo “positivo” numa rotação (ver secção de transformações) roda no sentido que visualmente parece o dos ponteiros do relógio, não o anti-horário matemático habitual.



canvas com 300x200 px (escala 1 unid. = 50 px) — origem sempre no canto superior esquerdo

Retângulos e as suas dimensões

Os métodos de criação de retângulos partilham sempre os mesmos quatro argumentos — **x**, **y** (canto superior esquerdo do retângulo) e **width**, **height** (largura e altura, para a direita/baixo a partir desse canto):

- `gc.strokeRect(x, y, width, height)` — traça apenas o contorno do retângulo.
- `gc.fillRect(x, y, width, height)` — preenche o interior do retângulo.
- `gc.clearRect(x, y, width, height)` — limpa (torna transparente) a área do retângulo, incluindo o que lá estivesse desenhado.

Exemplo resolvido

```
gc.strokeRect(40, 40, 100, 40); // só o contorno
gc.fillRect(40, 100, 100, 40); // preenchido
gc.fillRect(160, 40, 100, 100); // um quadrado maior preenchido...
gc.clearRect(180, 60, 60, 60); // ...com um "buraco" limpo por cima
```

O último `clearRect` recorta uma área retangular de 60×60 dentro do quadrado anterior, revelando o fundo do **canvas** (transparente por omissão) nesse retângulo interior.

O próprio elemento **canvas** reflete as suas dimensões nas propriedades `canvas.width` e `canvas.height` (em píxeis), úteis por exemplo para calcular o centro do **canvas**:

Exemplo resolvido

```
const xc = canvas.width / 2;
const yc = canvas.height / 2;

gc.fillRect(xc - 10, yc - 10, 20, 20); // um quadrado 20x20 centrado
```

Atenção

Nota adicional (não estava explícito nos slides): as dimensões devem ser definidas em **HTML** (atributos `width/height` do elemento), nunca em **CSS**. Se forem definidas em CSS, o `canvas` continua com a resolução interna por omissão (300×150) mas é **esticado** visualmente para o tamanho CSS pedido — o desenho fica distorcido/desfocado, porque o sistema de coordenadas interno não muda. Além disso, alterar as dimensões do `canvas` (mesmo para o mesmo valor que já tinha) provoca um *reset* total ao seu conteúdo e ao estado do contexto gráfico — é inclusivamente o truque usado para limpar o `canvas` por completo (ver secção de Animação).

Cores

As cores usadas em `fillStyle` (estilo de preenchimento) e `strokeStyle` (estilo de traço) são *strings*, nos mesmos formatos usados em CSS:

- nomes (ex: `'orange'`)
- RGB hexadecimal (ex: `'#228'`)
- RGB decimal, com ou sem transparência (`rgb(...)` / `rgba(...)`)
- HSL decimal, com ou sem transparência (`hsl(...)` / `hsla(...)`)

Exemplo resolvido

```
for (let hue = 0; hue < 240; hue++) {
  gc.fillStyle = 'hsl(' + hue + ',100%,50%)';
  gc.fillRect(30 + hue, 30, 1, 140);
}
gc.strokeStyle = '#228';
gc.strokeRect(30, 30, 240, 140);
```

Este ciclo desenha 240 retângulos de 1px de largura, cada um com uma matiz (*hue*) diferente do círculo cromático HSL, criando um gradiente de arco-íris — uma técnica comum para gerar gradientes de cor sem usar `CanvasGradient`.

Canvas — texto

Definição

O `canvas` permite desenhar texto de duas formas, controladas por propriedades semelhantes às do CSS:

- `gc.fillText(texto, x, y)` — **preenche** o interior das letras (o mais usado).
- `gc.strokeText(texto, x, y)` — **traça** apenas o contorno das letras.

Em ambos os métodos, `x` e `y` são a posição de referência do texto — que ponto exato do texto corresponde a essa posição depende do alinhamento (ver abaixo).

Exemplo resolvido

```
gc.font = '32pt Arial';
gc.fillStyle = 'orange';

gc.fillText('the quick', 30, 60);
gc.strokeText('brown fox', 30, 100);
gc.fillText('jumped over', 30, 140);
gc.strokeText('the lazy dog', 30, 180);
```

Propriedades de fonte

A propriedade `gc.font` caracteriza o tipo de letra e tem exatamente a mesma sintaxe da propriedade `font` do CSS, incluindo, por esta ordem (os campos são opcionais exceto tamanho e família):

- `font-style` — ex: `italic`
- `font-variant` — ex: `small-caps`
- `font-weight` — ex: `bold`
- `font-size` — ex: `32pt` (obrigatório)
- `font-family` — ex: `Arial` (obrigatório)

Exemplo resolvido

```
gc.font = '32pt Arial';
```

Alinhamento

A propriedade `gc.textAlign` define o alinhamento **horizontal** do texto face às coordenadas `x` fornecidas a `fillText`/`strokeText`. Valores possíveis:

- `"left"` — a coordenada `x` corresponde ao início do texto (**omissão**).
- `"center"` — a coordenada `x` corresponde ao centro do texto.
- `"right"` — a coordenada `x` corresponde ao fim do texto.
- `"start"` / `"end"` — equivalentes a `left`/`right` em textos da esquerda para a direita (LTR), mas invertem em textos RTL (ex: árabe).

Exemplo resolvido

```
gc.fillText('the quick', 120, 60);      // "left" (omissão): x é o início
gc.textAlign = 'left';
gc.fillText('brown fox', 120, 100);
gc.textAlign = 'center';
gc.fillText('jumped over', 120, 140);   // x é o centro do texto
gc.textAlign = 'right';
gc.fillText('the lazy dog', 120, 180);  // x é o fim do texto
```

Se se desenhar uma linha vertical em `x = 120`, os quatro textos ficam respetivamente: a começar na linha, a começar na linha (repetido), centrado na linha, e a acabar na linha.

Atenção

Nota adicional (não estava explícito nos slides de forma isolada, mas é usado nos exemplos do Relógio e das Transformações): existe também a propriedade `gc.textBaseline`, que faz o mesmo que `textAlign` mas no sentido **vertical** — define que ponto vertical do texto corresponde à coordenada `y`. Os valores mais usados são `"top"` (topo do texto), `"middle"` (meio, verticalmente centrado) e `"alphabetic"` (linha de base das letras, **valor por omissão** — importante porque letras como “g” ou “j” descem abaixo dela). O relógio usa `textBaseline = "middle"` e `textAlign = "center"` em conjunto precisamente para centrar os números do mostrador (horizontal e verticalmente) na posição calculada.

Medir texto

Antes de desenhar texto pode ser necessário saber a largura que vai ocupar — por exemplo para o alinhar ou desenhar algo a seguir a ele:

- `gc.measureText(texto)` — devolve um objeto `TextMetrics` com várias propriedades, mas a única relevante nestes apontamentos é `.width`, a largura (em píxeis) que o texto ocuparia com a fonte corrente.

Exemplo resolvido

```
gc.font = '32pt Arial';
gc.fillText('the quick', 40, 60);
```

```
const w = gc.measureText('the quick').width;
gc.strokeRect(40 + w, 40, 20, 20); // um quadrado logo a seguir ao texto
```

`measureText` não desenha nada — só calcula. É por isso preciso chamá-lo **antes** de decidir onde desenhar o que quer que dependa dessa largura (neste caso, o quadrado colocado imediatamente a seguir ao texto).

Canvas — caminhos e formas poligonais

Definição

Um **caminho** (*path*) é uma sequência de segmentos (retas ou curvas) que o **canvas** vai acumulando internamente, e que só produz efeito visível quando é explicitamente **traçado** ou **preenchido**. Definir um caminho, por si só, não desenha nada.

- `gc.beginPath()` — inicia um novo caminho vazio. Sem isto, novos segmentos ligar-se-iam ao último ponto do caminho anterior.
- `gc.moveTo(x, y)` — move a “caneta” para `(x,y)` **sem** desenhar, definindo o ponto inicial de um (sub)caminho.
- `gc.lineTo(x, y)` — acrescenta ao caminho um segmento de reta desde o último ponto até `(x,y)`.
- `gc.closePath()` — fecha o caminho ativo, ligando o **último** ponto ao **primeiro** com um segmento de reta. Afeta apenas o traçado da linha poligonal (`stroke()`) — não é preciso para preencher (`fill()`), que considera sempre o caminho fechado (ver abaixo).

Exemplo resolvido

```
gc.beginPath();
gc.moveTo(20, 20);
gc.lineTo(20, 140);
gc.lineTo(140, 20);
// (nada é desenhado até aqui)
gc.stroke();
```

Isto traça uma linha poligonal aberta com 2 segmentos: de (20,20) a (20,140), e de (20,140) a (140,20) — um “V” invertido, sem o terceiro lado do triângulo.

Traçar, preencher e fechar

- `gc.stroke()` — traça (desenha o contorno) do caminho **ativo**, com as propriedades de linha correntes (cor, largura, etc — ver secção seguinte).
- `gc.fill()` — preenche o **interior** do caminho ativo.

Exemplo resolvido

```
gc.beginPath();
gc.moveTo(20, 20);
gc.lineTo(20, 140);
gc.lineTo(140, 20);
gc.closePath();
gc.stroke();
```

Agora sim, com `closePath()` antes do `stroke()`, o terceiro segmento (de (140,20) de volta a (20,20)) é traçado, fechando o triângulo completo.

Atenção

Nota adicional (não estava explícito nos slides, mas é a fonte mais comum de confusão nesta secção): **fill()** **não precisa de closePath()** para preencher a figura completa — o `fill()` considera **sempre** o caminho como fechado (liga automaticamente o último ponto ao primeiro só para efeitos de cálculo da área a preencher), mesmo que visualmente o `stroke()` do mesmo caminho mostrasse uma linha aberta. Por isso:

```
gc.beginPath();
gc.moveTo(40, 40); gc.lineTo(40, 160); gc.lineTo(160, 40);
gc.fill();           // preenche o triângulo completo (3 lados)
// sem gc.stroke() aqui, nunca se veria o 3º lado como linha
```

A área preenchida é sempre o **interior delimitado pelo caminho fechado** — não é, por exemplo, “a área do polígono formado pelos pontos”, uma distinção que importa quando o caminho se **auto-intersecta** (um caminho em forma de estrela de 5 pontas desenhado sem levantar a caneta, por exemplo): o motor de desenho tem de decidir que regiões contar como “dentro” usando uma regra de preenchimento (**nonzero**, a regra por omissão, ou **evenodd**, que pode ser pedida com `gc.fill("evenodd")`) — algo que os slides não desenvolvem, mas que explica porque um caminho complexo pode preencher-se de forma inesperada.

Retângulos como caminhos

Além de `strokeRect`/`fillRect`/`clearRect` (atalhos vistos na secção de fundamentos), um retângulo pode também ser adicionado a um caminho com:

- `gc.rect(x, y, width, height)` — acrescenta ao caminho ativo um retângulo com os mesmos 4 argumentos (x,y,width,height) das restantes funções de retângulos. **Não** precisa de um `moveTo()` anterior

— ou fecha o último ponto aberto do caminho com uma reta até ao canto do retângulo, ou, se for o primeiro elemento do caminho, começa diretamente nele.

Exemplo resolvido

```
gc.beginPath();
gc.rect(40, 40, 100, 120);
gc.stroke();           // equivalente a strokeRect(40,40,100,120)

gc.beginPath();
gc.rect(160, 40, 100, 120);
gc.fill();             // equivalente a fillRect(160,40,100,120)
```

A diferença para `strokeRect`/`fillRect` é que, sendo um caminho, `rect()` pode ser **combinado** com outros segmentos no mesmo caminho (por exemplo um retângulo seguido de outras formas, tudo preenchido de uma vez).

Canvas — propriedades de linha

Definição

Um conjunto de propriedades do contexto gráfico controla o aspeto de qualquer linha traçada com `stroke()` (incluindo os contornos de retângulos e caminhos), até serem alteradas de novo:

- `gc.lineWidth` — largura da linha, em píxeis (valor numérico). Afeta todos os segmentos de todos os caminhos traçados depois de definida.
- `gc.lineJoin` — como são desenhadas as **junções** entre segmentos consecutivos de um mesmo caminho (só visível com `lineWidth` grande).
- `gc.lineCap` — como termina uma linha nas suas **extremidades** (início e fim de um segmento aberto).
- `gc.setLineDash(padrao)` — define um tracejado.

Junção de segmentos (`lineJoin`)

- `"miter"` — aguçado, os dois lados prolongam-se até se encontrarem num bico (**valor por omissão**).
- `"round"` — arredondado.
- `"bevel"` — cortado (chanfrado), como se o bico fosse serrado.

Exemplo resolvido

```
gc.lineWidth = 10;

gc.beginPath();
gc.moveTo(40,20); gc.lineTo(200,60); gc.lineTo(40,100);
gc.lineJoin = 'bevel';
gc.stroke();

gc.beginPath();
gc.moveTo(40,60); gc.lineTo(200,100); gc.lineTo(40,140);
gc.lineJoin = 'round';
gc.stroke();

gc.beginPath();
gc.moveTo(40,100); gc.lineTo(200,140); gc.lineTo(40,180);
gc.lineJoin = 'miter';
gc.stroke();
```

As três linhas em “V” desenharam o mesmo ângulo, mas com o vértice cortado (bevel), arredondado (round), ou aguçado (miter, o mais pontiagudo dos três) — o efeito só é visível porque `lineWidth = 10` é suficientemente grande.

Terminação de segmentos (lineCap)

- "butt" — cortado a direito, mesmo na ponta exata do segmento (**valor por omissão**).
- "round" — arredondado, a linha prolonga-se meia largura além da ponta com uma semicircunferência.
- "square" — quadrado, prolonga-se meia largura além da ponta mas com um corte reto (não arredondado).

Exemplo resolvido

```
gc.lineWidth = 20;

gc.beginPath(); gc.moveTo(40,60); gc.lineTo(200,60);
gc.lineCap = 'butt'; gc.stroke();

gc.beginPath(); gc.moveTo(40,100); gc.lineTo(200,100);
gc.lineCap = 'round'; gc.stroke();

gc.beginPath(); gc.moveTo(40,140); gc.lineTo(200,140);
gc.lineCap = 'square'; gc.stroke();
```

As três linhas horizontais têm exatamente o mesmo comprimento definido (`moveTo/lineTo`), mas a versão `round` e a `square` parecem ligeiramente mais compridas visualmente, porque “transbordam” meia largura de linha (10px, neste caso) para lá de cada ponta.

Linhas tracejadas

- `gc.setLineDash([cheio, vazio, ...])` — recebe uma lista (array) de tamanhos, alternando troços “cheios” (desenhados) e “vazios” (transparentes) ao longo da linha. Uma lista vazia `[]` termina o tracejado (volta a linha contínua).

Exemplo resolvido

```
gc.lineWidth = 10;

gc.beginPath(); gc.moveTo(30,30); gc.lineTo(210,30);
gc.setLineDash([1,1]); gc.stroke();    // ponteadado fino (1px cheio, 1px vazio)

gc.beginPath(); gc.moveTo(30,90); gc.lineTo(210,90);
gc.setLineDash([3,1,3]); gc.stroke();  // padrão com 3 elementos:
                                        // repete-se como [3,1,3,3,1,3,...]

gc.beginPath(); gc.moveTo(30,110); gc.lineTo(210,110);
gc.setLineDash([3,2,5,4]); gc.stroke(); // padrão irregular com 4 elementos
```

Quando a lista tem um número ímpar de elementos (como [3,1,3]), o motor de desenho **duplica-a** internamente ([3,1,3,3,1,3]) para conseguir alternar cheio/vazio de forma consistente.

Antisserrilhamento (*anti-aliasing*)**Atenção**

A largura de uma linha nem sempre corresponde exatamente ao valor definido em `lineWidth`, nem os seus contornos ficam perfeitamente nítidos. Isto deve-se à técnica de **antisserrilhamento**: o motor de desenho usa cores intermédias (semi-transparentes) na margem das linhas para “esconder” o aspeto em escada (*serrilhado*) que apareceria em linhas diagonais ou não alinhadas com a grelha de píxeis — mas em troca a linha fica ligeiramente mais larga e menos nítida do que o pretendido.

- Se as coordenadas usadas forem **inteiras**, é sempre aplicado antisserrilhamento, mesmo em linhas horizontais/verticais (porque uma linha fina de largura ímpar “cai entre” dois píxeis).
- Se as coordenadas estiverem deslocadas **meio píxel** (+0.5), esse efeito é evitado para linhas horizontais e verticais — é a técnica usada na grelha auxiliar destes próprios exemplos (`gc.moveTo(x+0.5, y+0.5)`), que faz com que as linhas de 1px fiquem perfeitamente nítidas.

Canvas — curvas**Definição**

Além de segmentos de reta (`lineTo`), um caminho pode incluir vários tipos de **segmentos curvos**: arcos de circunferência (como segmento, com `arcTo`, ou como forma completa, com `arc`), curvas quadráticas (1 ponto de controlo) e curvas de Bézier cúbicas (2 pontos de controlo).

Arco como segmento (`arcTo`)

- `gc.arcTo(xc, yc, x, y, raio)` — acrescenta ao caminho um segmento em arco.
 - `xc, yc` — coordenadas de um **ponto de controlo**.
 - `x, y` — coordenadas do **ponto final** do arco.
 - `raio` — raio da circunferência de que faz parte o arco.
- As coordenadas do **ponto inicial** são as últimas do caminho (definidas por um `moveTo`/`lineTo` anterior).
- A curva é **tangente** a duas retas: a que liga o ponto inicial ao ponto de controlo (no ponto inicial), e a que liga o ponto de controlo ao ponto final (no ponto final).

Exemplo resolvido

```
gc.beginPath();
gc.moveTo(20, 20);
gc.arcTo(140, 20, 140, 140, 100);
gc.stroke();
```

O caminho começa em (20,20). O ponto de controlo é (140,20) e o ponto final é (140,140), com raio 100. A curva sai de (20,20) tangente à reta horizontal até (140,20), e chega a (140,140) tangente à reta vertical que vem de (140,20) — na prática, um “canto arredondado” de raio 100 a ligar uma direção horizontal a uma vertical.

Curvas quadráticas (1 ponto de controlo)

- `gc.quadraticCurveTo(xc, yc, x, y)`
 - `xc, yc` — coordenadas do **ponto de controlo**.
 - `x, y` — coordenadas do **ponto final**.
- Ponto inicial: o último do caminho. Tangências: no ponto inicial, à reta que liga o ponto inicial ao ponto de controlo; no ponto final, à reta que liga o ponto final ao **mesmo** ponto de controlo (só há um).

Exemplo resolvido

```
gc.beginPath();
gc.moveTo(40, 100);
gc.quadraticCurveTo(200, 40, 260, 160);
gc.stroke();
```

Início em (40,100), ponto de controlo (200,40), fim em (260,160). O ponto de controlo “puxa” a curva na sua direção sem que ela lá passe: a curva vai de (40,100) na direção de (200,40), curva, e chega a (260,160) vinda também da direção de (200,40) — o ponto de controlo funciona como um íman único que atrai a curva inteira.

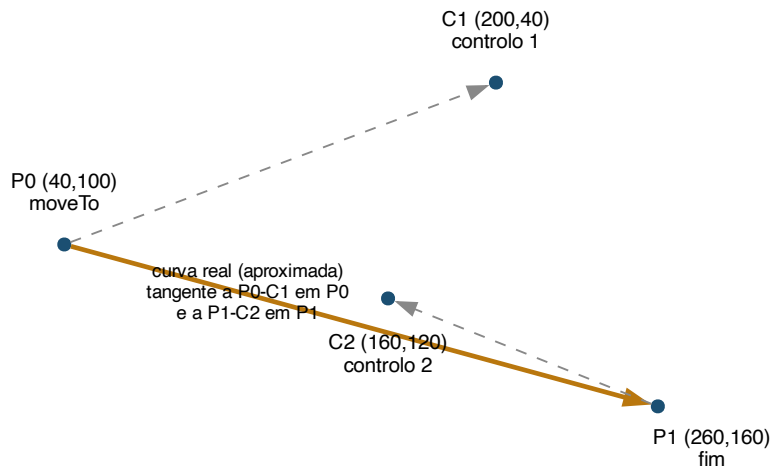
Curvas de Bézier cúbicas (2 pontos de controlo)

- `gc.bezierCurveTo(xc1, yc1, xc2, yc2, x, y)`
 - `xc1, yc1` — coordenadas do **ponto de controlo 1** (perto do início).
 - `xc2, yc2` — coordenadas do **ponto de controlo 2** (perto do fim).
 - `x, y` — coordenadas do **ponto final**.
- Ponto inicial: o último do caminho. Tangências: no ponto inicial, à reta que liga o ponto inicial ao **ponto de controlo 1**; no ponto final, à reta que liga o ponto final ao **ponto de controlo 2**.

Exemplo resolvido

```
gc.beginPath();
gc.moveTo(40, 100);
gc.bezierCurveTo(200, 40, 160, 120, 260, 160);
gc.stroke();
```

Ponto inicial P0=(40,100), controlo 1 C1=(200,40), controlo 2 C2=(160,120), ponto final P1=(260,160). Ao contrário da curva quadrática, aqui há **dois** pontos de controlo independentes: a curva sai de P0 na direção de C1 e chega a P1 vinda da direção de C2 — isto dá muito mais liberdade de forma (a curva pode até fazer uma espécie de “S”, como acontece nas velas do catavento mais à frente).



Arcos completos (circunferências e círculos)

- `gc.arc(x, y, raio, ângulo_inicial, ângulo_final, sentido)` — cria um caminho em forma de arco (ou circunferência completa). **Não** requer um ponto inicial (`moveTo`) — ou liga ao último ponto aberto do caminho, ou (com `beginPath()` antes) começa diretamente no arco.
 - `x, y` — coordenadas do **centro**.
 - `raio` — raio da circunferência que contém o arco.
 - `ângulo_inicial, ângulo_final` — em **radianos** (não graus!), com 0 a apontar para a direita (eixo X positivo) e a crescer no sentido dos ponteiros do relógio (porque Y cresce para baixo — ver nota do sistema de coordenadas).
 - `sentido` — booleano: `true` percorre o arco no sentido **anti-horário** (visualmente, dado o eixo Y invertido); `false` (ou omitido) no sentido horário.
- Um arco pode ser **traçado** (`stroke()` → dá uma circunferência ou arco de circunferência) ou **preenchido** (`fill()` → dá um círculo ou “fatia de tarte”).
- Um ângulo total de `2*Math.PI` radianos dá a circunferência completa.

Exemplo resolvido

```
gc.beginPath();
gc.arc(80, 80, 40, 0, 2*Math.PI, true);
gc.stroke();      // circunferência completa de raio 40 centrada em (80,80)

gc.beginPath();
gc.arc(180, 80, 40, 0, Math.PI, true);
gc.fill();        // meia-lua (metade "de cima", por causa do true)

gc.beginPath();
gc.arc(180, 100, 40, 0, Math.PI, false);
gc.fill();        // meia-lua (metade "de baixo", por causa do false)
```

Os dois últimos exemplos mostram como o argumento `sentido` decide qual das duas metades da circunferência é desenhada quando o arco vai só de 0 a `Math.PI` (meia volta) — a outra metade fica “por percorrer”.

Canvas — animação

Definição

Uma animação em **canvas** não é mais do que a execução **repetida** de código JS que redesenha o conteúdo, frame a frame, rápido o suficiente para o olho humano perceber movimento contínuo. Cada **frame** deve ser totalmente redesenhado — o **canvas** não tem noção de “objetos” que se possam simplesmente mover, é sempre a mesma tela em branco pintada de novo.

Base de tempo

Existem duas formas de temporizar a repetição:

- `id = setInterval(função, ms)` — chama *função* repetidamente, de *ms* em *ms* milissegundos, até ser cancelado.
- `id = setTimeout(função, ms) / clearTimeout(id)` — chama *função* **uma vez**, passados *ms* milissegundos. Chamar `setTimeout` de novo **dentro** da própria função (chamada recursiva) permite controlar o tempo até ao próximo frame de forma dinâmica (por exemplo, ajustar consoante o tempo que o frame anterior demorou a desenhar).
- `window.requestAnimationFrame(função)` — pede ao *browser* para chamar *função* mesmo antes do próximo repintar do ecrã. Chamado **recursivamente** dentro da própria função, cria uma animação. É a forma preferida para animações visuais porque:
 - o intervalo **ideal** entre frames depende do dispositivo, da carga do sistema, etc. — e é o *browser*, não o programador, que sabe qual é;
 - sincroniza-se com a taxa de atualização real do ecrã, evitando desenhar frames que nunca chegam a ser mostrados;
 - pausa automaticamente em separadores não visíveis, poupando bateria/CPU.

Exemplo resolvido

```
class Watch {
  constructor(canvasId) {
    this.base = document.getElementById(canvasId);
    this.gc = this.base.getContext('2d');
    // ...
    this.show();
    setInterval(this.show.bind(this), 1000); // um frame por segundo
  }
}

class WindMill {
  play() {
    this.base.width = this.base.width; // reset ao canvas (ver abaixo)
    this.show(this.gc);
    this.alpha -= WindMill.radialVelocity;
    requestAnimationFrame(this.play.bind(this)); // pede o próximo frame
  }
}
```

O relógio usa `setInterval` porque só precisa de se atualizar uma vez por segundo (a resolução do próprio relógio). O catavento usa `requestAnimationFrame` porque é uma animação contínua e suave, que quer correr o mais fluida possível.

Atenção

Nota adicional (não estava explícito nos slides): repare-se em `this.play.bind(this)` em vez de simplesmente `this.play`. Dentro de `play()`, `this` só continua a referir-se à instância se a função for chamada como método dessa instância (`objeto.play()`). Quando se passa `this.play` como referência de função para `setInterval` ou `requestAnimationFrame`, perde-se essa ligação — `bind(this)` cria uma nova função em que `this` fica permanentemente “colado” à instância correta, seja qual for a forma como essa função venha a ser chamada.

Redesenhar o canvas a cada frame

Como cada frame tem de ser **totalmente** redesenhado, é preciso limpar o conteúdo anterior no início de cada frame. Há duas formas:

- `canvas.width = canvas.width` — redefinir (mesmo que para o mesmo valor) qualquer propriedade de dimensão do `canvas` força um **reset completo**: limpa todo o conteúdo e repõe o estado do contexto gráfico (incluindo qualquer transformação, cor, etc. deixada de um frame anterior). É a forma mais simples e seguramente “limpa” de recomeçar um frame.
- `gc.clearRect(0, 0, canvas.width, canvas.height)` — limpa apenas o conteúdo visual da área indicada, mas **não** repõe as transformações nem outras propriedades do contexto gráfico. Só é seguro usar esta forma se o `gc` já estiver com as coordenadas/propriedades iniciais no início de cada frame (por exemplo, porque se teve o cuidado de usar `save()/restore()` — ver secção de transformações).

Exemplo resolvido

Três frames consecutivos de uma animação simples de rotação (como o catavento), ilustrando o padrão típico de cada `play()`:

Frame 1 (`alpha = 0`): `canvas.width = canvas.width` limpa tudo; o catavento é desenhado com as pás na posição de ângulo 0; `alpha` passa a `-Math.PI/100` (~ -0.0314 rad).

Frame 2 (`alpha  $\sim -0.0314$`): reset de novo ao `canvas`; o catavento é redesenhado do zero, mas agora com todas as pás rodadas ligeiramente ($\sim 1.8^\circ$) em relação ao frame 1; `alpha` passa a ~ -0.0628 .

Frame 3 (`alpha  $\sim -0.0628$`): o mesmo processo — reset, redesenhar com as pás rodadas mais $\sim 1.8^\circ$ em relação ao frame 2 ($\sim 3.6^\circ$ em relação ao frame 1); `alpha` passa a ~ -0.0942 .

Exemplo resolvido

A sequência de frames é sempre pedida com `requestAnimationFrame`, nunca com um ciclo `for/while` a correr sem parar: um ciclo infinito bloquearia a *thread* de JS do *browser*, impedindo-o de repintar o ecrã, processar eventos ou responder a interações — degradando por completo a experiência de utilização (UX). É por isso que a “recursão” via `requestAnimationFrame/setTimeout` (que devolve o controlo ao *browser* entre cada frame) é a única forma correta de fazer animação no `canvas`.

Canvas — embeber e eventos

Definição

O elemento `canvas` está bem integrado no *browser*, interagindo bem com HTML, CSS e JavaScript à sua volta (ao contrário de tecnologias mais antigas como *applets* ou *plugins*, que eram “caixas negras” isoladas do resto da página).

Sobrepor a outro conteúdo

- O `canvas` é **transparente** por omissão (o que não for explicitamente pintado deixa ver o que estiver por trás).

- Usando o posicionamento do CSS (`position: absolute` dentro de um contentor `position: relative`), pode ser sobreposto a outro conteúdo HTML e usado para desenhar por cima dele — por exemplo, para desenhar anotações, seleções ou destaques sobre um bloco de texto.

Exemplo resolvido

```
<div style="position: relative;">
  <div style="position: absolute; top: 0; left: 0;
    font: 20px Arial; width: 350px; height: 250px;
    line-height: 22px;">
    Lorem ipsum dolor sit amet, consectetur...
  </div>
  <canvas id="tela" width="350" height="250"
    style="position: absolute; top: 0; left: 0;"></canvas>
</div>
```

O texto e o `canvas` ocupam exatamente a mesma área (ambos `position: absolute; top:0; left:0` dentro do mesmo contentor `position: relative`), ficando o `canvas` sobreposto, transparente, pronto para desenhar por cima do texto sem o esconder.

Eventos no canvas

O `canvas` recebe eventos do rato como qualquer outro elemento — os *handlers* atuam sobre ele normalmente:

Exemplo resolvido

```
let start = null;

canvas.onmousedown = function(e) {
  start = relativeCoords(e);
};

canvas.onmousemove = function(e) {
  if (start !== null) {
    const p = relativeCoords(e);
    canvas.width = canvas.width; // reset + limpa
    lines(); // redesenha o fundo (linhas guia)
    gc.fillStyle = 'rgba(255,165,0,0.6)';
    gc.fillRect(start.x, (start.y / 20) * 19, p.x - start.x, 22);
  }
};

canvas.onmouseup = function(e) {
  start = null;
};
```

Este é o padrão clássico de um “arrastar para selecionar”: `mousedown` grava o ponto inicial, `mousemove` (enquanto o botão está premido, i.e. `start !== null`) redesenha continuamente um retângulo semi-transparente entre o ponto inicial e a posição atual do rato, e `mouseup` termina a seleção.

Converter coordenadas de evento para coordenadas do canvas

Atenção

Nota adicional (não estava explícito nos slides como conceito à parte, mas é essencial e é a causa mais comum de “o desenho aparece no sítio errado”): as coordenadas de um evento de rato (`event.clientX/event.clientY`) são relativas ao **ecrã/janela**, mas as coordenadas usadas pelo **canvas** (`fillRect`, `arc`, etc.) são relativas ao **próprio elemento**. É por isso sempre necessário converter umas nas outras antes de desenhar em resposta a um evento.

Exemplo resolvido

```
function relativeCoords(event) {  
  const bounds = event.target.getBoundingClientRect();  
  const x = event.clientX - bounds.left;  
  const y = event.clientY - bounds.top;  
  return { x, y };  
}
```

`getBoundingClientRect()` devolve a posição e dimensões do **canvas** relativas à janela (`.left`, `.top`, `.width`, `.height`, etc.). Subtrair `bounds.left` a `event.clientX` (e o mesmo para `y/top`) traduz a posição do evento — que veio em coordenadas de janela — para coordenadas relativas ao canto superior esquerdo do **canvas**, exatamente o referencial que `fillRect`, `arc`, etc. esperam.

Exemplo numérico: se o **canvas** estiver colocado a 50px do topo e 20px da esquerda da janela (`bounds = {left: 20, top: 50, ...}`), e o clique ocorrer no ponto de ecrã (120, 90), então: $x = 120 - 20 = 100$, $y = 90 - 50 = 40 \rightarrow$ o ponto (100,40) é a posição correta a usar em `gc.fillRect(100, 40, ...)` ou semelhante, mesmo que o **canvas** esteja deslocado ou a página tenha *scroll*.

Canvas — transformações

Definição

As transformações alteram o **sistema de coordenadas** usado pelas operações de desenho seguintes — não desenharam nada por si próprias. O **canvas** mantém internamente uma matriz de transformação (CTM — *current transformation matrix*), e cada transformação **compõe-se** com a que já lá estava, em vez de a substituir.

Translação

- `gc.translate(deltaX, deltaY)` — desloca a origem do referencial `deltaX` para a direita e `deltaY` para baixo. Tudo o que for desenhado depois usa esta nova origem.
- As translações (como todas as transformações) são **composicionais**: aplicar `translate(a,b)` seguido de `translate(c,d)` equivale a um único `translate(a+c, b+d)`.

Exemplo resolvido

```
sample('teste 1');           // desenhado na origem corrente
gc.translate(0, 60);
sample('teste 2');           // 60px mais abaixo
gc.translate(140, 0);
sample('teste 3');           // + 140px à direita = translate(140,60) acumulado
gc.translate(0, 60);
sample('teste 4');           // + 60px abaixo = translate(140,120) acumulado
gc.translate(-140, 0);
sample('teste 5');           // - 140px à esquerda = translate(0,120) acumulado
```

Cada `sample(...)` desenha sempre “na origem” do referencial corrente — é o referencial que se vai deslocando, não as coordenadas passadas a `sample`.

Rotação

- `gc.rotate(ângulo)` — roda o referencial `ângulo` radianos. Ângulos positivos rodam no sentido que, no ecrã, parece o dos ponteiros do relógio (consequência do eixo Y invertido); ângulos negativos, no sentido anti-horário aparente. Não há limitação a múltiplos de `Math.PI/4` — qualquer ângulo é válido.
- Tal como a translação, rotações **compõem-se**: `rotate(a)` seguido de `rotate(b)` equivale a `rotate(a+b)`.

Exemplo resolvido

```
sample('teste 1');
gc.translate(140, 120);      // desloca a origem - este será o centro de rotação
sample('teste 2');
gc.rotate(-Math.PI/2);      // roda 90° em torno da NOVA origem (140,120)
sample('teste 3');
gc.rotate(19/12*Math.PI);    // acumula com a rotação anterior
sample('teste 5');
```

Como a translação foi feita **antes** da rotação, é o ponto (140,120) — e não o (0,0) original do `canvas` — que serve de centro de rotação para tudo o que é desenhado depois. Isto ilustra a regra geral: cada transformação seguinte atua sempre sobre o referencial **já alterado** pelas anteriores.

Escala (homotetias)

- `gc.scale(razãoX, razãoY)` — escala o referencial pelos fatores dados em cada eixo. Um fator 1 não altera nada; 2 duplica; 0.5 reduz a metade; um fator **negativo** espelha o eixo correspondente.

Exemplo resolvido

```
sample('teste 1');
gc.translate(0, 40); gc.scale(2, 2);
sample('teste 2');           // desenhado ao dobro do tamanho
gc.translate(60, -20); gc.scale(0.5, 0.5);
sample('teste 3');           // a translação de (60,-20) já é medida
                             // no referencial 2x - desloca-se o dobro
                             // em pixels reais; a escala volta a 1x (2*0.5)
gc.translate(-120, 120); gc.scale(1, 0.5);
sample('teste 4');
gc.translate(240, 0); gc.scale(-0.5, 2);
sample('teste 5');           // eixo X espelhado (fator negativo)
```

O ponto crucial (e fácil de esquecer) é que, depois de um `scale(2,2)`, **qualquer translação seguinte também é escalada** — `translate(60,-20)` desloca na realidade 120px/-40px no ecrã, porque o referencial em que essa translação é interpretada já está ampliado 2×.

Transformações genéricas

Para além dos três atalhos anteriores, existem dois métodos que aplicam diretamente uma matriz de transformação 2D:

- `gc.transform(a, b, c, d, e, f)` — compõe a matriz dada com a transformação já existente (como `translate/rotate/scale`, que são todos casos particulares deste método mais genérico).
- `gc.setTransform(a, b, c, d, e, f)` — **substitui** a transformação corrente pela matriz dada (não compõe — é como fazer *reset* e depois aplicar).

Os seis parâmetros são as entradas da matriz de transformação 2D:

- `a` — escalamento horizontal
- `b` — torção (*shear*) horizontal
- `c` — torção vertical
- `d` — escalamento vertical
- `e` — deslocamento horizontal
- `f` — deslocamento vertical

Exemplo resolvido

```
gc.transform(1, Math.tan(Math.PI/6), 0, 1, 0, 0);
```

Esta matriz mantém as escalas a 1 (`a=1`, `d=1`) e o deslocamento a 0 (`e=0`, `f=0`), mas introduz uma torção horizontal de `tan(30°)` — o efeito visual é uma inclinação (itálico geométrico) do que for desenhado a seguir, como se o texto fosse “empurrado” para o lado à medida que desce.

A ordem importa

Atenção

Como cada transformação atua sobre o referencial **já alterado** pelas anteriores, `translate` seguido de `rotate` produz um resultado **diferente** de `rotate` seguido de `translate` — mesmo que sejam exatamente os mesmos dois comandos, com os mesmos argumentos, só que por ordem trocada.

Exemplo resolvido

Situação A — `translate(100,0)` depois `rotate(90°)`, e a seguir desenha-se um ponto na coordenada local (50,0):

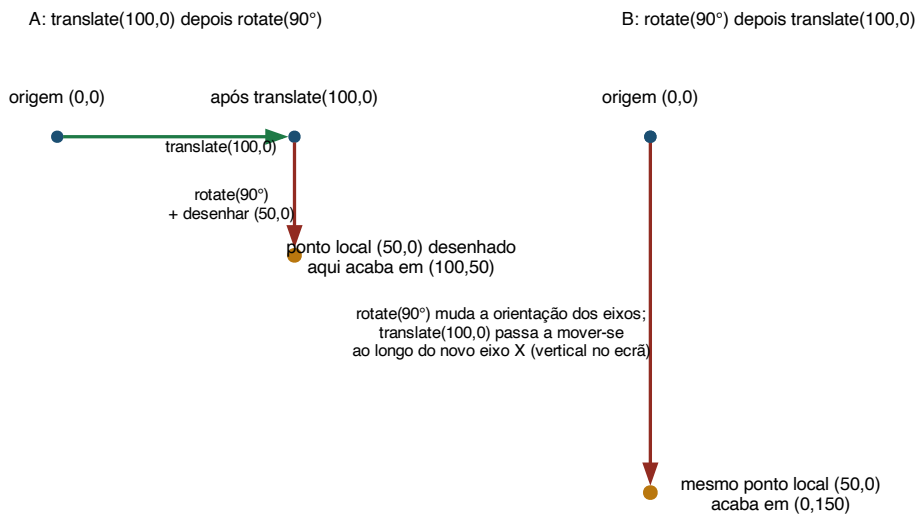
1. `translate(100,0)` move a origem do referencial para o que, no `canvas` original, é o ponto (100,0).
2. `rotate(Math.PI/2)` roda esse referencial (já deslocado) 90°: o eixo X local passa a apontar para onde antes apontava o eixo Y (para baixo).
3. Desenhar em (50,0) local significa “50 unidades ao longo do eixo X *já rodado*”, isto é, 50 unidades para baixo a partir da nova origem (100,0).
4. Posição final no `canvas` original: (100, 50).

Exemplo resolvido

Situação B — `rotate(90°)` depois `translate(100,0)`, com o mesmo ponto local (50,0):

1. `rotate(Math.PI/2)` roda o referencial **antes** de o deslocar: a origem continua em (0,0), mas o eixo X local já aponta para baixo.
2. `translate(100,0)` desloca a origem 100 unidades ao longo do eixo X **já rodado** — ou seja, 100 unidades **para baixo** no `canvas` original, chegando a (0,100).
3. Desenhar em (50,0) local soma mais 50 unidades nesse mesmo eixo X local (que continua a apontar para baixo): mais 50 para baixo.
4. Posição final no `canvas` original: (0, 150).

Duas sequências com os mesmos comandos e os mesmos argumentos, aplicadas ao mesmo ponto local (50,0), terminam em posições completamente diferentes do `canvas` — (100,50) contra (0,150) — só por troca da ordem.



Mesmo ponto local (50,0), mesmas transformações, ordem trocada → resultados diferentes
cada transformação seguinte atua no referencial já alterado pelas anteriores

Gravar e recuperar o estado — `save()/restore()`

Definição

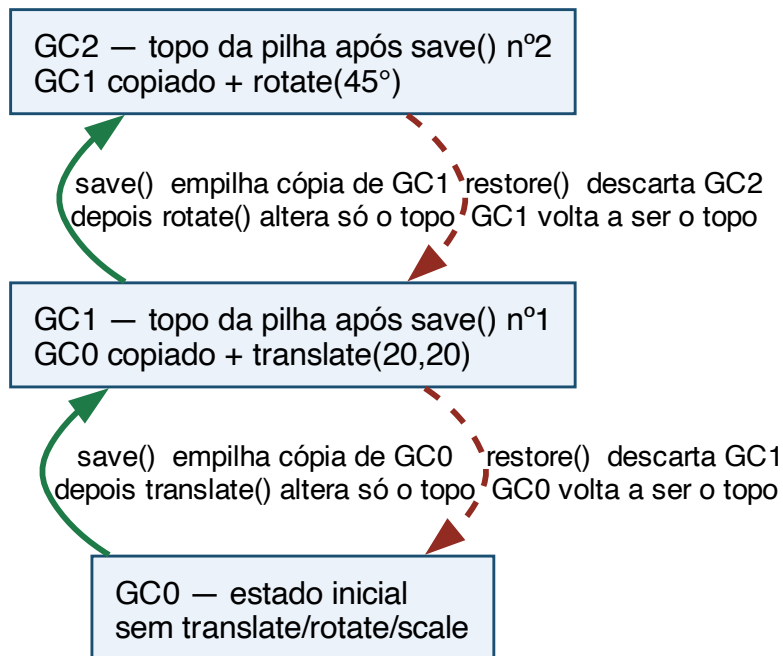
Transformações (e, em geral, qualquer propriedade do contexto gráfico) são difíceis de reverter manualmente — desfazer um `rotate(ângulo)` exigiria saber e aplicar `rotate(-ângulo)` na posição certa, o que se torna insustentável com várias transformações acumuladas. O `canvas` resolve isto com uma **pilha** de estados do contexto gráfico:

- `gc.save()` — empilha uma **cópia** do estado corrente do contexto gráfico (todas as propriedades: transformações, cores, largura de linha, fonte, etc.), sem alterar nada.
- `gc.restore()` — remove o estado no topo da pilha e repõe-o como corrente, **descartando** todas as alterações feitas desde o `save()` correspondente.
- Um `restore()` desfaz sempre o `save()` mais recente ainda não desfeito (comportamento de pilha, LIFO) — vários pares `save()/restore()` podem estar aninhados.

Exemplo resolvido

```
sample('teste 1');
gc.save();
gc.translate(0, 40);
sample('teste 2');    // desenhado 40px abaixo
gc.restore();         // desfaz o translate(0,40) - volta ao estado de "teste 1"
gc.translate(120, 0);
sample('teste 3');    // desenhado 120px à direita da posição ORIGINAL
                    // (não a partir de onde "teste 2" tinha ficado)
```

Sem o `save()/restore()`, o `translate(120,0)` acumular-se-ia com o `translate(0,40)` anterior, e “teste 3” apareceria deslocado nos dois eixos. Com `restore()`, o efeito da translação usada só para “teste 2” é completamente anulado antes de “teste 3” ser desenhado.



Frequente em exame

O padrão `save()` → alterar → desenhar → `restore()` é extremamente comum sempre que se quer aplicar uma transformação (ou mudar uma cor, fonte, etc.) **só** para uma parte do desenho, sem afetar o que vem a seguir — é usado exaustivamente nos dois exemplos completos seguintes (Relógio e Catavento), tipicamente uma vez por cada elemento repetido (cada marca do mostrador, cada pá do catavento).

Exemplos completos

Relógio

O relógio é uma classe `Watch` que junta transformações, posicionamento de texto e execução temporizada. Reconstituindo o código a partir dos fragmentos (campos, construtor, mostrador, ponteiro(s)):

```
class Watch {
  static numberSize = 14;
  static largeMarkSize = 10;
  static smallMarkSize = 5;

  constructor(canvasId) {
    this.base = document.getElementById(canvasId);
    this.gc = this.base.getContext('2d');
    this.center = { x: this.base.width / 2, y: this.base.height / 2 };
    this.length = Math.min(this.center.x, this.center.y);

    this.show();
    setInterval(this.show.bind(this), 1000);
  }
}
```

```
}

show() {
  const gc = this.gc;
  const now = new Date();

  this.base.width = this.base.width;          // reset ao canvas

  gc.translate(this.center.x, this.center.y); // origem no centro do mostrador
  gc.textAlign = 'center';
  gc.textBaseline = 'middle';
  gc.font = Watch.numberSize + 'pt Arial bold';

  this.face(gc);

  gc.strokeStyle = '#228';
  this.hand(gc, now.getHours() + now.getMinutes()/60, 12, this.length/2, 3);
  this.hand(gc, now.getMinutes(), 60, this.length - Watch.numberSize, 2);
  this.hand(gc, now.getSeconds(), 60, this.length - Watch.numberSize, 1);
}

face(gc) {
  gc.strokeStyle = '#228';
  gc.fillStyle = '#228';

  for (let minutes = 1; minutes <= 60; minutes++) {
    const angle = minutes * Math.PI/30 - Math.PI/2; // 0 rad aponta p/ direita
    const isHour = (minutes % 5) === 0;
    const len = this.length - Watch.numberSize - Watch.largeMarkSize;

    if (isHour) {
      const hour = minutes / 5;
      gc.save();
      gc.fillStyle = 'orange';
      gc.fillText(hour, len*Math.cos(angle), len*Math.sin(angle));
      gc.restore();
    }

    gc.save();
    gc.rotate(angle);
    gc.moveTo(this.length, 0);
    gc.lineTo(this.length - (isHour ? Watch.largeMarkSize : Watch.smallMarkSize), 0);
    gc.stroke();
    gc.restore();
  }
}

hand(gc, value, max, length, width) {
  const angle = (2*(value % max)/max - 1) * Math.PI;
  gc.save();
  gc.rotate(angle);
  gc.moveTo(width/2, 0);
  gc.lineTo(0, length);
}
```

```

    gc.lineTo(-width/2, 0);
    gc.lineWidth = width;
    gc.stroke();
    gc.restore();
  }
}

const watch = new Watch('base');
```

Como cada peça se encaixa:

- **Construtor** — guarda as referências ao `canvas`/contexto, calcula o centro e o comprimento (o menor entre metade da largura e metade da altura, para o relógio caber sempre no `canvas`), desenha o estado inicial e agenda a atualização a cada segundo com `setInterval`.
- **show()** — em cada segundo: reset total ao `canvas` (`base.width = base.width`), depois `translate` para o centro do relógio (assim todo o resto do código desenha relativo ao centro, não ao canto superior esquerdo), configura alinhamento e tipo de letra do texto uma única vez, desenha o mostrador e os três ponteiros.
- **face()** — percorre os 60 minutos do mostrador. Para cada minuto calcula o ângulo (`minutes * PI/30`, porque uma volta completa de 60 minutos tem 2PI , logo cada minuto vale $2\text{PI}/60 = \text{PI}/30$; o $-\text{PI}/2$ desloca a posição “0 minutos” do eixo X (à direita) para o topo). A cada 5 minutos (`isHour`), desenha o número da hora (`fillText` com `save()/restore()` para poder mudar o `fillStyle` sem afetar o resto) e desenha uma marca maior; nos restantes minutos, uma marca menor. Cada marca é desenhada sempre na **horizontal**, `save()`, `rotate(angle)` e `restore()` — é mais simples desenhá-la sempre a mesma marca horizontal e rodar o referencial do que calcular as suas coordenadas rodadas manualmente.
- **hand(gc, value, max, length, width)** — desenha um ponteiro genérico, parametrizado pela posição atual (`value`, ex: minutos), pelo total de posições possíveis (`max`, ex: 60), pelo comprimento e pela largura da base. Calcula o ângulo como uma percentagem do total (`value % max`) / `max` multiplicada por 2PI , roda o referencial (guardando/recuperando o estado à volta), e desenha o ponteiro sempre na “vertical” local (de (`largura/2,0`) a (`0,comprimento`) a (`-largura/2,0`)) — de novo, é a rotação do referencial que o posiciona corretamente, não o cálculo manual de senos/cosenos.

Catavento (*WindMill*)

O catavento é uma classe `WindMill` que combina animação contínua (`requestAnimationFrame`) com curvas de Bézier para desenhá-lo. Reconstituindo o código a partir dos fragmentos (campos, construtor, `vara`, `vela`, `animar`):

```

class WindMill {
  static radialVelocity = Math.PI/100;

  constructor(canvasId) {
    this.base = document.getElementById(canvasId);
    this.gc = this.base.getContext('2d');
    this.gc.fillStyle = 'orange';
    this.center = { x: this.base.width/2, y: this.base.height/2 };
    this.length = Math.min(this.center.x, this.center.y);
    this.alpha = 0;

    this.play();
  }

  play() {
    this.base.width = this.base.width;    // reset ao canvas (novo frame)
```

```
this.show(this.gc);
this.alpha -= WindMill.radialVelocity; // gira um pouco mais a cada frame
requestAnimationFrame(this.play.bind(this));
}

show(gc) {
  this.stick(gc);

  for (let beta = this.alpha; beta < this.alpha + 2*Math.PI; beta += Math.PI/2)
    this.sail(gc, beta); // 4 pás, a PI/2 (90°) umas das outras

  this.centerCircle(gc, this.length/5, '#228');
  this.centerCircle(gc, this.length/9, 'orange');
  this.centerCircle(gc, this.length/30, '#228');
}

stick(gc) {
  gc.beginPath();
  gc.moveTo(this.center.x, 2*this.center.y); // base da vara (fundo do canvas)
  gc.lineTo(this.center.x, this.center.y); // até ao centro
  gc.strokeStyle = '#228';
  gc.lineWidth = 10;
  gc.stroke();
}

centerCircle(gc, length, color) {
  gc.beginPath();
  gc.arc(this.center.x, this.center.y, length, 0, 2*Math.PI);
  gc.fillStyle = color;
  gc.fill();
}

sail(gc, beta) {
  const center = this.center, length = this.length;

  const a = WindMill.inLine(center, beta, length);
  const b = WindMill.inLine(center, beta - Math.PI/2, length/2);
  const c1 = WindMill.inLine(a, beta - 3*Math.PI/4, length/2);
  const c2 = WindMill.inLine(b, beta, length/2);
  const c3 = WindMill.inLine(center, beta - Math.PI/4, length/2);

  // metade "exterior" da pá
  gc.beginPath();
  gc.moveTo(center.x, center.y);
  gc.lineTo(a.x, a.y);
  gc.bezierCurveTo(c1.x, c1.y, c2.x, c2.y, b.x, b.y);
  gc.closePath();
  gc.fillStyle = '#228';
  gc.fill();

  // metade "interior" da pá
  gc.beginPath();
  gc.moveTo(center.x, center.y);
```

```

    gc.lineTo(a.x, a.y);
    gc.bezierCurveTo(c1.x, c1.y, c3.x, c3.y, center.x, center.y);
    gc.fillStyle = 'orange';
    gc.fill();
  }

  static inLine(p, a, l) {
    return { x: p.x + l*Math.cos(a), y: p.y + l*Math.sin(a) };
  }
}

const windmill = new WindMill('base');

```

Como cada peça se encaixa:

- **Campos** — `radialVelocity` é um campo **estático** (a mesma para todas as instâncias); `base`, `gc`, `alpha` (ângulo corrente) e `center/length` são campos de **instância**, dependentes das dimensões concretas do `canvas` usado.
- **Construtor** — inicializa os campos a partir do `canvasId` e arranca logo a animação chamando `play()`.
- `play()` — o ciclo de animação: reset ao `canvas`, desenha o frame corrente (`show`), decrementa `alpha` (fazendo o catavento rodar ao longo do tempo) e pede o próximo frame com `requestAnimationFrame`.
- `stick()` (“vara”) — desenha uma linha grossa (`lineWidth = 10`) da base do `canvas` até ao centro, representando o poste do catavento.
- `sail(gc, beta)` (“vela”) — desenha **uma** pá, na direção angular `beta`, usando pontos calculados em **coordenadas polares** a partir do centro com `inLine(ponto, ângulo, distância)` (um método **estático**, porque não depende de nenhum campo da instância — só faz trigonometria a partir dos argumentos). Cada pá é composta por **duas** curvas de Bézier partilhando o segmento reto do centro até ao ponto `a` (a ponta da pá): uma metade vai de `a` a `b` (com controlos `c1,c2`, fechada de volta ao centro — a metade “de fora”, pintada de azul escuro), a outra vai de `a` de volta ao centro (com controlos `c1,c3` — a metade “de dentro”, pintada de laranja). É esta assimetria de cores entre as duas metades da mesma pá que dá o efeito visual de “torção” típico de um catavento.
- `show()` — chama `stick`, depois `sail` quatro vezes (`beta` de `Math.PI/2` em `Math.PI/2`, ou seja 4 pás a 90° umas das outras à volta do catavento), e por fim três círculos concêntricos decrescentes no centro (`centerCircle`), que escondem a junção das pás e dão o efeito de “eixo” do catavento. A **ordem** de desenho (vara → velas → círculos) é relevante: os círculos do eixo são desenhados por cima das pás, escondendo a sua junção no centro.

O Servidor: Node.js

Primeiro servidor Node (“Olá mundo”)

Node.js é um ambiente *open source* de execução de JavaScript **fora do browser** — permite correr JS na linha de comandos ou, o que nos interessa aqui, como **servidor web**. As suas características distintivas marcam tudo o que se segue nesta secção:

Definição

- **Multiplataforma** — corre em Linux, Windows, macOS, etc.;
- **Single threading** — o código JavaScript da aplicação corre sempre numa única thread (não há concorrência por threads como em Java ou C);
- **Non-blocking I/O** (*event driven*) — as operações de entrada/saída (ler um ficheiro, esperar por dados de rede) não bloqueiam essa thread; são despoletadas e o resultado é entregue mais tarde através de um **callback** ou evento;
- **Motor JavaScript V8** — o mesmo motor que o Google Chrome usa para executar JS, o que dá a Node o desempenho de um JIT moderno;
- **Gestão de pacotes** — via **npm**, que não é tema desta secção mas é o mecanismo habitual para instalar módulos de terceiros.

Atenção

Nota adicional (não estava explícito nos slides): o modelo *single threading + non-blocking I/O* é a razão de ser de **tudo** o que vem a seguir — callbacks, *streams* com eventos **data/end/error**, a preferência pelas versões assíncronas de **fs**. Numa linguagem com threads (Java, C), ler um ficheiro bloqueia a thread que o pede, mas outras threads continuam a correr. Em Node só há uma thread de JavaScript: se ela bloqueasse à espera de um disco ou de uma rede, **todo o processo** parava de responder a qualquer outro pedido. A solução se é pedir a operação de I/O e **devolver imediatamente o controlo ao event loop**; quando o sistema operativo (por trás, com threads internas geridas pelo Node, invisíveis ao programador) termina a operação, o Node agenda a execução do *callback* correspondente, na única thread de JS, na próxima oportunidade livre. É exactamente isto que o diagrama mais à frente (comparação com CGI) ilustra.

O exemplo mínimo — um servidor “Olá mundo” — já contém todos os elementos estruturais de um servidor Node:

```
const http = require('http');

const server = http.createServer(function (request, response) {
  response.writeHead(200, {'Content-Type': 'text/plain'});
  response.end('Olá mundo\n');
});

server.listen(8008);
```

- O módulo **http** — interno ao Node — é incluído com `require('http')`; é ele que implementa o protocolo HTTP e expõe o necessário para criar um servidor;
- `http.createServer(callback)` cria uma instância de servidor; o argumento é uma função que **processa o ciclo pedido-resposta**: é invocada uma vez por cada pedido recebido, com dois parâmetros — **request** (o pedido) e **response** (a resposta a construir) — que agregam os métodos e propriedades relativos a cada uma dessas duas fases da comunicação;
- O valor retornado por `createServer()` é a **instância do servidor**, que se usa para o **lançar**: `server.listen(porta)` inicia a escuta de pedidos na porta indicada e **fica indefinidamente à espera** (não há **return**/fim de programa — o processo mantém-se vivo enquanto o servidor escuta).

Exemplo resolvido

Lançar e testar o servidor. Este código está guardado, por exemplo, em `ola.js`. Lança-se da linha de comandos com o comando `node` (tem de estar previamente instalado), passando o ficheiro com o código:

```
$ node ola.js
```

O processo fica a correr indefinidamente (CTRL+C cancela a execução). Para o testar, abre-se **outro** terminal e pede-se o URL correspondente com `curl`:

```
$ curl 'http://localhost:8008/'
```

```
Olá mundo
```

```
$
```

Notar: `localhost` refere-se à própria máquina (onde o servidor está a correr); a porta (8008) é a que foi indicada em `server.listen(...)` no código; é conveniente rodear o URL por plicas ('...') para evitar que a *shell* interprete caracteres especiais (como ? ou &) antes de chegarem ao `curl`.

Estruturar código em Node (módulos)

Um programa Node cresce depressa para lá de um único ficheiro. O mecanismo de organização de código é o **sistema de módulos**: um módulo implementa funcionalidades usadas por outras partes da aplicação, mas mantém **restritas** ao módulo as definições que não interessa expor, e **partilha** apenas as que interessam.

Definição

O mecanismo de partilha assenta em duas peças:

- **Inclusão** de módulos na aplicação, com a função `require()`;
- **Declaração**, dentro do módulo, dos nomes a exportar, atribuindo-os a `module.exports`.

```
const http = require('http');           // módulo instalado/interno
require('./modulo.js');                  // módulo próprio
```

`require()` inclui módulos, e o seu **argumento** identifica qual:

- módulos **instalados** (internos ao Node, ou de terceiros via `npm`) são referidos pelo **nome** (ex: `"http"`);
- módulos **próprios** são referidos pelo **nome de caminho** do ficheiro (ex: `"./modulo.js"`).

O **valor retornado** por `require()` é o que o módulo exportou — `require()` funciona simultaneamente como “carregar o módulo” e “obter a sua interface pública”. A interface entre aplicação e módulo depende inteiramente de como o módulo foi definido; há várias formas de o fazer, com um compromisso entre **simplicidade** e **expressividade**:

Definição — Variáveis globais (evitar)

Sem qualquer mecanismo de exportação, módulo e aplicação podem partilhar **nomes globais** — mas *apenas* valores estáticos e **funções anônimas** atribuídas a um nome global, nunca **declarações** (`function f() {}`, `class C {}`, `let/const`) porque essas ficam encapsuladas no âmbito do módulo:

```
// modulo.js
f = function() { /* ... */ }
x = 1
// NÃO fica partilhado: function g() {...}; const y = 2;

// aplicacao.js
require('./modulo.js');
f();
x;
```

Deve **evitar-se** este estilo — potencia colisões de nomes entre módulos diferentes que partilhem o mesmo espaço global.

Exemplo resolvido — Função anónima partilhada

```
// modulo.js
module.exports = function() {
  /* ... */
}

// aplicacao.js
const f = require('./modulo.js');
f();
```

Usa-se `module.exports` para declarar a função exportada; `require()` retorna esse valor, que pode ser invocado como função. Não polui o espaço de nomes global, mas só permite partilhar **um único** valor por módulo.

Exemplo resolvido — Função nomeada partilhada

```
// modulo.js
module.exports.f = function() {
  /* ... */
};
module.exports.x = 1;

// aplicacao.js
const mod = require('./modulo.js');
mod.f();
mod.x;
```

Cada **propriedade** de `module.exports` fica acessível no valor retornado por `require()`. Permite exportar **múltiplos** nomes, mas não dá jeito para orientação a objetos.

Exemplo resolvido — Classe (construtor) anônima compartilhada

```
// modulo.js
module.exports = class {
  constructor(...) {
    this.c = ...;
  }
  m() { /* ... */ }
}

// aplicacao.js
const T = require('./modulo.js');
const t = new T();
t.m();
t.c;
```

Permite usar construtores e protótipos (orientação a objetos completa); o construtor é exportado com `module.exports`, e instâncias podem ser criadas na aplicação com `new`. Está limitado a **uma única classe** por módulo.

Exemplo resolvido — Classes nomeadas compartilhadas

```
// modulo.js
module.exports.Type1 = class {
  m() { /* ... */ }
};
module.exports.Type2 = class {
  m() { /* ... */ }
}

// aplicacao.js
const mod = require('./modulo.js');
const t1 = new mod.Type1();
t1.m();
const t2 = new mod.Type2();
t2.m();
```

Múltiplos construtores atribuídos a `module.exports` — vários **tipos** de objetos exportados por um único módulo. É o interface mais complexo entre aplicação e módulo, mas também o mais expressivo.

Frequente em exame

Resumo dos 5 estilos de exportação, por ordem crescente de expressividade e complexidade: **variáveis globais** (evitar) → **função anônima** (um só valor) → **função nomeada** (vários valores/funções) → **construtor anônimo** (uma classe, orientação a objetos) → **construtores nomeados** (várias classes). A escolha depende de quanto a funcionalidade do módulo precisa de estado encapsulado em instâncias (classes) versus ser só utilidades sem estado (funções).

Módulos internos do Node

Node distribui, para além do que o programador escreve, um conjunto de **módulos internos** (ver a API do Node para a lista completa). Para um servidor HTTP básico interessam sobretudo quatro:

Definição

- **http** — processamento de pedidos e geração de respostas (o protocolo HTTP propriamente dito);
- **url** — utilitários para resolução e análise de URLs;
- **stream** — interface abstrata para processamento de fluxos de dados;
- **fs** — interação com o sistema de ficheiros, baseado em POSIX.

O módulo http

O **http** é o módulo fundamental: implementa o protocolo HTTP a **baixo nível**, para permitir múltiplas utilizações — expõe os cabeçalhos como objetos JS já decompostos (sem o programador ter de fazer o *parsing* do texto do protocolo), processa o corpo do pedido como *stream* (ver abaixo), suporta mensagens com corpo grande (*chunked transfer*) e permite negociar outros protocolos sobre a mesma ligação (ex: *upgrade* para WebSockets, usado no exemplo do jogo do galo mais à frente).

O módulo expõe vários tipos de objetos — **Agent** (gestão de ligações), **Client** (fazer pedidos HTTP *como cliente*) — mas aqui interessa apenas o tipo **Server**, para processar pedidos recebidos.

Definição — Classe Server

```
const http = require('http');

const server = http.createServer(function (request, response) {
  response.writeHead(200, {'Content-Type': 'text/plain'});
  response.write('Método: '+request.method+'\n');
  response.write('URL: '+request.url+'\n');
  response.end();
});

server.listen(8008);
```

- `http.createServer()` cria a instância; o argumento é a função processadora do ciclo pedido-resposta;
- `server.listen(porta)` inicia a escuta de pedidos nessa porta.

Definição — Objeto request

Propriedades principais:

- `request.method` — o **método HTTP** do pedido ('GET', 'POST', ...);
- `request.url` — o **URL do pedido, relativo ao servidor** (não inclui protocolo nem *host*): `http://servidor.up.pt/hello?nome=valor` chega ao handler como `request.url === '/hello?nome=valor'`.

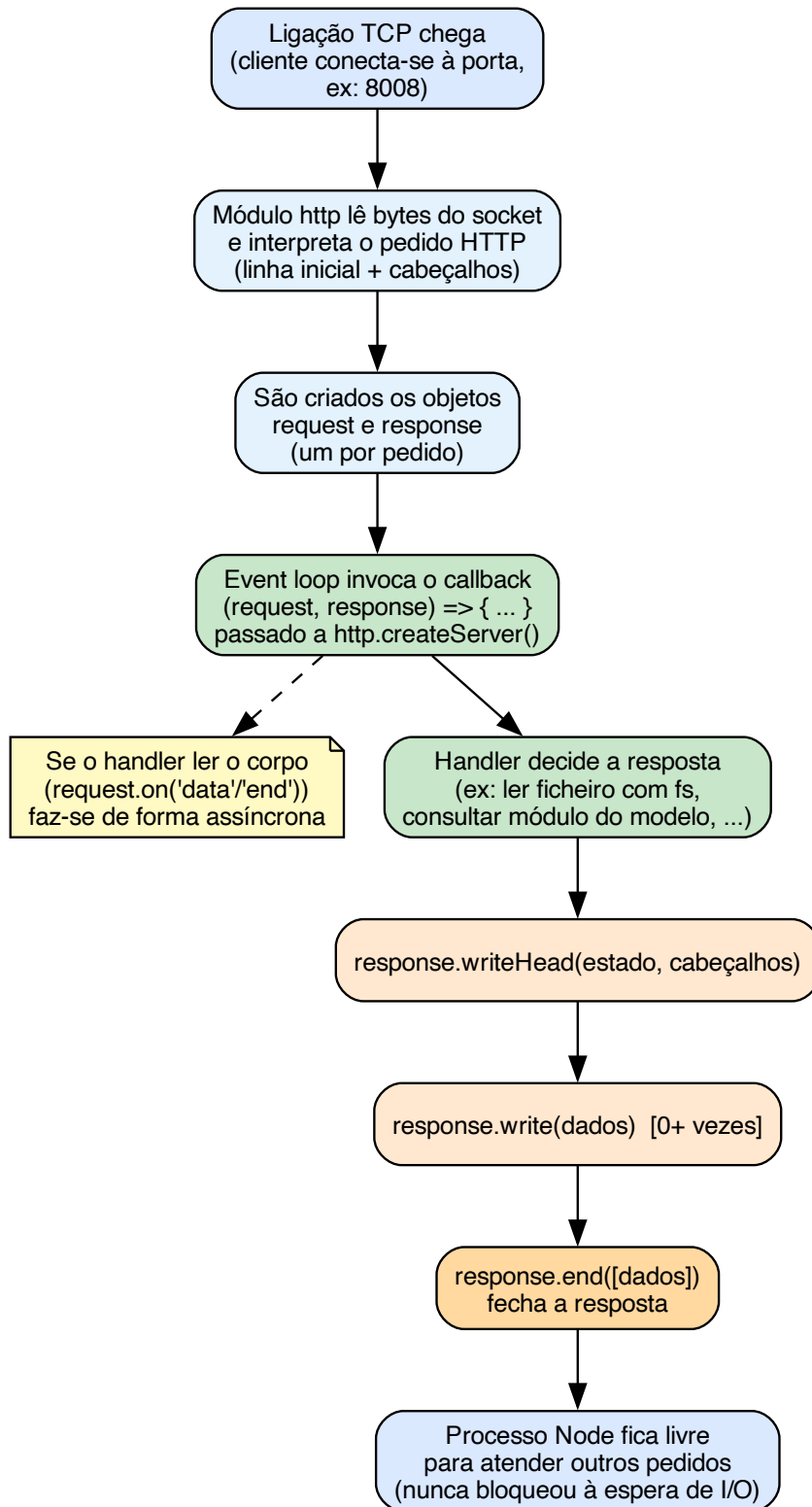
Além disso, `request` é também um **stream** (ver módulo **stream**, à frente) — é assim que se lê o corpo de um pedido POST.

Definição — Objeto response

Métodos principais:

- `response.writeHead(estado, cabeçalhos)` — cria o cabeçalho da resposta: o primeiro argumento é o **código de estado** (inteiro, ex: 200, 404); o segundo é um objeto com propriedades *nome/valor* que definem os cabeçalhos HTTP (ex: `{'Content-Type': 'text/plain'}`);
- `response.write(dados)` — escreve uma parte do **corpo** da resposta, como *string*; pode ser chamado várias vezes;
- `response.end([dados])` — **fecha** a comunicação; o argumento (uma *string*, opcional) é escrito como último pedaço do corpo antes de fechar.

O ciclo completo, desde a ligação TCP chegar ao socket até a resposta ser enviada e o processo Node ficar livre para o próximo pedido, é o seguinte:



O módulo url

Um URL tem sintaxe e componentes próprios — protocolo, servidor, porta, caminho e interrogação (*query-string*). O módulo `url` permite obter esses dados a partir de qualquer URL, incluindo os URLs **relativos** recebidos em `request.url`.

```
const url = require('url');

http.createServer(function (request, response) {
  const parsedUrl = url.parse(request.url, true);
  const pathname = parsedUrl.pathname;
  const query = parsedUrl.query;
  // ...
}).listen(8003);
```

- `url.parse(umUrl, true)` recebe um URL (tipicamente `request.url`) e retorna um **objeto** com os seus constituintes; o segundo argumento (`true`) pede que a *query-string* seja também decomposta num objeto (sem ele, `query` viria como a *string* crua, não decomposta);
- `parsedUrl.pathname` — o **nome de caminho** do URL, sempre a começar por `/`; serve tipicamente para **encaminhar** o pedido (`switch` sobre o seu valor);
- `parsedUrl.query` — reflete a *query-string*; é um objeto em que cada **propriedade** corresponde a um parâmetro do URL.

Exemplo resolvido

Decompor um URL completo. Considere-se o pedido a `http://exemplo.up.pt/hello?nome=Ana&idioma=pt`. O que chega ao servidor em `request.url` é apenas a parte relativa: `/hello?nome=Ana&idioma=pt`.

```
const parsedUrl = url.parse('/hello?nome=Ana&idioma=pt', true);
// parsedUrl.pathname === '/hello'
// parsedUrl.query === { nome: 'Ana', idioma: 'pt' }
// parsedUrl.query.nome === 'Ana'
// parsedUrl.query.idioma === 'pt'
```

Um handler pode então combinar as duas coisas — encaminhar por `pathname` e usar os parâmetros de `query`:

```
http.createServer(function (request, response) {
  const parsedUrl = url.parse(request.url, true);
  const query = parsedUrl.query;
  const name = query.name === undefined ? 'amig@' : query.name;
  let mensagem;

  switch (parsedUrl.pathname) {
    case '/hello':
      mensagem = 'Olá ' + name;
      break;
    case '/bye':
      mensagem = 'Adeus ' + name;
      break;
    default:
      mensagem = 'Podes repetir?';
  }
  // ... escrever mensagem na resposta
}).listen(8003);
```

Um pedido a `/hello?name=Rui` produz `pathname === '/hello'` e `query.name === 'Rui'`, logo `mensagem === 'Olá Rui'`. Um pedido a `/hello` (sem *query-string*) dá `query.name === undefined`, logo `name` cai no valor por omissão `'amig@'`.

O módulo stream

`stream` é um interface **abstrato** para trabalhar com fluxos de dados, usado por vários objetos internos do Node — nomeadamente `request`. É necessário sempre que é preciso **ler o corpo de um pedido** (tipicamente em POST), porque o corpo não chega todo de uma vez: chega aos bocados (*chunks*), à medida que a rede o vai entregando, de forma **assíncrona** e orientada a **eventos**.

Definição

Um objeto que suporta `Stream` expõe o método `on(evento, callback)`: o primeiro argumento é o **nome do evento**, o segundo é o *callback* invocado quando esse evento ocorre. `on()` **retorna o próprio objeto**, o que permite encadear várias chamadas (`stream.on(...).on(...).on(...)`).

Os três eventos relevantes aqui:

- **'data'** — ocorre sempre que um novo bloco de dados está disponível; o bloco é entregue ao *callback* como *string* ou *buffer* (um *buffer* fica convertido em *string* ao ser concatenado com `+=`);
- **'end'** — disparado quando a leitura **termina** (não há mais dados); é o momento correto para processar o conjunto de dados já acumulado;
- **'error'** — disparado se ocorrer um erro durante a leitura; pode simplesmente registar-se no *log*, ou gerar uma resposta HTTP de erro adequada (ex: estado 400).

Atenção

Nota adicional (não estava explícito nos slides): `fs.readFile()` (ver módulo `fs`, a seguir) usa também, internamente, um mecanismo de leitura por blocos — mas **não expõe** esse processo ao programador através dos eventos `data/end/error`; expõe apenas um único *callback* final (`err`, `data`) com o resultado completo. Os eventos `data/end/error` só são visíveis diretamente quando se trabalha com um objeto `Stream` “em bruto”, como é o caso de `request` (o corpo de um pedido HTTP) — ou, fora do que os slides cobrem, `fs.createReadStream()`. Vale a pena não confundir os dois estilos de API assíncrona que o Node oferece para o mesmo problema (ler dados aos poucos sem bloquear): *callback único* vs. *stream com eventos*.

Exemplo resolvido

Traçar a leitura do corpo de um pedido POST. O exemplo canônico dos slides é usar `request` como *stream* para ler um corpo JSON:

```
switch (request.method) {
  let body = '';
  case 'POST':
    request
      .on('data', (chunk) => { body += chunk; })
      .on('end', () => {
        try { query = JSON.parse(body); /* processar query */ }
        catch (err) { /* erros de JSON */ }
      })
      .on('error', (err) => { console.log(err.message); });
    break;
  // ...
}
```

Considere-se um pedido concreto: `POST /users` cujo corpo é o texto `{"nome":"Ana","idade":25}`, entregue pela rede em **dois** blocos (é normal o corpo não chegar de uma só vez):

1. **Bloco 1** chega → dispara `'data'` com `chunk === '{"nome":"Ana"}'` → o *callback* executa `body += chunk`, logo `body === '{"nome":"Ana"}'`;
2. **Bloco 2** chega → dispara `'data'` novamente, agora com `chunk === ', "idade":25}'` → `body` passa a `'{"nome":"Ana","idade":25}'`;
3. A ligação sinaliza que não há mais dados → dispara `'end'` → o *callback* de `'end'` corre `JSON.parse(body)`, que agora tem o JSON **completo** e válido, produzindo `query = {nome: 'Ana', idade: 25}`; o `try/catch` protege contra um corpo malformado (nesse caso o `catch` trataria o erro de *parsing*);
4. `'error'` não dispara neste caso (leitura correu sem falhas de rede); se a ligação fosse interrompida a meio, seria este *callback* a ser chamado, com o objeto de erro em `err`.

Note-se que o `case 'POST'` só entra no `switch` **uma vez**, mas os `.on(...)` registados **ficam pendentes**: o código não bloqueia à espera dos dados — regista os três *callbacks* e devolve o controlo ao event loop de imediato; os *callbacks* são invocados mais tarde, um de cada vez, à medida que os eventos ocorrem.

O módulo fs

`fs` (*file system*) trata da interação com o sistema de ficheiros — leitura e escrita de ficheiros, e obtenção de estatísticas sobre eles.

```
const fs = require('fs');
const fsp = require('fs').promises;
```

Há **duas formas de interação**: assíncrona (a preferida, com a possibilidade adicional de usar `Promises` através de `fsp`) e síncrona. O módulo tem muitas outras funções não cobertas aqui (listar diretorias, gerir ficheiros, criar *links* entre *inodes*).

Definição — Leitura assíncrona

```
fs.readFile('dados.txt', 'utf8', function (err, data) {
  if (!err) { /* processar data como string */ }
});

// equivalente com Promises
fsp.readFile('dados.txt', 'utf8').then((data) => { /* processar ... */ });
```

- Argumentos: nome de caminho do ficheiro, **codificação** (opcional — sem ela, os dados voltam como um *raw buffer*, não como *string*), e um *callback* (ou, no caso das **Promises**, o resultado chega ao argumento de `.then()`);
- **Callback**: invocado depois da leitura, com **dois** argumentos — (`err`, `data`); os dados só devem ser processados se **err não** estiver definido;
- **Promise**: os dados lidos chegam ao argumento de `then()`; eventuais erros chegam a `catch()`.

Quando se lê sem indicar codificação, os dados vêm como *buffer*; se o conteúdo for JSON, usa-se `JSON.parse(data.toString())` para o converter num objeto JS depois de garantidamente não haver erro:

```
fs.readFile('dados.json', function (err, data) {
  if (!err) {
    const dados = JSON.parse(data.toString());
    /* processar dados */
  }
});
```

Definição — Escrita assíncrona

```
fs.writeFile('dados.txt', dados_string, (err) => {
  if (err) { /* processa erro */ }
});

// equivalente com Promises
fsp.writeFile('dados.txt', dados).catch(() => { /* processa erro */ });
```

- Argumentos: nome de caminho, **dados a escrever** (como *string* ou *buffer* — `JSON.stringify(dados)` serializa um objeto JS para os poder escrever), e um *callback*/**Promise** que continua consoante o resultado;
- O *callback* é invocado depois da escrita com um eventual objeto de erro; o `.catch()` da **Promise** cumpre a mesma função.

Definição — Versões síncronas

```
let dados = fs.readFileSync('dados.txt');
// ...
fs.writeFileSync('dados.txt', dados);

// alternativa: async/await sobre as versões com Promise (continua não-bloqueante)
(async () => {
  let dados = await fsp.readFile('dados.txt');
  // ...
  await fsp.writeFile('dados.txt', dados);
})();
```

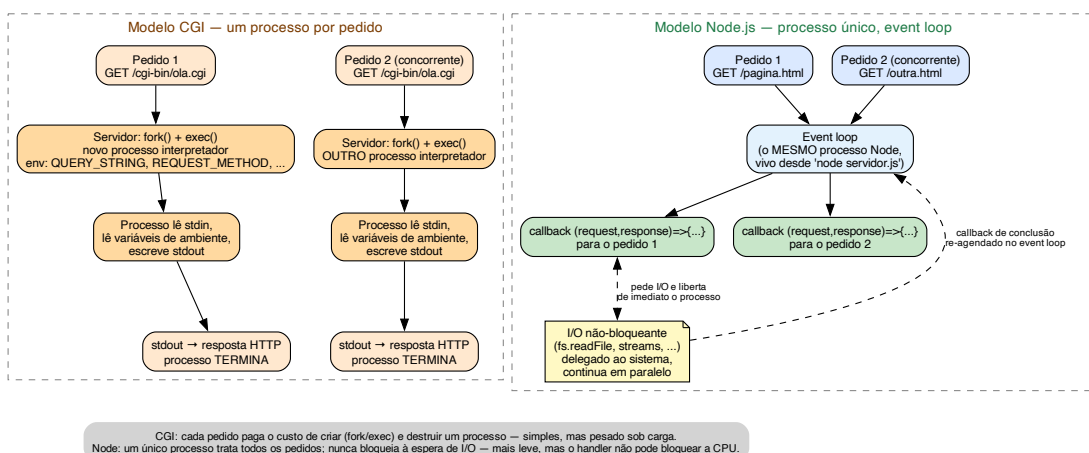
Existem versões **síncronas** de leitura e escrita (sufixo **Sync**), mas as versões **assíncronas** **devem ser preferidas** — uma leitura síncrona bloqueia a única thread de JavaScript, travando o servidor inteiro enquanto o disco não responde. Quando se quer um estilo de código sequencial *sem* bloquear, a alternativa correta é **async/await** sobre as versões com **Promise** (**fsp**), que continuam não-bloqueantes por trás.

Definição — Estatísticas de um ficheiro

```
fs.stat(pathname, (err, stats) => {
  if (err) throw err;
  console.log(pathname + ' tem ' + stats.size + ' bytes');
});
```

fs.stat() retorna (de forma assíncrona) uma instância de **Stat**, com propriedades como **size** (tamanho em bytes), **blocks** (tamanho em blocos), **birthtime** (momento de criação, um **Date**), **ctime/mtime/atime** (criação/modificação/acesso do *inode*), e métodos como **isFile()** e **isDirectory()**.

O contraste entre este modelo (um único processo, nunca bloqueado, callbacks/eventos para I/O) e o modelo **CGI** (estudado na secção seguinte, um processo novo por pedido) é uma das ideias mais importantes desta parte da disciplina:



Frequente em exame

A pergunta típica de exame por trás deste diagrama: **por que razão o Node não bloqueia a thread de JS a ler ficheiros ou a esperar por rede?** Resposta: porque as operações de I/O são delegadas (ao sistema operativo / *thread pool* interna do Node) de forma assíncrona — o código regista um *callback* e devolve logo o controlo ao *event loop*, que fica livre para atender outros pedidos entretanto; o *callback* só corre mais tarde, quando o resultado estiver pronto. O CGI resolve a concorrência de forma completamente diferente e mais simples de raciocinar (um processo do SO por pedido, isolado), mas paga o custo de criar e destruir um processo a cada pedido.

Exemplos completos de servidores

Os slides constroem três servidores completos por fragmentos incrementais. Reconstituem-se aqui, e traça-se o processamento de um pedido concreto em cada um.

Servidor de páginas estáticas

Um servidor HTTP básico que serve **dados estáticos** (ficheiros e diretorias do disco), respondendo apenas a GET sem *query string*, com diferentes estados de resposta (2XX, 3XX, 4XX, 5XX) consoante o caso.

Exemplo resolvido

Módulos e configuração.

```
// modulos.js (topo do servidor)
const http = require('http');
const path = require('path');
const url = require('url');
const fs = require('fs');
const conf = require('./conf.js');

// conf.js
module.exports.documentRoot = '/home/zp/public_html/about';
module.exports.defaultIndex = 'index.html';
module.exports.port = 8007;
module.exports.mediaTypes = {
  'txt': 'text/plain',
  'html': 'text/html',
  'css': 'text/css',
  'js': 'application/javascript',
  'png': 'image/png',
  'jpeg': 'image/jpeg',
  'jpg': 'image/jpeg',
};
```

A configuração (diretoria raiz dos documentos, nome do ficheiro índice de uma diretoria, porta de escuta, tipos de media por extensão) fica isolada num módulo próprio — num servidor real existem muitas mais destas opções.

Exemplo resolvido

Mapear caminhos e determinar o tipo de media.

```
function getPathname(request) {
  const purl = url.parse(request.url);
  let pathname = path.normalize(conf.documentRoot + purl.pathname);

  if (!pathname.startsWith(conf.documentRoot))
    pathname = null;

  return pathname;
}

function getMediaType(pathname) {
  const pos = pathname.lastIndexOf('.');
  let mediaType;

  if (pos !== -1)
    mediaType = conf.mediaTypes[pathname.substring(pos + 1)];
  if (mediaType === undefined)
    mediaType = 'text/plain';
  return mediaType;
}

function isText(mediaType) {
  return !mediaType.startsWith('image');
}
```

- `getPathname()` converte o caminho **relativo** do URL num caminho **absoluto** no disco, prefixando-o com `conf.documentRoot` e normalizando-o (`path.normalize` retira `./..` do meio do caminho);
- **Segurança:** se, depois de normalizado, o caminho já não começar por `documentRoot`, é inválido (`null`) — impede que um pedido do tipo `../../etc/passwd` escape da diretoria servida;
- `getMediaType()` deduz o tipo de media pela **extensão** do ficheiro, usando a tabela da configuração; sem extensão reconhecida, assume `text/plain`;
- `isText()` considera texto tudo o que **não** começa por `image` — usado a seguir para decidir a codificação de leitura.

Exemplo resolvido

Ler o ficheiro e responder / processar o pedido.

```
function doGetPathname(pathname, response) {
  const mediaType = getMediaType(pathname);
  const encoding = isText(mediaType) ? 'utf8' : null;

  fs.readFile(pathname, encoding, (err, data) => {
    if (err) {
      response.writeHead(404); // Not Found
      response.end();
    } else {
      response.writeHead(200, { 'Content-Type': mediaType });
      response.end(data);
    }
  });
}

function doGetRequest(request, response) {
  const pathname = getPathname(request);
  if (pathname === null) {
    response.writeHead(403); // Forbidden
    response.end();
  } else {
    fs.stat(pathname, (err, stats) => {
      if (err) {
        response.writeHead(500); // Internal Server Error
        response.end();
      } else if (stats.isDirectory()) {
        if (pathname.endsWith('/')) {
          doGetPathname(pathname + conf.defaultIndex, response);
        } else {
          response.writeHead(301, // Moved Permanently
            { 'Location': pathname + '/' });
          response.end();
        }
      } else {
        doGetPathname(pathname, response);
      }
    });
  }
}
```

- `doGetPathname()` lê o ficheiro com **codificação diferente** consoante seja texto ou não, gera 404 em caso de erro de leitura, e coloca o tipo de media no cabeçalho `Content-Type` em caso de sucesso;
- `doGetRequest()` primeiro valida/mapeia o caminho (403 se inválido); depois usa `fs.stat()` para saber se é ficheiro ou diretoria — pedidos de **diretoria** ou são redirecionados (301, se o URL não terminar em `/`) ou têm o `defaultIndex` acrescentado; erros de `stat` dão 500.

Exemplo resolvido**Servidor (continuação).**

```
http.createServer((request, response) => {
  switch (request.method) {
    case 'GET':
      doGetRequest(request, response);
      break;
    default:
      response.writeHead(501); // 501 Not Implemented
      response.end();
  }
}).listen(conf.port);
```

O servidor final só aceita GET (outros métodos dão 501 Not Implemented), e fecha a resposta de imediato nesse caso; escuta na porta configurada (conf.port).

Exemplo resolvido

Traçar GET /pagina.html (a existir em /home/zp/public_html/about/pagina.html):

1. Chega o pedido; request.method === 'GET' → entra em doGetRequest;
2. getPathname(request): url.parse('/pagina.html').pathname === '/pagina.html'; concatenado com documentRoot e normalizado dá /home/zp/public_html/about/pagina.html; começa por documentRoot, logo é válido (não é null);
3. fs.stat(pathname, ...) é assíncrono — o handler devolve o controlo ao event loop enquanto o disco não responde; quando responde, err é undefined e stats.isDirectory() é false (é um ficheiro);
4. Cai no else final → doGetPathname(pathname, response);
5. getMediaType('.../pagina.html') procura a extensão html na tabela → 'text/html'; isText('text/html') é true → encoding = 'utf8';
6. fs.readFile(pathname, 'utf8', ...) — novamente assíncrono; quando termina sem erro, response.writeHead(200, {'Content-Type': 'text/html'}) seguido de response.end(data) — a resposta é enviada com o conteúdo HTML e a ligação fecha-se.

Se o ficheiro **não existisse**, o passo 6 receberia err definido → response.writeHead(404); response.end(); — página não encontrada, sem corpo.

Servidor do contador partilhado

Um servidor que mantém um **contador partilhado** por todos os clientes ligados: POST para o incrementar/repor, GET para subscrever atualizações em tempo real via **Server-Sent Events (SSE)**.

Atenção

Nota adicional (não estava explícito nos slides): **SSE** é uma técnica em que o servidor mantém a ligação HTTP **aberta** (não chama response.end()) e vai escrevendo, ao longo do tempo, linhas no formato data: <mensagem>\n\n (reparar no **duplo** fim de linha — é o que separa um evento SSE do seguinte). O browser, do lado cliente, usa a API EventSource para ler estes eventos à medida que chegam, sem ter de fazer *polling*. Por isso o cabeçalho da resposta usa 'Content-Type': 'text/event-stream' e 'Connection': 'keep-alive' — é o que se vê a seguir em vars2.js.

Exemplo resolvido

Variáveis de topo e cabeçalhos.

```
// vars.js
"use strict";
let PORT = 8001;

let http    = require('http');
let url     = require('url');
let counter = require('./model.js');
let updater = require('./updater.js');

// vars2.js
const headers = {
  plain: {
    'Content-Type': 'application/json',
    'Cache-Control': 'no-cache',
    'Access-Control-Allow-Origin': '*'
  },
  sse: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Access-Control-Allow-Origin': '*',
    'Connection': 'keep-alive'
  }
};
```

counter implementa a contagem; **updater** gere as ligações abertas para difusão. Há dois conjuntos de cabeçalhos: **plain** para respostas POST (JSON simples, sem manter a ligação aberta), **sse** para a subscrição via GET (mantém a ligação, tipo de conteúdo próprio de SSE).

Exemplo resolvido

Modelo e *updater*.

```
// model.js
let count = 0;
module.exports.incr = function() { count++; };
module.exports.reset = function() { count = 0; };
module.exports.get = function() { return count; };

// updater.js
let responses = [];

module.exports.remember = function(response) {
  responses.push(response);
};
module.exports.forget = function(response) {
  let pos = responses.findIndex((resp) => resp === response);
  if (pos > -1) responses.splice(pos, 1);
};
module.exports.update = function(message) {
  for (let response of responses) {
    response.write('data: ' + message + '\n\n');
  }
};
```

O contador é um módulo simples com todas as funções necessárias exportadas — ilustra a **separação** do modelo em relação ao servidor. O *updater* gere a lista de canais de resposta abertos (**remember**, **forget**) e propaga, por difusão, uma linha **data:** a todos eles (**update**) — reparar no duplo fim de linha exigido pelo formato SSE.

Exemplo resolvido

Tratamento de GET/POST.

```
function doGet(pathname, request, response) {
  let answer = {};
  switch (pathname) {
    case '/update':
      updater.remember(response);
      request.on('close', () => updater.forget(response));
      setImmediate(() => updater.update(counter.get()));
      answer.style = 'sse';
      break;
    default:
      answer.status = 400;
      break;
  }
  return answer;
}

function doPost(pathname) {
  let answer = {};
  switch (pathname) {
    case '/incr':
      counter.incr();
      updater.update(counter.get());
      break;
    case '/reset':
      counter.reset();
      updater.update(counter.get());
      break;
    default:
      answer.status = 400;
      break;
  }
  return answer;
}
```

- `doGet()` só trata `/update`: memoriza a `response` no `updater` (para poder escrever nela mais tarde), registra `request.on('close', ...)` para a **esquecer** quando o cliente desliga, e usa `setImmediate(...)` para difundir a contagem atual **depois** de terminado este processamento (`setImmediate` é semelhante a `setTimeout(fn, 0)` — corre assim que o event loop tiver oportunidade); marca `answer.style = 'sse'`;
- `doPost()` mapeia `/incr` e `/reset` para as operações do `counter`, seguidas de difusão da nova contagem; outros URLs dão erro 400.

Exemplo resolvido

Servidor (continuação).

```

http.createServer(function (request, response) {
  const preq = url.parse(request.url, true);
  const pathname = preq.pathname;
  let answer = {};

  switch (request.method) {
    case 'GET':
      answer = doGet(pathname, request, response);
      break;
    case 'POST':
      answer = doPost(pathname);
      break;
    default:
      answer.status = 400;
  }

  if (answer.status === undefined) answer.status = 200;
  if (answer.style === undefined) answer.style = 'plain';

  response.writeHead(answer.status, headers[answer.style]);
  if (answer.style === 'plain')
    response.end();
}).listen(PORT);

```

O servidor tem um **ponto único** de escrita da resposta: define `status/style` por omissão (200/plain) se não tiverem sido definidos, escreve o cabeçalho adequado, e só fecha a resposta (`response.end()`) se **não** for SSE — nos pedidos SSE a ligação fica deliberadamente **aberta** para a difusão de atualizações futuras.

Exemplo resolvido

Traçar um POST /incr seguido de um GET /update de outro cliente.

1. Cliente A: POST /incr. `request.method === 'POST' → doPost('/incr') → counter.incr()` (contagem passa, digamos, de 3 para 4) → `updater.update(4)` — mas nesse momento a lista `responses` do `updater` já contém o cliente B (ver próximo passo, assumindo que B já tinha aberto o SSE antes) → escreve `'data: 4\n\n'` na `response` de B;
2. De volta ao handler de A: `answer` não tem `status` nem `style` definidos → assume 200/plain → `response.writeHead(200, headers.plain)` → como `style === 'plain'`, chama `response.end()` — a ligação de A fecha-se com uma resposta JSON simples e vazia;
3. Cliente B (já ligado antes, com GET /update pendente): tinha sido registado em `doGet` — `updater.remember(response)` guardou a sua `response` na lista; `response.writeHead(200, headers.sse)` já tinha sido enviado (cabeçalhos SSE, ligação keep-alive) e a `response` **nunca chegou a fechar** — é exactamente essa `response` que recebe, de forma assíncrona, a linha `'data: 4\n\n'` escrita no passo 1, que o browser de B processa via `EventSource` sem ter pedido nada de novo.

Se B fechasse a ligação (fechar o separador, por exemplo), o evento `'close'` do `request` de B dispararia `updater.forget(response)`, removendo-o da lista de difusão.

Servidor do jogo do galo

Um servidor **WebSockets** — um protocolo diferente de HTTP, embora tipicamente negociado a partir de um pedido HTTP inicial (*upgrade*) — que lê e difunde mensagens em **JSON**.

Atenção

Nota adicional (não estava explícito nos slides): o protocolo **WebSocket** começa por um pedido HTTP normal (GET) com cabeçalhos especiais (**Upgrade: websocket**); se o servidor aceitar, a ligação deixa de falar HTTP e passa a ser um canal **full-duplex** (ambos os lados podem enviar mensagens a qualquer momento, sem o modelo pedido/resposta). É por isso que o módulo `http` ainda é necessário no código abaixo — mesmo sem processar pedidos — para servir de base à negociação do *upgrade*, feita pela biblioteca `websocket`.

Exemplo resolvido

Módulos e servidor.

```
const WebSocketServer = require('websocket').server;
const http = require('http');

const httpServer = http.createServer(() => {}).listen(PORT);
const websocketServer = new WebSocketServer({ httpServer });
```

O módulo `http` é necessário mas não processa pedidos (o *callback* de `createServer` está vazio) — serve apenas de base para o `WebSocketServer` (da biblioteca externa `websocket`) negociar a ligação.

```
websocketServer.on('request', function (request) {
  const connection = request.accept(null, request.origin);
  remember(connection);
  connection.sendUTF(JSON.stringify({ board, current }));

  connection.on('message', function (message) {
    if (message.type === 'utf8')
      process(JSON.parse(message.utf8Data));
    else
      console.log('Unsupported message type: ' + message.type);
  });
  connection.on('close', function (connection) { forget(connection); });
});
```

- o `websocketServer` funciona como um **stream de pedidos** (`request`) — cada novo cliente que negocia a ligação gera um `'request'`; `request.accept(...)` cria a `connection` correspondente;
- a **conexão** obtida também funciona como um *stream*, mas de **mensagens** (`'message'`) em vez de `'data'/'end'` — o paralelismo com o módulo `stream` estudado atrás é intencional: eventos, não bloqueio;
- ao aceitar, o servidor logo envia (`sendUTF`) o estado atual do tabuleiro ao novo cliente, e regista-o (`remember`) para receber difusões futuras; mensagens UTF8 são interpretadas como JSON e passadas a `process()`; ao fechar, o cliente é esquecido (`forget`);
- nota nos slides: devia controlar-se a **origem** do pedido (*same origin policy*) — não é feito aqui por simplicidade.

Exemplo resolvido**Difusão e tabuleiro.**

```
// difundir.js
let connections = [];

function remember(connection) {
    connections.push(connection);
}

function forget(connection) {
    let pos = connections.findIndex((conn) => conn === connection);
    if (pos > -1) connections.splice(pos, 1);
}

function broadcast(data) {
    let json = JSON.stringify(data);
    for (let connection of connections) {
        connection.sendUTF(json);
    }
}

// tabuleiro.js
let current = 'X';
let board = undefined;

function initBoard() {
    board = new Array(9);
    for (let b = 0; b < 9; b++) board[b] = '&nbsp;';
}

function play(pos) {
    let piece = current;
    current = (current == 'X' ? 'O' : 'X');
    board[pos] = piece;
}

initBoard();
```

Exemplo resolvido

Processar mensagens (continuação).

```
// processar.js
function process(message) {
  switch (message.command) {
    case 'reset':
      initBoard();
      broadcast({ board, current });
      break;
    case 'play':
      if ('pos' in message) {
        play(message.pos);
        broadcast({ pos: message.pos, piece: current, current });
      } else
        console.log('pos expected in message');
      break;
    default:
      console.log('Invalid command: ' + message.command);
  }
}
```

- `broadcast()` faz o mesmo que o `updater.update()` do contador, mas sobre WebSockets: envia a todas as `connections` guardadas;
- `process()` reage ao campo `command` da mensagem recebida — `'reset'` reinicia o tabuleiro e difunde o novo estado completo; `'play'` requer a propriedade `pos`, joga a peça atual nessa posição, alterna `current` entre `'X'/'O'`, e difunde apenas a jogada (posição, peça, próximo a jogar) — não o tabuleiro inteiro.

Exemplo resolvido

Traçar uma jogada. Dois clientes (A e B) já ligados (`connections` tem as duas ligações); é a vez de `'X'` (`current === 'X'`); A envia a mensagem `{"command": "play", "pos": 4}` (jogar no centro do tabuleiro):

1. `connection.on('message', ...)` de A dispara; `message.type === 'utf8' → process(JSON.parse(message.utf8Data)) → process({command: 'play', pos: 4})`;
2. Dentro de `process`, cai no `case 'play'`; `'pos' in message` é verdade → `play(4): piece = 'X'` (valor de `current` antes de mudar), `current` passa a `'O'`, `board[4] = 'X'`;
3. `broadcast({pos: 4, piece: 'X', current: 'O'})` — percorre `connections` (A e B) e chama `sendUTF(json)` em **cada uma**;
4. Tanto A como B recebem a mesma mensagem JSON pela ligação WebSocket, já aberta e full-duplex — cada cliente atualiza a sua vista do tabuleiro (posição 4 preenchida com X) e sabe que agora é a vez de `O` (`current`).

Se, em vez disso, chegasse `{"command": "reset"}`: cairia no primeiro `case`, `initBoard()` reporia o tabuleiro a vazio e `current` continuaria o que estivesse, e `broadcast({board, current})` enviaria o **tabuleiro completo** (não só uma jogada) a A e B.

CGI e Alternativas

O modelo CGI

O **Common Gateway Interface (CGI)** foi a primeira tentativa de normalizar a interface entre servidores HTTP e aplicações — os primeiros servidores HTTP foram desenhados apenas para conteúdo **estático**, e rapidamente surgiu a necessidade de gerar conteúdo **dinâmico** através de programas externos, o que exigiu a definição de um protocolo de comunicação entre servidor e esses programas. Inicialmente pensado para CGIs escritos em C, mas com suporte para **qualquer linguagem** capaz de:

Definição

- ler dados pela entrada *standard* (`stdin`);
- escrever resultados na saída *standard* (`stdout`);
- ler **variáveis de ambiente**.

A ideia central do modelo de execução:

Definição — Um processo por pedido

```
#!/usr/bin/bash
echo "Content-type: text/html"
echo ""
echo "<h1>Olá mundo</h1>"
```

- o servidor HTTP lança **um processo CGI novo por cada pedido**;
- o **ambiente de execução** desse processo contém linhas de cabeçalho e informação genérica sobre o pedido, codificadas como **variáveis de ambiente** (ver secção seguinte);
- a **entrada standard** (`stdin`) do processo recebe o **corpo** do pedido HTTP;
- a **saída standard** (`stdout`) do processo é usada pelo programa para produzir: o final do cabeçalho, uma linha de separação, e o corpo da resposta (ver secção “Entrada e saída”).

A generalidade dos servidores HTTP suporta CGI (há inclusive um módulo Node.js para correr CGIs), embora hoje em dia existam alternativas mais eficientes ao CGI puro, como **Fast-CGI** e **SCGI** (*Simple Common Gateway Interface*) — não cobertas em detalhe aqui, mas que resolvem precisamente o problema de custo por pedido identificado a seguir.

Atenção

Nota adicional (não estava explícito nos slides): tecnicamente, “lançar um processo novo” significa que o servidor HTTP faz um `fork()` (duplica-se a si próprio) seguido de `exec()` (o processo filho substitui o seu código pelo do interpretador/programa CGI), passando as variáveis de ambiente através da chamada de sistema `execve`. Isto é uma operação relativamente **cara** ao nível do sistema operativo — envolve criar um novo espaço de endereçamento, carregar o interpretador (ex: arrancar todo o Python ou o Bash de novo), e destruir tudo isso outra vez quando o processo termina. É esse custo, pago **a cada pedido**, que motiva o contraste com o modelo Node visto atrás.

Este contraste — processo novo por pedido (CGI) vs. processo único com event loop (Node) — é exactamente o que o diagrama da secção anterior ilustra lado a lado: no CGI, cada pedido implica criar, correr e destruir um processo do zero; no Node, o mesmo processo, sempre vivo, atende todos os pedidos, delegando as operações de I/O de forma assíncrona para nunca ficar bloqueado.

Frequente em exame

Pergunta típica: **por que é que o modelo CGI escala pior que o modelo Node sob carga elevada?** Porque o custo de `fork()+exec()` (criar um processo do sistema operativo, com o seu próprio espaço de memória) é pago a **cada pedido**, independentemente de quão simples esse pedido seja — em contraste com o Node, onde o “custo fixo” (arrancar o processo Node) só é pago **uma vez**, e cada pedido subsequente só ocupa a CPU durante o tempo do seu próprio processamento em JavaScript.

Variáveis de ambiente CGI

As **variáveis de ambiente** são, de forma genérica, um conjunto de pares nome/valor usados para passar metadados, definidos pelo contexto em que um processo corre (ex: a *shell* que o lançou). O CGI usa-as como mecanismo principal para o servidor comunicar informação sobre o pedido ao programa.

Definição

O conjunto de variáveis definido pelo CGI evoluiu em duas versões:

- **CGI 1.0** — conjunto restrito, as variáveis “originais”;
- **CGI 1.1** — alargado, introduzindo **novos conjuntos**, cada um identificado por um prefixo próprio: **HTTP_** (cabeçalhos do pedido), variáveis relativas ao **cliente remoto**, e variáveis relativas ao **servidor**.

A tabela seguinte resume as mais importantes de cada conjunto:

Conjunto	Variável	Contém
Original (1.0)	DOCUMENT_ROOT	diretoria servida pelo servidor
	GATEWAY_INTERFACE	versão do CGI (ex: "CGI/1.1")
	HOME	casa do utilizador do processo do servidor
	PATH_INFO	caminho pedido (relativo)
	PATH_TRANSLATED	caminho correspondente no sistema de ficheiros
HTTP (1.1)	QUERY_STRING	parte do pedido depois do ?
	HTTP_ACCEPT	tipos de media aceites pelo cliente
	HTTP_ACCEPT_LANGUAGE	idiomas preferidos do cliente
	HTTP_HOST	<i>host</i> pedido (cabeçalho <i>Host</i>)
	HTTP_USER_AGENT	identificação do <i>browser</i> /cliente
Cliente remoto (1.1)	HTTP_CONNECTION	tipo de ligação (ex: keep-alive)
	REMOTE_ADDR	endereço IP do cliente
	REMOTE_PORT	porta de origem do cliente
	REQUEST_METHOD	método HTTP do pedido
	REQUEST_URI	URI completo pedido
Servidor (1.1)	CONTENT_TYPE / CONTENT_LENGTH	tipo e tamanho do corpo (ex: dados de formulário)
	REMOTE_USER / AUTH_TYPE	utilizador e tipo de autenticação HTTP, se aplicável
	SERVER_ADDR / SERVER_NAME	endereço/nome do servidor
	SERVER_PORT	porta onde o servidor escuta
	SERVER_PROTOCOL	versão do protocolo HTTP
	SERVER_SOFTWARE	identificação do software do servidor
	SERVER_ADMIN	contacto do administrador

Frequente em exame

Relação a memorizar (aparece explicitamente nos slides como nota de rodapé): **REQUEST_URI = PATH_INFO + QUERY_STRING** — o URI completo do pedido é a concatenação do caminho pedido com a *query-string* (separados pelo `?`, que não pertence a nenhuma das duas partes).

Entrada e saída num programa CGI

Input. Os dados recebidos no **corpo** de um pedido são passados ao CGI via **stdin** — necessário para processar pedidos **POST** (tipicamente submissões de formulários). É preciso decodificar esses dados consoante a sua codificação — as duas mais comuns são **multipart/form-data** e **application/x-www-form-urlencoded**. As variáveis de ambiente relevantes para saber **como** ler são **REQUEST_METHOD**, **CONTENT_TYPE** e **CONTENT_LENGTH** (para saber quantos bytes ler de **stdin**). Existem bibliotecas prontas para converter estes formatos, mas dependem sempre da linguagem de implementação do CGI (não há uma solução universal, ao contrário de um módulo **http** do Node que já entrega tudo decomposto).

Output. O CGI deve escrever a mensagem de resposta em **stdout**, respeitando o formato:

Definição

1. **Linha inicial** (opcional) — pode começar por **Status**: se for necessário indicar um código de estado diferente do razoável por omissão;
2. **Cabeçalhos** — o único **obrigatório** é **Content-type**;
3. **Linha em branco** — obrigatória, separa cabeçalhos do corpo (o paralelo direto da linha em branco de uma mensagem HTTP “a sério”);
4. **Corpo da mensagem** (opcional).

Exemplo resolvido

Script CGI completo (Bash, como nos slides) — lê variáveis de ambiente e escreve a resposta:

```
#!/usr/bin/bash

# "Input": não há corpo neste pedido GET, mas seria lido de stdin
# usando CONTENT_LENGTH para saber quantos bytes ler, se REQUEST_METHOD
# fosse POST. Aqui só se usam variáveis de ambiente do próprio pedido:

echo "Content-type: text/html"      # cabeçalho obrigatório
echo ""                             # linha em branco obrigatória
echo "<h1>Olá mundo</h1>"
echo "<p>Método: $REQUEST_METHOD</p>"
echo "<p>Caminho pedido: $PATH_INFO</p>"
echo "<p>Query string: $QUERY_STRING</p>"
echo "<p>Cliente: $REMOTE_ADDR</p>"
```

Um pedido GET /cgi-bin/ola.cgi?nome=Ana faria o servidor lançar este script com, entre outras, REQUEST_METHOD="GET", PATH_INFO="/ola.cgi" (ou caminho equivalente), QUERY_STRING="nome=Ana" e REMOTE_ADDR com o IP do cliente já definidas no ambiente do processo. O stdout do programa — visto pelo servidor — é exactamente o que este script echo-a: primeiro o cabeçalho Content-type, depois a linha em branco obrigatória, depois o corpo HTML. O servidor lê este stdout e reencaminha-o como resposta HTTP ao cliente, terminando o processo CGI logo a seguir.

Pseudocódigo equivalente para um POST com leitura de corpo:

```
ler CONTENT_LENGTH do ambiente      # ex: "27"
ler esse número de bytes de stdin   # ex: "nome=Ana&idade=25"
descodificar application/x-www-form-urlencoded → {nome: "Ana", idade: "25"}
```

escrever em stdout:

```
"Content-type: text/html"
""
"<p>Recebido nome=Ana, idade=25</p>"
```